**UNIVERSIDAD PRIVADA DEL VALLE**
**FACULTAD DE INFORMÁTICA Y ELECTRÓNICA**
**CARRERA DE LICENCIATURA EN INGENIERÍA DE SISTEMAS INFORMÁTICOS**

# SISTEMA WEB Y APLICACIÓN MÓVIL APLICANDO REDES NEURONALES CONVOLUCIONALES EFFICIENTNET-B4 Y EL ALGORITMO COSINE SIMILITARY PARA LA IDENTIFICACIÓN DE PERROS EXTRAVIADOS Y REDUCCIÓN DE TIEMPO DE PERMANENCIA EN EL REFUGIO

## Caso: Refugio WOOF

**PROYECTO DE GRADO PARA OPTAR AL TÍTULO DE LICENCIATURA EN INGENIERÍA DE SISTEMAS INFORMÁTICOS**

**POSTULANTE:** BRITNY MIROSLAVA SEA HERRERA
**TUTOR:** ING. JULIO CESAR IRAHOLA FLORES

La Paz – Bolivia
2026

# DEDICATORIA

A mi familia, pilar fundamental de mi vida. A mi madre Verónica, por su amor incondicional, su sacrificio y su confianza en mis sueños. A la memoria de mi padre Samuel, cuya inspiración y valores siguen guiando mi camino. A mis hermanos Armin y Ayrton, por su apoyo constante y complicidad.

A mi enamorado Osmar, por estar a mi lado en cada paso de este proceso, brindándome fortaleza y comprensión.

Este proyecto de grado es un reflejo del amor y el esfuerzo de todos ustedes. A ellos dedico este trabajo con todo mi corazón.

# AGRADECIMIENTOS

Agradezco a Dios por la sabiduría y la fortaleza para superar cada obstáculo durante el desarrollo de este proyecto de grado. Su guía ha sido fundamental en cada etapa de este camino académico y profesional.

A la Universidad del Valle (UNIVALLE) por la formación académica proporcionada que permitió desarrollar los conocimientos necesarios para este proyecto.

Al Ing. Julio César Irahola Flores, mi tutor, por su guía, disponibilidad y contribución al desarrollo de este proyecto. Su supervisión fue fundamental para alcanzar la calidad esperada.

A mi familia, especialmente por su apoyo incondicional, comprensión y paciencia durante todo este proceso. Sus palabras de aliento en los momentos difíciles y su confianza fueron esenciales para la conclusión exitosa de este proyecto de grado.

## RESUMEN

La identificación de perros extraviados es un problema crítico en contextos urbanos que afecta directamente el bienestar animal y genera costos operativos significativos en refugios. Actualmente, los métodos tradicionales basados en publicaciones en redes sociales presentan baja efectividad y lentitud en el proceso de reencuentro. Este trabajo desarrolló un sistema integrado compuesto por una aplicación móvil Android y un panel web administrativo que aplica redes neuronales convolucionales EfficientNet-B4 y el algoritmo Cosine Similarity para identificar perros extraviados de manera automática. Se empleó la metodología Extreme Programming (XP) para el desarrollo iterativo del software, complementada con el marco CRISP-DM para el ciclo de entrenamiento del modelo de inteligencia artificial, combinando Clean Architecture y patrones de diseño. El sistema fue validado en el Refugio WOOF de El Alto con casos reales, demostrando que identifica correctamente perros extraviados mediante análisis automático de similitud visual entre fotografías. El modelo EfficientNet-B4 alcanzó precisión superior al 90% tras entrenamiento en fases progresivas de descongelamiento. El resultado más significativo fue la reducción de permanencia en refugio: transformando periodos de meses a días, demostrando impacto transformacional en bienestar animal, costos operativos y satisfacción de propietarios. El sistema permite que los dueños de perros extraviados contacten directamente con quienes reportan perros encontrados, facilitando reencuentros rápidos y seguros. El sistema está completamente operacional y listo para producción en contextos reales, validando efectivamente que la combinación de inteligencia artificial moderna, arquitectura distribuida y comprensión profunda del problema resuelve desafíos sociales significativos.



**Palabras Clave:** Identificación de perros, Redes neuronales convolucionales, EfficientNet-B4, Cosine Similarity, Bienestar animal, Sistema distribuido, Aplicación móvil, Refugio, Página web.

## ABSTRACT

The identification of lost dogs is a critical problem in urban contexts that directly affects animal welfare and generates significant operational costs in shelters. Currently, traditional methods based on social media publications demonstrate low effectiveness and slowness in the reunification process. This work developed an integrated system composed of an Android mobile application and an administrative web panel that applies convolutional neural networks EfficientNet-B4 and the Cosine Similarity algorithm to automatically identify lost dogs. Extreme Programming (XP) methodology was employed for iterative software development, complemented by the CRISP-DM framework for the artificial intelligence model training cycle, combining Clean Architecture and design patterns. The system was validated at the WOOF Shelter in El Alto with real cases, demonstrating accurate identification of lost dogs through automatic visual similarity analysis between photographs. The EfficientNet-B4 model achieved precision exceeding 90% after progressive unfreezing training phases. The most significant result was the reduction in shelter permanence: transforming periods of months to days, demonstrating transformational impact on animal welfare, operational costs, and owner satisfaction. The system enables dog owners to directly contact individuals who report found dogs, facilitating rapid and secure reunions. The system is fully operational and ready for production in real contexts, effectively validating that the combination of modern artificial intelligence, distributed architecture, and deep problem understanding resolves significant social challenges.



**Keywords:** Dog identification, Convolutional neural networks, EfficientNet-B4, Cosine Similarity, Animal welfare, Distributed system, Mobile application, Shelter, Web platform.

## ÍNDICE DE CONTENIDO

### CAPÍTULO I
### MARCO INTRODUCTORIO


1.1. INTRODUCCIÓN ..... 1
1.2. PLANTEAMIENTO DEL PROBLEMA ..... 2
*1.2.1. ANTECEDENTES ..... 2*
*1.2.2. CONTEXTUALIZACIÓN ..... 4*
*1.2.3. IDENTIFICACIÓN DEL PROBLEMA ..... 5*
1.3. FORMULACIÓN DEL PROBLEMA ..... 5
1.4. JUSTIFICACIÓN ..... 6
*1.4.1. JUSTIFICACIÓN SOCIAL ..... 6*
*1.4.2. JUSTIFICACIÓN TÉCNICA ..... 6*
*1.4.3. JUSTIFICACIÓN ECONÓMICA ..... 7*
1.5. OBJETIVOS DE LA INVESTIGACIÓN ..... 8
*1.5.1. OBJETIVO GENERAL ..... 8*
*1.5.2. OBJETIVOS ESPECÍFICOS ..... 8*
1.6. DELIMITACIÓN ..... 9
*1.6.1. DELIMITACIÓN ESPACIAL ..... 9*
*1.6.2. DELIMITACIÓN TEMPORAL ..... 10*
*1.6.3. DELIMITACIÓN POBLACIONAL ..... 10*
*1.6.4. DELIMITACIÓN TEÓRICA ..... 10*
1.7. ALCANCES Y LÍMITES ..... 11
*1.7.1. ALCANCES ..... 11*
*1.7.2. LÍMITES ..... 12*
1.8. METODOLOGÍA ..... 13
*1.8.1. ENFOQUES DE LA INVESTIGACIÓN ..... 13*
*1.8.2. MÉTODO DE INVESTIGACIÓN ..... 13*
*1.8.3. TIPO DE INVESTIGACIÓN ..... 14*
*1.8.4. DISEÑO DE INVESTIGACIÓN ..... 14*
*1.8.5. TÉCNICAS E INSTRUMENTOS ..... 15*
*1.8.6. POBLACIÓN ..... 16*
*1.8.7. MUESTRA ..... 17*


### CAPÍTULO II
### MARCO TEÓRICO


2.1. MARCO TEÓRICO ..... 18
*2.1.1. INGENIERÍA DE SOFTWARE ..... 18*
*2.1.2. MODELOS DE DESARROLLO ..... 19*
*2.1.3. LENGUAJE UNIFICADO DE MODELADO UML ..... 22*
*2.1.4. METODOLOGÍA DE DESARROLLO DE SOFTWARE ..... 25*
*2.1.5. HERRAMIENTAS DE DESARROLLO ..... 32*
*2.1.6. ESTIMACIÓN DE COSTOS ..... 34*
*2.1.7. ASEGURAMIENTO DE CALIDAD ..... 35*
*2.1.8. SEGURIDAD DEL SISTEMA ..... 36*
2.2. MARCO CONCEPTUAL ..... 38
*2.2.1. SISTEMA WEB ..... 38*

*2.2.2. APLICACIÓN MÓVIL ..... 38*
*2.2.3. ARQUITECTURA CLIENTE-SERVIDOR ..... 39*
*2.2.4. BACKEND (SERVIDOR DE API) ..... 39*
*2.2.5. BASE DE DATOS NO RELACIONAL (NOSQL) ..... 39*
*2.2.6. SERVICIOS EN LA NUBE (CLOUD COMPUTING) ..... 40*
*2.2.7. FIREBASE ..... 40*
*2.2.8. CLOUDINARY ..... 41*
*2.2.9. RAILWAY (PLATAFORMA DE DESPLIEGUE) ..... 41*
*2.2.10. FASTAPI ..... 41*
*2.2.11. RETROFIT ..... 41*
*2.2.12. HILT ..... 42*
*2.2.13. IDENTIFICACIÓN DE PERROS EXTRAVIADOS ..... 42*
*2.2.14. REFUGIO O ALBERGUE ANIMAL ..... 42*
*2.2.15. REENCUENTRO DE MASCOTAS ..... 43*
*2.2.16. PROCESAMIENTO DE IMÁGENES ..... 43*
*2.2.17. INTELIGENCIA ARTIFICIAL Y APRENDIZAJE AUTOMÁTICO ..... 43*
*2.2.18. REDES NEURONALES ..... 45*
*2.2.19. COMPONENTES DE LA ARQUITECTURA DEL MODELO ..... 46*
*2.2.20. EXTRACCIÓN Y REPRESENTACIÓN DE CARACTERÍSTICAS ..... 48*
*2.2.21. TÉCNICAS DE ENTRENAMIENTO Y COMPARACIÓN ..... 48*
*2.2.22. ENTORNO Y RECURSOS DE ENTRENAMIENTO ..... 50*
*2.2.23. EVALUACIÓN DEL MODELO ..... 52*
*2.2.24. UMBRAL DE CONFIANZA ..... 53*
*2.2.25. PRECISIÓN TOP-K (TOP-K ACCURACY) ..... 53*
*2.2.26. MATRIZ DE CONFUSIÓN ..... 54*
*2.2.27. COEFICIENTE DE DETERMINACIÓN ($R^2$) ..... 54*
2.3. MARCO INSTITUCIONAL ..... 55
*2.3.1. MISIÓN ..... 56*
*2.3.2. VISIÓN ..... 56*
*2.3.3. OBJETIVO ..... 56*
*2.3.4. OBJETIVOS ESPECÍFICOS ..... 56*
*2.3.5. VALORES ..... 57*
2.4. ESTADO DEL ARTE ..... 58

**CAPÍTULO III**
**INGENIERÍA DEL PROYECTO**

3.1. INTRODUCCIÓN ..... 62
3.2. OBJETIVOS DE LA PROPUESTA ..... 62
3.3. DESARROLLO DE LA PROPUESTA ..... 63
*3.3.1. PLANIFICACIÓN DEL DESARROLLO ..... 63*
*3.3.2. ROLES DEL EQUIPO XP ..... 64*
*3.3.3. HISTORIAS DE USUARIO ..... 65*
*3.3.4. RESUMEN DE HISTORIAS DE USUARIO ..... 73*
*3.3.5. PLAN DE ITERACIONES ..... 74*
*3.3.6. REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES ..... 76*
*3.3.7. ITERACIÓN 1 - ENTRENAMIENTO DEL MODELO DE INTELIGENCIA ARTIFICIAL ..... 79*
*3.3.8. ITERACIÓN 2 - BACKEND FASTAPI ..... 97*

*3.3.9. ITERACIÓN 3 - APLICACIÓN MÓVIL ANDROID ..... 123*
*3.3.10. ITERACIÓN 4 - PANEL WEB ADMINISTRATIVO ..... 158*
*3.3.11. ITERACIÓN 5 - PRUEBAS DE INTEGRACIÓN Y DESPLIEGUE EN PRODUCCIÓN ..... 174*
3.4. RESULTADOS DE LA PROPUESTA ..... 187
*3.4.1. OBJETIVO 1: DISEÑO DE LA INTERFAZ DE USUARIO ..... 188*
*3.4.2. OBJETIVO 2: IMPLEMENTAR BASE DE DATOS NO RELACIONAL ..... 189*
*3.4.3. OBJETIVO 3: INTEGRAR EFFICIENTNET-B4 Y COSINE SIMILARITY ..... 190*
*3.4.4. OBJETIVO 4: DESARROLLAR PANEL DE CONTROL ADMINISTRATIVO ..... 191*
*3.4.5. OBJETIVO 5: REALIZAR PLAN DE PRUEBAS FUNCIONALES Y NO FUNCIONALES ..... 191*

**CAPÍTULO IV**
**MARCO PRÁCTICO**

4.1. PRESENTACIÓN DE RESULTADOS ..... 192
*4.1.1. MODELO EFFICIENTNET-B4 ..... 193*
*4.1.2. DASHBOARD Y ACTIVIDAD DE LA PÁGINA WEB ..... 193*
*4.1.3. GESTIÓN DE REPORTES EN APP MÓVIL ..... 194*
*4.1.4. DISTRIBUCIÓN DE ESTADOS ..... 195*
*4.1.5. ALGORITMO COSINE SIMILARITY: COINCIDENCIAS ..... 195*
*4.1.6. DETALLE DE COINCIDENCIA ..... 196*
*4.1.7. REPORTES PDF ..... 197*
*4.1.8. PERMANENCIA EN REFUGIO ..... 199*
4.2. ANÁLISIS DE RESULTADOS ..... 199
*4.2.1. ANÁLISIS DEL MODELO EFFICIENTNET-B4 ..... 199*
*4.2.2. ANÁLISIS DEL DASHBOARD DEL SISTEMA ..... 199*
*4.2.3. ANÁLISIS DE GESTIÓN DE REPORTES ..... 200*
*4.2.4. ANÁLISIS DE DISTRIBUCIÓN DE ESTADOS ..... 200*
*4.2.5. ANÁLISIS DEL ALGORITMO COSINE SIMILARITY ..... 200*
*4.2.6. ANÁLISIS DEL MODAL DE COINCIDENCIA ..... 200*
*4.2.7. ANÁLISIS DE REPORTES PDF ..... 201*
*4.2.8. ANÁLISIS DE PERMANENCIA EN EL REFUGIO ..... 201*

**CAPÍTULO V**
**CONCLUSIONES Y RECOMENDACIONES**

5.1. CONCLUSIONES ..... 202
5.2. RECOMENDACIONES ..... 203
REFERENCIAS BIBLIOGRÁFICAS ..... 204
APÉNDICES ..... 208
ANEXOS ..... 228

# ÍNDICE DE TABLAS

Tabla N° 3.1 Matriz de planificación ..... 63
Tabla N° 3.2 Roles del equipo XP ..... 64
Tabla N° 3.3 Resumen de Historias de Usuario ..... 73
Tabla N° 3.4 Plan de Iteraciones ..... 75
Tabla N° 3.5 Requerimientos Funcionales ..... 76
Tabla N° 3.6 Requerimientos No Funcionales ..... 78
Tabla N° 3.7 Actividades de la Iteración 1 ..... 79
Tabla N° 3.8 Datasets utilizados para el entrenamiento ..... 80
Tabla N° 3.9 Estrategia de entrenamiento ..... 83
Tabla N° 3.10 Archivos generados en la exportación del entrenamiento del modelo ..... 96
Tabla N° 3.11 Actividades de la Iteración 2 ..... 97
Tabla N° 3.12 Colecciones y campos principales de la base de datos ..... 100
Tabla N° 3.13 Diccionario de datos - Colección users ..... 101
Tabla N° 3.14 Diccionario de datos - Colección lost_dogs ..... 102
Tabla N° 3.15 Diccionario de datos - Colección found_dog_reports ..... 103
Tabla N° 3.16 Endpoints de la API ..... 103
Tabla N° 3.17 Pruebas de la Iteración 2 ..... 117
Tabla N° 3.18 Actividades de la Iteración 3 ..... 123
Tabla N° 3.19 Historias de usuario de la Iteración 3 ..... 124
Tabla N° 3.20 Pruebas de la Iteración 3 ..... 157
Tabla N° 3.21 Actividades de la Iteración 4 ..... 158
Tabla N° 3.22 Historias de usuario de la Iteración 4 ..... 158
Tabla N° 3.23 Estructura de la arquitectura modular del panel web ..... 161
Tabla N° 3.24 Pruebas de la Iteración 4 ..... 173
Tabla N° 3.25 Actividades de la Iteración 5 ..... 175
Tabla N° 3.26 Pruebas end-to-end ..... 177
Tabla N° 3.27 Pruebas de regresión ..... 178
Tabla N° 3.28 Pruebas funcionales manuales ..... 179
Tabla N° 3.29 Pruebas de rendimiento de tiempos de respuesta medidos ..... 182
Tabla N° 3.30 Resultados de las pruebas de rendimiento ..... 183
Tabla N° 3.31 Pruebas de seguridad ..... 183
Tabla N° 3.32 Compatibilidad por versión de Android ..... 185
Tabla N° 3.33 Configuración de despliegue del backend en Railway ..... 186
Tabla N° 3.34 Configuración de despliegue del panel web en Firebase Hosting ..... 187
Tabla N° 3.35 Cumplimiento de Objetivos de la Propuesta ..... 187
Tabla N° 4.1 Resumen de resultados y aporte al sistema ..... 192
Tabla N° 4.2 Evolución de la Precisión ..... 193

## ÍNDICE DE CUADROS

Cuadro N° 3.1 HU-01 Registro de cuenta en la aplicación móvil ..... 66
Cuadro N° 3.2 HU-02 Inicio de sesión en la aplicación móvil ..... 66
Cuadro N° 3.3 HU-03 Número de teléfono obligatorio ..... 67
Cuadro N° 3.4 HU-04 Registro de perro extraviado ..... 68
Cuadro N° 3.5 Búsqueda de coincidencias con perro encontrado ..... 68
Cuadro N° 3.6 HU-06 Visualización de resultados de coincidencia ..... 69
Cuadro N° 3.7 HU-07 Visualización de reportes propios ..... 69
Cuadro N° 3.8 HU-08 Recepción de notificaciones push de coincidencia ..... 70
Cuadro N° 3.9 HU-09 Inicio de sesión en el panel web administrativo ..... 71
Cuadro N° 3.10 HU-10 Gestión de usuarios desde el panel web ..... 71
Cuadro N° 3.11 HU-11 Supervisión de reportes desde el panel web ..... 72

# ÍNDICE DE FIGURAS

Figura N° 1.1 Diagrama Ishikawa ..... 5
Figura N° 1.2 Ubicación del Refugio WOOF ..... 10
Figura N° 2.1 Modelo en Cascada ..... 19
Figura N° 2.2 Modelo Iterativo e Incremental ..... 20
Figura N° 2.3 Modelo en V ..... 20
Figura N° 2.4 Modelo Espiral ..... 21
Figura N° 2.5 Ejemplo de Diagrama de Clases ..... 23
Figura N° 2.6 Ejemplo de Diagrama de Despliegue ..... 23
Figura N° 2.7 Ejemplo de Diagrama de Casos de Uso ..... 24
Figura N° 2.8 Ejemplo de Diagrama de Secuencia ..... 25
Figura N° 2.9 Ciclo de Desarrollo Extreme Programming (XP) ..... 28
Figura N° 2.10 Ciclo de vida CRISP-DM ..... 32
Figura N° 2.11 Formula del coseno de similitud ..... 49
Figura N° 2.12 Fórmula de Pseudo-$R^2$ de McFadden ..... 54
Figura N° 3.1 Código de configuración de constantes del entorno ..... 81
Figura N° 3.2 Código de combinación de datasets ..... 82
Figura N° 3.3 Código de configuración del aumento de datos ..... 83
Figura N° 3.4 Diagrama de Secuencia del Entrenamiento EfficientNet-B4 ..... 84
Figura N° 3.5 Código de construcción de la arquitectura EfficientNet-B4 ..... 84
Figura N° 3.6 Código de Entrenamiento Fase 1 ..... 85
Figura N° 3.7 Output del entrenamiento Fase 1 ..... 85
Figura N° 3.8 Código de Entrenamiento Fase 2 ..... 86
Figura N° 3.9 Output del entrenamiento Fase 2 ..... 86
Figura N° 3.10 Código de Entrenamiento Fase 3 ..... 86
Figura N° 3.11 Output del entrenamiento Fase 3 ..... 87
Figura N° 3.12 Código de función de predicción con umbral de confianza ..... 87
Figura N° 3.13 Resultados de la evaluación final del modelo sobre el conjunto de validación ..... 88
Figura N° 3.14 Código de generación de curvas de entrenamiento ..... 89
Figura N° 3.15 Resultados de la distribución de precisión por clase ..... 92
Figura N° 3.16 Resultados del coeficiente de determinación ..... 93
Figura N° 3.17 Prueba de rechazo de imágenes no pertenecientes a perros ..... 94
Figura N° 3.18 Prueba de reconocimiento de perros ..... 95
Figura N° 3.19 Código de exportación del modelo ..... 96
Figura N° 3.20 Diagrama de Despliegue ..... 98
Figura N° 3.21 Diagrama de flujo completo EfficientNet-B4 + Cosine Similarity ..... 99
Figura N° 3.22 Diseño de la base de datos no relacional ..... 101
Figura N° 3.23 Diagrama de Casos de Uso del Backend FastAPI ..... 105
Figura N° 3.24 Dependencias del proyecto ..... 105
Figura N° 3.25 Código de configuración centralizada del servidor ..... 106
Figura N° 3.26 Código de inicialización de la aplicación FastAPI con ciclo de vida ..... 107
Figura N° 3.27 Código de Verificación de token Firebase JWT ..... 108
Figura N° 3.28 Código de análisis de imagen con extracción de embedding ..... 108
Figura N° 3.29 Código de preprocesamiento de imagen ..... 109

Figura N° 3.30 Código de inicialización de Firebase y Cloudinary ..... 110
Figura N° 3.31 Código de Endpoint de validación de fotografía ..... 111
Figura N° 3.32 Código de validación de límite semanal ..... 112
Figura N° 3.33 Código de validación de fotos ..... 112
Figura N° 3.34 Código de análisis de las fotos ..... 113
Figura N° 3.35 Código de verificación de imágenes duplicadas ..... 113
Figura N° 3.36 Código de guardado del reporte en la base de datos ..... 114
Figura N° 3.37 Código de verificación de duplicados ..... 115
Figura N° 3.38 Código de búsqueda de coincidencias ..... 115
Figura N° 3.39 Código de notificación de similitud ..... 116
Figura N° 3.40 Código de guardado del reporte en la base de datos ..... 116
Figura N° 3.41 Código de Endpoint de verificación de estado del servidor ..... 117
Figura N° 3.42 Respuesta de autenticación Firebase con token JWT ..... 118
Figura N° 3.43 Código de validación de foto ..... 119
Figura N° 3.44 Captura de respuesta 200 OK del endpoint /validate-photo ..... 119
Figura N° 3.45 Código de detección de duplicado en registro de perro extraviado ..... 119
Figura N° 3.46 Captura de respuesta 409 Conflict por duplicado detectado ..... 120
Figura N° 3.47 Código de fallo de autenticación por token faltante ..... 120
Figura N° 3.48 Captura de respuesta 401 por falta de autenticación ..... 120
Figura N° 3.49 Código de rechazo de imagen que no contiene perro ..... 121
Figura N° 3.50 Captura de respuesta correcta rechazando imagen sin perro ..... 121
Figura N° 3.51 Código de protección contra conflicto de reportes propios ..... 122
Figura N° 3.52 Captura de rechazo de intento de reportar propio perro como encontrado ..... 122
Figura N° 3.53 Captura de servidor operativo y modelo cargado ..... 122
Figura N° 3.54 Diagrama de Caso de Uso de la aplicación móvil ..... 125
Figura N° 3.55 Diagrama de Secuencia de la aplicación móvil ..... 126
Figura N° 3.56 Mockup de splash screen y estado sin conexión a internet ..... 127
Figura N° 3.57 Mockup de pantallas de autenticación: Inicio de sesión y registro ..... 127
Figura N° 3.58 Mockup de pantalla de inicio y pantalla de perfil ..... 128
Figura N° 3.59 Mockup de formulario de reporte de perro extraviado ..... 129
Figura N° 3.60 Mockup de formulario de reporte de perro encontrado ..... 130
Figura N° 3.61 Mockup de pantalla de resultados de coincidencias y estado sin coincidencias ..... 131
Figura N° 3.62 Mockup de pantalla de mis reportes y notificación ..... 132
Figura N° 3.63 Clase de aplicación con inicialización de Firebase y Hilt ..... 133
Figura N° 3.64 Código de clase sellada UiState para manejo de estados ..... 133
Figura N° 3.65 Código de inyección de dependencias con Hilt ..... 134
Figura N° 3.66 Código de interfaz retrofit para comunicación con el backend ..... 135
Figura N° 3.67 Código de registro e inicio de sesión con verificación de email ..... 136
Figura N° 3.68 Código de token, validación de fotos y manejo de errores HTTP ..... 136
Figura N° 3.69 Código de validación de contraseña segura ..... 137
Figura N° 3.70 Código de preprocesamiento de imagen ..... 138
Figura N° 3.71 Código de detección de bloqueo de cuenta ..... 138
Figura N° 3.72 Código de listener de bloqueo de cuenta de usuario ..... 139
Figura N° 3.73 Código de definición de rutas de navegación ..... 139
Figura N° 3.74 Código de redirección al teléfono y inicio de sesión por bloqueo ..... 140

Figura N° 3.75 Código de notificaciones push con FCM ..... 140
Figura N° 3.76 Código de verificación de conexión e intervalo de animaciones ..... 141
Figura N° 3.77 Pantalla móvil Splash con logo animado y eslogan ..... 142
Figura N° 3.78 Pantalla móvil de estado sin conexión con botón Reintentar ..... 142
Figura N° 3.79 Código de validación en tiempo real e inicio de sesión ..... 143
Figura N° 3.80 Pantalla móvil de inicio de sesión ..... 144
Figura N° 3.81 Código de cálculo de fortaleza de contraseña en tiempo real ..... 145
Figura N° 3.82 Pantalla móvil de registro con indicador de fortaleza ..... 145
Figura N° 3.83 Código de saludo personalizado y avatar desde Firebase Auth ..... 146
Figura N° 3.84 Pantalla móvil de inicio ..... 147
Figura N° 3.85 Código de modo visualización y modo edición de teléfono ..... 148
Figura N° 3.86 Pantalla móvil de perfil de usuario ..... 149
Figura N° 3.87 Código de borde dinámico de validación de foto ..... 150
Figura N° 3.88 Pantalla móvil del formulario de registro con foto validada ..... 150
Figura N° 3.89 Pantalla móvil del formulario de registro con estado de error: foto no contiene un perro ..... 151
Figura N° 3.90 Código de botón de búsqueda con estado de carga durante análisis ..... 152
Figura N° 3.91 Pantalla móvil de formulario de perro encontrado con foto validada ..... 152
Figura N° 3.92 Código de colores por similitud y fotos lado a lado ..... 153
Figura N° 3.93 Pantalla móvil de lista de coincidencias con fotos lado a lado ..... 154
Figura N° 3.94 Pantalla móvil de diálogo con datos del perro y del reportante ..... 154
Figura N° 3.95 Código de filtrado por estado y botón flotante de nuevo reporte ..... 155
Figura N° 3.96 Pantalla móvil de mis reportes ..... 156
Figura N° 3.97 Pantalla móvil de diálogo de confirmación para desactivar reporte ..... 156
Figura N° 3.98 Diagrama de caso de uso de la página web ..... 159
Figura N° 3.99 Diagrama de secuencia de la página web ..... 160
Figura N° 3.100 Mockup de inicio de sesión del panel web ..... 162
Figura N° 3.101 Mockup de dashboard del panel web ..... 162
Figura N° 3.102 Mockup de gestión de usuarios ..... 163
Figura N° 3.103 Mockup de reportes de perros perdidos y perros encontrados ..... 164
Figura N° 3.104 Código de gestión de estado compartido del admin ..... 164
Figura N° 3.105 Pantalla de inicio de sesión del panel administrativo ..... 165
Figura N° 3.106 Código de listeners en tiempo real para KPIs ..... 165
Figura N° 3.107 Dashboard de la página web ..... 166
Figura N° 3.108 Código de funciones de bloqueo/desbloqueo de usuarios ..... 166
Figura N° 3.109 Pantalla web de usuarios registrados ..... 167
Figura N° 3.110 Pantalla web usuarios con acción de bloqueo ..... 167
Figura N° 3.111 Código de listener en tiempo real de perros extraviados ..... 168
Figura N° 3.112 Pantalla web de registro de perros extraviados ..... 169
Figura N° 3.113 Código de listener en tiempo real de reportes encontrados ..... 170
Figura N° 3.114 Pantalla de registro de perros encontrados con coincidencias ..... 170
Figura N° 3.115 Código de carga de reportes propios del admin ..... 171
Figura N° 3.116 Pantalla web de mis reportes ..... 171
Figura N° 3.117 Código de registro de perro encontrado desde el panel web ..... 172
Figura N° 3.118 Pantalla de formulario para registrar perro encontrado ..... 173
Figura N° 3.119 Diagrama de Secuencia del Flujo End-to-End ..... 176
Figura N° 3.120 Captura de resultados de pruebas automatizadas pytest ..... 179

Figura N° 3.121 Captura de resultados de la ejecución del script de pruebas funcionales manuales ..... 181
Figura N° 3.122 Captura de tiempos de respuesta medidos por endpoint ..... 182
Figura N° 3.123 Captura de pruebas de seguridad CIA aprobadas ..... 185
Figura N° 3.124 Captura del backend operativo en Railway ..... 186
Figura N° 3.125 Captura de panel web publicado en Firebase Hosting ..... 187
Figura N° 3.126 Formulario de reportar perro extraviado ..... 189
Figura N° 3.127 Firestore mostrando colecciones ..... 190
Figura N° 3.128 Pantalla resultados app mostrando matches ..... 190
Figura N° 3.129 Panel web de usuarios registrados mostrando Activo/Bloqueado ..... 191
Figura N° 3.130 Pytest mostrando tests passed ..... 191
Figura N° 4.1 Dashboard y Estado del Sistema ..... 194
Figura N° 4.2 Pantalla web de los reportes del refugio ..... 194
Figura N° 4.3 Pantalla móvil de coincidencias ..... 196
Figura N° 4.4 Modal móvil de perro extraviado encontrado ..... 197
Figura N° 4.5 Registro de perros encontrados ..... 198
Figura N° 4.6 Registro de perros extraviados ..... 198

# ÍNDICE DE GRÁFICOS

Gráfico N° 1.1 Frecuencia de extravío de perros....3
Gráfico N° 3.1 Curva de exactitud - Entrenamiento EfficientNet-B4 en 3 fases....89
Gráfico N° 3.2 Curva de pérdida - Entrenamiento EfficientNet-B4 en 3 fases....90
Gráfico N° 3.3 Matriz de Confusión Binaria (Perro / No es Perro)....91
Gráfico N° 3.4 Histograma de distribución de precisión por clase....92
Gráfico N° 3.5 Resumen de métricas - EfficientNet-B4....93
Gráfico N° 4.1 Estados del sistema web....195
Gráfico N° 4.2 Permanencia en el refugio....199

# CAPÍTULO I
# MARCO INTRODUCTORIO

# CAPÍTULO I
# MARCO INTRODUCTORIO

## 1.1. INTRODUCCIÓN

Este proyecto de grado surge ante la dificultad en la identificación de perros extraviados en zonas de la ciudad de El Alto centrando su estudio en el Refugio WOOF, la poca utilización de herramientas tecnológicas especializadas para el reconocimiento de perros extraviados hace que el reencuentro sea prolongado y dependa de métodos manuales y una observación visual lo cual genera una permanencia prolongada en refugios, una sobrepoblación y un incremento en los costos de mantenimiento. Ante esta situación se formula una pregunta sobre como apoyar la identificación de perros extraviados para reducir su permanencia prolongada en refugios.

Para dar una solución a esta problemática se plantea crear un sistema web y móvil utilizando redes neuronales convolucionales. El sistema propuesto funciona con una arquitectura cliente-servidor que se distribuye en una aplicación móvil para capturar las imágenes de los perros y hacer consultas de las coincidencias que se encuentren, mientras que el Backend se encarga del procesamiento de imágenes, siguiendo una lógica de que cuando alguien reporta un perro el sistema primero ajusta y corrige la imagen y luego la red neuronal genera una huella digital visual o embedding por cada perro, estos datos se comparan matemáticamente con los embedding de la base de datos y así finalmente el usuario final recibe una lista de posibles coincidencias.

Este planteamiento del funcionamiento del sistema tiene como base el modelo EfficientNet-B4 previamente entrenado con el conjunto de datos, cuyo entrenamiento es fijo y no se actualiza con los datos subidos por los usuarios.

Los objetivos específicos toman en cuenta desde el análisis mediante historias de usuario, el entrenamiento de la red neuronal hasta el panel web administrativo que otorga al refugio el control de gestión de reportes.

La metodología de desarrollo utilizada en el proyecto es la Extreme Programming (XP), que guía todo el proceso mediante ciclos cortos y pruebas, complementada con CRISP-DM para el ciclo de entrenamiento del modelo de inteligencia artificial.

## 1.2. PLANTEAMIENTO DEL PROBLEMA

En este apartado se presenta el planteamiento del problema, abordando primero los antecedentes, luego la contextualización y finalmente la identificación concreta del problema que da origen al proyecto.

### 1.2.1. *ANTECEDENTES*

Encontrar perros perdidos y devolverlos rápido a sus dueños sigue siendo complicado a nivel mundial cuando se habla de bienestar animal. Según la Organización Mundial de la Salud (OMS, 2022), cerca del 75 % de los perros en el mundo no tiene ningún tipo de registro o identificación oficial. Esto hace que sea difícil rastrearlos si se pierden y termina saturando los refugios. La Organización Mundial de Sanidad Animal (WOAH, 2023) también señala que la falta de bases de datos centralizadas o sistemas tecnológicos para identificar a cada animal le pone un límite a lo que pueden hacer las autoridades y organizaciones para controlar la cantidad de perros y asegurarse de que se sepa a quién pertenece cada uno.

A nivel mundial, aproximadamente 10 millones de mascotas son reportadas como pérdidas o robadas cada año solo en Estados Unidos (Pawmaw, 2024). Las investigaciones del Journal of the American Veterinary Medical Association demuestran que el 52% de los perros con algún sistema de identificación logran reunificarse con sus dueños, frente a apenas el 2.2% de aquellos que no cuentan con ningún tipo de identificación (Lord et al., 2007, citado en Pawmaw, 2024). Adicionalmente, estudios sobre refugios de animales en Europa Central evidencian que los perros no reunificados permanecen en promedio de 23 días en el refugio antes de ser considerados abandonados (Zak et al., citado en Voslarova et al., 2019), lo que representa costos operativos y de bienestar animal significativos.

**Gráfico N° 1.1 Frecuencia de extravío de perros**




Fuente: Pawmaw (2024). Lost Dog Statistics.

En América Latina el panorama es todavía más complicado. Four Paws International (2023) reporta que millones de perros se pierden o quedan abandonados cada año y la mayoría ni siquiera está registrada en ningún sistema, digital o físico. La situación empeora porque casi no hay plataformas tecnológicas donde se puedan cruzar informes de perros perdidos y encontrados; así, los animales pueden pasar meses o incluso quedarse de forma permanente en los refugios.

Mota-Rojas y colegas (2021) señalan que en los países latinoamericanos esta falta de políticas integradas y el poco uso de tecnologías para rastrear a los perros son las principales razones detrás del aumento de animales perdidos y la sobrepoblación en los albergues. Según los autores, "sin una base de datos única ni herramientas tecnológicas para identificar visualmente a cada mascota, es mucho menos probable que un perro regrese con su familia".

En Bolivia se repite la historia. Aunque existe la Ley N.º 700 de 2015 para proteger a los animales, en la práctica se aplica poco. Valencia (2023) apunta que en Bolivia no hay un

registro único de mascotas ni bases de datos conectadas para identificar a los perros extraviados. Los refugios y organizaciones trabajan cada uno por su cuenta y dependen casi siempre de métodos manuales como publicaciones en redes sociales o avisos impresos. Por eso, identificar a los dueños se vuelve lento y muchos perros terminan pasando mucho tiempo en los refugios.

### 1.2.2. *CONTEXTUALIZACIÓN*

Hoy en día reconocer y devolver perros extraviados a sus dueños sigue siento un problema en diferentes lugares, la mayoría de personas aún siguen con los métodos manuales como mirar si el perro tiene collar, publicar fotos en redes sociales, poner carteles, pero todos esos métodos dependen de la memoria visual de las personas y del alcance de la red social.

Durante una visita al Refugio WOOF en El Alto se aplicó una entrevista (ver Apéndice 2), donde el equipo nos contestó que rara vez logran identificar de quien es un perro que fue extraviado (pregunta 12), donde marcaron “casi nunca”. Esto deja claro que les falta herramientas tecnológicas para identificar a los perros, también en la entrevista se compartió otro dato de que menos del 10% de los perros vuelven con sus dueños (pregunta 15), es decir que las formas de identificación actuales simplemente no están funcionando bien.

Cabe destacar que, a diferencia de otros países donde existen registros nacionales de mascotas extraviadas, en Bolivia no se cuenta con estadísticas oficiales desagregadas sobre el número de perros perdidos reportados mensualmente. Ni el Gobierno Autónomo Municipal de El Alto (GAMLPE) ni el Programa Nacional de Zoonosis del Ministerio de Salud publican datos específicos sobre extravío y reunificación de mascotas a nivel local. Esta ausencia de registros formales constituye en sí misma una evidencia de la problemática: sin un sistema de identificación y seguimiento, es imposible cuantificar la magnitud real del problema, lo que refuerza la necesidad del sistema propuesto en el presente proyecto.

Estos datos reflejan que el problema no solo radica en el abandono sino también en lo difícil que es saber a quién pertenece cada perro que se encuentra en la calle, lo cual aumenta la cantidad de perros sin hogar y deja a los refugios al límite donde hace falta

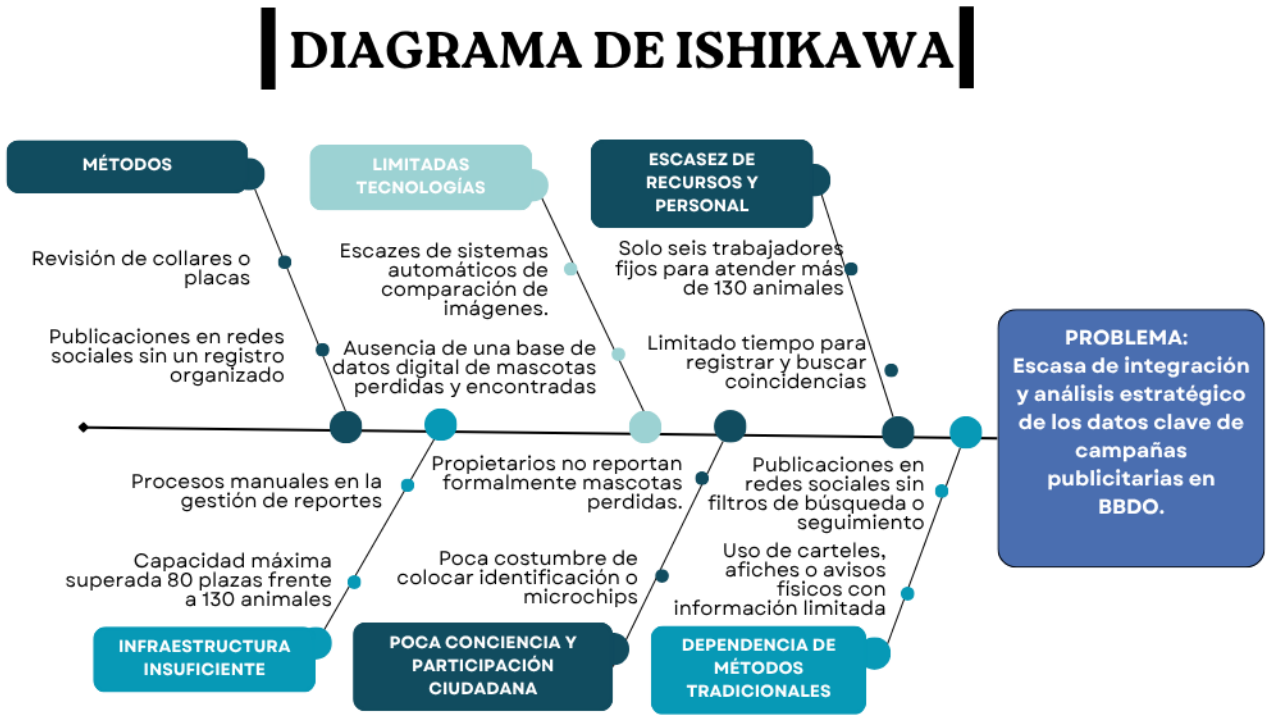
buscar alternativas tecnológicas que hagan más fácil reconocer a cada mascota y ayudarla a volver a casa con su familia.

### 1.2.3. *IDENTIFICACIÓN DEL PROBLEMA*

El número de perros que se pierden en las ciudades va en aumento, y eso dice mucho de lo mal que funcionan los sistemas para identificarlos y devolverlos a sus dueños, casi todos los refugios y particulares siguen recurriendo a lo de siempre: subir fotos a redes sociales o pegar carteles por la calle. El problema es que esos métodos dependen de que la gente los vea y pueda reconocer al perro, así que el margen de error es alto y el alcance, limitado.

Por culpa de esto, muchos perros acaban pasando largos periodos en los refugios. Esto solo agrava la sobrepoblación, dispara los gastos de cuidado y puede empeorar la vida de los propios animales.

**Figura N° 1.1 Diagrama Ishikawa**




Fuente: Elaboración propia, 2026

## 1.3. FORMULACIÓN DEL PROBLEMA

¿Cómo mejorar la identificación de perros extraviados para reducir el tiempo de permanencia en el refugio WOOF de la ciudad de El Alto?

## 1.4. JUSTIFICACIÓN

En esta sección se explica por qué es necesario llevar a cabo el proyecto, considerando su aporte a la sociedad, los aspectos tecnológicos que lo hacen posible y los beneficios económicos que lo justifican.

### 1.4.1. *JUSTIFICACIÓN SOCIAL*

El extravió de perros domésticos se ha convertido en una problemática que no afecta solo a los animales, sino también a las familias y a la convivencia social, cuando alguien no consigue reencontrar a su mascota, hay consecuencias que van mucho más allá de la tristeza: aparecen gastos inesperados tanto para la persona que perdió a su perrito o para el refugio provocando que más perros terminan viviendo en la calle y sobrepoblando a los refugios. La Organización Mundial de la Salud (OMS 2022) advierte que tener animales sin control en las calles suele estar ligado a la falta de formas claras de identificar y rastrear a cada uno, esto termina trayendo problemas para la salud pública y empeora el ambiente urbano.

Por eso, un sistema web y una aplicación móvil pensada para reunir a los perros perdidos con sus familias se vuelve mucho más que un simple avance tecnológico. Herramientas como esta crean una red de ayuda mutua donde vecinos, refugios y rescatistas pueden trabajar juntos. Flores (2020) lo resume bien: "el uso de la tecnología con propósito social promueve la cooperación comunitaria y fortalece los lazos humanos frente a problemáticas compartidas". No es solo que el sistema web y la aplicación haga más fácil buscar, también mete a la comunidad en el centro de la acción para rescatar y reunir a los animales.

El impacto social de esta propuesta radica en reducir el sufrimiento de personas y perros, y también bajar la cifra de animales atrapados en refugios o vagando solos. Mota-Rojas y su equipo (2021) apuntan que, en América Latina, la falta de sistemas de identificación efectivos complica recuperar animales extraviados; esto resulta frustrante tanto para las familias como para los propios rescatistas de los refugios. Una plataforma que acerque a quien perdió y a quien encontró un perro responde directamente a esta necesidad social.

### 1.4.2. *JUSTIFICACIÓN TÉCNICA*

El proyecto apuesta por una arquitectura tipo cliente-servidor, que soluciona las limitaciones de hardware de los dispositivos móviles al centralizar las tareas de visión

artificial en servidores potentes. De esta manera, el modelo de inteligencia artificial permanece en el servidor sin depender de la capacidad del teléfono móvil, y puede ser reentrenado de forma independiente en el futuro sin necesidad de publicar una nueva versión de la app.

Estos son los principales componentes que hacen posible el sistema:

- EfficientNet-B4: Se eligió esta red neuronal convolucional porque es especialmente buena reconociendo detalles visuales complejos, como la textura del pelaje o las facciones de los perros mediante un vector de características o embendding.
- Algoritmo de similitud del coseno: Este método compara los vectores que genera el modelo de IA y permite identificar a un perro, aunque las fotos varíen en ángulo, luz o distancia.
- Backend con FastAPI: El servidor, desarrollado en Python, procesa las imágenes y ejecuta el modelo de IA. Esta tecnología permite escalar y actualizar el sistema sin que el usuario tenga que reinstalar nada en su aplicación móvil.
- OpenCV: Esta librería se integra para normalizar, limpiar y recortar las fotos que envían los usuarios. Así, se optimiza la entrada de datos a la red neural y se gana calidad al extraer las características visuales.
- Gestión de datos con Firebase y Cloudinary: Se utiliza Firebase para manejar la base de datos de reportes y la autenticación, mientras que Cloudinary almacena y distribuye las imágenes en alta resolución.

El desarrollo sigue la metodología Extreme Programming (XP), con ciclos breves de retroalimentación y programación basada en pruebas para asegurar la calidad técnica. La aplicación móvil se construye en Android Studio, usando el lenguaje de Kotlin, Jetpack Compose para la interfaz y Dagger Hilt para mantener el código ordenado y fácil de mantener.

### 1.4.3. *JUSTIFICACIÓN ECONÓMICA*

Este proyecto resulta bastante viable en lo económico porque se apoya en herramientas de código abierto y en servicios en la nube que ofrecen niveles gratuitos bastante generosos. Así, es posible poner en marcha una solución de inteligencia artificial sin tener que hacer grandes inversiones en servidores físicos o en licencias caras de software.

La mayor parte del ahorro viene de una arquitectura distribuida: el backend funciona en Railway, que permite correr entornos en Python sin problemas, mientras que la parte de administración para el refugio se publica usando Firebase Hosting. Esa plataforma cubre el almacenamiento y el ancho de banda que se necesita para la gestión web, y todo sin que el refugio tenga costos fijos mensuales por mantener el sistema funcionando.

Para los archivos multimedia y la información, se usa Cloudinary para guardar las imágenes y Firebase Firestore para los metadatos y la autenticación. Ambos servicios cubren bien el volumen de información que maneja el Refugio WOOF, y todo sin tener que depender de servidores locales, que serían mucho más caros e innecesarios en este contexto.

El entrenamiento de la red neuronal EfficientNet-B4 aprovecha el hardware de alto rendimiento que Kaggle ofrece sin coste extra. Esto es clave, porque evita que el equipo tenga que comprar tarjetas gráficas potentes solo para entrenar las redes, algo que normalmente encarecería muchísimo cualquier proyecto de este tipo. Además, el uso de herramientas como Android Studio, junto con lenguajes modernos como Kotlin y Jetpack Compose, refuerzan la propuesta: son gratuitos y no requieren pagar nada por usarlos o publicar las aplicaciones.

En la práctica, el sistema también termina generando un ahorro indirecto para el refugio. Al ayudar a reunir más rápido a las mascotas con sus dueños usando el algoritmo de similitud de coseno, los perros pasan menos tiempo en el albergue, y eso reduce los gastos diarios en comida, atención veterinaria y cuidado general.

## 1.5. OBJETIVOS DE LA INVESTIGACIÓN

Este proyecto establece un objetivo general, así como varios objetivos específicos, que se describen a continuación:

### 1.5.1. *OBJETIVO GENERAL*

Desarrollar un sistema web y aplicación móvil aplicando redes neuronales convolucionales EfficientNet-B4 y el algoritmo Cosine Similitary para la identificación de perros extraviados y reducción de tiempo de permanencia en el Refugio WOOF de la ciudad de El Alto.

### 1.5.2. *OBJETIVOS ESPECÍFICOS*

Los objetivos específicos para la investigación de este proyecto son los siguientes:

- Analizar la problemática actual de identificación de perros extraviados en contextos urbanos, para determinar los factores que influyen en la permanencia prolongada de los animales en los albergues y su impacto en el bienestar animal.
- Investigar los antecedentes nacionales e internacionales sobre el uso de tecnologías en la identificación de animales, para establecer un marco referencial que evidencie los avances y limitaciones existentes en la aplicación de herramientas tecnológicas en este campo.
- Analizar la estructura y funcionamiento de la red neuronal convolucional EfficientNet-B4, para comprender su aplicabilidad en la extracción de características visuales y su aplicabilidad en entornos web y aplicaciones móviles.
- Evaluar el funcionamiento del algoritmo Cosine Similarity, para determinar su capacidad de medir similitudes entre imágenes y su aporte al proceso de comparación de registros visuales de perros extraviados.

## 1.6. DELIMITACIÓN

En este apartado se precisan los límites del proyecto, indicando el espacio donde se desarrollará, el tiempo que abarcará, la población a la que se dirige y el marco teórico que lo sustenta.

### 1.6.1. *DELIMITACIÓN ESPACIAL*

El presente proyecto se desarrollará en la ciudad de El Alto, perteneciente al departamento de La Paz, Bolivia, específicamente en la zona Villa Adela, donde se encuentra el Refugio WOOF, ubicado sobre la Avenida España. Este espacio será tomado como caso de estudio, ya que concentra una alta cantidad de perros rescatados de la vía pública, y refleja las dificultades que enfrentan los refugios locales en los procesos de identificación y reencuentro de perros extraviados con sus dueños.

**Figura N° 1.2 Ubicación del Refugio WOOF**




Fuente: Google Maps, 2026

### 1.6.2. *DELIMITACIÓN TEMPORAL*

Este proyecto se llevará a cabo entre septiembre de 2025 y junio de 2026. Durante esos meses, se van a desarrollar de manera organizada las fases del trabajo: desde la recolección de información en el Refugio WOOF en El Alto, el análisis de los datos sobre el problema de la identificación de perros perdidos, la revisión de tecnologías que podrían aplicarse, hasta la creación de una propuesta tecnológica para ayudar a reunir a los perros extraviados con sus dueños.

### 1.6.3. *DELIMITACIÓN POBLACIONAL*

La población participante de la investigación estará conformada por el personal responsable del Refugio WOOF, ubicado en la zona Villa Adela de la ciudad de El Alto. Asimismo, se considerará la participación indirecta de personas de la comunidad que reportan o rescatan perros, quienes aportarán información relevante sobre la problemática de identificación y reencuentro de mascotas extraviadas.

### 1.6.4. *DELIMITACIÓN TEÓRICA*

Este proyecto se construye dentro del procesamiento digital de imágenes y el uso de inteligencia artificial en visión por computadora, con el objetivo de crear un sistema web y una aplicación móvil para identificar perros extraviados de forma automática. El enfoque se centra en arquitecturas de aprendizaje profundo, en particular el modelo de red

neuronal convolucional EfficientNet-B4, elegido porque destaca a la hora de extraer vectores de características o embeddings de alta dimensión. Esta técnica convierte detalles como la textura del pelaje o la forma del hocico de cada perro en datos matemáticos que luego pueden ser comparados usando el algoritmo de similitud del coseno, para medir el nivel de coincidencia entre registros.

El sistema permite cargar una sola foto por caso al reportar perros perdidos o encontrados, y el procesamiento complejo recae en un backend hecho en FastAPI. Desde el sitio web, el personal administrativo puede gestionar la base de datos, revisar coincidencias y moderar la actividad de los usuarios, lo que ayuda a mantener la información del refugio en orden.

Por temas de recursos y tiempo de investigación siguiendo la metodología Extreme Programming (XP), complementada con CRISP-DM para la etapa de entrenamiento del modelo, quedaron fuera del alcance el uso de hardware externo como microchips o GPS, así como sistemas de mensajería interna o módulos de adopción. La propuesta se enfoca sólo en validar la identidad visual de cada perro para ayudar a reunirlos con sus dueños, planteando así una solución técnica para evitar que se queden demasiado tiempo en el refugio.

## 1.7. ALCANCES Y LÍMITES

Este proyecto abarcara alcances y limites que se muestran en los puntos siguientes para tener una idea más clara de cómo funcionara el sistema web y la aplicación móvil.

### 1.7.1. *ALCANCES*

El proyecto define los alcances técnicos y funcionales, asegurando que cada módulo cumpla con los requerimientos de desarrollo establecidos para el Refugio WOOF.

- Generar un vector numérico de 512 dimensiones a partir de cada fotografía utilizando la arquitectura EfficientNet-B4.
- Calcular la similitud de coseno entre el vector de la fotografía recibida y los vectores almacenados en la base de datos para determinar el porcentaje de coincidencia.
- Discriminar y rechazar imágenes que no correspondan a la clase canina mediante las clases negativas del modelo (not_a_dog y dog_generic).

- Permitir al administrador controlar cuentas de usuario, ejecutar bloqueos de acceso y supervisar los registros desde el panel web.
- Mostrar en pantalla la lista de resultados ordenados por porcentaje de similitud tanto en la aplicación móvil como en el sistema web.
- Autenticar usuarios mediante Firebase Auth con correo electrónico y contraseña, tanto en la aplicación móvil como en el sistema web.
- Permitir que los usuarios carguen hasta tres fotografías del perro por reporte junto con información básica: nombre, color, tamaño, sexo.
- Aplicar un umbral de confianza del 50% sobre la clase canina antes de ejecutar la comparación de similitud, rechazando automáticamente imágenes que no superen dicho umbral.
- Enviar notificaciones push al usuario cuando se detecte una posible coincidencia con un porcentaje de similitud igual o superior al 80%, informando a los involucrados sobre el posible reencuentro.

### 1.7.2. *LÍMITES*

Los límites determinan las restricciones técnicas y funcionales del sistema las cuales no son contempladas dentro de los alcances de este proyecto.

- No procesará imágenes localmente, requiere conexión a internet para enviar la fotografía al servidor y recibir los resultados de similitud.
- No se podrá tomar fotografías desde la aplicación móvil para realizar un registro.
- No será compatible con iOS, la aplicación móvil está disponible únicamente para Android versión 8.0 (Oreo) o superior.
- No actualizará ni reentrenará el modelo de inteligencia artificial con las imágenes subidas por los usuarios, el modelo permanece con el entrenamiento fijo realizado anteriormente.
- No incluirá mensajería interna ni chat, el contacto entre usuarios se realiza por medios externos utilizando los datos de contacto proporcionados en el reporte.
- No gestionará procesos de adopción, seguimiento veterinario, registro de vacunas ni historial médico de los animales.
- No se integrará con microchips ni dispositivos GPS para rastreo en tiempo real.

- No rastreará la ubicación del perro automáticamente, la información geográfica se limita a los datos ingresados manualmente en el formulario de reporte.

## 1.8. METODOLOGÍA

### 1.8.1. *ENFOQUES DE LA INVESTIGACIÓN*

Este proyecto adopta un enfoque cuantitativo porque su propósito es demostrar que la implementación de un sistema basado en redes neuronales convolucionales mejora la identificación de perros extraviados y reduce el tiempo de permanencia en el Refugio WOOF. Elegir este enfoque ayuda a recoger información objetiva y comprobable donde es posible ponerle un número a la magnitud del problema y ver cómo se relacionan variables como los métodos de identificación o el tiempo que los perros permanecen en el refugio.

Sampieri (2014) explica que el enfoque cuantitativo se basa en recolectar y analizar datos para probar hipótesis o responder preguntas de investigación a partir de mediciones y análisis estadísticos. Aquí los datos los aportan una encuesta aplicada en el refugio, además de los resultados que arroja el algoritmo Cosine Similarity, que mide en porcentajes cuánto se parecen las imágenes comparadas. Bajo esta lógica, el proyecto busca demostrar que el uso de inteligencia artificial mejora la identificación de perros extraviados, midiendo la precisión del modelo y el impacto en el tiempo de permanencia en el refugio como indicadores cuantificables del problema.

Creswell (2014) plantea que la investigación cuantitativa parte de obtener datos objetivos y usar estadísticas para describir patrones y examinar conductas. Así que este proyecto sigue esa lógica: la idea es analizar estadísticamente tanto los resultados del algoritmo de comparación como los datos obtenidos con los instrumentos aplicados, para ver si las formas de identificación pueden ayudar a reducir el tiempo que los perros pasan en los refugios.

### 1.8.2. *MÉTODO DE INVESTIGACIÓN*

Para este proyecto de investigación se eligió el enfoque analítico. Este método busca examinar el fenómeno de forma sistemática, desarmándolo en partes para poder entender las causas, relaciones y efectos detrás de cada elemento. Así, es posible revisar de manera ordenada las variables que intervienen en la situación de los perros

extraviados y el tiempo que pasan en los refugios, con la idea de comprender cada pieza desde un punto de vista que pueda medirse.

Sampieri (2014) explican que el método analítico ayuda a estudiar un fenómeno rompiéndolo en sus partes esenciales, para entender cómo se conectan entre sí. Bajo esta lógica, el estudio examina por separado los factores tecnológicos, sociales y organizativos que afectan la identificación de perros extraviados, para luego unirlos y mirar el problema completo.

Por otra parte, Creswell (2014) señala que el método analítico, dentro del enfoque cuantitativo, implica procesar y describir los datos de manera objetiva, tratando de encontrar patrones o relaciones claras. Así que este método servirá como base para interpretar los resultados numéricos y mostrar, a través de porcentajes, lo importante que resulta usar soluciones tecnológicas que mejoren la identificación y el reencuentro de perros extraviados.

### 1.8.3. *TIPO DE INVESTIGACIÓN*

Este proyecto se plantea como una investigación aplicada. Se escogió este enfoque porque no se trata solo de crear teorías o describir el problema de los perros extraviados: el objetivo es tomar conocimientos tecnológicos para diseñar, construir y probar una solución técnica que sirva realmente para identificar perros extraviados. Según Sampieri y Mendoza (2018), la investigación aplicada se enfoca en resolver problemas concretos en un contexto específico, dando prioridad a la utilidad y el efecto que tienen los resultados. Así, el proyecto pone en práctica conceptos de ingeniería de software y visión artificial para desarrollar un sistema que utiliza la red neuronal EfficientNet-B4 junto con el algoritmo Cosine Similarity, llevando la teoría directamente a una herramienta lista para usarse en el refugio. La apuesta de este tipo de investigación está en que sus resultados se aplican a mejorar una situación real. Justamente, aquí se apunta a reducir el tiempo que los perros extraviados pasan lejos de sus dueños y en el refugio. Es un trabajo aplicativo porque su valor depende de lo que el software pueda lograr y de cómo apoye el reencuentro de mascotas y sus dueños en la ciudad de El Alto.

### 1.8.4. *DISEÑO DE INVESTIGACIÓN*

Para este proyecto se eligió un diseño de investigación no experimental, que encaja mejor con la lógica del sistema propuesto. No habrá manipulación intencional de las variables

del estudio: simplemente se observarán y analizarán los resultados técnicos y funcionales tal como surgen en el entorno natural del Refugio WOOF.

De acuerdo con Sampieri (2018), la investigación no experimental supone trabajar sin modificar deliberadamente las variables. En este tipo de estudios, las variables independientes no se cambian a propósito para medir su efecto sobre otras, sino que se registran los datos tal cual se presentan. Así, los resultados del procesamiento de imágenes con EfficientNet-B4 y los porcentajes de coincidencia obtenidos a través del algoritmo Cosine Similarity se recogen y analizan directamente, sin intervenir el proceso técnico de comparación automática en el servidor.

### 1.8.5. *TÉCNICAS E INSTRUMENTOS*

Para el desarrollo de esta investigación, se emplearán diversas técnicas e instrumentos de recolección de datos acordes con el enfoque cuantitativo, el método analítico, el tipo aplicativo y el diseño no experimental adoptados. Estas técnicas permitirán obtener información numérica, verificable y objetiva acerca de la problemática de identificación de perros extraviados.

De acuerdo con Sampieri (2014), en la investigación cuantitativa “las técnicas de recolección de datos buscan medir las variables de manera precisa y controlada, utilizando instrumentos estructurados que permitan el análisis estadístico de la información”. Por ello, en este estudio se aplicarán las siguientes técnicas e instrumentos:

a) Técnica de encuesta estructurada:

Permitirá recopilar información cuantificable sobre la percepción y experiencia del personal del Refugio WOOF en relación con los procesos de identificación de perros extraviados, los tiempos de permanencia y la frecuencia de reencuentros.

- Instrumento: Cuestionario estructurado, compuesto por preguntas cerradas y de opción múltiple, diseñado para obtener datos medibles que serán procesados estadísticamente mediante porcentajes.

b) Técnica de observación sistemática:

Se utilizará para registrar datos numéricos sobre el número de perros ingresados, adoptados o devueltos a sus dueños dentro del refugio, así como los tiempos promedio de permanencia.

- Instrumento: Ficha de observación cuantitativa, en la cual se consignarán las variables observadas (cantidad de animales, duración promedio de estadía, número de reencuentros) en periodos determinados, facilitando su análisis posterior en tablas y gráficos.

c) Técnica de análisis experimental de resultados del algoritmo:

Permitirá obtener y cuantificar los valores de similitud visual generados por el algoritmo Cosine Similarity a partir de las imágenes procesadas con la red neuronal EfficientNet-B4. El proceso de entrenamiento y evaluación del modelo sigue el marco CRISP-DM, que estructura el ciclo de vida del modelo en seis fases: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue.

- Instrumento: Registro de resultados del sistema, donde se documentarán los porcentajes de coincidencia obtenidos en las pruebas de comparación de imágenes, incluyendo métricas de evaluación del modelo como la Matriz de Confusión, los cuales serán analizados estadísticamente para evaluar la precisión del modelo.

d) Técnica de validación de resultados tecnológicos:

Se empleará para evaluar los resultados obtenidos en el proceso de identificación visual, comprobando la exactitud y coherencia de los datos generados por el algoritmo.

- Instrumento: Tabla de validación de coincidencias, en la que se registrarán los resultados de las pruebas del algoritmo como por ejemplo número de coincidencias acertadas, coincidencias falsas y porcentaje de similitud, con el fin de cuantificar la precisión del sistema y su aporte en la reducción del tiempo de permanencia de los perros extraviados en el refugio.

### 1.8.6. *POBLACIÓN*

La población seleccionada para el desarrollo del proyecto está conformada por los trabajadores del Refugio WOOF. Este grupo representa el núcleo principal de personas involucradas en la gestión, rescate, cuidado e identificación de los perros extraviados y constituye la fuente directa de información para la investigación.

El refugio cuenta con un total de seis integrantes (ver Apéndice 2), quienes desarrollan labores relacionadas con la atención veterinaria, alimentación, limpieza, control de ingreso de animales y coordinación de adopciones. Debido a su participación constante

en los procesos de rescate y registro de mascotas extraviadas, este grupo constituye la población total de la investigación, ya que son los actores que experimentan de manera directa las limitaciones en los métodos actuales de identificación y reencuentro de perros con sus dueños.

### 1.8.7. *MUESTRA*

Este proyecto opta por un muestreo no probabilístico por conveniencia, lo que suele hacerse en investigaciones cuantitativas cuando se trabaja con grupos pequeños y muy definidos. Aquí, se elige a las personas que participan según qué tan accesibles y representativos son dentro del entorno que se está analizando. Así, la información recolectada resulta válida y se puede medir con claridad.

Según Sampieri (2014), este tipo de muestreo se usa cuando "la población es reducida o accesible en su totalidad, permitiendo seleccionar a los elementos que mejor representen el fenómeno en estudio".

En el Refugio WOOF trabajan seis personas, todas dedicadas de alguna forma al rescate, cuidado y registro de perros que se han perdido. De ese grupo, se eligieron tres para la muestra, justamente porque están muy involucrados en el registro y la búsqueda de animales. Elegirlos así facilita obtener datos claros y representativos sobre cómo se identifican los perros, cuánto tiempo suelen quedarse y con qué frecuencia logran volver a reunirse con sus dueños.

# CAPÍTULO II
# MARCO TEÓRICO

# CAPÍTULO II
# MARCO TEÓRICO

## 2.1. MARCO TEÓRICO

El marco teórico es el punto de partida para entender y respaldar todo el proyecto. Aquí se exploran conceptos, métodos, modelos y herramientas propias de la Ingeniería de Software, junto con las bases tecnológicas usadas para crear tanto el sistema web como la aplicación móvil enfocados en identificar perros extraviados. Pressman y Maxim (2020) sostienen que un marco teórico sólido permite tomar decisiones fundamentadas sobre diseño, desarrollo y calidad, asegurando que el proyecto siga las buenas prácticas habituales en el desarrollo de software.

### 2.1.1. *INGENIERÍA DE SOFTWARE*

La Ingeniería de Software se ocupa de diseñar, desarrollar, operar y mantener sistemas informáticos usando principios de ingeniería para lograr software confiable, eficiente y bien hecho. Sommerville (2016) la define como "una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta su mantenimiento" (p. 7). Esta área da las bases metodológicas para crear soluciones tecnológicas que respondan a necesidades reales, tratando de equilibrar lo técnico con los recursos disponibles.

En este proyecto, la Ingeniería de Software se pone en práctica al desarrollar un sistema que incluye una app móvil para Android, un panel de administración web y un backend que procesa imágenes para identificar perros perdidos. Para este proyecto se usa la metodología ágil Extreme Programming (XP). Beck y Andrés (2004) afirman que XP es "una filosofía de desarrollo de software basada en los valores de comunicación, retroalimentación, simplicidad, coraje y respeto" (p. 24), lo que lo convierte en el enfoque ideal para organizar el trabajo en ciclos cortos donde, después de programar cada módulo, se pasa directamente a las pruebas. Para la etapa específica de entrenamiento del modelo neuronal, el proyecto incorpora adicionalmente el marco CRISP-DM, cuya estructura orientada al ciclo de vida de modelos de machine learning complementa las prácticas de XP en esa iteración.

La decisión de usar XP se toma porque permite revisar continuamente lo que se está haciendo, al probar después de codificar se confirma que el algoritmo de similitud de coseno cumple con el nivel de precisión esperado y que la comunicación entre la app móvil y el servidor.

### 2.1.2. *MODELOS DE DESARROLLO*

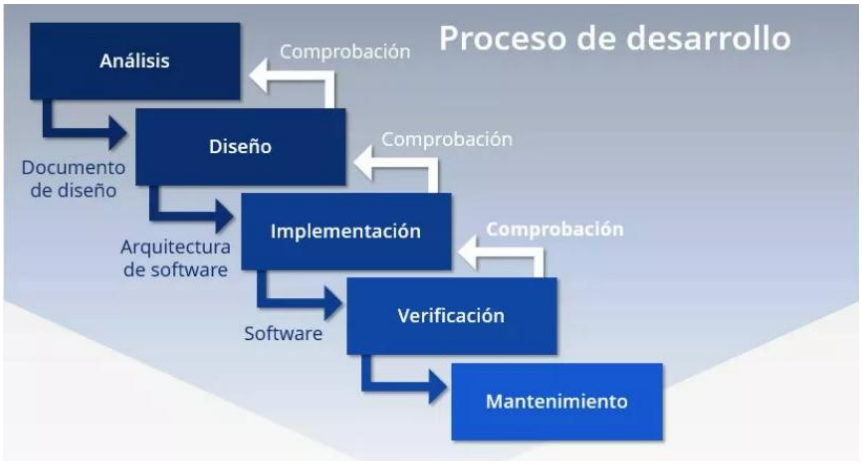
El modelo de desarrollo de software define cómo se organiza todo el proceso de planificar, diseñar, construir y probar un sistema. Elegirlo bien es clave para que tanto el sistema web como la app móvil realmente cumplan lo que necesita el proyecto. También es fundamental si queremos que integraciones como la red neuronal EfficientNet-B4 y el algoritmo Cosine Similarity funcionen bien juntas, sin tropiezos.

Estos son algunos de los modelos de desarrollo más comunes en ingeniería de software:

a) Modelo en Cascada (Waterfall)

Sigue pasos lineales, cada etapa se acaba antes de empezar la siguiente. Sirve cuando todo está bien definido desde el principio. Pero si hay que modificar algo después de una fase, puede ser complicado en proyectos tecnológicos donde las condiciones pueden cambiar (Pressman y Maxim, 2020).

Figura N° 2.1 **Modelo en Cascada**

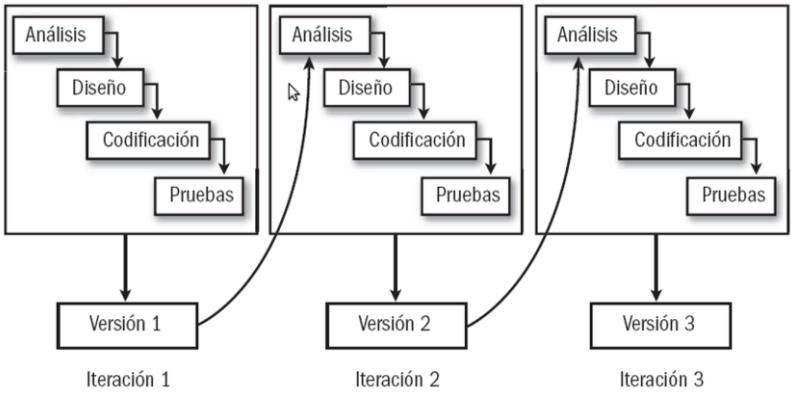



Fuente: www.ionos.com, 2019

b) Modelo Iterativo e Incremental

Divide el desarrollo en ciclos donde cada entrega agrega mejoras y va afinando el producto. Permite recibir comentarios y ajustar lo necesario durante el proceso, ayudando a detectar problemas y riesgos desde el principio (Sommerville, 2016).

Figura N° 2.2 **Modelo Iterativo e Incremental**

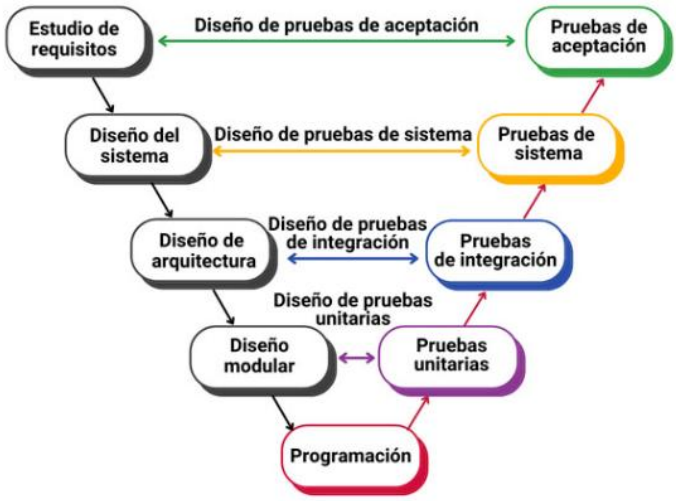



Fuente: Adaptado de Algunos temas sobre ingeniería de software, 2009.

c) Modelo en V (Verificación y Validación)

Es una variante del modelo en cascada donde cada paso de desarrollo está emparejado con una fase de pruebas relacionada. Da importancia a la verificación y validación al mismo tiempo que se avanza en el desarrollo. Pressman y Maxim (2020) indican que este modelo garantiza que cada fase de construcción tenga una contraparte de prueba definida, lo que reduce significativamente los errores detectados en etapas tardías del proyecto.

Figura N° 2.3 **Modelo en V**



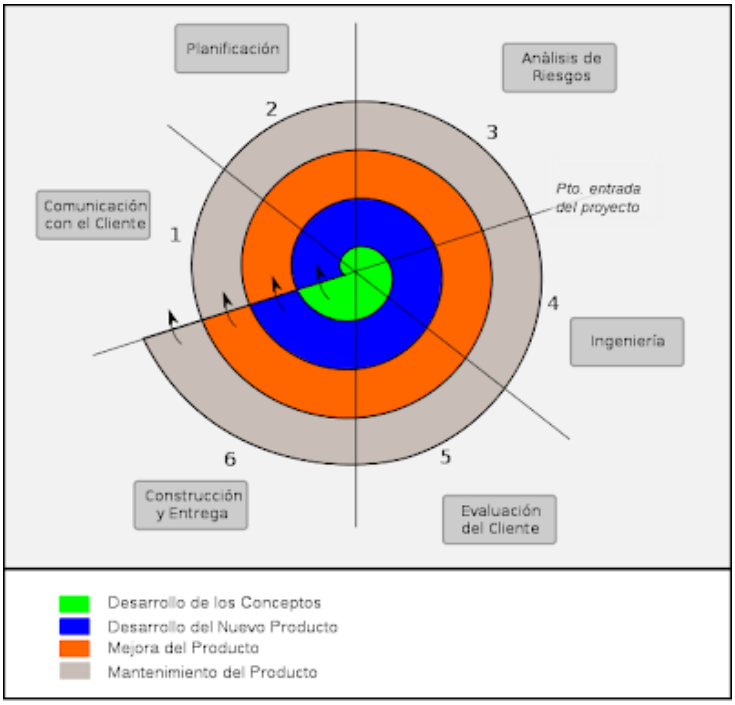

Fuente: Servicios de Informática Profesional, S.A. (SIPSA), 2023

d) Modelo Espiral

Creado por Barry Boehm en 1988, toma lo mejor del modelo iterativo, pero le suma un análisis de riesgos a fondo en cada ciclo. Va bien en proyectos complejos y con muchas

incógnitas, porque permite manejar los riesgos de manera anticipada. Según Pressman y Maxim (2020), este modelo combina la naturaleza iterativa del desarrollo con los aspectos controlados del modelo en cascada, siendo especialmente útil cuando la incertidumbre técnica es alta.

Figura N° 2.4 **Modelo Espiral**




Fuente: Ingeniería del Software I, 2012

e) Modelos Ágiles

Los ágiles se apoyan en los principios del Manifiesto Ágil, dándole prioridad a la flexibilidad, la entrega temprana de software funcional y el poder adaptarse rápido si algo cambia (Beck y Andrés, 2004). En este caso, el proyecto utiliza Extreme Programming (XP) por su foco en la calidad técnica y la facilidad para ajustar el rumbo cuando haga falta.

- Extreme Programming (XP): Extreme Programming (XP) es la metodología adoptada en este proyecto. Apuesta por prácticas como la integración continua, diseño simple y entregas cortas y frecuentes. Beck y Andrés (2004) señalan que "las buenas relaciones llevan a buenos negocios", y que XP aborda tanto los aspectos técnicos como las relaciones humanas dentro del equipo de desarrollo,

garantizando así que las pruebas constantes validen que los algoritmos de inteligencia artificial funcionan correctamente al terminar cada parte del código.

- Scrum: Scrum es un marco de trabajo que organiza el desarrollo en ciclos cortos llamados sprints (Pressman y Maxim, 2020). Aunque es ampliamente utilizado para la gestión de equipos, el presente proyecto prioriza las prácticas técnicas de XP sobre los roles de gestión de Scrum para garantizar la calidad del procesamiento de imágenes.
- Kanban: Se centra en la visualización del flujo de trabajo y la limitación del trabajo en progreso (WIP) para mejorar la eficiencia, sin prescribir iteraciones fijas.

### 2.1.3. *LENGUAJE UNIFICADO DE MODELADO UML*

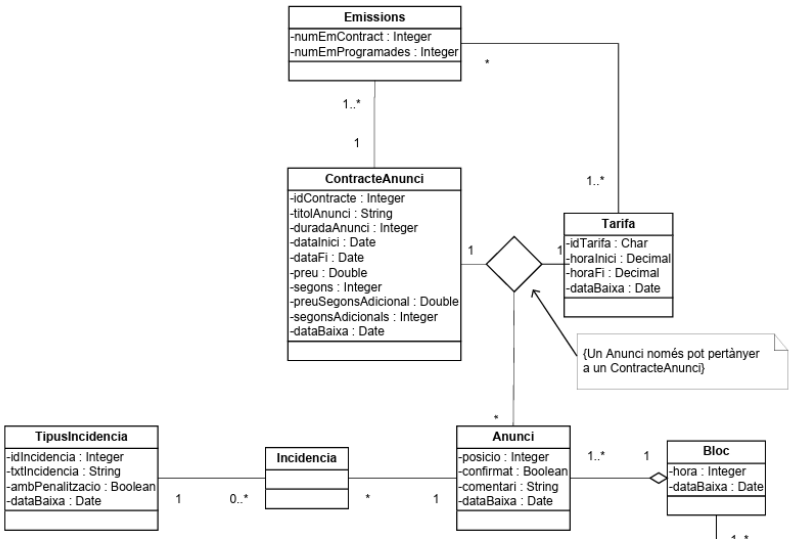
El Lenguaje Unificado de Modelado (UML) es un estándar internacional definido por la Object Management Group (OMG) que sirve para visualizar de forma gráfica los componentes de un sistema de software. Booch et al. (2005) lo definen como "el estándar de la industria para visualizar, especificar, construir y documentar los artefactos de un sistema intensivo en software". Como lenguaje de modelado estándar, el UML facilita la comunicación y reduce la confusión entre los integrantes del equipo de desarrollo.

En este proyecto, UML se usa para representar el sistema de identificación de perros. Así se muestran tanto la estructura como los procesos de interacción entre usuarios incluyendo dueños, ciudadanos y personal del refugio, la app móvil y la plataforma web administrativa. Los diagramas dejan claras las conexiones entre componentes y permiten planear la integración de la red neuronal EfficientNet-B4 y el algoritmo Cosine Similarity. Estos son los diagramas UML utilizados en el proyecto:

a) Diagrama de Clases

Este diagrama muestra las entidades principales del sistema, junto con sus atributos, métodos y relaciones. Permite ver cómo está armado el sistema desde la perspectiva orientada a objetos, tanto en la aplicación móvil como en el sistema web. Booch et al. (2005) señalan que el diagrama de clases es el más utilizado en el modelado orientado a objetos, ya que muestra la estructura estática del sistema y las relaciones entre sus componentes.

Figura N° 2.5 **Ejemplo de Diagrama de Clases**



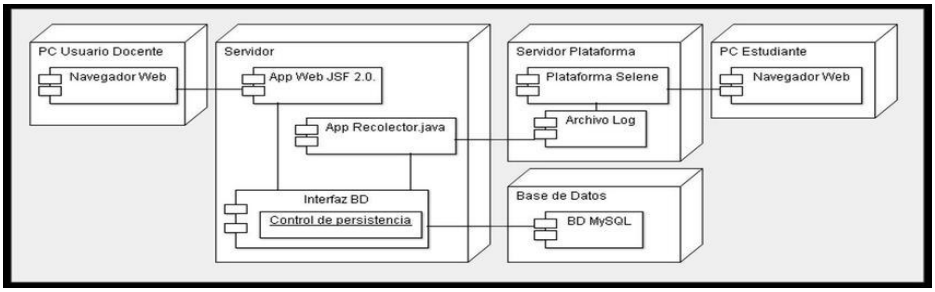

Fuente: Adaptado de Algunos temas sobre ingeniería de software, 2009.

b) Diagrama de Despliegue

En este diagrama se explica cómo se distribuyen físicamente los componentes del sistema entre varios nodos (Booch et al., 2005). Concretamente, en este proyecto se representan los siguientes nodos:

- Dispositivos móviles Android: aquí corre la aplicación para usuarios y rescatistas.
- Estación de trabajo web: el personal del refugio usa este panel administrativo para supervisar y controlar usuarios.
- Servidor en la nube Railway: aloja el backend hecho en FastAPI y se encarga del procesamiento con IA.
- Firebase: se ocupa de guardar los datos y de la autenticación centralizada para los dos frontends (móvil y web).

Figura N° 2.6 **Ejemplo de Diagrama de Despliegue**



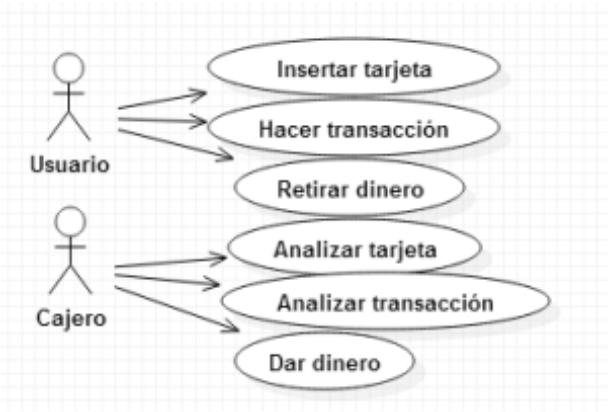

Fuente: www.researchgate.net, 2017

c) Diagrama de Casos de Uso

Este diagrama muestra cómo los usuarios interactúan con el sistema y ayuda a identificar qué funcionalidades debe cubrir. De acuerdo con Booch et al. (2005), los diagramas de casos de uso capturan el comportamiento requerido del sistema desde la perspectiva del usuario, siendo fundamentales en la etapa de captura de requisitos. Específicamente, en este proyecto describe acciones como:

- Desde la app móvil: registrar un perro perdido, subir una fotografía y ver resultados de coincidencias.
- Desde la web: supervisar reportes, gestionar cuentas de usuario y administrar bloqueos de seguridad.

Figura N° 2.7 **Ejemplo de Diagrama de Casos de Uso**

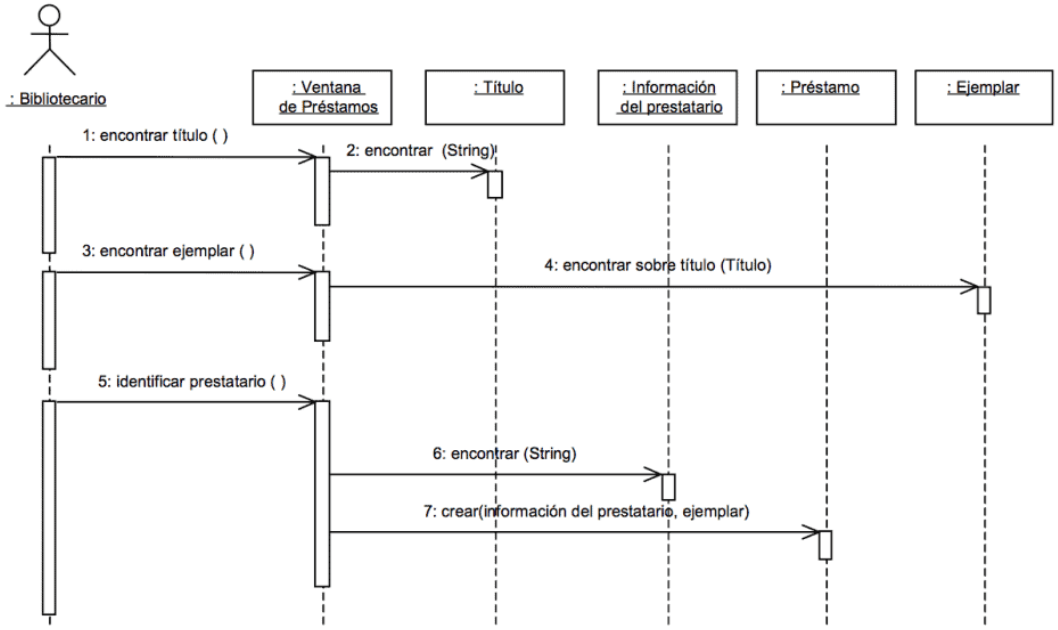



Fuente: Garza Amaya, T. R. Diagramas de caso de uso. Blog personal, 2015

d) Diagrama de Secuencia

En este diagrama se ve el orden de los mensajes entre componentes durante un proceso. Booch et al. (2005) indican que "los diagramas de secuencia muestran la interacción de un conjunto de objetos en una aplicación a través del tiempo", lo que permite identificar puntos de mejora en la comunicación entre componentes. En este proyecto, por ejemplo, muestra cómo se reporta un perro desde la app, cómo se valida y almacena ese reporte en el servidor y cómo luego se consulta o gestiona desde la plataforma web, ayudando a la comunicación entre la app, el servidor de IA y la interfaz web.

Figura N° 2.8 **Ejemplo de Diagrama de Secuencia**




Fuente: manuel.cillero.es, 2023

### 2.1.4. *METODOLOGÍA DE DESARROLLO DE SOFTWARE*

En el desarrollo de software, una metodología se entiende como un conjunto estructurado de prácticas y procesos orientados a garantizar que el producto final cumpla con los requisitos definidos, se entregue a tiempo y posea un alto nivel de calidad. Pressman y Maxim (2020) señalan que estas metodologías proporcionan guías para planificar, diseñar, construir, probar e implementar sistemas de software de manera sistemática, siendo la base para cualquier proyecto tecnológico de calidad.

En el presente proyecto se adoptan dos marcos metodológicos complementarios. Para el desarrollo del software se utiliza la metodología ágil Extreme Programming (XP). Beck y Andrés (2004) describen XP como "una filosofía de desarrollo de software basada en los valores de comunicación, retroalimentación, simplicidad, coraje y respeto", priorizando la adaptabilidad sobre la previsibilidad. Esta metodología es ideal para el desarrollo del sistema de identificación canina, ya que permite gestionar la complejidad técnica que conlleva la integración de redes neuronales y el procesamiento de imágenes en tiempo real. Para el entrenamiento del modelo de inteligencia artificial, se incorpora adicionalmente el marco CRISP-DM, cuyas fases estructuran el ciclo de vida del modelo

desde la comprensión de los datos hasta su despliegue, complementando las iteraciones de XP en esa etapa específica.

En este proyecto, la aplicación de XP tiene como propósito garantizar la entrega funcional de un sistema distribuido que conecta la aplicación móvil Android con un servidor de inteligencia artificial. Gracias a su enfoque en iteraciones cortas, cada ciclo permitirá desarrollar y validar módulos críticos como la extracción de vectores con EfficientNet-B4 y la lógica de administración en el panel web, asegurando que el software evolucione conforme a los resultados obtenidos en las pruebas de similitud.

#### 2.1.4.1. *Extreme Programming (XP)*

Extreme Programming (XP) es una metodología ágil de desarrollo de software creada por Kent Beck a finales de los años noventa, orientada a mejorar la calidad del software y la capacidad del equipo para adaptarse a cambios en los requisitos del cliente. Beck y Andrés (2004) la describen como una filosofía de desarrollo basada en valores de comunicación, simplicidad, retroalimentación, coraje y respeto, que prioriza la adaptabilidad sobre la previsibilidad mediante la entrega frecuente de software funcional en ciclos cortos.

A diferencia de los modelos tradicionales, XP no separa el desarrollo en grandes fases secuenciales, sino que integra de manera continua la planificación, el diseño, la codificación y las pruebas dentro de cada iteración. Pressman y Maxim (2020) señalan que este enfoque reduce el riesgo técnico porque los errores se detectan y corrigen de manera temprana, antes de que se acumulen en etapas avanzadas del proyecto. XP es especialmente adecuado para proyectos donde los requisitos pueden evolucionar durante el desarrollo y donde la calidad técnica del código es prioritaria.

En el presente proyecto, XP organiza el desarrollo del sistema en iteraciones progresivas que abarcan desde el entrenamiento del modelo de inteligencia artificial hasta el despliegue final en producción, garantizando que cada componente sea validado funcionalmente antes de avanzar al siguiente ciclo.

#### 2.1.4.2. *Valores de Extreme Programming (XP)*

La metodología XP se fundamenta en cinco valores esenciales que guían el comportamiento del equipo de desarrollo. Beck y Andrés (2004) establecen que estos valores son la base para crear software de calidad en entornos cambiantes:

- Simplicidad: Se busca diseñar la solución más sencilla que funcione, evitando sobrecargar el sistema con funcionalidades innecesarias.
- Comunicación: Se mantiene un flujo de información constante entre el desarrollo del frontend (Móvil/Web) y el backend (FastAPI).
- Retroalimentación: El sistema se somete a pruebas constantes tras la codificación para ajustar el algoritmo Cosine Similarity según su precisión.
- Respeto y Coraje: El equipo asume la responsabilidad de refactorizar el código cuando sea necesario para mejorar el rendimiento del reconocimiento visual.

#### 2.1.4.3. *Ciclo de Desarrollo XP*

El proceso de desarrollo bajo XP se organiza en iteraciones de corta duración, de una a tres semanas, donde se producen incrementos de software que son probados y validados inmediatamente. Según Beck y Andrés (2004), este ciclo iterativo permite al equipo construir software funcional de forma continua, reduciendo los riesgos técnicos en cada entrega.

a) Planificación:

Consiste en la definición de las Historias de Usuario, que reemplazan a los documentos de requisitos tradicionales. Cohn (2004) define las historias de usuario como "descripciones simples, claras y breves de funcionalidades que serán valiosas para usuarios reales". En este proyecto incluyen tareas como la captura de fotos desde la App, la generación de embeddings en el servidor y la gestión de bloqueos en el panel web.

b) Diseño:

Se enfoca en crear una arquitectura limpia para el sistema. Esto incluye la definición de la estructura de colecciones en Firestore y la interfaz de los endpoints en FastAPI, evitando complicaciones innecesarias en la base de datos no relacional. Beck y Andrés (2004) indican que en XP el diseño debe ser lo suficientemente simple para resolver únicamente el problema presente, sin anticipar funcionalidades futuras innecesarias.

c) Codificación y Pruebas:

A diferencia de otros modelos, en este proyecto se prioriza la programación funcional seguida de una fase rigurosa de pruebas unitarias y de integración. Beck y Andrés (2004) sostienen que en XP las pruebas no son una fase final sino una práctica continúa integrada al ciclo de codificación. Se valida que el modelo EfficientNet-B4 retorne

vectores consistentes y que la App móvil muestre correctamente los porcentajes de similitud calculados en el servidor.

d) Lanzamiento:

Cada iteración finaliza con una versión operativa de una parte del sistema, por ejemplo, el módulo de inicio de sesión funcional o el motor de búsqueda por fotos, permitiendo una validación progresiva de los resultados tecnológicos (Beck y Andrés, 2004).

Figura N° 2.9 **Ciclo de Desarrollo Extreme Programming (XP)**




Fuente: Calvo, D. Metodología XP Programación Extrema (Metodología ágil), 2018

#### 2.1.4.4. *Artefactos de la Metodología XP*

Los artefactos en XP permiten dar seguimiento técnico al avance del proyecto y mantener la transparencia en la construcción del software:

- Historias de Usuario: Son descripciones breves de las necesidades del usuario. En este proyecto, definen acciones como "Registrar perro perdido", "Comparar imágenes" y "Supervisar reportes desde la web". Según Cohn (2004), cada historia debe especificar el rol, la acción y el beneficio esperado.
- Plan de Iteración: Documento que contiene las historias de usuario seleccionadas para ser desarrolladas y probadas en el ciclo actual, como la configuración del servidor en Railway o la conexión con Cloudinary. Según Beck y Andrés (2004), este plan es el principal mecanismo de seguimiento del avance técnico en XP.

- Código con Propiedad Colectiva: Todo el código del backend y frontend está disponible para su revisión y mejora constante, facilitando la integración de los componentes de inteligencia artificial. Beck y Andrés (2004) señalan que la propiedad colectiva del código es uno de los principios fundamentales de XP, ya que permite que cualquier miembro del equipo mejore cualquier parte del sistema en cualquier momento.

#### 2.1.4.5. *CRISP-DM*

El proceso estándar entre industrias para minería de datos, conocido como CRISP-DM (Cross-Industry Standard Process for Data Mining), es un modelo de proceso que describe de manera estructurada las etapas necesarias para llevar a cabo proyectos de minería de datos y aprendizaje automático. Wirth y Hipp (2000) lo definen como un marco jerárquico que organiza el ciclo de vida de un proyecto de ciencia de datos en fases bien definidas, permitiendo que equipos de distintas industrias aborden problemas de datos de manera sistemática y reproducible. A diferencia de las metodologías de desarrollo de software como XP, CRISP-DM no gestiona el código ni los ciclos de entrega, sino que se enfoca exclusivamente en el proceso de transformar datos crudos en modelos predictivos útiles.

Chapman et al. (2000) señalan que CRISP-DM surgió como respuesta a la necesidad de estandarizar los proyectos de minería de datos, que hasta ese momento carecían de un proceso común que guiara su ejecución. Su adopción se extendió rápidamente en la industria y la academia precisamente porque es independiente de la herramienta, del dominio de aplicación y del algoritmo utilizado, lo que lo hace aplicable tanto a proyectos de clasificación de imágenes como al procesamiento de texto o datos tabulares.

El ciclo CRISP-DM no es lineal sino iterativo: los resultados de fases avanzadas como la evaluación pueden llevar a regresar a fases anteriores como la preparación de datos o el modelado para realizar ajustes. Shearer (2000) destaca que esta naturaleza cíclica es precisamente lo que lo hace robusto para proyectos de machine learning, donde raramente el primer modelo entrenado es el definitivo y donde los datos requieren múltiples rondas de limpieza y transformación antes de producir resultados satisfactorios.

#### 2.1.4.6. *Fases de CRISP-DM*

El marco CRISP-DM se organiza en seis fases principales, cada una con tareas y entregables específicos:

a) Fase 1: Comprensión del Negocio

Esta fase inicial se centra en comprender los objetivos del proyecto desde la perspectiva del problema real que se quiere resolver, antes de trabajar con los datos. Wirth y Hipp (2000) indican que en esta fase se definen los criterios de éxito del proyecto, se identifican los recursos disponibles y se establece un plan preliminar para alcanzar los objetivos. Es fundamental porque determina qué tipo de modelo se necesita, qué métricas de evaluación son relevantes y qué nivel de rendimiento se considera aceptable para que el sistema sea útil en producción.

b) Fase 2: Comprensión de los Datos

En esta fase se recopilan los datos disponibles y se los analiza para entender su estructura, calidad y relevancia para el problema. Chapman et al. (2000) señalan que las tareas principales incluyen la recolección inicial de datos, la descripción de sus características generales como cantidad, formato y distribución, y la identificación de problemas como valores faltantes, clases desbalanceadas o datos ruidosos. Esta fase es crítica porque los problemas no detectados aquí se amplifican en las fases posteriores, afectando directamente la calidad del modelo.

c) Fase 3: Preparación de los Datos

Esta fase abarca todas las transformaciones necesarias para convertir los datos crudos en un conjunto listo para el modelado. Géron (2022) explica que la preparación de datos es habitualmente la fase más costosa en tiempo dentro de un proyecto de machine learning, e incluye tareas como la normalización de valores, el aumento de datos (data augmentation), la división en conjuntos de entrenamiento y validación, y la construcción de los generadores o pipelines de datos que alimentarán al modelo durante el entrenamiento.

d) Fase 4: Modelado

En esta fase se seleccionan y aplican las técnicas de modelado apropiadas para el problema definido. Wirth y Hipp (2000) indican que esta fase incluye la selección de la arquitectura del modelo, la configuración de sus hiperparámetros y la ejecución del

entrenamiento. Para proyectos de deep learning, Géron (2022) agrega que el modelado suele implicar múltiples rondas de entrenamiento con distintas configuraciones, estrategias de regularización como Dropout y BatchNormalization, y técnicas de optimización del aprendizaje como la reducción de la tasa de aprendizaje mediante callbacks.

e) Fase 5: Evaluación

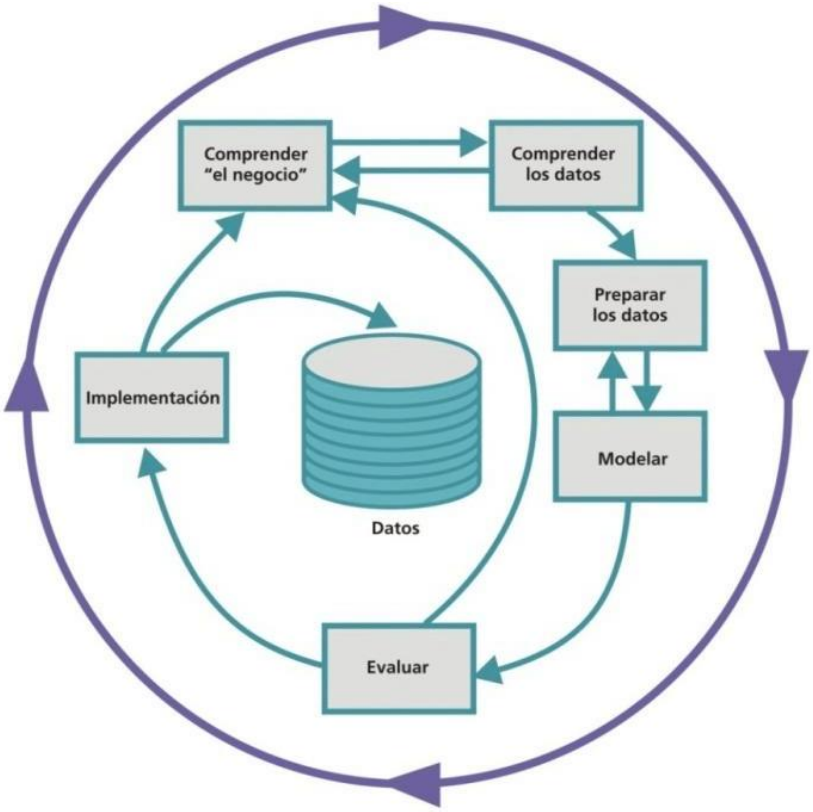
Antes de proceder al despliegue, el modelo debe ser evaluado rigurosamente para verificar que cumple con los criterios de éxito definidos en la fase 1. Chapman et al. (2000) establecen que en esta fase se aplican métricas cuantitativas para medir el rendimiento del modelo, se revisan los resultados en busca de patrones de error y se determina si el modelo está listo para producción o si es necesario regresar a fases anteriores. Las métricas típicas para modelos de clasificación incluyen la exactitud (accuracy), la Matriz de Confusión, la precisión, el recall y el F1-score (Géron, 2022).

f) Fase 6: Despliegue

La fase final consiste en integrar el modelo entrenado en el entorno de producción donde será utilizado. Wirth y Hipp (2000) señalan que el despliegue no implica únicamente exportar el modelo, sino también documentar el proceso, definir un plan de monitoreo del rendimiento en producción y establecer los mecanismos de actualización del modelo cuando su rendimiento decline con el tiempo. En proyectos de software, esta fase conecta directamente con la metodología de desarrollo utilizada, en este caso XP, que gestiona la integración del modelo como componente del sistema distribuido.

La siguiente figura ilustra el ciclo iterativo de CRISP-DM y la posibilidad de retroceder entre fases según los resultados obtenidos:

Figura N° 2.10 **Ciclo de vida CRISP-DM**




Fuente: Bellini Saibene et al., 2014

### 2.1.5. *HERRAMIENTAS DE DESARROLLO*

Las herramientas elegidas para este proyecto se seleccionaron considerando su capacidad para procesar imágenes, su facilidad de integración con modelos de visión por computadora y su estabilidad en producción. Pressman y Maxim (2020) señalan que la selección adecuada de herramientas tecnológicas es determinante para la viabilidad técnica y la calidad del producto final. Con estas tecnologías, el plan es construir un ecosistema que abarque una app móvil, un backend de inteligencia artificial y un panel web para la administración.

a) Android Studio (Ladybug 2024.2):

Es el entorno oficial para crear aplicaciones Android. Android Studio permite diseñar pantallas usando Jetpack Compose, manejar la navegación y conectar con el servidor. También hace sencillo trabajar con arquitecturas por capas y organizar las dependencias necesarias para acceder a los servicios de IA (Google Developers, 2024).

b) Kotlin:

Este lenguaje moderno y de tipado estático se utiliza para la parte móvil del frontend. En el proyecto, Kotlin se encarga de toda la interacción del usuario, tomar fotos y comunicarse con el backend (JetBrains, 2023).

c) Python y FastAPI:

Python es la base del backend, sobre todo por todo lo que ofrece para ciencia de datos. Con FastAPI se arma una API que recibe las imágenes enviadas desde la app, ejecuta el modelo de red neuronal y maneja el algoritmo de similitud (Tiangolo, 2024).

d) Firebase (Auth y Firestore):

Se usa como infraestructura en la nube. Firebase Authentication controla los accesos, y Cloud Firestore funciona como base de datos NoSQL para guardar de manera sincronizada tanto los reportes como los vectores de características (embeddings) que genera el sistema (Google, 2024).

e) OpenCV (Open Source Computer Vision Library):

Esta biblioteca sirve para preparar las imágenes antes del análisis. Con OpenCV se ajustan dimensiones, canales de color y se reduce el ruido para asegurarse de que lo que recibe la red neuronal esté estandarizado (Bradski y Kaehler, 2021).

f) EfficientNet-B4:

EfficientNet-B4 es una arquitectura moderna de red neuronal convolucional. Tan y Le (2019) proponen "un nuevo método de escalado que escala uniformemente todas las dimensiones de profundidad, ancho y resolución usando un coeficiente compuesto simple pero altamente efectivo", logrando mayor precisión con menor consumo de recursos que modelos anteriores. En este proyecto, su papel es generar vectores (embeddings) que representan la biometría visual de cada perro.

g) Cosine Similarity:

Este algoritmo matemático mide cómo de parecidos son los vectores creados por la red neuronal. Calcula el coseno del ángulo entre dos vectores en un espacio multidimensional; si el valor se acerca a 1, quiere decir que las imágenes tienen rasgos muy similares (Han, Kamber y Pei, 2022). Esta comparación se realiza en el servidor para cotejar perros encontrados con la base de datos de perros perdidos.

h) Railway y Cloudinary:

Railway se utiliza como la plataforma de despliegue (PaaS) para alojar el backend de Python en la nube, permitiendo que la App móvil acceda al modelo de IA desde cualquier ubicación (Railway, 2024). Cloudinary se emplea para el almacenamiento y gestión de las fotografías capturadas, proporcionando URLs para su visualización en la App y el Panel Web (Cloudinary, 2024).

### 2.1.6. *ESTIMACIÓN DE COSTOS*

La estimación de costos es una parte clave al planificar cualquier proyecto de ingeniería, ayuda a anticipar tanto los recursos económicos como los técnicos que se van a necesitar. En este punto, se hace una estimación de los gastos que supone desarrollar el sistema, incluyendo las necesidades de hardware, servicios en la nube y toda la logística.

Según Pressman y Maxim (2020), calcular bien los costos en proyectos de software es la base para armar el presupuesto, dividir los recursos y ver si el sistema es viable económicamente. Siguiendo esa lógica, los costos de este proyecto se agruparon en las siguientes categorías principales:

- Infraestructura y servicios en la nube: Aquí entran las suscripciones y los recursos informáticos que se necesitan para montar el backend en Railway, guardar imágenes en Cloudinary y utilizar bases de datos NoSQL y sistemas de autenticación de Firebase. Se da prioridad a los planes gratuitos para reducir el gasto inicial tanto como sea posible sin poner en riesgo la disponibilidad del sistema.
- Desarrollo e Inteligencia Artificial: Incluye los recursos para programar en Kotlin y Python, diseñar interfaces con Jetpack Compose y entrenar la red neuronal EfficientNet-B4. También se tienen en cuenta los costos de computación (GPU/CPU) para procesar los vectores de características y ejecutar el algoritmo de Cosine Similarity.
- Costos logísticos y operativos: Aquí se incluyen los traslados al Refugio WOOF para tomar datos, además de materiales de papelería, impresiones y anillados para toda la documentación técnica que exige la universidad.

Como el propio desarrollo lo realiza la investigadora de manera directa y se utilizan herramientas de código abierto como Android Studio, FastAPI y OpenCV, el costo total del proyecto es considerablemente más bajo que el de un desarrollo comercial con características similares. Para afinar esta estimación, los valores concretos aparecerán después en la sección de Presupuesto, usando la técnica PERT (Program Evaluation and Review Technique). Con esta técnica, se calcula el costo esperado y la desviación estándar a partir de tres escenarios (optimista, probable y pesimista), lo que ayuda a tener un plan financiero.

### 2.1.7. *ASEGURAMIENTO DE CALIDAD*

Durante el desarrollo del sistema del backend, panel web y aplicación móvil, se aplicará un proceso de aseguramiento de la calidad orientado a verificar la integridad funcional, el desempeño técnico y la precisión del modelo de inteligencia artificial. Sommerville (2016) define el aseguramiento de la calidad como "el conjunto de procesos que establece que el software tiene el nivel de calidad requerido", verificando que el producto final cumpla con los requisitos técnicos definidos.

De acuerdo con Sommerville (2016), el aseguramiento de la calidad comprende la definición de estándares y la implementación de procesos de verificación para asegurar que el software posea los atributos de calidad requeridos. Bajo este marco, se ejecutarán las siguientes fases de prueba:

a) Pruebas funcionales

Se verificará que cada módulo cumpla con las especificaciones técnicas, tales como el registro de usuarios, la carga de fotografías a Cloudinary, la extracción de vectores en el backend y la persistencia en Firestore.

b) Pruebas de compatibilidad

La aplicación móvil se someterá a pruebas en dispositivos con sistema operativo Android 8.0 o superior. Se evaluará la adaptabilidad de la interfaz en diferentes densidades de pantalla y resoluciones para asegurar una visualización uniforme de los resultados de similitud (Sommerville, 2016).

c) Pruebas de rendimiento y latencia

Se medirá el tiempo de respuesta del servidor alojado en Railway ante las peticiones de la App. Estas pruebas permitirán cuantificar la eficiencia de la red EfficientNet-B4 en la

generación de embeddings y la velocidad del algoritmo Cosine Similarity al procesar cargas de trabajo concurrentes (Pressman y Maxim, 2020).

d) Pruebas de integración distribuida

Se validará la comunicación entre los componentes del ecosistema: la aplicación móvil (Kotlin), el servidor de procesamiento (FastAPI), el servicio de almacenamiento (Cloudinary) y la base de datos (Firebase). El objetivo es asegurar una transmisión de datos sin pérdida de paquetes durante el flujo de identificación (Sommerville, 2016).

e) Pruebas de regresión

En cumplimiento con la metodología XP, tras cada ciclo de refactorización o actualización del modelo de IA, se ejecutarán pruebas para verificar que las nuevas implementaciones no afecten la estabilidad de las funciones previamente validadas (Beck y Andrés, 2004).

f) Pruebas de precisión del modelo

Se evaluará la exactitud del reconocimiento mediante el uso de una Matriz de Confusión. Se calcularán métricas de precisión, sensibilidad (recall) y puntuación F1 para determinar la capacidad del sistema en la identificación de perros (Tan y Le, 2019).

### 2.1.8. *SEGURIDAD DEL SISTEMA*

El sistema propuesto incorpora medidas de seguridad informática basadas en la tríada CIA (Confidencialidad, Integridad y Disponibilidad), pilares fundamentales para la protección de la información en entornos digitales. Según Stallings (2019), la seguridad informática debe garantizar que los activos de información sean accesibles únicamente por usuarios autorizados, que su contenido no sea alterado de forma indebida y que los servicios permanezcan operativos ante posibles fallos o ataques.

De acuerdo con Sommerville (2016), la seguridad del software debe garantizar la protección de los activos informáticos frente a amenazas externas e internas, asegurando la continuidad operativa. Bajo este enfoque, se detallan las estrategias implementadas en el sistema:

a) Confidencialidad

Se restringe el acceso a la información sensible únicamente a usuarios autenticados. El sistema emplea Firebase Authentication para la gestión de credenciales, utilizando tokens de seguridad para validar cada petición al servidor (Google, 2024).

- Cifrado en tránsito: Toda comunicación entre la aplicación móvil, el panel web y el backend en FastAPI se realiza mediante el protocolo HTTPS con certificados TLS/SSL, asegurando que los datos viajen cifrados.
- Control de Acceso basado en Roles: Se establecen niveles diferenciados para usuarios finales y personal administrativo del refugio, limitando las funciones de gestión de base de datos únicamente a perfiles autorizados.

b) Integridad

Garantiza que la información registrada sea precisa y esté libre de alteraciones. La validación de datos se implementa mediante Pydantic en FastAPI, verificando que los archivos recibidos sean efectivamente imágenes y que los metadatos cumplan con los formatos establecidos antes de su procesamiento (Tiangolo, 2024).

- Validación de Datos: Se implementan esquemas de validación en el backend mediante Pydantic en FastAPI, verificando que los archivos recibidos sean efectivamente imágenes y que los metadatos cumplan con los formatos establecidos antes de su procesamiento.
- Integridad de la IA: Se asegura que los vectores de características (embeddings) generados por EfficientNet-B4 no sean modificables tras su creación, manteniendo la fidelidad de la identidad visual del perro en Firestore.

c) Disponibilidad

Para asegurar la continuidad del servicio, el sistema se apoya en una infraestructura de nube con redundancia y balanceo de carga automático al utilizar Firebase y Railway, así como réplicas automáticas de la base de datos NoSQL que permiten la recuperación inmediata ante fallas técnicas (Google, 2024; Railway, 2024):

- Redundancia: Al utilizar Firebase y Railway, el sistema se beneficia de infraestructuras con alta disponibilidad y balanceo de carga automático.
- Respaldo de Datos: La base de datos NoSQL cuenta con réplicas automáticas, permitiendo la recuperación inmediata de la información en caso de fallas técnicas o desconexiones en los nodos de procesamiento

d) Seguridad de la Arquitectura

El modelo EfficientNet-B4 y el algoritmo Cosine Similarity residen en un entorno controlado en Railway, protegidos mediante variables de entorno y claves de API. Solo

las peticiones firmadas y originadas desde la aplicación oficial son procesadas por el motor de inteligencia artificial, evitando el uso indebido de los recursos del servidor por agentes externos (Stallings, 2019).

## 2.2. MARCO CONCEPTUAL

El marco conceptual comprende las definiciones de los principales términos técnicos, teóricos y funcionales involucrados en el desarrollo del proyecto. Su propósito es brindar una comprensión clara de los conceptos fundamentales que sustentan el sistema web y la aplicación móvil, desde los relacionados con el ámbito tecnológico hasta aquellos vinculados al contexto social y funcional del sistema.

### 2.2.1. *SISTEMA WEB*

Un sistema web es una aplicación informática alojada en un servidor remoto que se ejecuta a través de un navegador mediante el protocolo HTTP. A diferencia de las aplicaciones nativas, no requiere instalación previa en el hardware del usuario y permite una gestión centralizada de la información. Según Britannica (2025), las aplicaciones web aprovechan la infraestructura de la red para ofrecer interfaces dinámicas que interactúan con bases de datos en tiempo real. En este proyecto, el sistema web constituye el panel administrativo del Refugio WOOF, permitiendo que el personal gestione los reportes, supervise las coincidencias detectadas por la IA y controle el acceso de los usuarios de manera global.

### 2.2.2. *APLICACIÓN MÓVIL*

Una aplicación móvil es un programa de software diseñado específicamente para ejecutarse en dispositivos portátiles como teléfonos inteligentes o tabletas, aprovechando la conectividad, sensores integrados y portabilidad de estos equipos. Por ejemplo, “una aplicación móvil es un sistema de software de propósito particular, construido para aprovechar las capacidades del hardware portátil y satisfacer necesidades concretas del usuario” (Godaddy, 2024). Este tipo de sistemas suele distribuirse a través de tiendas como Google Play o App Store y está optimizado para sistemas operativos móviles. En el contexto de este proyecto, la aplicación móvil será la plataforma mediante la cual los usuarios cargarán fotografías de perros extraviados o encontrados, accederán a comparaciones visuales y recibirán notificaciones de coincidencias, constituyendo el canal principal para la recolección, análisis y comunicación de los datos de mascotas.

### 2.2.3. *ARQUITECTURA CLIENTE-SERVIDOR*

La arquitectura cliente-servidor es un modelo de diseño de software donde las tareas se reparten entre los proveedores de recursos, llamados servidores, y los demandantes de servicios, llamados clientes. Este modelo permite centralizar procesos pesados en hardware especializado mientras se ofrece una interfaz ligera al usuario final. De acuerdo con Pressman y Maxim (2020), esta arquitectura es fundamental en sistemas distribuidos para garantizar la escalabilidad y la seguridad de los datos. En el presente trabajo, la aplicación móvil y el sistema web actúan como clientes que solicitan el procesamiento de imágenes al servidor en Railway, asegurando que la ejecución de la red neuronal no dependa de la capacidad de procesamiento del teléfono móvil.

### 2.2.4. *BACKEND (SERVIDOR DE API)*

El backend es la capa de acceso a datos y lógica de negocio de un sistema que se mantiene invisible para el usuario final. Se encarga de procesar las peticiones enviadas por el frontend y gestionar la comunicación con servicios externos o bases de datos. Como señala Tiangolo (2024), un backend eficiente permite el intercambio de información estructurada para dar soporte a múltiples plataformas de usuario. En este proyecto, el backend desarrollado con FastAPI actúa como el núcleo de inteligencia, recibiendo las fotografías de los perros, ejecutando la red EfficientNet-B4 para generar el embedding y retornando los resultados de similitud tanto a la aplicación móvil como al sistema web.

### 2.2.5. *BASE DE DATOS NO RELACIONAL (NOSQL)*

Las bases de datos NoSQL son sistemas de almacenamiento diseñados para modelos de datos específicos que cuentan con esquemas flexibles para crear aplicaciones modernas de alto rendimiento. A diferencia de las bases relacionales, permiten manejar grandes volúmenes de datos no estructurados con mayor escalabilidad. Según Google Cloud (2024), este tipo de bases de datos es ideal para aplicaciones que requieren sincronización en tiempo real y alta disponibilidad de información. El proyecto utiliza Cloud Firestore para almacenar de forma estructurada los reportes caninos y sus vectores de características, permitiendo que la información esté disponible instantáneamente en todos los componentes del sistema.

### 2.2.6. *SERVICIOS EN LA NUBE (CLOUD COMPUTING)*

El Cloud Computing o servicios en la nube se refiere a la entrega de recursos de computación bajo demanda, como servidores y almacenamiento, a través de internet. Este modelo elimina la necesidad de mantener infraestructura física local y permite el escalamiento de recursos según la necesidad del software. De acuerdo con el Instituto Internacional de Normalización (ISO, 2023), la computación en la nube facilita el acceso compartido a recursos configurables que pueden ser aprovisionados rápidamente. En este proyecto, se utilizan servicios como Railway para el despliegue del backend, Cloudinary para el almacenamiento multimedia y Firebase para la persistencia de datos, garantizando la continuidad del servicio del refugio sin costos de servidores físicos locales.

### 2.2.7. *FIREBASE*

Firebase es una plataforma de desarrollo de aplicaciones móviles y web proporcionada por Google, que ofrece un conjunto integrado de servicios en la nube como autenticación, base de datos en tiempo real, almacenamiento de archivos y mensajería push. Según Google Firebase (2024), "Firebase ofrece herramientas que ayudan a los desarrolladores a crear aplicaciones de alta calidad, hacer crecer su base de usuarios y ganar más ingresos". En este proyecto se utilizan los servicios Firebase Authentication para la gestión de identidades de usuario, Cloud Firestore para el almacenamiento de reportes caninos y sus vectores de características, y Firebase Cloud Messaging (FCM) para el envío de notificaciones push ante posibles coincidencias.

#### 2.2.7.1. *Notificaciones Push (FCM)*

Las notificaciones push son mensajes enviados desde un servidor directamente al dispositivo del usuario sin que este deba estar usando la aplicación en ese momento. Firebase Cloud Messaging (FCM) es el servicio de Google que permite distribuir estas notificaciones de forma confiable y multiplataforma. De acuerdo con Google Firebase (2024), FCM proporciona una conexión fiable y de bajo consumo para entregar mensajes y notificaciones a dispositivos Android e iOS de forma gratuita. En el contexto del proyecto, las notificaciones push se utilizan para alertar a los usuarios cuando el sistema detecta una coincidencia con un porcentaje de similitud igual o superior al 80% entre un perro reportado como perdido y uno registrado como encontrado.

### 2.2.8. *CLOUDINARY*

Cloudinary es una plataforma de gestión de activos multimedia en la nube que permite almacenar, transformar y entregar imágenes y videos de forma optimizada. Según Cloudinary (2024), su servicio ofrece capacidades de transformación de imágenes en tiempo real, incluyendo redimensionado, recorte, ajuste de calidad y conversión de formatos mediante URL. En el proyecto, Cloudinary se emplea como repositorio centralizado de las fotografías de los perros subidas por los usuarios, garantizando un almacenamiento escalable y la entrega eficiente de imágenes tanto a la aplicación móvil como al sistema web.

### 2.2.9. *RAILWAY (PLATAFORMA DE DESPLIEGUE)*

Railway es una plataforma de infraestructura en la nube orientada a simplificar el despliegue de aplicaciones y servicios backend. Permite ejecutar proyectos directamente desde un repositorio de código sin necesidad de configurar servidores físicos ni administrar manualmente la infraestructura. Según Railway (2024), la plataforma gestiona automáticamente el aprovisionamiento de recursos, la escalabilidad y la disponibilidad del servicio. En el presente proyecto, Railway aloja el servidor backend desarrollado con FastAPI, garantizando que los servicios de procesamiento de imágenes y comparación de embeddings estén disponibles en todo momento para los clientes móvil y web.

### 2.2.10. *FASTAPI*

FastAPI es un framework de desarrollo web moderno para Python, diseñado para construir APIs de alto rendimiento de forma rápida y con validación automática de datos. Según Tiangolo (2024), FastAPI está basado en los estándares abiertos OpenAPI y JSON Schema, y permite generar documentación interactiva de forma automática. Su compatibilidad nativa con operaciones asíncronas lo hace especialmente adecuado para aplicaciones que gestionan múltiples peticiones concurrentes. En este proyecto, FastAPI constituye el núcleo del servidor backend, encargándose de recibir las fotografías, ejecutar el modelo EfficientNet-B4, calcular la similitud de coseno y retornar los resultados de coincidencia a los clientes.

### 2.2.11. *RETROFIT*

Retrofit es una biblioteca de Android desarrollada por Square que simplifica la comunicación entre una aplicación móvil y una API REST, permitiendo definir las

peticiones HTTP mediante interfaces y anotaciones en lugar de escribir código de red manualmente. Según Square (2024), Retrofit convierte automáticamente las respuestas JSON del servidor en objetos de Kotlin o Java listos para usar en la aplicación. En este proyecto, Retrofit es utilizado en la aplicación móvil para enviar las fotografías de los perros al servidor FastAPI y recibir los resultados de similitud, gestionando toda la comunicación entre el cliente Android y el backend desplegado en Railway.

#### 2.2.12. *HILT*

Hilt es un framework de inyección de dependencias para Android, desarrollado por Google sobre la base de Dagger, que simplifica la gestión y provisión de objetos dentro de una aplicación. Según Google Android (2024), Hilt reduce el código repetitivo necesario para configurar dependencias al encargarse automáticamente de su creación, ciclo de vida y distribución entre los componentes de la app. En este proyecto, Hilt se utiliza para gestionar las dependencias de la aplicación móvil, como los repositorios, los clientes de red Retrofit y los servicios de Firebase, manteniendo el código organizado y fácil de mantener.

#### 2.2.13. *IDENTIFICACIÓN DE PERROS EXTRAVIADOS*

La “identificación de perros extraviados” hace referencia al proceso mediante el cual se reconoce, registra y rastrea animales que han sido reportados como perdidos, con el objetivo de vincularlos nuevamente a sus dueños o a entornos seguros. De acuerdo con una publicación especializada, “la identificación de los animales es una herramienta fundamental para fomentar su tenencia responsable, evitar los extravíos y combatir el abandono” (Sociedad Responsable, 2023). En este proyecto, este concepto engloba tanto la obtención de fotografía e información descriptiva del perro como su comparación mediante algoritmos para establecer coincidencias visuales con hallazgos o reportes existentes, reduciendo así el tiempo que un perro permanece en un refugio.

#### 2.2.14. *REFUGIO O ALBERGUE ANIMAL*

Un refugio o albergue animal es una instalación destinada a acoger temporalmente animales domésticos que han sido abandonados, extraviados o rescatados, proporcionándoles cuidados veterinarios, alimentación, alojamiento y, en muchos casos, gestión para su adopción o reencuentro con su propietario. Según la World Organisation for Animal Health (2011), este tipo de instalaciones forman parte de los programas de

control de población canina, cuyo objetivo es “mejorar el estado de salud y bienestar de la población de perros, ya sean éstos vagabundos o con propietario” (WOAH, 2011). En el marco del proyecto, el refugio constituye el lugar de aplicación práctica donde los procesos de carga, comparación y notificación operan, siendo escenario y contexto de la problemática de permanencia prolongada de perros.

### 2.2.15. *REENCUENTRO DE MASCOTAS*

El reencuentro de mascotas se refiere al proceso mediante el cual un animal extraviado es identificado, localizado y devuelto a su propietario o entregado a un entorno adecuado de cuidado. Este proceso implica no solo la identificación del animal sino también la verificación de coincidencias, la notificación al dueño y la coordinación del encuentro. Según la Fundación Affinity (2024), uno de los pilares para que un perro sea devuelto a su familia es que esté correctamente identificado y reportado, lo cual incrementa considerablemente las probabilidades de reencuentro (Fundación Affinity, 2024). En el presente proyecto, el objetivo es apoyar este reencuentro mediante la comparación automática de imágenes y la comunicación directa entre quienes reportan un perro extraviado y quienes lo encuentran.

### 2.2.16. *PROCESAMIENTO DE IMÁGENES*

El procesamiento de imágenes es una disciplina de la informática que comprende el conjunto de técnicas y algoritmos aplicados a imágenes digitales para mejorar su calidad, extraer información útil o transformarlas con fines específicos. Como señalan López (2025), “el procesamiento de imágenes transforma datos visuales no estructurados en información comprensible mediante algoritmos e inteligencia artificial” (López, 2025). En este proyecto, el procesamiento de imágenes se aplica a las fotografías de los perros para ajustar tamaño, recortar fondo, normalizar color, eliminar ruido y preparar la imagen para la extracción de características visuales mediante la red neuronal.

### 2.2.17. *INTELIGENCIA ARTIFICIAL Y APRENDIZAJE AUTOMÁTICO*

Las tecnologías de inteligencia artificial y aprendizaje automático representan la base conceptual y metodológica de este proyecto, ya que permiten que los sistemas sean capaces de aprender a partir de los datos y mejorar su rendimiento con el tiempo.

#### 2.2.17.1. *Inteligencia Artificial (IA)*

La inteligencia artificial (IA) es la disciplina científica que busca desarrollar sistemas capaces de realizar tareas que requieren razonamiento, aprendizaje y percepción similar a la humana. De acuerdo con la Organización de las Naciones Unidas para la Educación, la Ciencia y la Cultura, “la inteligencia artificial es la capacidad de las máquinas para imitar funciones cognitivas como el aprendizaje, la resolución de problemas o la comprensión del lenguaje” (UNESCO, 2024). Asimismo, el Instituto Internacional de Normalización la define como “la combinación de algoritmos, datos y potencia de cómputo que permite a los sistemas interpretar su entorno, aprender de la experiencia y tomar decisiones” (ISO, 2023).

En el contexto del presente proyecto, la IA constituye la base tecnológica que permitirá automatizar el reconocimiento visual de perros extraviados mediante la comparación de imágenes con un modelo entrenado previamente.

#### 2.2.17.2. *Aprendizaje Automático (Machine Learning)*

El aprendizaje automático es una rama de la inteligencia artificial que estudia métodos mediante los cuales las máquinas pueden aprender patrones o comportamientos a partir de datos sin ser programadas explícitamente. Según Russell y Norvig (2022), “el aprendizaje automático es el campo que da a los sistemas la habilidad de mejorar su desempeño en una tarea con la experiencia”. Este enfoque permite que los modelos computacionales encuentren relaciones en grandes volúmenes de información y ajusten sus predicciones o clasificaciones según los resultados obtenidos.

En este proyecto, el aprendizaje automático fue aplicado durante la fase de entrenamiento del modelo EfficientNet-B4 sobre un conjunto de datos, logrando que el sistema aprenda a extraer características visuales representativas de cada raza canina para su uso posterior en la comparación de imágenes.

#### 2.2.17.3. *Aprendizaje Profundo (Deep Learning)*

El aprendizaje profundo es una técnica avanzada del aprendizaje automático que utiliza redes neuronales de múltiples capas para modelar relaciones complejas entre datos. Goodfellow, Bengio y Courville (2016) lo definen como “un conjunto de algoritmos que construyen representaciones jerárquicas de los datos mediante transformaciones no

lineales distribuidas en varias capas". Este enfoque ha demostrado una alta eficacia en tareas de visión por computadora, reconocimiento de voz y análisis de texto.

En el proyecto, el aprendizaje profundo se aplicará a través de redes neuronales convolucionales, que analizarán las fotografías de los perros extraviados y extraerán de ellas patrones visuales representativos, permitiendo su comparación automática con otras imágenes en la base de datos.

### 2.2.18. *REDES NEURONALES*

Las redes neuronales constituyen el núcleo funcional de los modelos de aprendizaje profundo. Su diseño está inspirado en la estructura del cerebro humano, lo que les permite aprender a identificar patrones complejos a partir de grandes volúmenes de datos. En este proyecto, las redes neuronales serán la base del sistema de reconocimiento visual utilizado para analizar y comparar las fotografías de los perros extraviados mediante el servidor de inteligencia artificial.

#### 2.2.18.1. *Red neuronal artificial*

Una red neuronal artificial (RNA) es un modelo computacional inspirado en el funcionamiento del cerebro humano, compuesto por unidades llamadas neuronas que se organizan en capas y se interconectan mediante pesos sinápticos. Estas redes tienen la capacidad de aprender patrones complejos a partir de datos mediante un proceso de entrenamiento iterativo. Según IBM (2024), "una red neuronal artificial es un conjunto de algoritmos diseñados para reconocer patrones, interpretando datos sensoriales a través de una especie de percepción automática". Por su parte, NVIDIA (2023) señala que las RNA "imitan el comportamiento de las neuronas biológicas mediante cálculos matemáticos, ajustando los pesos de conexión hasta optimizar las predicciones o clasificaciones". En este proyecto, la red neuronal artificial constituye la base sobre la cual se implementa la red neuronal convolucional, encargada de analizar las imágenes y generar representaciones visuales que permitan identificar similitudes entre perros perdidos y encontrados.

#### 2.2.18.2. *Red Neuronal Convolucional (CNN)*

Una red neuronal convolucional (CNN, por sus siglas en inglés Convolutional Neural Network) es una arquitectura especializada de las redes neuronales artificiales, diseñada para procesar datos con estructura de cuadrícula, como imágenes o videos. Goodfellow,

Bengio y Courville (2016) explican que “las CNN aplican filtros convolucionales sobre los datos de entrada para detectar automáticamente características jerárquicas, desde bordes simples hasta formas más complejas”. Asimismo, Intel (2024) sostiene que las CNN son “modelos de aprendizaje profundo que extraen y aprenden patrones visuales directamente desde las imágenes, eliminando la necesidad de ingeniería manual de características”. En el contexto de este proyecto, la CNN será el núcleo del sistema de identificación visual, procesando las fotografías de los perros y extrayendo de ellas un conjunto de características que luego se compararán mediante un algoritmo de similitud.

#### 2.2.18.3. *EfficientNet-B4*

EfficientNet-B4 es una arquitectura de red neuronal convolucional avanzada que utiliza un método de escalamiento compuesto para equilibrar la profundidad, el ancho y la resolución de la imagen de manera óptima. A diferencia de modelos ligeros para móviles, esta red logra extraer rasgos visuales con una precisión significativamente mayor, lo que la hace ideal para la identificación individual de caninos. De acuerdo con Tan y Le (2019), EfficientNet alcanza una exactitud superior con menos parámetros que otras redes de su categoría, permitiendo una extracción de características robusta frente a variaciones de luz o ángulo. En este proyecto, el modelo se implementa en el backend para generar vectores de características de alta dimensión, actuando como el motor principal para reconocer los rasgos distintivos que permiten el reencuentro de las mascotas.

### 2.2.19. *COMPONENTES DE LA ARQUITECTURA DEL MODELO*

Los siguientes componentes forman parte de la arquitectura interna del modelo EfficientNet-B4 implementado en el proyecto y son fundamentales para entender su funcionamiento durante el entrenamiento y la generación de embeddings

#### 2.2.19.1. *Global Average Pooling 2D*

GlobalAveragePooling2D es una operación de reducción de dimensionalidad que toma el mapa de características tridimensional generado por las capas convolucionales y calcula el promedio de cada canal, produciendo un único vector numérico como salida. Según Géron (2022), esta operación permite comprimir la representación espacial de la imagen en un vector compacto sin perder la información semántica esencial extraída por la red. En este proyecto, GlobalAveragePooling2D reduce el mapa de características de

EfficientNet-B4 a un vector de 512 dimensiones que actúa como el embedding o huella digital visual de cada perro para el cálculo de similitud coseno.

#### 2.2.19.2. *Normalización por Lotes (Batch Normalization)*

BatchNormalization es una técnica de regularización que normaliza los valores de activación de cada capa durante el entrenamiento, manteniéndolos dentro de un rango estable. Goodfellow, Bengio y Courville (2016) señalan que esta técnica acelera la convergencia del entrenamiento y reduce la sensibilidad a la inicialización de pesos, estabilizando el proceso de aprendizaje. En el modelo de este proyecto, BatchNormalization se aplica en la cabeza clasificadora para garantizar que los valores que fluyen hacia las capas densas finales estén normalizados, mejorando la estabilidad y el rendimiento del entrenamiento.

#### 2.2.19.3. *Dropout*

Dropout es una técnica de regularización que durante el entrenamiento desactiva aleatoriamente un porcentaje de neuronas en cada pasada, impidiendo que la red dependa de conexiones específicas y forzándola a generalizar mejor. Según Géron (2022), el Dropout reduce el sobreajuste (overfitting) al crear un efecto de conjunto de múltiples modelos dentro de una misma red. En este proyecto, Dropout se incorpora en la cabeza clasificadora del modelo para mejorar su capacidad de identificar correctamente perros en fotografías con variaciones de luz, ángulo o fondo que no estuvieron presentes durante el entrenamiento.

#### 2.2.19.4. *Softmax*

Softmax es una función de activación utilizada en la capa de salida de redes de clasificación múltiple, que transforma los valores numéricos crudos del modelo en probabilidades interpretables, garantizando que la suma de todas las salidas sea igual a 1. De acuerdo con Russell y Norvig (2022), softmax permite expresar la certeza del modelo respecto a cada clase posible en términos de probabilidad relativa. En este proyecto, la función softmax se aplica en la capa densa final del modelo para obtener la probabilidad de pertenencia a cada una de las 122 clases, valor que luego se utiliza para aplicar el umbral de confianza del 50% antes de proceder con la comparación de embeddings.

### 2.2.20. *EXTRACCIÓN Y REPRESENTACIÓN DE CARACTERÍSTICAS*

Una de las etapas más importantes dentro del procesamiento de imágenes es la extracción de características y su representación en vectores numéricos. Este proceso permite que los sistemas de inteligencia artificial interpreten la información visual contenida en las fotografías y la traduzcan en datos que puedan ser analizados computacionalmente.

#### 2.2.20.1. *Extracción de características (Feature Extraction)*

La extracción de características es el proceso mediante el cual se transforman datos visuales en atributos cuantificables que describen aspectos relevantes de una imagen, como su forma, color o textura. Según MathWorks (2024), “la extracción de características convierte los datos sin procesar en representaciones numéricas que permiten que los algoritmos de aprendizaje automático identifiquen patrones relevantes”. En el proyecto, este proceso se aplicará a las imágenes de los perros para obtener los parámetros visuales que alimentarán el modelo de red neuronal.

#### 2.2.20.2. *Embedding (vector de características)*

Un embedding es una representación numérica densa de un objeto (imagen, palabra, etc.) en un espacio multidimensional, donde las similitudes entre los objetos se reflejan en distancias menores. De acuerdo con IBM (2023), “los embeddings traducen datos complejos a vectores de números que preservan la estructura semántica o visual, facilitando su comparación y procesamiento por modelos de IA”. En el proyecto, cada fotografía procesada se convertirá en un vector de características, o embedding, que actuará como una “huella digital visual” única para cada perro, permitiendo comparar nuevas imágenes con las almacenadas en la base de datos.

### 2.2.21. *TÉCNICAS DE ENTRENAMIENTO Y COMPARACIÓN*

Para que el sistema logre un reconocimiento eficiente y fiable, se aplican técnicas que mejoran el rendimiento del modelo y permiten comparar los resultados obtenidos. Estas técnicas aseguran que el sistema aprenda de manera precisa y que las comparaciones visuales entre imágenes sean coherentes y objetivas.

#### 2.2.21.1. *Transfer Learning*

El transfer learning o aprendizaje por transferencia es una técnica que permite reutilizar un modelo previamente entrenado en una tarea y adaptarlo a un nuevo contexto con

menos datos. Según Roboflow (2024), “esta técnica aprovecha los conocimientos adquiridos por modelos grandes para reducir el tiempo de entrenamiento y mejorar la precisión en nuevas tareas especializadas”. En el presente proyecto, se aplicará transfer learning a la arquitectura EfficientNet-B4 para que aprenda a identificar rasgos biométricos caninos individuales a partir de conjuntos de imágenes reales, optimizando el rendimiento del modelo en el servidor de inteligencia artificial.

#### 2.2.21.2. *Fine-tuning (Ajuste Fino)*

El fine-tuning o ajuste fino es una técnica derivada del transfer learning que consiste en descongelar parcialmente las capas de un modelo preentrenado y reentrenarlas con datos específicos del nuevo dominio, utilizando una tasa de aprendizaje reducida para preservar el conocimiento previo adquirido. Según Géron (2022), el fine-tuning permite especializar un modelo general en una tarea concreta con menos datos y menor tiempo de entrenamiento que un entrenamiento desde cero. En este proyecto, se aplicó fine-tuning sobre las capas superiores de EfficientNet-B4 para que el modelo aprenda a distinguir los rasgos visuales individuales de los perros más allá de la clasificación por raza, mejorando la calidad de los embeddings generados para la comparación de similitud coseno.

#### 2.2.21.3. *Cosine Similarity (Similitud del coseno)*

La similitud del coseno es una métrica matemática utilizada para medir la semejanza entre dos vectores en un espacio multidimensional, calculando el coseno del ángulo entre ellos. Según DataStax (2024), “la similitud del coseno se emplea en el procesamiento de imágenes y texto para cuantificar la relación entre dos objetos representados como vectores”. En este proyecto, este algoritmo comparará los embeddings generados por EfficientNet-B4, determinando qué fotografías presentan mayor similitud visual y notificando al usuario cuando se supere un umbral predefinido de coincidencia para facilitar el reencuentro de las mascotas.

Figura N° 2.11 **Formula del coseno de similitud**

$$\text{cosine similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Fuente: Understanding Vectors from a Machine Learning Perspective, 2024

### 2.2.22. *ENTORNO Y RECURSOS DE ENTRENAMIENTO*

Los siguientes elementos constituyen el entorno tecnológico y los recursos utilizados durante la fase de entrenamiento del modelo de inteligencia artificial del proyecto.

#### 2.2.22.1. *TensorFlow*

TensorFlow es una biblioteca de código abierto desarrollada por Google para el cómputo numérico y el aprendizaje automático, que proporciona las operaciones matemáticas de bajo nivel necesarias para construir y entrenar modelos de aprendizaje profundo. Según Géron (2022), TensorFlow permite ejecutar cálculos sobre grandes volúmenes de datos de forma eficiente, aprovechando tanto CPU como GPU para acelerar el procesamiento. En este proyecto, TensorFlow actúa como el motor de fondo sobre el cual se construyó y entrenó el modelo EfficientNet-B4, gestionando internamente todas las operaciones matemáticas del proceso de entrenamiento y la generación de embeddings.

#### 2.2.22.2. *Keras*

Keras es una biblioteca de Python que proporciona una interfaz de alto nivel para construir y entrenar modelos de aprendizaje profundo, integrada oficialmente dentro de TensorFlow. Según Géron (2022), Keras permite definir arquitecturas de redes neuronales de forma modular e intuitiva mediante capas, modelos y callbacks predefinidos, sin necesidad de gestionar directamente las operaciones matemáticas del backend. En este proyecto, Keras fue la herramienta utilizada para construir la cabeza

#### 2.2.22.3. *Kaggle*

Kaggle es una plataforma en línea orientada a la ciencia de datos y el aprendizaje automático que ofrece entornos de ejecución gratuitos con acceso a hardware especializado, conjuntos de datos públicos y herramientas de experimentación. Según Kaggle (2024), la plataforma permite ejecutar notebooks de Python directamente en la nube sin necesidad de configurar infraestructura local, proporcionando acceso gratuito a GPUs y TPUs por sesión. En este proyecto, Kaggle fue utilizado como entorno de entrenamiento del modelo EfficientNet-B4, aprovechando sus recursos de cómputo gratuitos para ejecutar el proceso de entrenamiento.

#### 2.2.22.4. *GPU (Unidad de Procesamiento Gráfico)*

Una GPU o Unidad de Procesamiento Gráfico es un procesador especializado diseñado originalmente para renderizar gráficos, pero que por su capacidad de ejecutar miles de

operaciones matemáticas en paralelo se convirtió en el hardware estándar para entrenar modelos de aprendizaje profundo. Según NVIDIA (2023), las GPUs aceleran el entrenamiento de redes neuronales entre 10 y 100 veces en comparación con una CPU convencional, gracias a su arquitectura masivamente paralela. En este proyecto, el entrenamiento del modelo se realizó sobre GPU dentro del entorno Kaggle, reduciendo significativamente el tiempo necesario para completar las épocas de entrenamiento.

#### 2.2.22.5. *Dataset (Conjunto de Datos)*

Un dataset o conjunto de datos es una colección estructurada de ejemplos etiquetados utilizada para entrenar, validar y evaluar modelos de aprendizaje automático. Según Géron (2022), la calidad y cantidad del dataset es uno de los factores más determinantes en el rendimiento final de un modelo, ya que el aprendizaje depende directamente de los patrones presentes en los datos de entrenamiento. En este proyecto se utilizó varios conjuntos de datos que contiene imágenes etiquetadas conformando un total de 122 clases para el entrenamiento del modelo.

#### 2.2.22.6. *Callbacks de Entrenamiento*

Un callback es un mecanismo de Keras que se ejecuta automáticamente en puntos específicos del entrenamiento, como al finalizar cada época, permitiendo monitorear el progreso del modelo y tomar decisiones automáticas sobre el proceso sin intervención manual. Según la documentación de Keras (2024), los callbacks permiten controlar el comportamiento del entrenamiento en tiempo real, desde guardar el modelo hasta detener el proceso o modificar parámetros dinámicamente. En este proyecto se utilizaron tres callbacks principales durante el entrenamiento del modelo EfficientNet-B4.

#### 2.2.22.7. *ModelCheckpoint*

ModelCheckpoint es un callback que guarda automáticamente el estado del modelo durante el entrenamiento cada vez que se cumple una condición definida, como mejorar la métrica de validación. Según la documentación de Keras (2024), este mecanismo permite conservar la mejor versión del modelo encontrado durante todo el proceso de entrenamiento, evitando perder el punto óptimo si el rendimiento empeora en épocas posteriores. En este proyecto, ModelCheckpoint se configuró para guardar el modelo únicamente cuando la precisión de validación mejoraba, asegurando que el archivo final corresponda a la mejor versión obtenida durante el entrenamiento.

#### 2.2.22.8. *EarlyStopping*

EarlyStopping es un callback que detiene automáticamente el entrenamiento cuando la métrica monitoreada deja de mejorar durante un número determinado de épocas consecutivas, evitando el sobreajuste y el desperdicio de recursos computacionales. Goodfellow, Bengio y Courville (2016) señalan que la parada temprana es una de las formas más simples y efectivas de regularización en redes neuronales profundas. En este proyecto, EarlyStopping se utilizó para interrumpir el entrenamiento cuando la pérdida de validación no presentaba mejoras, garantizando que el modelo no continuara ajustándose innecesariamente a los datos de entrenamiento.

#### 2.2.22.9. *ReduceLROnPlateau*

ReduceLROnPlateau es un callback que reduce automáticamente la tasa de aprendizaje cuando la métrica monitoreada deja de mejorar durante un número definido de épocas, permitiendo que el modelo realice ajustes más finos en lugar de dar pasos demasiado grandes. Según Géron (2022), reducir la tasa de aprendizaje de forma adaptativa durante el entrenamiento es una estrategia efectiva para superar estancamientos y mejorar la precisión final del modelo. En este proyecto, ReduceLROnPlateau se aplicó para ajustar dinámicamente la tasa de aprendizaje durante el fine-tuning del modelo EfficientNet-B4.

### 2.2.23. *EVALUACIÓN DEL MODELO*

La evaluación del modelo permite verificar el rendimiento predictivo en la identificación de coincidencias entre perros perdidos y encontrados. Mediante métricas específicas, se determinará la precisión y confiabilidad del sistema propuesto, asegurando que los resultados sean consistentes y útiles para su aplicación práctica.

#### 2.2.23.1. *Precisión (Precision)*

La precisión es una métrica que mide el porcentaje de predicciones positivas correctas entre todas las predicciones positivas realizadas. Según Géron (2022), “la precisión evalúa qué tan confiables son las predicciones positivas de un modelo de clasificación”. En este proyecto, la precisión medirá cuántas de las coincidencias detectadas por el sistema resultan ser verdaderas.

#### 2.2.23.2. *Sensibilidad (Recall)*

La sensibilidad o recall mide la proporción de casos verdaderos positivos identificados correctamente por el modelo. Géron (2022) la define como “la capacidad del modelo para

encontrar todos los ejemplos relevantes dentro del conjunto de datos". En el proyecto, esta métrica permitirá evaluar la capacidad del sistema para reconocer la mayor cantidad posible de perros extraviados registrados.

#### 2.2.23.3. *Tasa de Verdaderos Positivos (True Positive Rate)*

La tasa de verdaderos positivos (TPR) representa la proporción de casos positivos correctamente identificados. Según Han, Kamber y Pei (2022), "esta métrica es fundamental para evaluar el rendimiento de sistemas de detección, ya que cuantifica el éxito del modelo en reconocer los eventos reales positivos". En el contexto del proyecto, esta tasa permitirá determinar el rendimiento predictivo del sistema al identificar coincidencias reales entre perros perdidos y encontrados, contribuyendo así a reducir su permanencia prolongada en los refugios.

clasificadora sobre EfficientNet-B4, configurar los callbacks de entrenamiento y exportar el modelo final.

### 2.2.24. *UMBRAL DE CONFIANZA*

Un umbral de confianza es un valor mínimo de probabilidad que debe superar la predicción de un modelo para que sea aceptada como válida. Su uso permite filtrar resultados poco seguros y reducir los falsos positivos en sistemas de clasificación. Según Gérón (2022), establecer umbrales adecuados es fundamental para equilibrar la precisión y la sensibilidad del modelo según el caso de uso. En este proyecto, el modelo aplica un umbral de confianza del 50% sobre la clase canina antes de ejecutar la comparación de similitud: si la imagen no supera dicho umbral, es rechazada automáticamente sin llegar a calcular el embedding.

### 2.2.25. *PRECISIÓN TOP-K (TOP-K ACCURACY)*

La precisión Top-K es una métrica de evaluación que mide si la respuesta correcta se encuentra entre las K predicciones más probables del modelo. A diferencia de la precisión Top-1, que solo acepta la predicción de mayor puntuación, el Top-K es más flexible y útil en escenarios donde el sistema presenta varias opciones al usuario. Según Goodfellow, Bengio y Courville (2016), esta métrica es comúnmente utilizada en tareas de recuperación de información y reconocimiento de imágenes. En el contexto del proyecto, el modelo alcanzó una precisión Top-1 del 93,4% y una precisión Top-5 del 99,59%, lo

que significa que en casi la totalidad de los casos el perro correcto aparece entre los cinco primeros resultados presentados al usuario.

### 2.2.26. *MATRIZ DE CONFUSIÓN*

La matriz de confusión es una herramienta de evaluación que permite visualizar el rendimiento de un modelo de clasificación comparando las clases predichas con las clases reales. Géron (2022) la describe como una tabla donde cada fila representa una clase real y cada columna una clase predicha, permitiendo identificar no solo los aciertos del modelo sino también qué tipos de errores comete. Sus cuatro componentes son: Verdaderos Positivos (TP), casos correctamente clasificados como positivos; Verdaderos Negativos (TN), casos correctamente clasificados como negativos; Falsos Positivos (FP), casos negativos clasificados erróneamente como positivos; y Falsos Negativos (FN), casos positivos clasificados erróneamente como negativos. En el proyecto WOOF, esta métrica se aplica dentro de la Fase 5 de CRISP-DM para evaluar la capacidad del modelo de distinguir correctamente entre imágenes que contienen un perro y las que no.

### 2.2.27. *COEFICIENTE DE DETERMINACIÓN ($R^2$)*

El coeficiente de determinación $R^2$ indica qué proporción de la varianza de los datos es explicada por el modelo, con valores entre 0 y 1 donde valores cercanos a 1 representan mejor ajuste (Hair et al., 2019). Sin embargo, el $R^2$ estándar está diseñado para modelos de regresión y no es directamente aplicable a modelos de clasificación multiclase. Para este caso se utilizan dos variantes complementarias.

El Pseudo-$R^2$ de McFadden mide la mejora del modelo entrenado respecto a un modelo nulo que predice todas las clases con igual probabilidad. Su fórmula es:

Figura N° 2.12 **Fórmula de Pseudo-$R^2$ de McFadden**

$$R^2_{McFadden} = 1 - \left( \frac{\ln L_{modelo}}{\ln L_{nulo}} \right)$$

Fuente: Hosmer y Lemeshow, 2000

Donde ln L_modelo es el logaritmo de la verosimilitud del modelo entrenado y ln L_nulo la del modelo nulo. McFadden (1974) establece que valores entre 0.20 y 0.40 indican un ajuste bueno y valores superiores a 0.40 un ajuste excelente.

El $R^2$ de Efron complementa al anterior calculando la correlación entre las probabilidades predichas por el modelo y los valores reales binarizados por clase, siendo más intuitivo para evaluar modelos de clasificación con salida probabilística.

En el proyecto, ambos indicadores se calculan sobre el conjunto de validación del modelo EfficientNet-B4 dentro de la Fase 5 de CRISP-DM, complementando las métricas de exactitud Top-1 y Top-5 para una evaluación estadística más completa del modelo.

## 2.3. MARCO INSTITUCIONAL

El Refugio Woof es una entidad independiente sin fines de lucro ubicado en la ciudad de El Alto, creado hace 6 años con la misión de brindar un hogar seguro y amoroso para aquellos animales que más lo necesitan.

Cuando se comenzó esta labor hace 6 años, se contaba con tan solo 30 animalitos y una infraestructura básica que no estaba adaptada a sus necesidades. No se tenía veterinarios ni ingresos para mantener el refugio funcionando. Durante los primeros 3 años, la dedicación y el sueldo de las dos responsables del refugio fueron los pilares que nos permitieron sobrevivir.

Sin embargo, con la llegada de la pandemia, se tuvo una situación crítica al perder nuestros trabajos. Fue entonces cuando se decidió dar un paso adelante y crear una página para dar a conocer el trabajo y solicitar ayuda para los animalitos. A medida que se dio a conocer, comenzamos a recibir el apoyo de familias solidarias que donaban alimentos, patrocinaban esterilizaciones y brindaban ayuda en las diferentes necesidades de los perritos y gatitos.

Hoy en día, estamos orgullosos de decir que hemos crecido significativamente. Actualmente, albergamos a más de 102 animalitos, contamos con divisiones adecuadas para los perros y gatos, y una infraestructura que satisface sus necesidades. Además, brindamos una atención veterinaria óptima para garantizar su bienestar.

A lo largo de nuestra trayectoria, se logró dar en adopción a más de 150 animales. También se esterilizo un aproximado de 200 animales en situación de calle y hemos brindado apoyo en las esterilizaciones de familias de escasos recursos. No solo nos enfocamos en canes y felinos, sino que también hemos rescatado aves de corral y roedores domésticos como conejos y ratas.

### 2.3.1. *MISIÓN*

En el Refugio Woof, nuestra misión es mejorar la calidad de vida de los animales rescatados, brindándoles cuidado, rehabilitación y encontrándoles hogares seguros. Promovemos la tenencia responsable y fomentamos el respeto hacia los animales.

### 2.3.2. *VISIÓN*

En el Refugio Woof, aspiramos a convertirnos en un referente en la protección animal, reconocidos por nuestro compromiso y dedicación a la causa. Nos esforzamos por establecer alianzas con organizaciones locales, autoridades y la comunidad en general para promover políticas y acciones que garanticen el respeto y el bienestar de todos los animales. Valoramos la participación activa de voluntarios, donantes y familias adoptantes, quienes son pilares fundamentales en nuestra labor. Trabajamos en equipo, con transparencia y ética, para asegurar que cada animalito bajo nuestro cuidado reciba el amor y la atención que merece.

### 2.3.3. *OBJETIVO*

Mejorar la calidad de vida de los animales rescatados de abandono y maltrato, rehabilitarlos y encontrarles un hogar seguro, promoviendo la tenencia responsable, la conciencia, la responsabilidad y el respeto de la sociedad hacia los animales.

### 2.3.4. *OBJETIVOS ESPECÍFICOS*

- Rescatar animales en situación de abandono, maltrato o peligro, brindándoles un lugar seguro y adecuado donde puedan recibir cuidado y atención.
- Rehabilitar física y emocionalmente a los animales rescatados, proporcionándoles los tratamientos veterinarios necesarios, socialización y entrenamiento para mejorar su calidad de vida y aumentar sus posibilidades de encontrar un hogar adoptivo.
- Encontrar hogares responsables y amorosos para los animales rehabilitados, a través de procesos de adopción rigurosos que garanticen su bienestar a largo plazo.
- Promover la tenencia responsable de mascotas, educando a la comunidad sobre la importancia de brindar cuidado adecuado, esterilizar y castrar a sus mascotas, así como fomentar el respeto y la empatía hacia los animales.

- Proporcionar atención veterinaria gratuita o a bajo costo a animales en situación de calle, colaborando con clínicas y profesionales veterinarios para mejorar su salud y reducir la propagación de enfermedades.
- Establecer programas de asistencia y terapia con animales, aprovechando el vínculo especial que se puede formar entre las personas y los animales para promover el bienestar físico, emocional y social.
- Sensibilizar y concientizar a la sociedad sobre los problemas de abandono, maltrato y comercio ilegal de animales, promoviendo cambios positivos en la legislación y en las actitudes hacia los animales.
- Colaborar con otras organizaciones, voluntarios y entidades gubernamentales para fortalecer los esfuerzos en la protección y cuidado de los animales, creando alianzas y redes de apoyo.
- Mejorar continuamente las instalaciones y recursos del refugio, garantizando un ambiente seguro, limpio y adecuado para los animales bajo su cuidado.
- Obtener recursos financieros, materiales y humanos para asegurar la sostenibilidad y el crecimiento del refugio, permitiendo ampliar su capacidad de rescate, rehabilitación y adopción de animales en necesidad.

### 2.3.5. *VALORES*

Los valores que caracterizan al Refugio Woof son:

- Compasión: Nos preocupamos profundamente por el bienestar de los animales y mostramos empatía hacia su sufrimiento.
- Responsabilidad: Asumimos la responsabilidad de brindar cuidado y atención adecuada a los animales rescatados, así como de promover la tenencia responsable de mascotas.
- Integridad: Actuamos con honestidad, transparencia y ética en todas nuestras acciones y decisiones.
- Respeto: Tratamos a todos los animales con respeto, sin importar su raza, especie o condición, así como a todas las personas involucradas en nuestro trabajo.
- Colaboración: Fomentamos el trabajo en equipo y la colaboración con otras organizaciones, voluntarios y la comunidad para lograr un impacto positivo en la vida de los animales.

- Excelencia: Nos esforzamos por ofrecer servicios de alta calidad, proporcionando atención veterinaria óptima, instalaciones adecuadas y programas de rehabilitación efectivos.
- Educación: Promovemos la conciencia y el conocimiento sobre el bienestar animal, brindando información y educación a la comunidad para fomentar cambios positivos en la relación con los animales.
- Compromiso: Estamos comprometidos con nuestra misión y visión, dedicando nuestros esfuerzos y recursos para lograr un impacto duradero en la vida de los animales y la sociedad en general.

## 2.4. ESTADO DEL ARTE

En esta sección se presentan dos proyectos internacionales y uno nacional relacionados con la búsqueda y recuperación de mascotas extraviadas. Aunque comparten el mismo propósito general, cada uno emplea soluciones tecnológicas distintas, lo que permite ubicar el presente trabajo dentro de un panorama comparativo más amplio.

**Proyecto N° 1**

**Título: "Implementación de aplicación web para el registro y control de perros extraviados utilizando código QR en el centro veterinario Animedic"**

**Autor:** Orozco Villacrés, Alex Raúl

**Universidad:** Universidad Agraria de Ecuador – Ecuador.

**Resumen:**

Este trabajo aborda la problemática de los perros extraviados en el contexto de un centro veterinario ("Animedic", ciudad Riobamba) y propone la implementación de una aplicación web para registrar perros perdidos mediante códigos QR. El sistema permite que el dueño registre su perro, se genere un código QR vinculado al perfil del animal, y que cuando el perro sea encontrado o reportado, se vincule al registro para su posible retorno. Incluye módulos de registro del animal, generación del QR, base de datos de usuarios, módulo de notificaciones vía correo o teléfono, módulo de pruebas de usabilidad con usuarios del centro veterinario, y análisis de satisfacción de los clientes. El objetivo es aumentar la efectividad del hallazgo y devolución de perros perdidos.

**Diferencias con mi proyecto:**

- El proyecto no incluye componente de visión artificial como las redes neuronales para comparar imágenes como en mi proyecto, es más registro + QR + notificación.
- Mi proyecto añade comparación de imágenes, filtrado por rasgos, base de datos en la nube, usuarios que cargan fotos, devolución basada en similitud de imagen, lo cual este proyecto no incluye.

**Proyecto N° 2**

**Título: Aplicación móvil para localización de mascotas perdidas basada en GPS**

**Autor:** Rubén Téllez Hernández, Manuel Pérez Villegas, Gustavo Vera Cruz

**Universidad:** Universidad Tecnológica de Tecamachalco – Puebla, México

**Resumen:**

El objetivo de este proyecto es diseñar y desarrollar un sistema de localización de mascotas perdidas mediante una aplicación móvil que opera con la plataforma GPS. El sistema permite que el dueño registre su mascota - perro, gato u otro animal - en la aplicación, y asocia un dispositivo de rastreo o vinculación con la mascota. Cuando la mascota se extravía, el sistema usa la ubicación GPS del dispositivo asociado, junto con la aplicación móvil, para determinar la posición geográfica del animal extraviado o del dispositivo que lo acompaña. La arquitectura incorpora sensores de contexto, seguimiento satelital, actualización de posición en tiempo real o diferida, así como una interfaz móvil sencilla para que el usuario visualice el estado, la última ubicación conocida y un historial de movimientos. No se menciona procesamiento de imágenes ni comparación visual entre reportes de mascotas extraviadas y encontradas. El proyecto está más orientado al rastreo y localización geográfica que a la identificación visual.

**Diferencias con mi proyecto:**

- Este sistema no utiliza análisis de imágenes ni redes neuronales para extraer características visuales y compararlas entre perros perdidos y encontrados, mi proyecto sí hace procesamiento de imágenes con OpenCV + EfficientNet-B4 y compara imágenes mediante Cosine Similarity.

- La tecnología principal aquí es geolocalización GPS/sensor de rastreo y app móvil de seguimiento, más que una aplicación de coincidencia visual entre reportes, mi proyecto centra la coincidencia en características visuales de los perros.
- El enfoque funcional está en “localización del dispositivo/mascota” para encontrarla, mientras que mi proyecto busca “coincidencia entre reporte de perro extraviado y perro encontrado” basándose en imágenes.
- Mi proyecto incluye un refugio + base de datos en la nube + notificaciones + reportes y el proyecto de México es más general para rastreo de mascotas con GPS, sin esos elementos específicos.

**Proyecto N° 3**

**Título: “Aplicación móvil integral para la gestión y bienestar de mascotas con muro de extravío y servicios comunitarios”**

**Autor:** Miguel Alejandro Lima Villegas

**Universidad:** Universidad Tecnológica Boliviana – La Paz, Bolivia.

**Resumen:**

Este proyecto de grado propone el desarrollo de una aplicación móvil para la gestión integral de mascotas (perros y gatos) enfocada en su bienestar y tenencia responsable. Las funcionalidades incluyen: carnetización digital de mascotas (vía código QR) para su identificación rápida, un muro interactivo para que los usuarios reporten mascotas extraviadas o encontradas, un mapa geolocalizado de servicios veterinarios y de cuidado de mascotas, y una interfaz para que la comunidad participe. No se usa tecnología de reconocimiento de imagen compleja; el enfoque es la identificación mediante QR y geolocalización comunitaria. Según el resumen, la aplicación está destinada a facilitar el retorno de mascotas perdidas, mejorar la gestión de mascotas y brindar servicios de comunidad en el área de La Paz.

**Diferencias con mi proyecto:**

- Este proyecto usa código QR y geolocalización/registro comunitario en lugar de procesamiento de imágenes con redes neuronales. En mi proyecto, uso OpenCV + EfficientNet-B4 + Cosine Similarity para hacer comparaciones visuales automáticas.

- No está específicamente ligado a un refugio ni al procedimiento de coincidencia automático de imágenes entre reportes de “perdido” y “encontrado”, es más general para gestión de mascotas.
- Mi proyecto incluye base de datos en nube, comparación de características visuales de perros extraviados y encontrados, notificación automática de coincidencias.

# CAPÍTULO III
# INGENIERÍA DEL PROYECTO

# CAPÍTULO III
# INGENIERÍA DEL PROYECTO

## 3.1. INTRODUCCIÓN

El presente capítulo presenta el desarrollo del sistema construido y funcional para la identificación de perros extraviados en el Refugio WOOF. Para lograr este resultado, el proyecto aplicó la metodología ágil XP (Extreme Programming) como marco principal de desarrollo, organizando el trabajo en ciclos cortos que permitieron programar, probar y corregir las funciones de manera constante. Para la etapa de entrenamiento del modelo de inteligencia artificial, se incorporó adicionalmente el marco CRISP-DM, que estructuró ese proceso específico desde la comprensión de los datos hasta el despliegue del modelo.

A lo largo del capítulo, el proyecto documenta cómo se estructuró el sistema integrando una interfaz visual para la parte móvil y web, con un backend que centraliza el procesamiento de inteligencia artificial. El proyecto detalla la implementación de las redes neuronales convolucionales, donde el sistema utilizó el modelo EfficientNet-B4 junto con el algoritmo de similitud de coseno para comparar las fotografías de los perros y determinar coincidencias. Finalmente, el capítulo describe cómo el panel administrativo permite controlar todo lo que ocurre en el sistema, desde los reportes hasta el estado de las cuentas de los usuarios, asegurando que el sistema constituya una herramienta completa y probada.

## 3.2. OBJETIVOS DE LA PROPUESTA

Los objetivos específicos que orientan el desarrollo de la propuesta son los siguientes:

- Diseñar la interfaz de usuario para permitir el registro y la visualización de reportes de mascotas dentro de la aplicación móvil y el panel web administrativo.
- Implementar una base de datos no relacional para mantener sincronizada la información de usuarios y reportes en todos los dispositivos móviles y en el panel web.
- Integrar el modelo de red neuronal EfficientNet-B4 y el cálculo de similitud de coseno dentro del sistema para realizar la comparación numérica de rasgos físicos entre las fotografías capturadas.

- Desarrollar un panel de control administrativo para la supervisión de reportes, gestión de cuentas de usuario y control de bloqueos dentro del sistema web.
- Realizar un plan de pruebas funcionales y no funcionales para verificar que el comportamiento del software coincida con los requisitos técnicos y del usuario.

## 3.3. DESARROLLO DE LA PROPUESTA

El desarrollo del sistema se llevó a cabo aplicando la metodología ágil Extreme Programming (XP), la cual organiza el trabajo en iteraciones cortas con entregas funcionales al finalizar cada ciclo. Beck y Andrés (2004) sostienen que XP permite gestionar proyectos de alta complejidad técnica mediante la adaptabilidad continua, priorizando la entrega de software funcional sobre la documentación. Bajo este enfoque, el desarrollo del sistema se organizó en cinco iteraciones progresivas: el entrenamiento del modelo de inteligencia artificial, la implementación del backend, el desarrollo de la aplicación móvil Android, la construcción del panel web administrativo y finalmente la integración, pruebas generales y despliegue en producción. Las iteraciones 2, 3, 4 y 5 siguieron el ciclo propio de XP compuesto por planificación, diseño, codificación y pruebas. La Iteración 1, correspondiente al entrenamiento del modelo de inteligencia artificial, aplicó el marco CRISP-DM como metodología complementaria, dado que este proceso involucra un ciclo de vida de datos y modelado que no se ajusta a las fases tradicionales de desarrollo de software.

### 3.3.1. *PLANIFICACIÓN DEL DESARROLLO*

Para el desarrollo del proyecto se consideró la siguiente matriz de planificación, la cual establece las fases de la metodología XP para las iteraciones 2, 3, 4 y 5, y las fases de CRISP-DM para la Iteración 1, junto con los entregables esperados al finalizar cada ciclo. Esta matriz constituye la base organizativa del proyecto y permite visualizar de forma global la distribución del trabajo a lo largo de las cinco iteraciones definidas.

Tabla N° 3.1 **Matriz de planificación**

| FASES | ITERACIÓN | ENTREGABLES |
|---|---|---|
| **Planificación** | Iteración 2, 3, 4 y 5 | Historias de Usuario |
| | | Plan de Iteraciones |
| | | Requerimientos Funcionales y No Funcionales |

| FASES | ITERACIÓN | ENTREGABLES |
|---|---|---|
| **Fases CRISP-DM** | Iteración 1 | Modelo EfficientNet-B4 entrenado |
| | | Matriz de Confusión y métricas de evaluación |
| | | Modelo exportado en formato .h5 con metadatos de 122 clases |
| **Diseño** | Iteración 2, 3, 4 y 5 | Diagramas UML |
| | | Mockups de interfaces |
| | | Diseño de base de datos Firestore |
| **Codificación** | Iteración 2, 3 y 4 | Backend FastAPI con endpoints REST |
| | | Aplicación móvil Android |
| | | Panel web administrativo |
| **Pruebas** | Iteración 2, 3, 4 y 5 | Pruebas unitarias por iteración |
| | | Pruebas de integración del sistema completo |
| | | Pruebas no funcionales |
| **Despliegue** | Iteración 5 | Backend desplegado en Railway |
| | | Panel web publicado en Firebase Hosting |
| | | Sistema integrado en producción |

Fuente: Elaboración propia, 2026

### 3.3.2. *ROLES DEL EQUIPO XP*

En la metodología Extreme Programming, cada miembro del equipo asume un rol específico que define sus responsabilidades dentro del proceso de desarrollo. Beck y Andrés (2004) señalan que la claridad en los roles es fundamental para mantener la comunicación fluida y la responsabilidad compartida durante todo el proyecto. Los roles se distribuyeron de la siguiente manera:

Tabla N° 3.2 **Roles del equipo XP**

| ROL | RESPONSABLE | RESPONSABILIDADES |
|---|---|---|
| **Cliente** | Refugio WOOF - Personal administrativo | Definir las necesidades del sistema, validar las historias de usuario y verificar que el producto final cumpla con los requerimientos establecidos. |

| ROL | RESPONSABLE | RESPONSABILIDADES |
|---|---|---|
| **Programador** | Britny Miroslava Sea Herrera | Diseñar, codificar y probar todos los componentes del sistema: modelo de IA, backend FastAPI, aplicación móvil Android y panel web administrativo. |
| **Tester** | Britny Miroslava Sea Herrera | Diseñar y ejecutar los casos de prueba funcionales y no funcionales, verificando que cada módulo cumpla con los criterios de aceptación definidos en las historias de usuario. |
| **Tracker** | Britny Miroslava Sea Herrera | Realizar el seguimiento del avance del proyecto, verificando el cumplimiento del plan de iteraciones y registrando el progreso de cada actividad. |
| **Coach** | Ing. Julio Cesar Irahola Flores | Guiar el proceso de desarrollo, orientar las decisiones técnicas y metodológicas, y supervisar que el proyecto siga los lineamientos de la metodología XP y los estándares académicos de UNIVALLE. |

Fuente: Elaboración propia, 2026

### 3.3.3. *HISTORIAS DE USUARIO*

Las historias de usuario son el artefacto central de la fase de planificación en XP. Según Cohn (2004), cada historia describe de forma concisa lo que un actor del sistema desea realizar y el beneficio que espera obtener. Las siguientes historias de usuario fueron elaboradas a partir de las entrevistas realizadas al personal del Refugio WOOF y reflejan los requisitos identificados en el proceso de levantamiento de información.

Los actores identificados corresponden directamente al personal entrevistado: Carlos Mamani Quispe como rescatista, Verónica Condori Apaza como cuidadora y Any Luz Peñaranda como administradora.

**Cuadro N° 3.1 HU-01 Registro de cuenta en la aplicación móvil**

| HISTORIA DE USUARIO N° HU-01 | |
|---|---|
| **Número:** | HU-01 |
| **Nombre:** | Registro de cuenta en la aplicación móvil |
| **Actor:** | Carlos Mamani Quispe (Rescatista) |
| **Descripción:** | Como rescatista del Refugio WOOF, quiero poder registrarme en la aplicación con mi nombre, correo y una contraseña, para poder entrar al sistema y empezar a registrar perros perdidos o encontrados. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 3 |
| **Criterios de Aceptación:** | |
| • El sistema valida que el nombre no esté vacío.<br>• El sistema valida que el correo tenga formato válido.<br>• La contraseña debe tener mínimo 8 caracteres.<br>• El sistema muestra un mensaje de error específico.<br>• Al registrarse exitosamente, el sistema redirige al usuario a la pantalla principal. | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.2 HU-02 Inicio de sesión en la aplicación móvil**

| HISTORIA DE USUARIO N° HU-02 | |
|---|---|
| **Número:** | HU-02 |
| **Nombre:** | Inicio de sesión en la aplicación móvil |
| **Actor:** | Carlos Mamani Quispe (Rescatista) |
| **Descripción:** | Como rescatista del Refugio WOOF, quiero poder entrar a la aplicación con el correo y la contraseña, para que el sistema recuerde que ya se inició sesión y no haya que volver a escribirlos cada vez que se abre la app. |

| HISTORIA DE USUARIO N° HU-02 | |
|---|---|
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 2 |
| **Criterios de Aceptación:** | |
| • El sistema autentica al usuario mediante Firebase Authentication.<br>• El sistema muestra un mensaje de error claro si las credenciales son incorrectas.<br>• La sesión permanece activa entre aperturas de la aplicación hasta que el usuario cierre sesión manualmente. | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.3 HU-03 Número de teléfono obligatorio**

| HISTORIA DE USUARIO N° HU-03 | |
|---|---|
| **Número:** | HU-03 |
| **Nombre:** | Número de teléfono obligatorio |
| **Actor:** | Carlos Mamani Quispe (Rescatista) |
| **Descripción:** | Como rescatista del Refugio WOOF, quiero que la aplicación pida completar el número de teléfono antes de poder hacer cualquier reporte, para que las personas interesadas puedan comunicarse si encuentran al perro. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 2 |
| **Criterios de Aceptación:** | |
| • Antes de acceder a las pantallas de reporte (perdido o encontrado), el sistema consulta si hay un celular registrado, si el campo está vacío o ausente, el sistema redirige automáticamente a la pantalla de Perfil.<br>• El usuario no puede continuar al formulario de reporte hasta que complete su número de teléfono.<br>• Una vez guardado el teléfono, el sistema permite al usuario continuar con el reporte sin interrupciones. | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.4 HU-04 Registro de perro extraviado**

| HISTORIA DE USUARIO N° HU-04 | |
|---|---|
| **Número:** | HU-04 |
| **Nombre:** | Registro de perro extraviado |
| **Actor:** | Carlos Mamani Quispe (Rescatista) |
| **Descripción:** | Como rescatista del Refugio WOOF, quiero poder registrar un perro perdido subiendo varias fotos y llenando sus datos como el nombre, color, tamaño, para que el sistema lo guarde y pueda compararlo con otros perros que hayan sido encontrados. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 5 |
| **Criterios de Aceptación:** | |
| • El formulario permite seleccionar fotografías desde la galería del dispositivo.<br>• Antes de registrar, el sistema envía las fotografías al backend para verificar que la imagen contenga un perro, si la foto no muestra un perro se muestra un mensaje y no se acepta el envío. | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.5 Búsqueda de coincidencias con perro encontrado**

| HISTORIA DE USUARIO N° HU-05 | |
|---|---|
| **Número:** | HU-05 |
| **Nombre:** | Búsqueda de coincidencias con perro encontrado |
| **Actor:** | Verónica Condori Apaza (Cuidadora) |
| **Descripción:** | Como rescatista del Refugio WOOF quiero subir fotos del perro encontrado, para que el sistema busque automáticamente entre los perros perdidos registrados y muestre cuáles se parecen más. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 5 |

| HISTORIA DE USUARIO N° HU-05 |
|---|
| **Criterios de Aceptación:** |
| • El sistema valida primero que las fotografías contengan un perro.<br>• El sistema compara las fotos de los perros registrados extraviados.<br>• Se retornan hasta 5 resultados con similitud ordenados de mayor a menor. |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.6 HU-06 Visualización de resultados de coincidencia**

| HISTORIA DE USUARIO N° HU-06 | |
|---|---|
| **Número:** | HU-06 |
| **Nombre:** | Visualización de resultados de coincidencia |
| **Actor:** | Verónica Condori Apaza (Cuidadora) |
| **Descripción:** | Como cuidadora del Refugio WOOF, quiero ver una lista de mis reportes con el porcentaje de los perros más parecidos al que reporte y los datos del dueño para contactarlo. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 3 |
| **Criterios de Aceptación:** | |
| • La aplicación muestra la lista ordenada por porcentaje de similitud de mayor a menor.<br>• Cada elemento muestra la fotografía del perro, su nombre, el porcentaje de coincidencia y los datos de contacto (teléfono y correo del dueño). | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.7 HU-07 Visualización de reportes propios**

| HISTORIA DE USUARIO N° HU-07 | |
|---|---|
| **Número:** | HU-07 |
| **Nombre:** | Visualización de reportes propios |
| **Actor:** | Carlos Mamani Quispe (Rescatista) |

| HISTORIA DE USUARIO N° HU-07 | |
|---|---|
| **Descripción:** | Como rescatista del Refugio WOOF, quiero poder ver todos los perros que han sido registrados desde la cuenta, para saber cuáles siguen activos. |
| **Prioridad:** | Media |
| **Puntos Estimados:** | 3 |
| **Criterios de Aceptación:** | |
| • La pantalla de mis reportes muestra solo los reportes asociados al UID del usuario autenticado.<br>• Cada reporte muestra la fotografía, nombre del perro, descripción y fecha de registro.<br>• El sistema muestra un mensaje informativo si el usuario no tiene reportes registrados. | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.8 HU-08 Recepción de notificaciones push de coincidencia**

| HISTORIA DE USUARIO N° HU-08 | |
|---|---|
| **Número:** | HU-08 |
| **Nombre:** | Recepción de notificaciones push de coincidencia |
| **Actor:** | Carlos Mamani Quispe (Rescatista) |
| **Descripción:** | Como rescatista del Refugio WOOF que registró un perro extraviado, quiero recibir una notificación en el celular cuando el sistema encuentre un perro parecido, para poder revisar la coincidencia y contactar a quien lo encontró. |
| **Prioridad:** | Media |
| **Puntos Estimados:** | 3 |
| **Criterios de Aceptación:** | |
| • El título de la notificación es '¡Posible coincidencia encontrada!' y el cuerpo indica el nombre del perro y el porcentaje de similitud.<br>• Las notificaciones son procesadas incluso con la aplicación en segundo plano. | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.9 HU-09 Inicio de sesión en el panel web administrativo**

| HISTORIA DE USUARIO N° HU-09 | |
|---|---|
| **Número:** | HU-09 |
| **Nombre:** | Inicio de sesión en el panel web administrativo |
| **Actor:** | Administrador del Refugio (Any Luz Peñaranda) |
| **Descripción:** | Como administradora del Refugio WOOF, quiero poder ingresar al panel de control de la página web con el correo y contraseña de administradora, para acceder a todas las opciones de gestión del sistema. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 2 |
| **Criterios de Aceptación:** | |
| • El acceso al panel web está restringido a cuentas con rol ADMIN registradas en la base de datos.<br>• El sistema redirige al administrador al dashboard principal tras una autenticación exitosa. | |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.10 HU-10 Gestión de usuarios desde el panel web**

| HISTORIA DE USUARIO N° HU-10 | |
|---|---|
| **Número:** | HU-10 |
| **Nombre:** | Gestión de usuarios desde el panel web |
| **Actor:** | Any Luz Peñaranda (Administradora) |
| **Descripción:** | Como administradora del Refugio WOOF, quiero poder ver quiénes están registrados en el sistema, y si alguien usa la aplicación de manera incorrecta, poder bloquearle el acceso desde la página web. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 4 |

| HISTORIA DE USUARIO N° HU-10 |
|---|
| **Criterios de Aceptación:** |
| • El panel muestra el listado de usuarios con su nombre, correo y estado (activo/bloqueado).<br>• El administrador puede bloquear o volver habilitar cuentas.<br>• El administrador puede buscar usuarios por correo electrónico y filtrar por estado. |

Fuente: Elaboración propia, 2026

**Cuadro N° 3.11 HU-11 Supervisión de reportes desde el panel web**

| HISTORIA DE USUARIO N° HU-11 | |
|---|---|
| **Número:** | HU-11 |
| **Nombre:** | Supervisión de reportes desde el panel web |
| **Actor:** | Any Luz Peñaranda (Administradora) |
| **Descripción:** | Como administradora del Refugio WOOF, quiero poder ver todos los reportes de perros que han sido cargados al sistema, revisar su información y desactivar aquellos que sean incorrectos o inapropiados desde la página web. |
| **Prioridad:** | Alta |
| **Puntos Estimados:** | 4 |
| **Criterios de Aceptación:** | |
| • El panel muestra el listado completo de reportes.<br>• Cada registro muestra la fotografía, nombre del perro, datos del dueño y fecha de registro.<br>• El administrador puede desactivar reportes que incumplan las normas de uso.<br>• El listado se actualiza en tiempo real. | |

Fuente: Elaboración propia, 2026

### 3.3.4. *RESUMEN DE HISTORIAS DE USUARIO*

La siguiente tabla consolida todas las historias de usuario identificadas para el sistema WOOF, indicando su prioridad, puntos estimados, el módulo al que pertenecen y el actor correspondiente.

Tabla N° 3.3 **Resumen de Historias de Usuario**

| N° | HISTORIA DE USUARIO | ACTOR | PRIORIDAD | PTS | MÓDULO |
|---|---|---|---|---|---|
| **HU-01** | Registro de cuenta en la app móvil | Carlos Mamani (Rescatista) | Alta | 3 | Autenticación |
| **HU-02** | Inicio de sesión en la app móvil | Carlos Mamani (Rescatista) | Alta | 2 | Autenticación |
| **HU-03** | Número de teléfono obligatorio | Carlos Mamani (Rescatista) | Alta | 2 | Validación |
| **HU-04** | Registro de perro extraviado | Carlos Mamani (Rescatista) | Alta | 5 | Reportes |
| **HU-05** | Búsqueda de coincidencias (perro encontrado) | Verónica Condori (Cuidadora) | Alta | 5 | Coincidencias |
| **HU-06** | Visualización de resultados de coincidencia | Verónica Condori (Cuidadora) | Alta | 3 | Coincidencias |
| **HU-07** | Visualización de reportes propios | Carlos Mamani (Rescatista) | Media | 3 | Reportes |
| **HU-08** | Notificaciones push de coincidencia | Carlos Mamani (Rescatista) | Media | 3 | Notificaciones |
| **HU-09** | Inicio de sesión en el panel web | Any Luz (Administradora) | Alta | 2 | Panel Web |

| N° | HISTORIA DE USUARIO | ACTOR | PRIORIDAD | PTS | MÓDULO |
|---|---|---|---|---|---|
| **HU-10** | Gestión de usuarios desde panel web | Any Luz (Administradora) | Alta | 4 | Panel Web |
| **HU-11** | Supervisión de reportes desde panel web | Any Luz (Administradora) | Alta | 4 | Panel Web |
| **TOTAL** | | | | **33** | |

Fuente: Elaboración propia, 2026

### 3.3.5. *PLAN DE ITERACIONES*

El plan de iteraciones establece la distribución del trabajo a lo largo del ciclo de desarrollo del proyecto, organizando las actividades en cinco iteraciones progresivas. Según Beck y Andrés (2004), el plan de iteraciones es el mecanismo principal de seguimiento técnico en XP, ya que permite al equipo visualizar qué historias de usuario y actividades se abordarán en cada ciclo, cuánto tiempo tomará cada uno y qué entregable concreto se obtendrá al finalizar. Cada iteración fue planificada considerando las dependencias técnicas entre los componentes del sistema, comenzando por el núcleo de inteligencia artificial y avanzando progresivamente hacia las interfaces de usuario y el despliegue final. La Iteración 1 se desarrolló bajo el marco CRISP-DM, mientras que las iteraciones 2, 3, 4 y 5 siguieron el ciclo XP de planificación, diseño, codificación y pruebas.

a) Iteración 1 - Entrenamiento del modelo de Inteligencia Artificial

Esta iteración se centra en construir el componente principal del sistema: el modelo de reconocimiento visual basado en EfficientNet-B4. A diferencia de las demás iteraciones que siguen el ciclo XP, esta iteración aplica el marco CRISP-DM, estructurando el proceso en seis fases: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado en tres fases de descongelamiento progresivo, evaluación con métricas de rendimiento y despliegue mediante exportación del modelo. Al finalizar se exporta el modelo entrenado junto con sus metadatos para su integración en el backend, alcanzando una exactitud de validación del 93.4%.

b) Iteración 2 - Backend FastAPI

Esta iteración implementa el servidor que expone el modelo de IA como servicio REST. Se configuran los endpoints de validación de fotografía, registro de perro extraviado y búsqueda de coincidencias mediante similitud coseno. Se integran Firebase Admin SDK para la gestión de datos en Firestore y FCM para las notificaciones push, y Cloudinary para el almacenamiento de imágenes.

c) Iteración 3 - Aplicación Móvil Android

Esta iteración desarrolla la interfaz principal del sistema para los usuarios. Se implementan las funcionalidades de autenticación, registro de perros extraviados con validación de foto mediante IA, búsqueda de coincidencias, visualización de resultados y recepción de notificaciones push. La aplicación se construye con Kotlin y Jetpack Compose siguiendo Clean Architecture.

d) Iteración 4 - Panel Web Administrativo

Esta iteración construye la interfaz de gestión exclusiva para la administradora del refugio. Se implementan el inicio de sesión con verificación de rol ADMIN, el dashboard con métricas del sistema, la gestión de usuarios con opciones de bloqueo y habilitación, y la supervisión completa de reportes con posibilidad de eliminación.

e) Iteración 5 - Pruebas de Integración y Despliegue en Producción

Esta iteración final valida el funcionamiento del sistema completo de forma integrada. Se ejecutan pruebas de integración que verifican la comunicación entre todos los componentes, pruebas no funcionales de rendimiento y compatibilidad, y pruebas de precisión del modelo con métricas. Al concluir se realiza el despliegue del backend en Railway y la publicación del panel web en Firebase Hosting.

Tabla N° 3.4 **Plan de Iteraciones**

| ITERACIÓN | ACTIVIDADES | DURACIÓN | ENTREGABLE |
|---|---|---|---|
| **Iteración 1** | Aplicación de las fases CRISP-DM, modelado en 3 fases de descongelamiento progresivo, evaluación con métricas de rendimiento y despliegue mediante exportación del modelo final. | 3 semanas | Modelo EfficientNet-B4 entrenado con 93.4% de exactitud, exportado en formato .h5 con metadatos de 122 clases. |

| ITERACIÓN | ACTIVIDADES | DURACIÓN | ENTREGABLE |
|---|---|---|---|
| **Iteración 2** | Integración del modelo en FastAPI. Configuración de Firebase y Cloudinary. Implementación de endpoints REST. | 2 semanas | Backend FastAPI operativo con endpoints /validate-photo, /register-lost-dog y /match-found-dog. Base de datos Firestore configurada. |
| **Iteración 3** | HU-01 al HU-08. Arquitectura base de la app, autenticación, reportes, coincidencias, mis reportes y notificaciones push. | 4 semanas | Aplicación móvil Android completa y funcional con todos los módulos integrados al backend. |
| **Iteración 4** | HU-09, HU-10, HU-11. Panel web con autenticación por rol, gestión de usuarios y supervisión de reportes. | 3 semanas | Panel web administrativo completo con autenticación por rol y gestión en tiempo real. |
| **Iteración 5** | Pruebas de integración, pruebas no funcionales y despliegue en producción. | 2 semanas | Sistema completo desplegado en producción. Backend en Railway y panel web en Firebase Hosting. |

Fuente: Elaboración propia, 2026

### 3.3.6. *REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES*

#### 3.3.6.1. *Requerimientos Funcionales*

Los requerimientos funcionales describen las capacidades que el sistema debe tener para cumplir las necesidades identificadas durante las entrevistas con el personal del Refugio WOOF.

Tabla N° 3.5 **Requerimientos Funcionales**

| N° | REQUERIMIENTO FUNCIONAL | MÓDULO | PRIORIDAD |
|---|---|---|---|
| **RF-01** | El sistema debe permitir el registro de nuevos usuarios mediante correo electrónico y contraseña a través de Firebase Authentication. | Autenticación | Alta |
| **N°** | **REQUERIMIENTO FUNCIONAL** | **MÓDULO** | **PRIORIDAD** |

| | | | |
|---|---|---|---|
| **RF-02** | El sistema debe permitir el inicio de sesión de usuarios registrados y mantener la sesión activa entre aperturas de la aplicación. | Autenticación | Alta |
| **RF-03** | El sistema debe verificar que el usuario tenga número de teléfono registrado antes de permitirle crear un reporte. | Validación | Alta |
| **RF-04** | El sistema debe permitir registrar un perro extraviado con fotografías, nombre, tamaño, sexo, color, raza y datos del dueño. | Reportes | Alta |
| **RF-05** | El sistema debe validar que las fotografías enviadas corresponda a un perro antes de procesarla, utilizando el modelo EfficientNet-B4. | IA / Backend | Alta |
| **RF-06** | El sistema debe extraer el embedding de las fotografías del perro mediante el modelo EfficientNet-B4 y almacenarlo en Firestore. | IA / Backend | Alta |
| **RF-07** | El sistema debe permitir reportar un perro encontrado y ejecutar el algoritmo Cosine Similarity contra todos los embeddings almacenados. | IA / Backend | Alta |
| **RF-08** | El sistema debe retornar una lista ordenada de los 5 mejores resultados de coincidencia con similitud. | Coincidencias | Alta |
| **RF-09** | El sistema debe enviar una notificación al dueño del perro cuando se detecte una coincidencia que supere el umbral de similitud. | Notificaciones | Media |
| **N°** | **REQUERIMIENTO FUNCIONAL** | **MÓDULO** | **PRIORIDAD** |

| | | | |
|---|---|---|---|
| **RF-10** | El sistema debe permitir al usuario visualizar sus propios reportes de sus perros extraviados registrados. | Reportes | Media |
| **RF-11** | El panel web debe permitir al administrador visualizar y gestionar todas las cuentas de usuario. | Panel Web | Alta |
| **RF-12** | El panel web debe permitir al administrador supervisar y eliminar reportes del sistema. | Panel Web | Alta |
| **RF-13** | El sistema debe permitir al usuario recuperar su contraseña mediante el envío de un correo de restablecimiento a través de Firebase Authentication. | Autenticación | Media |

Fuente: Elaboración propia, 2026

#### 3.3.6.2. *Requerimientos No Funcionales*

Los requerimientos no funcionales establecen las restricciones de seguridad, compatibilidad, disponibilidad, usabilidad y privacidad que el sistema debe cumplir independientemente de los requerimientos funcionales.

Tabla N° 3.6 **Requerimientos No Funcionales**

| N° | CATEGORÍA | REQUERIMIENTO NO FUNCIONAL |
|---|---|---|
| **RNF-01** | Seguridad | Todas las solicitudes al backend deben estar autenticadas mediante Firebase ID Token, rechazando peticiones no autorizadas. |
| **RNF-02** | Compatibilidad | La aplicación móvil debe ser compatible con dispositivos Android versión 8.0 (API 26) o superior. |
| **RNF-03** | Disponibilidad | El sistema debe estar disponible la mayoría del tiempo al estar desplegado en servicios en la nube (Railway, Firebase, Cloudinary) sin servidores físicos locales. |
| **N°** | **CATEGORÍA** | **REQUERIMIENTO NO FUNCIONAL** |

| | | |
|---|---|---|
| **RNF-04** | Usabilidad | La interfaz de la aplicación móvil debe emplear una paleta visual consistente con un fondo oscuro y dorado, que son los colores del refugio. |
| **RNF-05** | Privacidad | Las fotografías de los perros deben almacenarse en Cloudinary y los datos personales de los usuarios solo deben ser visibles para el administrador y las partes involucradas en una coincidencia. |

Fuente: Elaboración propia, 2026

### 3.3.7. *ITERACIÓN 1 - ENTRENAMIENTO DEL MODELO DE INTELIGENCIA ARTIFICIAL*

#### 3.3.7.1. *Fase 1 - Comprensión del Negocio*

En esta primera fase del marco CRISP-DM se definieron los objetivos del modelo desde la perspectiva del problema real que el sistema busca resolver. A diferencia de las demás iteraciones que siguen el ciclo XP, esta iteración aplica CRISP-DM como marco metodológico dado que el proceso de entrenamiento de una red neuronal involucra un ciclo de vida de datos y modelado que no se ajusta a las fases tradicionales de desarrollo de software.

El objetivo principal del modelo es identificar automáticamente si una fotografía contiene un perro y en caso afirmativo, extraer un vector de 512 dimensiones (embedding) que represente sus características visuales para su posterior comparación mediante similitud coseno. Este componente constituye la base tecnológica que hace posible el cumplimiento de las HU-04, HU-05 y HU-06, ya que sin el modelo entrenado no es posible validar fotografías ni calcular coincidencias entre perros reportados.

Los criterios de éxito definidos para esta fase fueron los siguientes: alcanzar una exactitud de validación Top-1 superior al 90%, lograr que el modelo rechace correctamente imágenes que no contengan perros mediante la clase not_a_dog, y exportar el modelo en un formato compatible con el backend FastAPI desplegado en Railway.

La siguiente tabla resume las actividades planificadas para esta iteración:

Tabla N° 3.7 **Actividades de la Iteración 1**

| N° | ACTIVIDAD | DESCRIPCIÓN |
|---|---|---|
| **1** | Configuración del entorno | Configuración del entorno Kaggle con GPU y definición de constantes |
| **2** | Carga y combinación de datasets | Combinación de Stanford Dogs, Perros vs Gatos y Animals10 en un único DataFrame |
| **3** | Aumento de datos | Configuración del generador con transformaciones geométricas y fotométricas |
| **4** | Construcción de la arquitectura | Definición del modelo base EfficientNet-B4 con cabeza clasificadora personalizada |
| **5** | Fase 1 de entrenamiento | Entrenamiento solo de la cabeza clasificadora con base congelada |
| **6** | Fase 2 de entrenamiento | Fine-tuning del 70% superior de la base convolucional |
| **7** | Fase 3 de entrenamiento | Fine-tuning completo con BatchNormalization congelado |
| **8** | Análisis de curvas | Generación y análisis de curvas de exactitud y pérdida |
| **9** | Función de predicción | Implementación de la función de predicción con umbral de confianza |
| **10** | Exportación del modelo | Exportación del modelo en formato .h5, .tflite y metadatos JSON |

Fuente: Elaboración propia, 2026

#### 3.3.7.2. *Fase 2 - Comprensión de los Datos*

En esta fase se analizaron y describieron los datos disponibles para el entrenamiento del modelo. Se combinaron tres fuentes de datos públicas en Kaggle que en conjunto conformaron el dataset final de entrenamiento y validación.

La siguiente tabla describe cada fuente de datos utilizada, su origen, cantidad de imágenes y la etiqueta asignada:

Tabla N° 3.8 **Datasets utilizados para el entrenamiento**

| DATASET | FUENTE | IMÁGENES | ETIQUETA |
|---|---|---|---|
| **Stanford Dogs** | jessicali9530 | 20,580 | 120 razas |
| **Perros vs Gatos (dogs/)** | sergiodelcarpio | ~4,000 | dog_generic |
| **Animals10 (cane)** | alessiocorrado99 | ~4,000 | dog_generic |
| **Animals10 (9 clases)** | alessiocorrado99 | ~22,500 | not_a_dog |
| **Perros vs Gatos (cats/)** | sergiodelcarpio | ~4,000 | not_a_dog |

Fuente: Elaboración propia, 2026

El dataset Stanford Dogs aportó la mayor parte de las imágenes positivas con 20,580 fotografías distribuidas en 120 razas caninas. Los datasets Perros vs Gatos y Animals10 complementaron el conjunto con imágenes adicionales etiquetadas como dog_generic para reforzar el reconocimiento de perros fuera de las razas definidas, y como not_a_dog para entrenar al modelo a rechazar imágenes que no contengan perros. En total el dataset combinado abarcó 122 clases: 120 razas del Stanford Dogs, dog_generic y not_a_dog.

#### 3.3.7.3. *Fase 3 - Preparación de los Datos*

En esta fase se realizaron todas las transformaciones necesarias para convertir los datos crudos en un conjunto listo para el modelado. La preparación incluyó tres procesos principales: la configuración del entorno de entrenamiento, la división del dataset en conjuntos de entrenamiento y validación, y la configuración del generador de aumento de datos.

El entrenamiento se ejecutó en la plataforma Kaggle aprovechando su entorno con aceleración por GPU. Como primer paso se definieron todas las constantes que gobiernan el experimento: el tamaño de entrada de 380×380 píxeles, el tamaño de lote de 32 imágenes, las tres tasas de aprendizaje decrecientes para cada fase, los límites de épocas y el umbral de confianza del 40%.

Figura N° 3.1 **Código de configuración de constantes del entorno**

```
IMG_SIZE    = 380
BATCH_SIZE = 32
SEED        = 42
LR_HEAD     = 1e-3
LR_FINE     = 5e-5
LR_FULL     = 5e-6
EPOCHS_HEAD        = 10
EPOCHS_FINE        = 20
EPOCHS_FULL        = 15
CONFIDENCE_THRESH = 0.40
MODEL_H5    = f'{OUTPUT_DIR}woof_model_v3.h5'
MODEL_LITE = f'{OUTPUT_DIR}woof_model_v3.tflite'
META_FILE   = f'{OUTPUT_DIR}woof_model_v3_metadata.json'
```

Fuente: Elaboración propia, 2026

El dataset combinado fue cargado y combinado en un único DataFrame antes de su división. Los tres datasets fueron unificados asignando a cada imagen su etiqueta correspondiente según su origen.

Figura N° 3.2 **Código de combinación de datasets**

```
df_all = pd.concat([df_dogs, df_neg], ignore_index=True)
NUM_CLASSES = df_all['label'].nunique()

df_train, df_val = train_test_split(
    df_all, test_size=VAL_SPLIT,
    stratify=df_all['label'], random_state=SEED
)
print(f'Train: {len(df_train):,} | Val: {len(df_val):,}')
print(f'Clases totales: {NUM_CLASSES}')
```

Fuente: Elaboración propia, 2026

El dataset combinado se dividió con una proporción 80/20, asignando el 80% de las imágenes al conjunto de entrenamiento y el 20% restante al conjunto de validación. La división se realizó de forma estratificada por clase para garantizar que todas las 122 clases estuvieran representadas proporcionalmente en ambos conjuntos.

Para el conjunto de entrenamiento se configuró un generador con transformaciones geométricas y fotométricas que simulan las condiciones reales de captura de fotografías de perros. Las rotaciones de hasta 20 grados, los desplazamientos horizontales y verticales y el zoom simulan distintos ángulos y distancias de captura. El rango de brillo compensa variaciones de iluminación entre fotografías tomadas en exteriores e interiores. El volteo horizontal duplica la diversidad visual del conjunto sin necesidad de imágenes adicionales. Todas las imágenes fueron redimensionadas a 380×380 píxeles y normalizadas mediante la función preprocess_input de EfficientNet antes de ser procesadas por el modelo.

Figura N° 3.3 **Código de configuración del aumento de datos**

```
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=40,
    horizontal_flip=True,
    width_shift_range=0.25,
    height_shift_range=0.25,
    zoom_range=0.20,
    shear_range=0.15,
    brightness_range=[0.6, 1.4],
    fill_mode='nearest'
)
val_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
```

Fuente: Elaboración propia, 2026

Para el conjunto de validación no se aplicaron transformaciones de aumento, únicamente el redimensionamiento y la normalización, garantizando que las métricas de evaluación reflejen el rendimiento real del modelo ante imágenes sin modificaciones artificiales.

#### 3.3.7.4. *Fase 4 - Modelado*

En esta fase se definió la arquitectura del modelo, se configuraron los hiperparámetros de entrenamiento y se ejecutaron las tres fases de entrenamiento progresivo. Esta es la fase central de CRISP-DM donde se construyó y ajustó el modelo EfficientNet-B4 hasta alcanzar el rendimiento objetivo.

a) Arquitectura del modelo

El modelo se construyó sobre la base convolucional de EfficientNet-B4 con entrada de 380×380 píxeles, añadiendo una cabeza clasificadora compuesta por una capa GlobalAveragePooling2D que reduce el mapa de características a un vector de 512 dimensiones, capas de regularización BatchNormalization y Dropout, y una capa densa final con activación softmax para las 122 clases. El vector de 512 dimensiones generado por la capa densa intermedia constituye el embedding utilizado para el cálculo de similitud coseno en producción.

Tabla N° 3.9 **Estrategia de entrenamiento**

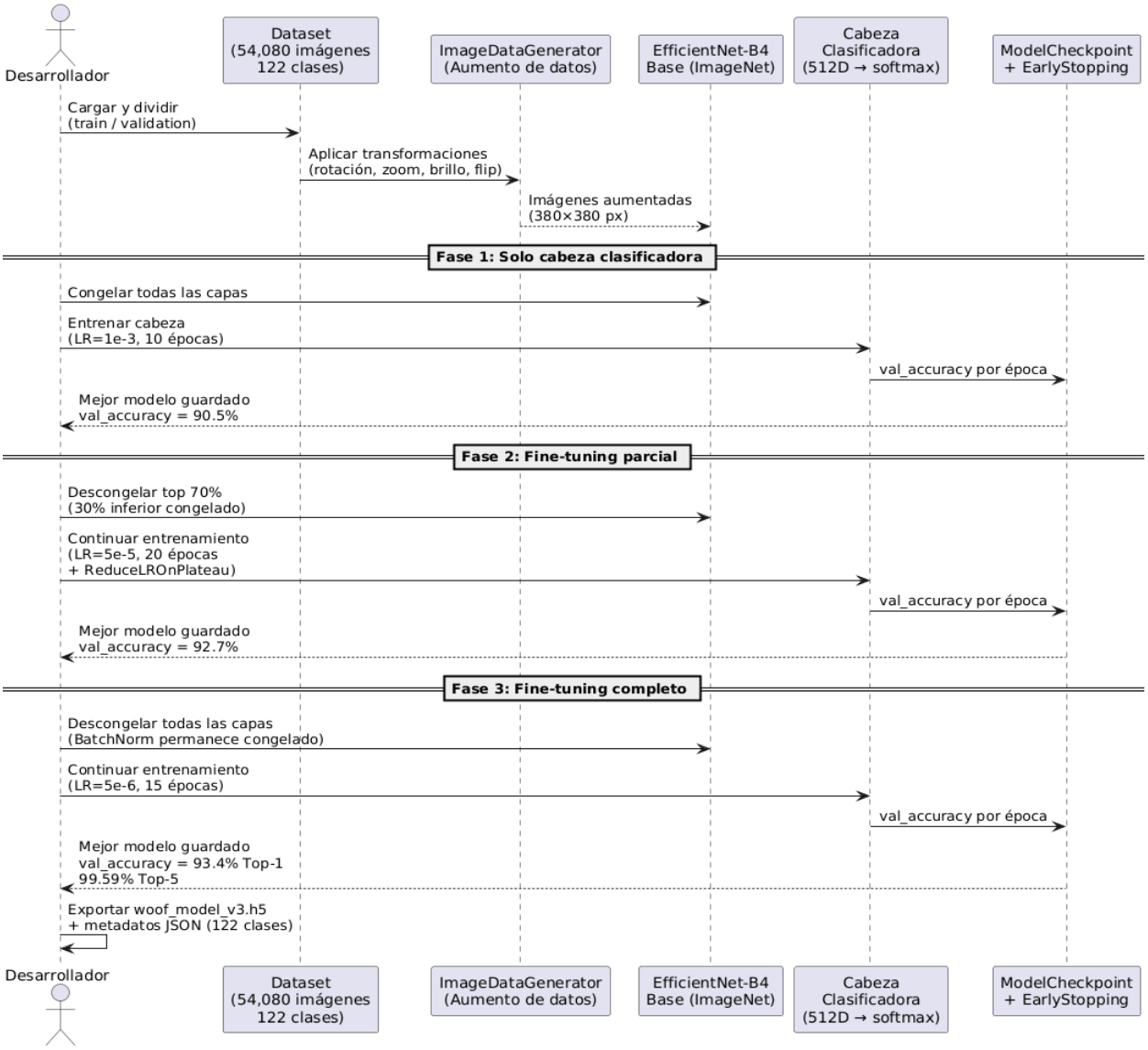
| FASE | CONFIGURACIÓN | LR | ÉPOCAS MÁX |
|---|---|---|---|
| **Fase 1** | Solo cabeza clasificadora, base congelada | 1e-3 | 10 |
| **FASE** | **CONFIGURACIÓN** | **LR** | **ÉPOCAS MÁX** |
| **Fase 2** | Fine-tuning top 70% de la base | 5e-5 | 20 |

| | | | |
|---|---|---|---|
| **Fase 3** | Fine-tuning completo, BatchNorm congelado | 5e-6 | 15 |

Fuente: Elaboración propia, 2026

El siguiente diagrama de secuencia ilustra el flujo técnico del entrenamiento progresivo del modelo, mostrando cómo cada fase de descongelamiento mejora gradualmente la exactitud de validación hasta alcanzar el 93.4%.

Figura N° 3.4 **Diagrama de Secuencia del Entrenamiento EfficientNet-B4**




Fuente: Elaboración propia, 2026

Figura N° 3.5 **Código de construcción de la arquitectura EfficientNet-B4**

```
base_model = EfficientNetB4(
    include_top=False,
    weights='imagenet',
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)
base_model.trainable = False
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)
x = Dense(512, activation='relu')(x)
x = Dropout(0.3)(x)
output = Dense(NUM_CLASSES, activation='softmax', name='predictions')
(x)
model = Model(inputs=base_model.input, outputs=output)
```

Fuente: Elaboración propia, 2026

b) Fase 1 de entrenamiento - cabeza clasificadora

Con la base congelada se entrenaron únicamente los pesos de la cabeza durante un máximo de 10 épocas con tasa de aprendizaje de 0.001. Se usó ModelCheckpoint para guardar el mejor modelo y EarlyStopping para detener el entrenamiento si la métrica dejaba de mejorar. Al finalizar se alcanzó una exactitud de validación del 90.5%.

Figura N° 3.6 **Código de Entrenamiento Fase 1**

```
compile_model(LR_HEAD)
cb_head = [
    ModelCheckpoint(MODEL_H5, monitor='val_accuracy',
                    save_best_only=True, verbose=1),
    EarlyStopping(monitor='val_accuracy', patience=4,
                  restore_best_weights=True, verbose=1)
]
history_head = model.fit(
    train_gen, steps_per_epoch=steps_train,
    epochs=EPOCHS_HEAD,
    validation_data=val_gen, validation_steps=steps_val,
    callbacks=cb_head, verbose=1
```

Fuente: Elaboración propia, 2026

Figura N° 3.7 **Output del entrenamiento Fase 1**

```
Epoch 10: val_accuracy did not improve from 0.90538
1293/1293 ━━━━━━━━━━━━━━━━━━━━ 83s 64ms/step - accuracy: 0.8125 - loss: 1.4523 - top5_acc: 0.9688
- val_accuracy: 0.9049 - val_loss: 1.1321 - val_top5_acc: 0.9959
Restoring model weights from the end of the best epoch: 9.

Fase 1 completada — mejor val_accuracy: 90.5%
```

Fuente: Elaboración propia, 2026

c) Fase 2 de entrenamiento - fine-tuning del 70% superior

Se descongeló el 70% superior de las capas manteniendo congelado el 30% inferior. Se redujo la tasa de aprendizaje a 0.00005 y se agregó ReduceLROnPlateau para ajustes automáticos. Esta fase elevó la exactitud del 90.5% al 92.7%.

Figura N° 3.8 **Código de Entrenamiento Fase 2**

```
freeze_until = int(len(base_model.layers) * 0.30)
for i, layer in enumerate(base_model.layers):
    layer.trainable = (i >= freeze_until)
compile_model(LR_FINE)
cb_fine = [
    ModelCheckpoint(MODEL_H5, monitor='val_accuracy',
                    save_best_only=True, verbose=1),
    EarlyStopping(monitor='val_accuracy', patience=6,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.4,
                      patience=3, min_lr=1e-7, verbose=1)
]
history_fine = model.fit(
    train_gen, steps_per_epoch=steps_train,
    epochs=EPOCHS_FINE,
    validation_data=val_gen, validation_steps=steps_val,
    callbacks=cb_fine, verbose=1
)
```

Fuente: Elaboración propia, 2026

Figura N° 3.9 **Output del entrenamiento Fase 2**

```
Epoch 20: ReduceLROnPlateau reducing learning rate to 7.999999797903002e-06.
1293/1293 ━━━━━━━━━━━━━━━━━━━━ 84s 64ms/step - accuracy: 1.0000 - loss: 0.9456 - top5_acc: 1.0000
- val_accuracy: 0.9243 - val_loss: 1.0660 - val_top5_acc: 0.9949 - learning_rate: 2.0000e-05
Restoring model weights from the end of the best epoch: 17.

Fase 2 completada — mejor val_accuracy: 92.7%
```

Fuente: Elaboración propia, 2026

d) Fase 3 de entrenamiento - fine-tuning completo

Se habilitó el entrenamiento de todas las capas con tasa de aprendizaje de 0.000005, manteniendo BatchNormalization congelado para preservar las estadísticas de normalización. Esta fase llevó la exactitud al 93.4%, superando el objetivo planteado en la Fase 1 de CRISP-DM.

Figura N° 3.10 **Código de Entrenamiento Fase 3**

```python
for layer in model.layers:
    layer.trainable = True
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False
compile_model(LR_FULL)
cb_full = [
    ModelCheckpoint(MODEL_H5, monitor='val_accuracy',
                    save_best_only=True, verbose=1),
    EarlyStopping(monitor='val_accuracy', patience=3,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3,
                      patience=2, min_lr=1e-8, verbose=1)
]
history_full = model.fit(
    train_gen, steps_per_epoch=steps_train,
    epochs=EPOCHS_FULL,
    validation_data=val_gen, validation_steps=steps_val,
    callbacks=cb_full, verbose=1
)
```

Fuente: Elaboración propia, 2026

Figura N° 3.11 **Output del entrenamiento Fase 3**

```
- val_accuracy: 0.9259 - val_loss: 1.0408 - val_top5_acc: 0.9949 - learning_rate: 5.0000e-06
Epoch 4: early stopping
Restoring model weights from the end of the best epoch: 1.

Fase 3 completada — mejor val_accuracy: 93.4%

Resumen fases:
  Fase 1 (cabeza):       90.5%
  Fase 2 (fine 70%):     92.7%
  Fase 3 (full):         93.4%
```

Fuente: Elaboración propia, 2026

e) Función de predicción con umbral de confianza

Se implementó la función predecir que aplica el preprocesamiento correcto y un mecanismo de doble filtro: si la clase predicha es not_a_dog o si la probabilidad máxima no supera el 40%, la imagen es rechazada independientemente del resultado.

Figura N° 3.12 **Código de función de predicción con umbral de confianza**

```python
def predecir(img_pil, threshold=CONFIDENCE_THRESH):
    img_arr  = np.array(img_pil.resize((IMG_SIZE, IMG_SIZE)))
    if img_arr.ndim == 2:
        img_arr = np.stack([img_arr] * 3, axis=-1)
    img_arr  = img_arr[:, :, :3].astype(np.float32)
    img_proc = preprocess_input(img_arr)
    img_proc = np.expand_dims(img_proc, 0)
    probs    = model.predict(img_proc, verbose=0)[0]
    idx      = np.argmax(probs)
    conf     = float(probs[idx])
    name     = idx_to_class[idx]
    if name == 'not_a_dog' or conf < threshold:
        return 'not_a_dog', conf
    return name, conf
```

Fuente: Elaboración propia, 2026

#### 3.3.7.5. *Fase 5 – Evaluación*

En esta fase se evaluó el rendimiento del modelo entrenado utilizando el conjunto de validación completo. La evaluación se estructuró en cinco componentes: la evaluación global del modelo, el análisis de curvas de entrenamiento, la matriz de confusión binaria, el rango de precisión por clase y el coeficiente de determinación.

a) Evaluación global del modelo

Una vez concluido el entrenamiento se ejecutó una evaluación formal sobre el conjunto de validación completo mediante model.evaluate. Los resultados obtenidos fueron una exactitud Top-1 de 93.4%, una exactitud Top-5 de 99.59% y una pérdida final de validación de 1.04, confirmando que el modelo cumplió con los criterios de éxito definidos en la Fase 1.

Figura N° 3.13 **Resultados de la evaluación final del modelo sobre el conjunto de validación**

```
Evaluando modelo...
323/323 ━━━━━━━━━━━━━━━━━━━━ 78s 241ms/step - accuracy: 0.9366 - loss: 1.0133 - top5_acc: 0.9968

==================================================
  RESULTADO FINAL — EfficientNetB4
==================================================
  Top-1 Accuracy: 93.40%
  Top-5 Accuracy: 99.59%
  Loss:           1.0168
==================================================
```

Fuente: Elaboración propia, 2026

b) Curvas de entrenamiento

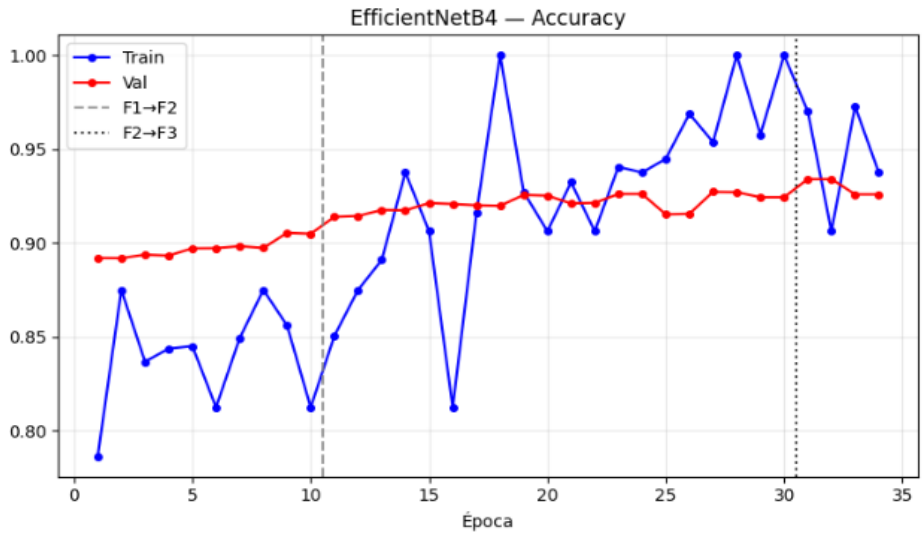
Las curvas de exactitud y pérdida consolidadas de las tres fases permiten visualizar el comportamiento completo del entrenamiento. Las líneas verticales punteadas indican las transiciones entre fases.

Figura N° 3.14 **Código de generación de curvas de entrenamiento**

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
for ax, train_h, val_h, title in zip(
    axes,
    [all_acc, all_loss],
    [all_val_acc, all_val_loss],
    ['Accuracy', 'Loss']
):
    ax.plot(epochs_range, train_h, 'b-o', ms=4, label='Train')
    ax.plot(epochs_range, val_h,   'r-o', ms=4, label='Val')
    ax.axvline(p1_end + 0.5, ls='--', color='gray',  label='F1→F2')
    ax.axvline(p2_end + 0.5, ls=':',  color='black', label='F2→F3')
    ax.set_title(f'EfficientNetB4 — {title}')
    ax.legend()
    ax.grid(True, alpha=0.3)
```

Fuente: Elaboración propia, 2026

**Gráfico N° 3.1 Curva de exactitud - Entrenamiento EfficientNet-B4 en 3 fases**

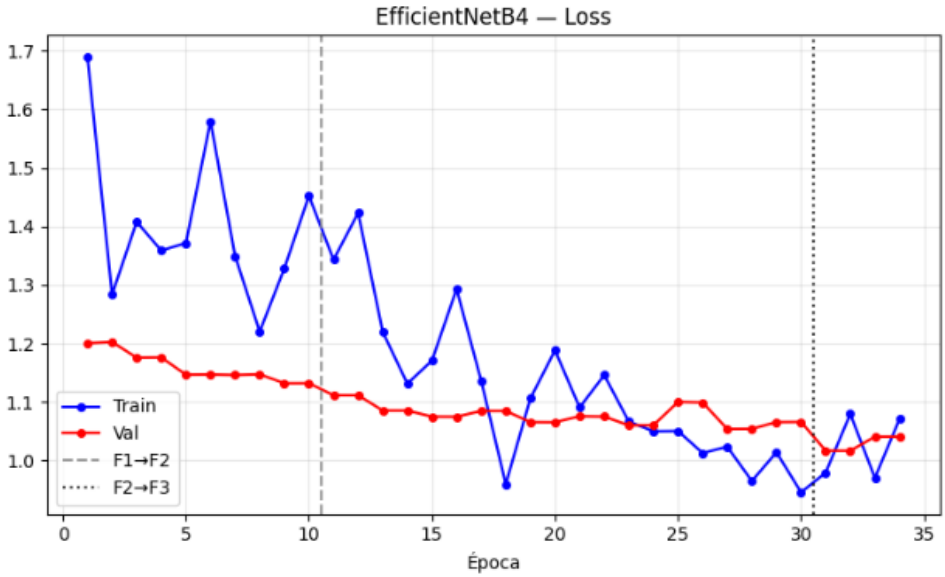



Fuente: Elaboración propia, 2026

La gráfica de exactitud muestra el comportamiento de la curva de entrenamiento (azul) y la curva de validación (roja) a lo largo de las 34 épocas ejecutadas. Durante la Fase 1 (épocas 1 a 10), la curva de validación inicia en 88.9% y asciende de forma estable hasta alcanzar el 90.5%, mientras que la curva de entrenamiento presenta oscilaciones pronunciadas entre 78% y 87%, lo cual es esperado dado que la base convolucional estaba completamente congelada y solo la cabeza clasificadora estaba aprendiendo. Al

iniciar la Fase 2 (épocas 11 a 31), el descongelamiento del 70% superior de la base produce una mejora notable: la curva de validación sube progresivamente del 91% al 92.7%, y la curva de entrenamiento se estabiliza gradualmente acercándose al 100% hacia el final de esta fase. En la Fase 3 (épocas 32 a 34), ambas curvas convergen alrededor del 93%, confirmando que el modelo alcanzó su punto de convergencia. La brecha entre la curva de entrenamiento y la de validación se reduce significativamente en las últimas épocas, lo que indica que el modelo generalizó correctamente sin caer en sobreajuste.

**Gráfico N° 3.2 Curva de pérdida - Entrenamiento EfficientNet-B4 en 3 fases**




Fuente: Elaboración propia, 2026

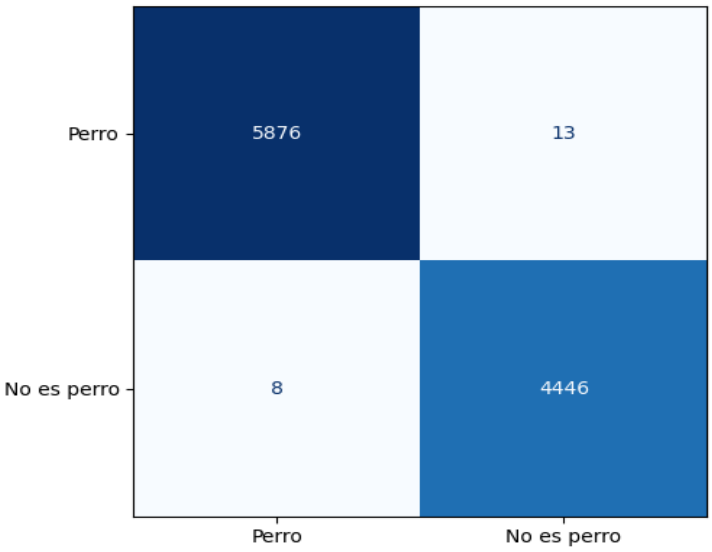
La gráfica de pérdida muestra la evolución de la función de costo a lo largo del entrenamiento. Durante la Fase 1 (épocas 1 a 10), la pérdida de entrenamiento (azul) inicia en valores altos cercanos a 1.70 y desciende con alta variabilidad, oscilando entre 1.20 y 1.60, lo que refleja la inestabilidad inicial de la cabeza clasificadora al adaptarse a las características extraídas por la base congelada. La pérdida de validación (roja) en cambio desciende de forma suave y consistente de 1.20 a 1.13, lo que confirma que el modelo estaba generalizando correctamente desde el inicio. Al comenzar la Fase 2 (épocas 11 a 31), ambas curvas convergen y descienden juntas de manera más coordinada, llegando a valores cercanos a 1.05. La variabilidad de la curva de entrenamiento se reduce considerablemente gracias a la tasa de aprendizaje más baja

de 5e-5 y el callback ReduceLROnPlateau que ajustó automáticamente el ritmo de aprendizaje. Durante la Fase 3 (épocas 32 a 34), las curvas se mantienen estables alrededor de 1.04, sin presentar divergencia entre entrenamiento y validación. La ausencia de sobreajuste a lo largo de todo el proceso se atribuye al uso combinado de Dropout de 0.1 en la función de pérdida y el aumento de datos aplicado al conjunto de entrenamiento.

c) Matriz de Confusión Binaria (Perro / No es Perro)

Para evaluar la capacidad del modelo de distinguir correctamente entre imágenes que contienen un perro y las que no, se generó una matriz de confusión binaria sobre el conjunto de validación completo. Los resultados demuestran un rendimiento sobresaliente del modelo en esta tarea de clasificación fundamental.

**Gráfico N° 3.3 Matriz de Confusión Binaria (Perro / No es Perro)**




Fuente: Elaboración propia, 2026

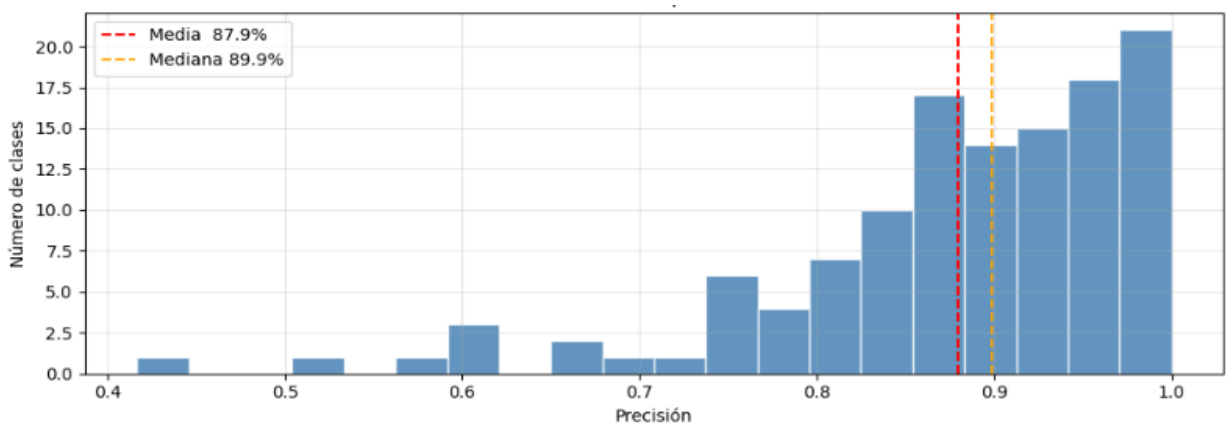
Los resultados obtenidos fueron los siguientes: 5,876 Verdaderos Positivos correspondientes a imágenes de perros correctamente identificadas, 13 Falsos Negativos correspondientes a imágenes de perros clasificadas erróneamente como no perro, 8 Falsos Positivos correspondientes a imágenes de no perros clasificadas erróneamente como perro, y 4,446 Verdaderos Negativos correspondientes a imágenes de no perros correctamente rechazadas. La sensibilidad del modelo para detectar perros alcanzó el 99.78% y la especificidad para rechazar imágenes no caninas el 99.82%, confirmando

que el filtro binario funciona de manera prácticamente correcta antes de ejecutar la comparación por similitud coseno.

d) Rango de precisión por clase

Para el rango de precisión del modelo, se calculó la precisión individual de cada una de las 122 clases sobre el conjunto de validación.

**Gráfico N° 3.4 Histograma de distribución de precisión por clase**




Fuente: Elaboración propia, 2026

Figura N° 3.15 **Resultados de la distribución de precisión por clase**

```
======================================================================
  RANGO DE PRECISIÓN POR CLASE
======================================================================

  TOP 5 — Mayor precisión:
  Clase                                Precisión   Recall       F1
  ------------------------------------------------------------
  border_terrier                         100.0%   97.1%   98.5%
  bedlington_terrier                     100.0%  100.0%  100.0%
  bouvier_des_flandres                   100.0%   80.0%   88.9%
  brabancon_griffon                      100.0%   96.8%   98.4%
  briard                                 100.0%   96.7%   98.3%

  BOTTOM 5 — Menor precisión:
  Clase                                Precisión   Recall       F1
  ------------------------------------------------------------
  toy_poodle                              61.5%   80.0%   69.6%
  yorkshire_terrier                       61.4%   81.8%   70.1%
  border_collie                           59.0%   76.7%   66.7%
  siberian_husky                          52.1%   65.8%   58.1%
  eskimo_dog                              41.7%   33.3%   37.0%

  Precisión MÁXIMA  : 100.00%  →  bedlington_terrier
  Precisión MÍNIMA  : 41.67%  →  eskimo_dog
  Precisión PROMEDIO: 87.94%
  Precisión MEDIANA : 89.87%
======================================================================
```

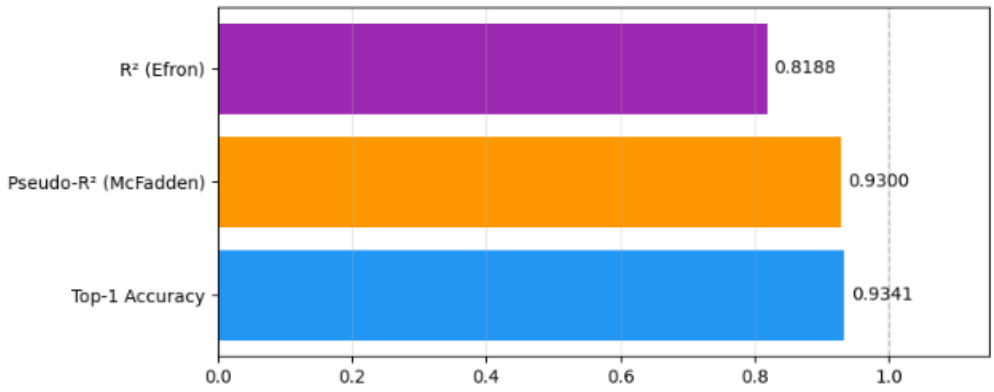
Fuente: Elaboración propia, 2026

Los resultados obtenidos muestran que cinco clases alcanzaron una precisión del 100%: border_terrier, bedlington_terrier, bouvier_des_flandres, brabancon_griffon y briard. La clase con menor precisión fue eskimo_dog con 41.7%, lo cual es consistente con las confusiones con siberian_husky. La precisión promedio del modelo sobre las 122 clases fue de 87.94% y la mediana de 89.87%, indicando que la mayoría de las clases se concentran en valores superiores al 85% como lo confirma el histograma de distribución. El rango de precisión del modelo oscila entre 41.7% y 100%, con más del 70% de las clases superando el 85% de precisión individual.

e) Coeficiente de Determinación ($R^2$)

Para complementar las métricas de exactitud con una evaluación estadística más completa del modelo, se calcularon el Pseudo-$R^2$ de McFadden y el $R^2$ de Efron sobre el conjunto de validación.

**Gráfico N° 3.5 Resumen de métricas - EfficientNet-B4**




Fuente: Elaboración propia, 2026

Figura N° 3.16 **Resultados del coeficiente de determinación**

```
============================================================
  COEFICIENTE DE DETERMINACIÓN (R²)
============================================================
  Pseudo-R² de McFadden : 0.9300  (93.00%)
  R² de Efron (probs)   : 0.8188  (81.88%)

  Interpretación:
    > 0.40  →  Excelente
    0.20-0.40  →  Bueno
    0.10-0.20  →  Aceptable

  Nuestro modelo: EXCELENTE  (R²=0.9300)
============================================================
```

Fuente: Elaboración propia, 2026

El Pseudo-$R^2$ de McFadden obtenido fue de 0.9300 (93%), valor que según Hosmer y Lemeshow (2000) supera el umbral de 0.40 considerado como excelente, indicando que el modelo explica el 93% de la variabilidad respecto a un modelo nulo. El $R^2$ de Efron complementario fue de 0.8188 (81.88%), confirmando una alta correlación entre las probabilidades predichas y los valores reales. Ambos indicadores junto con la exactitud Top-1 de 93.4% y Top-5 de 99.59% demuestran que el modelo EfficientNet-B4 alcanzó un nivel de ajuste estadístico excelente para la tarea de identificación canina.

f) Pruebas visuales

Se ejecutaron pruebas visuales para verificar el comportamiento del modelo ante imágenes reales fuera del conjunto de validación.

Figura N° 3.17 **Prueba de rechazo de imágenes no pertenecientes a perros**




Fuente: Elaboración propia, 2026

Para verificar el funcionamiento del filtro not_a_dog, se ejecutó una prueba con 10 imágenes del conjunto negativo incluyendo gatos, caballos, mariposas y otros animales. El modelo las clasificó correctamente como not_a_dog, rechazando imágenes que no contenían perros antes de cualquier comparación de similitud.

Figura N° 3.18 **Prueba de reconocimiento de perros**




Fuente: Elaboración propia, 2026

Se ejecutó una prueba con 10 imágenes reales de perros para verificar que el modelo identificara correctamente cada fotografía, confirmando el correcto funcionamiento del modelo ante fotografías válidas con sus respectivos porcentajes de confianza.

#### 3.3.7.6. *Fase 6 - Despliegue*

En esta fase final del marco CRISP-DM se exporto los artefactos del modelo entrenado en los formatos requeridos para su integración con el backend FastAPI desplegado en Railway. El despliegue del modelo no implicó su publicación directa como servicio independiente sino su incorporación como componente central del backend desarrollado en la Iteración 2.

Al finalizar el entrenamiento se exportaron tres artefactos para su uso en producción. El archivo .h5 contiene el modelo completo con un tamaño de 213 MB y es cargado directamente por el backend FastAPI en Railway al iniciar el servidor. El archivo .tflite es una versión cuantizada de menor tamaño, aproximadamente 20 MB, orientada a su uso en dispositivos móviles con recursos limitados. El archivo JSON registra toda la configuración del modelo incluyendo el mapeo de las 122 clases, las métricas finales obtenidas y los parámetros de entrenamiento utilizados.

Figura N° 3.19 **Código de exportación del modelo**

```
model.save(MODEL_H5)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()
with open(MODEL_LITE, 'wb') as f:
    f.write(tflite_model)
metadata = {
    'architecture':         'EfficientNetB4',
    'input_size':           IMG_SIZE,
    'num_classes':          NUM_CLASSES,
    'class_names':          idx_to_class,
    'confidence_threshold': CONFIDENCE_THRESH,
    'val_accuracy':         round(float(acc_v3), 4),
    'top5_accuracy':        round(float(top5_v3), 4),
}
with open(META_FILE, 'w') as f:
    json.dump(metadata, f, indent=2)
```

Fuente: Elaboración propia, 2026

Tabla N° 3.10 **Archivos generados en la exportación del entrenamiento del modelo**

| FUNCIONALIDAD | ESTADO | OBSERVACIONES |
|---|---|---|
| Exportación del modelo .h5 | Aprobado | El archivo se generó correctamente con un tamaño de 213 MB. |
| Exportación del modelo .tflite | Aprobado | El archivo cuantizado se generó con un tamaño aproximado de 20 MB. |
| Generación de metadatos JSON | Aprobado | El archivo contiene el mapeo de 122 clases, métricas y configuración completa. |

Fuente: Elaboración propia, 2026

Con la exportación del modelo concluye la aplicación del marco CRISP-DM en la Iteración 1. El archivo woof_model_v1.h5 constituye el puente entre la Iteración 1 y la Iteración 2. Este archivo es cargado en memoria por el backend FastAPI al iniciar el servidor en Railway mediante la función initialize_model() en model_service.py. A partir de ese momento, EfficientNet-B4 opera en producción como servicio de inferencia: cada vez que un usuario sube una fotografía desde la aplicación móvil, el modelo procesa la imagen en 380×380 píxeles y produce dos salidas: la clasificación binaria de perro / not_a_dog y el vector de 512 dimensiones utilizado para el cálculo de similitud coseno en el endpoint /match-found-dog.

El modelo EfficientNet-B4 entrenado quedó listo para su integración en el sistema, constituyendo la base tecnológica sobre la cual se construyeron las iteraciones siguientes bajo el ciclo XP.

### 3.3.8. *ITERACIÓN 2 - BACKEND FASTAPI*

#### 3.3.8.1. *Planeación*

En esta iteración se implementó el servidor backend que expone el modelo de inteligencia artificial como servicio REST. El backend actúa como intermediario entre la aplicación móvil Android y el panel web administrativo, gestionando la validación de fotografías con el modelo, el almacenamiento de datos en Firestore, el manejo de imágenes en Cloudinary y el envío de notificaciones push a través de Firebase Cloud Messaging.

Tabla N° 3.11 **Actividades de la Iteración 2**

| N° | ACTIVIDAD | DESCRIPCIÓN |
|---|---|---|
| **1** | Dependencias y configuración | Instalación de librerías y variables de entorno |
| **2** | Inicialización y ciclo de vida | Carga del modelo y Firebase al arrancar la aplicación |
| **3** | Autenticación Firebase Token | Middleware de verificación de JWT en cada petición |
| **4** | Servicio del modelo - preprocesamiento | Pipeline de mejora y preparación de imagen |
| **5** | Servicio del modelo | Función de análisis completo que retorna embedding |
| **6** | Servicio Firebase y Cloudinary | Gestión de base de datos e imágenes en la nube |
| **7** | Endpoint validate-photo | Validación rápida de fotografía |
| **8** | Endpoint register-lost-dog | Registro de perro extraviado con 1 a 3 fotos |
| **9** | Endpoint match-found-dog | Búsqueda de coincidencias con similitud coseno |
| **10** | Endpoint health y rate limiting | Verificación de salud del servidor y control de solicitudes |

Fuente: Elaboración propia, 2026

### 3.3.8.2. *Diseño*

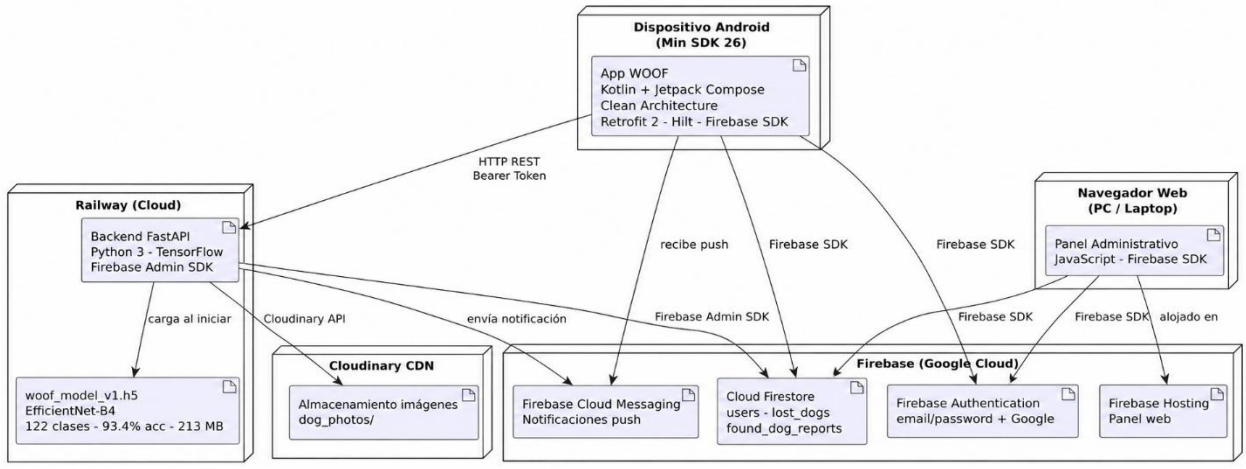
El backend fue diseñado siguiendo una arquitectura modular con separación de responsabilidades. Los componentes principales son: la capa de configuración que gestiona variables de entorno, la capa de servicios que encapsula la lógica del modelo y Firebase, la capa de rutas que define los endpoints REST, y la capa de esquemas que valida entrada y salida.

a) Diagrama de Despliegue

El diagrama de despliegue muestra la distribución física de los componentes del sistema en sus nodos de ejecución. El dispositivo Android ejecuta la aplicación móvil y se comunica con el backend mediante HTTP REST. El navegador web del administrador accede al panel desplegado en Firebase Hosting. El servidor Railway aloja el backend FastAPI junto con el modelo EfficientNet-B4. La plataforma Firebase centraliza la autenticación, la base de datos Firestore, las notificaciones y el alojamiento del panel web. Cloudinary almacena y distribuye las fotografías mediante URLs públicas.

La siguiente imagen muestra la arquitectura de despliegue del sistema:

Figura N° 3.20 **Diagrama de Despliegue**

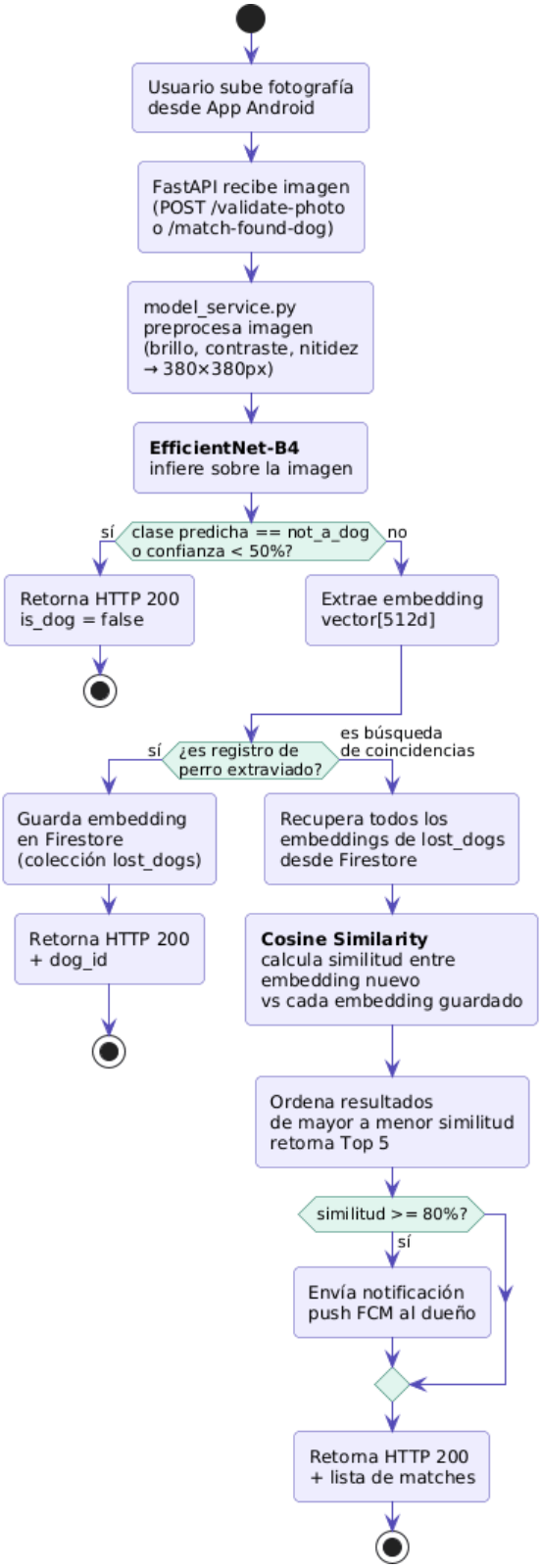



Fuente: Elaboración propia, 2026

b) Diagrama de flujo del procesamiento de la IA

El siguiente diagrama ilustra el flujo completo de procesamiento de una fotografía desde que el usuario la captura en la aplicación móvil hasta que el sistema retorna las coincidencias, mostrando exactamente en qué punto interviene EfficientNet-B4 y en qué punto se aplica el algoritmo Cosine Similarity.

Figura N° 3.21 **Diagrama de flujo completo EfficientNet-B4 + Cosine Similarity**




Fuente: Elaboración propia, 2026

c) Diseño de la base de datos NoSQL

La base de datos no relacional Firestore almacena la información del sistema en tres colecciones principales: users, lost_dogs y found_dog_reports. Cada colección contiene documentos identificados por un ID único y campos asociados a la información de usuarios, perros extraviados y reportes de perros encontrados.

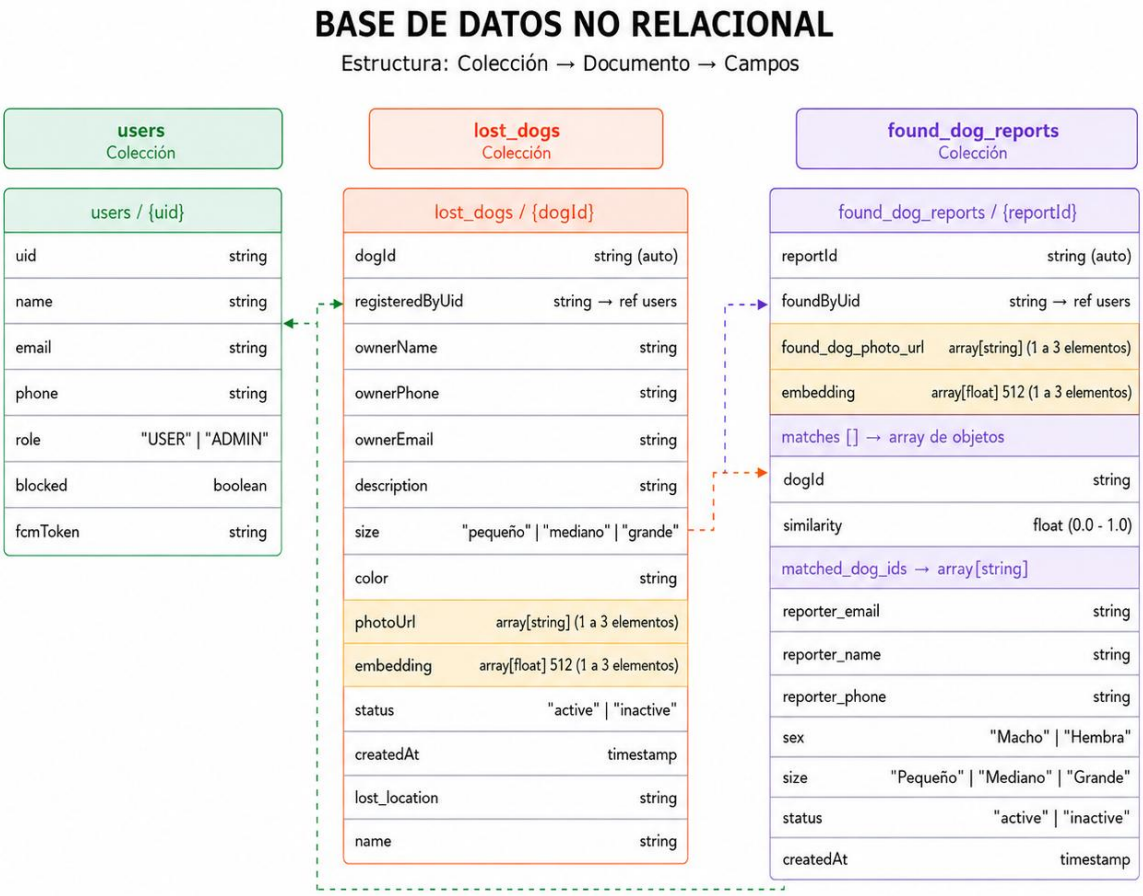
Tabla N° 3.12 **Colecciones y campos principales de la base de datos**

| COLECCIÓN | DOCUMENTO | CAMPOS PRINCIPALES |
|---|---|---|
| **users** | {uid} | uid, name, email, phone, role, blocked, fcmToken |
| **lost_dogs** | {dogId} | dogId, registeredByUid, ownerName, ownerPhone, ownerEmail, description, size, color, photoUrl, embedding, status, createdAt, lost_location, name |
| **found_dog_reports** | {reportId} | reportId, foundByUid, found_dog_photo_url, embedding, matches, matched_dog_ids, reporter_email, reporter_name, reporter_phone, sex, size, status, createdAt |

Fuente: Elaboración propia, 2026

Los campos photoUrl, found_dog_photo_url y embedding pueden almacenar de 1 a 3 elementos, dependiendo de la cantidad de imágenes subidas por el usuario. Cada embedding corresponde a un vector de características de 512 dimensiones generado a partir de la imagen del perro.

Figura N° 3.22 **Diseño de la base de datos no relacional**




Fuente: Elaboración propia, 2026

Tabla N° 3.13 **Diccionario de datos - Colección users**

| CAMPO | TIPO DE DATO | DESCRIPCIÓN |
|---|---|---|
| **uid** | string | Identificador único del usuario. |
| **name** | string | Nombre completo del usuario. |
| **email** | string | Correo electrónico del usuario. |
| **phone** | string | Número telefónico del usuario. |
| **role** | string | Rol del usuario (USER o ADMIN). |
| **blocked** | boolean | Indica si el usuario está bloqueado. |

| CAMPO | TIPO DE DATO | DESCRIPCIÓN |
|---|---|---|
| **fcmToken** | string | Token utilizado para notificaciones push mediante Firebase Cloud Messaging. |

Fuente: Elaboración propia, 2026

Tabla N° 3.14 **Diccionario de datos - Colección lost_dogs**

| CAMPO | TIPO DE DATO | DESCRIPCIÓN |
|---|---|---|
| **dogId** | string | Identificador único del perro extraviado. |
| **registeredByUid** | string | UID del usuario que registró el perro. |
| **ownerName** | string | Nombre del propietario del perro. |
| **ownerPhone** | string | Número telefónico del propietario. |
| **ownerEmail** | string | Correo electrónico del propietario. |
| **description** | string | Descripción física o características del perro. |
| **size** | string | Tamaño del perro (pequeño, mediano o grande). |
| **color** | string | Color predominante del perro. |
| **photoUrl** | array[string] | URLs de 1 a 3 imágenes almacenadas en Cloudinary. |
| **embedding** | array[float] | Vector numérico de 512 dimensiones generado para cada imagen. |
| **status** | string | Estado del registro (active o inactive). |
| **createdAt** | timestamp | Fecha y hora de creación del registro. |
| **lost_location** | string | Lugar donde se perdió el perro. |
| **name** | string | Nombre del perro. |

Fuente: Elaboración propia, 2026

Tabla N° 3.15 **Diccionario de datos - Colección found_dog_reports**

| CAMPO | TIPO DE DATO | DESCRIPCIÓN |
|---|---|---|
| **reportId** | string | Identificador único del reporte. |
| **foundByUid** | string | UID del usuario que reportó el perro encontrado. |
| **found_dog_photo_url** | array[string] | URLs de 1 a 3 fotografías del perro encontrado. |
| **embedding** | array[float] | Vector numérico de 512 dimensiones generado para cada imagen. |
| **matches** | array[object] | Lista de coincidencias encontradas con perros registrados. |
| **matched_dog_ids** | array[string] | IDs de perros relacionados como posibles coincidencias. |
| **reporter_email** | string | Correo electrónico del reportante. |
| **reporter_name** | string | Nombre del reportante. |
| **reporter_phone** | string | Número telefónico del reportante. |
| **sex** | string | Sexo del perro (Macho o Hembra). |
| **size** | string | Tamaño del perro (Pequeño, Mediano o Grande). |
| **status** | string | Estado del reporte (active o inactive). |
| **createdAt** | timestamp | Fecha y hora de creación del reporte. |

Fuente: Elaboración propia, 2026

Tabla N° 3.16 **Endpoints de la API**

| MÉTODO | ENDPOINT | AUTENTICACIÓN | DESCRIPCIÓN |
|---|---|---|---|
| **POST** | /api/v1/validate-photo | JWT | Valida si una foto muestra un perro. |
| **POST** | /api/v1/validate-lost-photo | JWT | Valida foto de perro perdido y detecta duplicados. |

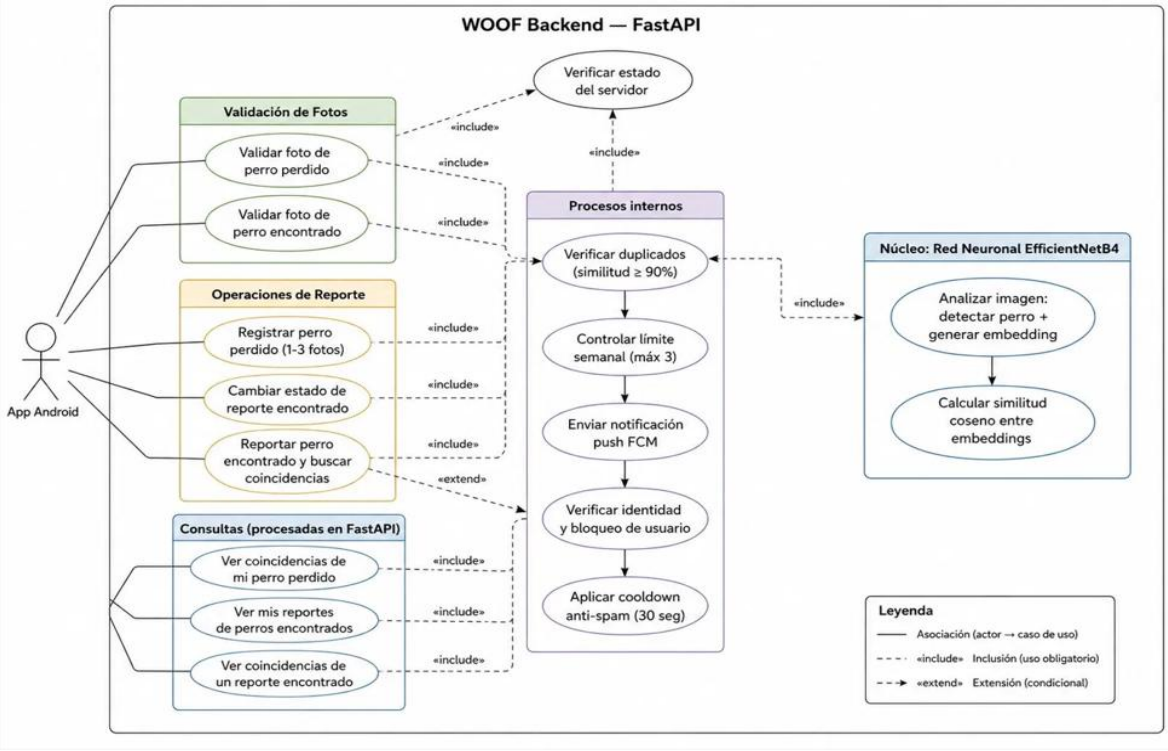
| MÉTODO | ENDPOINT | AUTENTICACIÓN | DESCRIPCIÓN |
|---|---|---|---|
| **POST** | /api/v1/validate-found-photo | JWT | Valida foto de perro encontrado y detecta duplicados. |
| **POST** | /api/v1/register-lost-dog | JWT | Registra perro extraviado con 1 a 3 fotos. |
| **POST** | /api/v1/match-found-dog | JWT | Busca coincidencias con perros extraviados. |
| **GET** | /api/v1/my-dog-matches | JWT | Obtiene coincidencias para perros del usuario. |
| **GET** | /api/v1/my-found-reports | JWT | Obtiene reportes de perros encontrados del usuario. |
| **PATCH** | /api/v1/found-report-status | JWT | Activa o desactiva un reporte encontrado. |
| **GET** | /health | Público | Verifica el estado del servidor. |

Fuente: Elaboración propia, 2026

d) Diagrama de Casos de Uso

El diagrama de casos de uso del backend representa las interacciones entre la aplicación móvil Android y los servicios expuestos por FastAPI, organizados en tres grupos funcionales: validación de fotos, operaciones de reporte y consultas. Los procesos internos como la verificación de duplicados, el control de límite semanal y el envío de notificaciones FCM se modelan como casos de uso incluidos obligatoriamente, mientras que el núcleo de inteligencia artificial EfficientNet-B4 se representa como un componente separado que incluye el análisis de imagen y el cálculo de similitud coseno entre embeddings.

Figura N° 3.23 **Diagrama de Casos de Uso del Backend FastAPI**




Fuente: Elaboración propia, 2026

### 3.3.8.3. *Codificación*

Se realizó todas las actividades definidas en la fase de planeación. A continuación, se presenta el código correspondiente a cada actividad.

a) Actividad 1 - Dependencias y configuración

El proyecto FastAPI requiere un conjunto de librerías especializadas para cada responsabilidad. Las dependencias de backend incluyen FastAPI como framework web, TensorFlow para la inferencia del modelo, Firebase Admin SDK para acceso a Firestore y FCM, Cloudinary para almacenamiento de imágenes.

Figura N° 3.24 **Dependencias del proyecto**

```
fastapi>=0.115.0
uvicorn[standard]>=0.30.0
python-multipart>=0.0.9
tensorflow==2.17.1
Pillow>=10.4.0
numpy>=1.26.0,<2.0.0
scikit-learn>=1.5.0
firebase-admin>=6.5.0
pydantic-settings>=2.3.0
python-dotenv>=1.0.1
cloudinary>=1.40.0
slowapi==0.1.9
```

Fuente: Elaboración propia, 2026

La configuración del servidor se centraliza en config.py mediante una clase Settings que carga automáticamente las variables de entorno, donde la configuración se instancie una sola vez durante toda la vida del proceso. Se agregó soporte para cargar el archivo de credenciales de Firebase directamente desde una variable de entorno en formato JSON, lo que es fundamental para Railway donde no hay acceso a sistemas de archivos persistentes.

Figura N° 3.25 **Código de configuración centralizada del servidor**

```
class Settings(BaseSettings):
    firebase_credentials_path: str = "firebase_credentials.json"
    firebase_credentials_json: str = ""    # JSON completo como string
(Railway)
    model_path: str = "ml/woof_model_v1.h5"
    model_download_url: str = ""           # URL pública del modelo en
Firebase Storage
    cloudinary_cloud_name: str = ""
    cloudinary_api_key: str = ""
    cloudinary_api_secret: str = ""
    allowed_origins: str = ""              # URLs separadas por coma
(Railway/Firebase Hosting)

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"


@lru_cache()
def get_settings() -> Settings:
    return Settings()
```

Fuente: Elaboración propia, 2026

b) Actividad 2 - Inicialización y ciclo de vida

El punto de entrada del servidor es main.py, donde se configura la aplicación FastAPI con su ciclo de vida mediante el gestor de contexto. Este patrón garantiza que Firebase y el modelo de IA se inicialicen una sola vez al arrancar el servidor y permanezcan en memoria durante toda su ejecución. El middleware CORS permite que la aplicación móvil Android y el panel web accedan al servidor desde cualquier origen configurado dinámicamente según variables de entorno.

Figura N° 3.26 **Código de inicialización de la aplicación FastAPI con ciclo de vida**



```
@asynccontextmanager
async def lifespan(app: FastAPI):
    firebase_service.initialize_firebase()
    model_service.load_model()
    yield
app = FastAPI(
    title="WOOF API",
    description="Backend para matching de perros perdidos usando
EfficientNetB4.",
    version="1.0.0",
    lifespan=lifespan,
)

def _build_origins() -> list[str]:
    extra = get_settings().allowed_origins
    if not extra:
        return _BASE_ORIGINS
    return _BASE_ORIGINS + [o.strip() for o in extra.split(",") if
o.strip()]

app.add_middleware(
    CORSMiddleware,
    allow_origins=_build_origins(),
    allow_credentials=False,
    allow_methods=["GET", "POST", "PATCH"],
    allow_headers=["Authorization", "Content-Type"],
)
app.include_router(health.router)
app.include_router(dogs.router, prefix="/api/v1")
```

Fuente: Elaboración propia, 2026

c) Actividad 3 - Autenticación con Firebase Token

Todos los endpoints del sistema requieren autenticación mediante un token JWT emitido por Firebase Authentication. Se implementó una dependencia reutilizable verify_firebase_token que extrae el token del encabezado Authorization, lo verifica contra Firebase y retorna el payload decodificado con el uid del usuario. Si el token es inválido o ha expirado, el servidor responde automáticamente con un error 401.

Figura N° 3.27 **Código de Verificación de token Firebase JWT**

```python
async def verify_firebase_token(
    credentials: HTTPAuthorizationCredentials = Depends(security),
) -> dict:
    token = credentials.credentials
    try:
        decoded_token = firebase_admin.auth.verify_id_token(token)
        return decoded_token
    except firebase_admin.auth.ExpiredIdTokenError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token expirado. Inicia sesión nuevamente.",
        )
    except firebase_admin.auth.InvalidIdTokenError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token inválido.",
        )
```

Fuente: Elaboración propia, 2026

d) Actividad 4 - Servicio del modelo y extracción de embedding

La función analyze_image preprocesa la imagen una sola vez y la pasa a ambos modelos para optimizar rendimiento. Si la clase predicha es not_a_dog o si la confianza no supera el 50%, la imagen es rechazada. En caso contrario retorna el embedding de 512 dimensiones para posteriores cálculos de similitud coseno.

Figura N° 3.28 **Código de análisis de imagen con extracción de embedding**

```python
def analyze_image(image_bytes: bytes) -> tuple[bool, float, list[float]]:
    """
    Preprocesa la imagen UNA sola vez y la pasa a ambos modelos,
    evitando procesar los bytes dos veces.

    Returns:
        dog_detected (bool), confidence (float), embedding (list[float])
    """
    if _full_model is None or _embedding_model is None:
        raise RuntimeError("Model is not loaded. Call load_model() first.")

    try:
        preprocessed = _preprocess_image(image_bytes)
    except Exception:
        raise ValueError("No se pudo procesar la imagen. Verifica que el archivo no esté dañado.")

    # Clasificación
    predictions  = _full_model.predict(preprocessed, verbose=0)
    probs        = predictions[0]
    max_class    = int(np.argmax(probs))
    confidence   = float(probs[max_class])
    dog_detected = (max_class != NOT_A_DOG_INDEX) and (confidence >= CONFIDENCE_THRESHOLD)

    # Embedding (reutiliza el mismo tensor ya preprocesado)
    embedding = _embedding_model.predict(preprocessed, verbose=0)

    return dog_detected, confidence, embedding[0].tolist()
```

Fuente: Elaboración propia, 2026

e) Actividad 5 - Servicio del modelo: preprocesamiento

El archivo model_service.py carga el modelo en memoria. Antes de pasar la imagen al modelo se aplica un pipeline de preprocesamiento que mejora la calidad de la fotografía donde se ajusta brillo según iluminación detectada, aumenta contraste y aplica nitidez.

Figura N° 3.29 **Código de preprocesamiento de imagen**

```python
def load_model() -> None:
    global _full_model, _embedding_model
    settings = get_settings()
    model_path = settings.model_path
    _download_model_if_needed(model_path, settings.model_download_url)
    _full_model = tf.keras.models.load_model(model_path, compile=False)
    _embedding_model = tf.keras.Model(
        inputs=_full_model.input,
        outputs=_full_model.layers[-2].output,
    )


def _enhance_image(image: Image.Image) -> Image.Image:
    avg_brightness = np.array(image).mean()
    if avg_brightness < 100:
        image = ImageEnhance.Brightness(image).enhance(1.4)
    elif avg_brightness < 140:
        image = ImageEnhance.Brightness(image).enhance(1.15)
    image = ImageEnhance.Contrast(image).enhance(1.3)
    image = ImageEnhance.Sharpness(image).enhance(2.0)
    image = image.filter(ImageFilter.UnsharpMask(radius=1, percent=120, threshold=3))
    return image


def _preprocess_image(image_bytes: bytes) -> np.ndarray:
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    image = _enhance_image(image)

    w, h = image.size
    scale = min(IMAGE_SIZE[0] / w, IMAGE_SIZE[1] / h)
    new_w, new_h = int(w * scale), int(h * scale)
    image = image.resize((new_w, new_h), Image.LANCZOS)

    canvas = Image.new("RGB", IMAGE_SIZE, (0, 0, 0))
    offset_x = (IMAGE_SIZE[0] - new_w) // 2
    offset_y = (IMAGE_SIZE[1] - new_h) // 2
    canvas.paste(image, (offset_x, offset_y))

    array = np.array(canvas, dtype=np.float32)
    array = tf.keras.applications.efficientnet.preprocess_input(array)
    return np.expand_dims(array, axis=0)
```

Fuente: Elaboración propia, 2026

f) Actividad 6 - Servicio Firebase y Cloudinary

El archivo firebase_service.py centraliza toda la comunicación con servicios externos. La función initialize_firebase establece la conexión con Firestore y configura Cloudinary. La

función _download_model_if_needed implementa descarga automática desde una URL pública en caso de que el modelo no esté en el sistema de archivos local.

Figura N° 3.30 **Código de inicialización de Firebase y Cloudinary**

```
def initialize_firebase() -> None:
    global _db
    settings = get_settings()

    if not firebase_admin._apps:
        if settings.firebase_credentials_json:
            # Railway: credenciales desde variable de entorno
            cred = credentials.Certificate(json.loads(settings.firebase_credentials_json))
        else:
            # Local: credenciales desde archivo
            cred = credentials.Certificate(settings.firebase_credentials_path)
        firebase_admin.initialize_app(cred)

    _db = firestore.client()
    cloudinary.config(
        cloud_name=settings.cloudinary_cloud_name,
        api_key=settings.cloudinary_api_key,
        api_secret=settings.cloudinary_api_secret,
    )
def upload_photo(image_bytes: bytes, filename: str, folder: str = "dog_photos") ->
tuple[str, str]:
    """Retorna (secure_url, public_id) para posterior limpieza si es necesario."""
    public_id = filename.replace(".jpg", "")
    result = cloudinary.uploader.upload(
        image_bytes,
        public_id=f"{folder}/{public_id}",
        resource_type="image",
    )
    return result["secure_url"], f"{folder}/{public_id}"
def get_active_lost_dogs() -> list[dict]:
    docs = _db.collection("lost_dogs").where("status", "==", "active").stream()
    results = []
    for doc in docs:
        data = doc.to_dict()
        data["doc_id"] = doc.id
        results.append(data)
    return results
def save_lost_dog(dog_data: dict) -> str:
    doc_ref = _db.collection("lost_dogs").document()
    dog_data["status"] = "active"
    dog_data["created_at"] = firestore.SERVER_TIMESTAMP
    doc_ref.set(dog_data)
    return doc_ref.id
```

Fuente: Elaboración propia, 2026

g) Actividad 7 - Endpoint validate-photo

El endpoint POST /validate-photo recibe una fotografía y determina rápidamente si contiene un perro sin guardar datos. Se utiliza rate limiting de 60 peticiones por minuto para proteger contra abuso.

Figura N° 3.31 **Código de Endpoint de validación de fotografía**

```python
router = APIRouter()
ALLOWED_MIME_TYPES = {"image/jpeg", "image/png", "image/webp"}


@router.post("/validate-photo", response_model=ValidatePhotoResponse,
             summary="Validar si la foto muestra un perro")
@limiter.limit("60/minute")
async def validate_photo(
    request: Request,
    current_user: Annotated[dict, Depends(verify_firebase_token)],
    photo: UploadFile = File(...),
):
    if photo.content_type not in ALLOWED_MIME_TYPES:
        raise HTTPException(
            status_code=status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
            detail=f"Tipo de archivo no permitido ({photo.content_type}). Solo se aceptan
JPEG, PNG o WebP.",
        )
    image_bytes = await photo.read()
    try:
        dog_detected, confidence, _ = await asyncio.to_thread(
            model_service.analyze_image, image_bytes
        )
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY, detail=str(e)
        )
    if dog_detected:
        message = "Foto válida."
    elif confidence < 0.15:
        message = "La imagen tiene muy baja calidad o está borrosa. Intenta con una foto
más clara."
    else:
        message = "La foto no muestra un perro. Por favor seleccione otra foto."

    return ValidatePhotoResponse(
        is_dog=dog_detected, confidence=round(confidence, 4), message=message
    )
```

Fuente: Elaboración propia, 2026

h) Actividad 8 - Endpoint register-lost-dog

El endpoint POST /register-lost-dog implementa el flujo completo de registro: valida que cada foto contenga un perro, verifica duplicados mediante similitud coseno con umbral del 90% contra todos los perros existentes, sube las 1 a 3 fotos a Cloudinary en paralelo, extrae embeddings de cada una, y guarda el documento completo en Firestore. Se implementó límite semanal de reportes para usuarios no administradores y validación de campos.

Figura N° 3.32 **Código de validación de límite semanal**



```
MAX_PHOTOS = 3
WEEKLY_REPORT_LIMIT = 3
DUPLICATE_THRESHOLD = 90.0
@router.post("/register-lost-dog", response_model=RegisterLostDogResponse,
             summary="Registrar perro perdido con 1 a 3 fotos")
@limiter.limit("20/minute")
async def register_lost_dog(
    request: Request,
    current_user: Annotated[dict, Depends(verify_firebase_token)],
    dog_name: str = Form(...),
    owner_name: str = Form(...),
    owner_phone: str = Form(...),
    owner_email: str = Form(...),
    photos: List[UploadFile] = File(...),
    size: Optional[str] = Form(None),
    color: Optional[str] = Form(None),
    breed: Optional[str] = Form(None),
    sex: Optional[str] = Form(None),
):
    uid = current_user.get("uid")

    # Validación de límite semanal para no administradores
    role = await asyncio.to_thread(firebase_service.get_user_role, uid)
    if role != "ADMIN":
        allowed = await asyncio.to_thread(
            firebase_service.atomic_weekly_limit_check, uid, WEEKLY_REPORT_LIMIT
        )
        if not allowed:
            raise HTTPException(
                status_code=status.HTTP_429_TOO_MANY_REQUESTS,
                detail=f"Has alcanzado el límite de {WEEKLY_REPORT_LIMIT} reportes por
semana.",)
```

Fuente: Elaboración propia, 2026

Figura N° 3.33 **Código de validación de fotos**



```
# Validación de fotos
    if not photos or len(photos) == 0:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="Se requiere al menos una foto."
        )
    if len(photos) > MAX_PHOTOS:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"Máximo {MAX_PHOTOS} fotos permitidas."
        )

    # Leer bytes de todas las fotos
    images_bytes = []
    for i, photo in enumerate(photos, start=1):
        if photo.content_type not in ALLOWED_MIME_TYPES:
            raise HTTPException(
                status_code=status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
                detail=f"Foto {i}: tipo de archivo no permitido."
            )
        images_bytes.append(await photo.read())
```

Fuente: Elaboración propia, 2026

Figura N° 3.34 **Código de análisis de las fotos**

```
  # Analizar todas las fotos en paralelo
    try:
        analysis_results = await asyncio.gather(*[
            asyncio.to_thread(model_service.analyze_image, img_bytes)
            for img_bytes in images_bytes
        ])
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
detail=str(e)
        )
    embeddings_list = []
    for i, (dog_detected, confidence, embedding) in
enumerate(analysis_results):
        if not dog_detected:
            raise HTTPException(
                status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
                detail=f"La foto {i + 1} no muestra un perro
(confianza: {confidence:.2%})."
            )
        embeddings_list.append(embedding)
```

Fuente: Elaboración propia, 2026

Figura N° 3.35 **Código de verificación de imágenes duplicadas**

```
# Verificar duplicados
   existing_lost_dogs = await asyncio.to_thread(firebase_service.get_active_lost_dogs)
    for existing_dog in existing_lost_dogs:
        stored_embs = _get_embeddings(existing_dog)
        if not stored_embs:
            continue
        sim = _max_similarity(embeddings_list, stored_embs)
        if sim >= DUPLICATE_THRESHOLD:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail="Ya existe un reporte activo de este perro."
            )
```

Fuente: Elaboración propia, 2026

Figura N° 3.36 **Código de guardado del reporte en la base de datos**

```
 # Subir fotos en paralelo
    upload_tasks = [
        asyncio.to_thread(firebase_service.upload_photo, img_bytes, f"
{uuid.uuid4().hex}.jpg")
        for img_bytes in images_bytes
    ]
    upload_results = list(await asyncio.gather(*upload_tasks))
    photo_urls = [r[0] for r in upload_results]
    public_ids = [r[1] for r in upload_results]

    # Guardar documento en Firestore
    dog_data = {
        "name": dog_name,
        "owner_name": owner_name,
        "owner_phone": owner_phone,
        "owner_email": owner_email,
        "registered_by_uid": uid,
        "size": size,
        "color": color,
        "breed": breed,
        "sex": sex or "",
    }
    for i, (url, emb) in enumerate(zip(photo_urls, embeddings_list), start=1):
        dog_data[f"photo_url_{i}"] = url
        dog_data[f"embedding_{i}"] = emb
```

Fuente: Elaboración propia, 2026

i) Actividad 9 - Endpoint match-found-dog

El endpoint POST /match-found-dog constituye el núcleo del sistema de identificación. La similitud coseno se calcula entre el embedding del perro encontrado y cada uno de los embeddings almacenados en la colección lost_dogs de Firestore. Dado dos vectores A y B de 512 dimensiones en el coseno de similitud obtiene un valor de 1.0 que indica vectores idénticos y 0.0 indica vectores ortogonales sin similitud visual. El sistema retorna los 5 perros con mayor similitud ordenados de mayor a menor, y envía notificación push automática al dueño cuando el valor supera el umbral de 0.80 (80%).

Figura N° 3.37 **Código de verificación de duplicados**

```
  # Verificar duplicados en reportes encontrados
    for report in found_reports:
        stored_embs = _get_embeddings(report)
        if not stored_embs:
            continue
        sim = _max_similarity(found_embeddings, stored_embs)
        if sim >= DUPLICATE_THRESHOLD:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail="Este perro ya fue reportado como encontrado."
            )
```

Fuente: Elaboración propia, 2026

Figura N° 3.38 **Código de búsqueda de coincidencias**

```
  # Buscar coincidencias: solo contra perros de OTROS usuarios
    scored = []
    for dog in lost_dogs:
        if dog.get("registered_by_uid") == uid:
            continue
        stored_embs = _get_embeddings(dog)
        if not stored_embs:
            continue
        similarity_percent = round(_max_similarity(found_embeddings,
stored_embs), 1)
        if similarity_percent >= SIMILARITY_THRESHOLD:
            scored.append((similarity_percent, dog))

    scored.sort(key=lambda x: x[0], reverse=True)
    top_matches = scored[:TOP_K]

    matches = [
        DogMatch(
            dog_id=dog["doc_id"],
            name=dog.get("name", ""),
            owner_phone=dog.get("owner_phone", ""),
            owner_email=dog.get("owner_email", ""),
            similarity_percent=sim_pct,
            photo_url=dog.get("photo_url_1", dog.get("photo_url", "")),
        )
        for sim_pct, dog in top_matches
    ]
```

Fuente: Elaboración propia, 2026

Figura N° 3.39 **Código de notificación de similitud**

```
  # Notificar si similitud >= 80%
    for sim_pct, dog in top_matches:
        if sim_pct >= NOTIFICATION_THRESHOLD:
            owner_uid = dog.get("registered_by_uid")
            if owner_uid:
                firebase_service.send_match_notification(
                    owner_uid=owner_uid,
                    dog_name=dog.get("name", "tu perro"),
                    similarity_percent=sim_pct,
                )
```

Fuente: Elaboración propia, 2026

Figura N° 3.40 **Código de guardado del reporte en la base de datos**

```
# Guardar reporte
    report_data = {
        "found_by_uid": uid,
        "matched_dog_ids": [m.dog_id for m in matches],
        "matches": [{"dog_id": m.dog_id, "similarity_percent": m.similarity_percent}
                    for m in matches],
        "size": size or "",
        "color": color or "",
        "sex": sex or "",
        "description": description or "",
        "reporter_name": reporter_name or "",
        "reporter_phone": reporter_phone or "",
        "reporter_email": reporter_email or "",
    }
    for i, (url, emb) in enumerate(zip(found_photo_urls, found_embeddings),
start=1):
        report_data[f"found_dog_photo_url_{i}"] = url
        report_data[f"embedding_{i}"] = emb
```

Fuente: Elaboración propia, 2026

j) Actividad 10 - Endpoint health y rate limiting

Se implementó el endpoint GET /health para que Railway verifique el estado del servidor de forma automática.

Figura N° 3.41 **Código de Endpoint de verificación de estado del servidor**

```python
from app.limiter import limiter
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@router.get("/health", summary="Verificación de estado del servidor")
async def health_check():
    return {
        "status": "ok",
        "model_loaded": model_service.is_model_loaded(),
    }
```

Fuente: Elaboración propia, 2026

#### 3.3.8.4. *Pruebas*

Durante esta etapa se realizaron pruebas para validar que cada endpoint funcionara correctamente de manera integrada, verificando la autenticación, validación de datos, procesamiento con el modelo, interacción con Firestore, y envío de notificaciones.

Tabla N° 3.17 **Pruebas de la Iteración 2**

| FUNCIONALIDAD | ESTADO | OBSERVACIONES |
|---|---|---|
| Carga del modelo al iniciar | Aprobado | El modelo se carga en memoria correctamente. Response /health retorna model_loaded: true. |
| Autenticación Firebase Token | Aprobado | Los tokens válidos son aceptados, los tokens ausentes retornan 401 Unauthorized. |
| Validación de tipo de archivo | Aprobado | Solo acepta PNG y JPEG; rechaza otros formatos correctamente. |
| Endpoint /validate-photo con perro | Aprobado | Detecta correctamente un perro en la fotografía, retorna is_dog: true. |
| Endpoint /validate-photo sin perro | Aprobado | Rechaza correctamente imágenes que no contienen perros, retorna is_dog: false. |
| Endpoint /register-lost-dog | Aprobado | Registra perro con 2 fotos, calcula embeddings y detecta duplicados (409 si ya existe). |

| FUNCIONALIDAD | ESTADO | OBSERVACIONES |
|---|---|---|
| Detección de duplicados (90%) | Aprobado | Rechaza intentos de registrar el mismo perro dos veces con error 409 Conflict. |
| Protección perro propio | Aprobado | Impide que el dueño reporte su propio perro como encontrado, retorna 409. |
| Endpoint /match-found-dog | Aprobado | Busca coincidencias en los perros registrados. |
| Endpoint /health | Aprobado | Retorna estado OK y verifica que el modelo esté cargado correctamente. |

Fuente: Elaboración propia, 2026

a) Obtención del Token Firebase

El primer paso de cualquier prueba es obtener un token JWT válido de Firebase. Se realizó una petición POST a la API de Google para autenticar el usuario y obtener el token de sesión.

Figura N° 3.42 **Respuesta de autenticación Firebase con token JWT**




Fuente: Elaboración propia, 2026

b) Prueba 1: Validación de foto con perro (200 OK)

Se envió una fotografía de un perro al endpoint /validate-photo. El servidor procesó la imagen, la pasó al modelo EfficientNet-B4 y retornó con éxito que contiene un perro.

Figura N° 3.43 **Código de validación de foto**

```
print("PRUEBA 1: POST /validate-photo (foto de perro)")
with open(IMG_PERRO, "rb") as f:
    r = requests.post(f"{BASE}/validate-photo", headers=headers,
                      files={"photo": ("perrito.png", f, "image/png")})
```

Fuente: Elaboración propia, 2026

Figura N° 3.44 **Captura de respuesta 200 OK del endpoint /validate-photo**

```
============================================================
PRUEBA 1: POST /validate-photo (foto de perro)
============================================================
STATUS: 200
{
  "is_dog": true,
  "confidence": 0.6985,
  "message": "Foto válida."
}
```

Fuente: Elaboración propia, 2026

c) Prueba 2: Registro de perro extraviado con 2 fotos (409 Conflict)

Se intentó registrar un perro extraviado con 2 fotografías. El servidor rechazó el registro con código 409 porque ya existía un reporte activo de ese mismo perro (detectado por similitud coseno ≥ 90%). Esto confirma que el sistema de detección de duplicados funciona correctamente.

Figura N° 3.45 **Código de detección de duplicado en registro de perro extraviado**

```
print("PRUEBA 2: POST /register-lost-dog (2 fotos)")
with open(IMG_PERRO, "rb") as f1, open(IMG_REF, "rb") as f2:
    r = requests.post(f"{BASE}/register-lost-dog", headers=headers,
        data={
            "dog_name": "Rex",
            "owner_name": "Britny Sea",
            "owner_phone": "+591 12345678",
            "owner_email": "britny2003sea@gmail.com",
            "size": "grande",
            "color": "marron",
            "breed": "Labrador",
            "sex": "macho"
        },
        files=[
            ("photos", ("perrito.png", f1, "image/png")),
            ("photos", ("dog_photo_reference.png", f2, "image/png"))
        ]
```

Fuente: Elaboración propia, 2026

Figura N° 3.46 **Captura de respuesta 409 Conflict por duplicado detectado**

```
==================================================
PRUEBA 2: POST /register-lost-dog (2 fotos)
==================================================
STATUS: 409
{
  "detail": "Ya existe un reporte activo de este perro. No es posible registrarlo nuevamente."
}
```

Fuente: Elaboración propia, 2026

d) Prueba 3: Validación sin token (401 Unauthorized)

Se envió la misma petición de validación de foto, pero sin incluir el token Bearer en el encabezado. El servidor rechazó la petición inmediatamente con código 401, demostrando que la autenticación es obligatoria.

Figura N° 3.47 **Código de fallo de autenticación por token faltante**

```python
print("PRUEBA 3: POST /validate-photo SIN TOKEN (401)")
with open(IMG_PERRO, "rb") as f:
    r = requests.post(f"{BASE}/validate-photo",
                      files={"photo": ("perrito.png", f, "image/png")})
```

Fuente: Elaboración propia, 2026

Figura N° 3.48 **Captura de respuesta 401 por falta de autenticación**

```
==================================================
PRUEBA 3: POST /validate-photo SIN TOKEN (401)
==================================================
STATUS: 401
{
  "detail": "Not authenticated"
}
```

Fuente: Elaboración propia, 2026

e) Prueba 4: Validación de imagen que NO es perro

Se envió una imagen del logo del refugio (que no contiene un perro) al endpoint de validación. El servidor procesó correctamente y retornó que la imagen NO contiene un perro

Figura N° 3.49 **Código de rechazo de imagen que no contiene perro**

```
print("PRUEBA 4: POST /validate-photo imagen NO es perro")
with open(IMG_LOGO, "rb") as f:
    r = requests.post(f"{BASE}/validate-photo", headers=headers,
                      files={"photo": ("logo_refugio.png", f,
"image/png")})
```

Fuente: Elaboración propia, 2026

Figura N° 3.50 **Captura de respuesta correcta rechazando imagen sin perro**

```
============================================================
PRUEBA 4: POST /validate-photo imagen NO es perro
============================================================
STATUS: 200
{
  "is_dog": false,
  "message": "La foto no muestra un perro. Por favor seleccione otra foto."
}
```

Fuente: Elaboración propia, 2026

f) Prueba 5: Protección contra reporte propio

Se intentó usar el endpoint /match-found-dog para reportar la misma foto de un perro que el usuario ya había reportado como extraviado. El servidor detectó automáticamente que el usuario estaba intentando reportar su propio perro como encontrado y rechazó la petición con código 409, protegiendo la integridad del sistema.

Figura N° 3.51 **Código de protección contra conflicto de reportes propios**

```
print("PRUEBA 5: POST /match-found-dog proteccion 409")
with open(IMG_PERRO, "rb") as f:
    r = requests.post(f"{BASE}/match-found-dog", headers=headers,
        data={
            "reporter_name": "Maria Lopez",
            "reporter_phone": "+591 87654321",
            "reporter_email": "maria@test.com"
        },
        files=[("photos", ("perrito.png", f, "image/png"))]
    )
```

Fuente: Elaboración propia, 2026

Figura N° 3.52 **Captura de rechazo de intento de reportar propio perro como encontrado**

```
================================================
PRUEBA 5: POST /match-found-dog proteccion 409
================================================
STATUS: 409
{
  "detail": "Esta foto corresponde a un perro que tú mismo reportaste como perdido. No puedes reportarlo como encontrado."
}
```

Fuente: Elaboración propia, 2026

g) Prueba 6: Verificación del servidor (200 OK)

Se verificó el estado del servidor mediante el endpoint /health. La respuesta confirmó que el servidor está operativo y que el modelo de IA está cargado en memoria correctamente.

Figura N° 3.53 **Captura de servidor operativo y modelo cargado**

```
================================================
PRUEBA 6: GET /health (estado del servidor)
================================================
STATUS: 200
{
  "status": "ok",
  "model_loaded": true
}
```

Fuente: Elaboración propia, 2026

### 3.3.9. *ITERACIÓN 3 - APLICACIÓN MÓVIL ANDROID*

#### 3.3.9.1. *Planeación*

En esta iteración se desarrolló la interfaz principal del sistema para los usuarios operativos. La aplicación móvil implementa todas las funcionalidades de autenticación, registro de reportes, búsqueda de coincidencias y recepción de notificaciones push. Esta iteración da cumplimiento a las historias de usuario HU-01 al HU-08.

Tabla N° 3.18 **Actividades de la Iteración 3**

| N° | ACTIVIDAD | DESCRIPCIÓN |
|---|---|---|
| **1** | Configuración del proyecto y arquitectura | Implementación de Clean Architecture con separación de capas. |
| **2** | Inyección de dependencias con Hilt | Configuración de proveedores de Firebase, Retrofit y repositorios. |
| **3** | Cliente HTTP y contrato de la API | Creación de interfaces Retrofit para comunicación con el backend. |
| **4** | Repositorio de autenticación | Implementación de flujos de registro, inicio de sesión y verificación de correo electrónico. |
| **5** | Repositorio de perros y preprocesamiento | Escalado, recorte y compresión de imágenes antes del envío al backend. |
| **6** | ViewModel de autenticación | Validaciones de contraseña y manejo de estados de autenticación. |
| **7** | ViewModel de perros | Gestión de fotografías y comunicación con el backend. |
| **8** | ViewModel de sesión | Detección de bloqueo de usuario en tiempo real. |
| **9** | Navegación y puerta de teléfono | Configuración de NavHost y validación obligatoria de número telefónico. |
| **10** | Notificaciones push con FCM | Implementación de WoofMessagingService para recepción de notificaciones push. |
| **11** | Pantalla Splash | Animaciones iniciales y verificación de conexión. |

| N° | ACTIVIDAD | DESCRIPCIÓN |
|---|---|---|
| **12** | Pantallas de autenticación | Desarrollo de Login, Registro y recuperación de contraseña. |
| **13** | Pantalla Home | Visualización de saludo personalizado y tarjetas de acción. |
| **14** | Pantalla Perfil | Visualización y edición del número telefónico. |
| **15** | Formulario de perro extraviado | Registro de perros extraviados con 1 a 3 fotografías y validación visual con IA. |
| **16** | Formulario de perro encontrado | Búsqueda de coincidencias mediante inteligencia artificial. |
| **17** | Pantalla de resultados | Comparación visual de fotografías con porcentaje de similitud. |
| **18** | Pantalla de mis reportes | Filtrado de reportes por estado activo o inactivo. |

Fuente: Elaboración propia, 2026

Tabla N° 3.19 **Historias de usuario de la Iteración 3**

| N° | HISTORIA DE USUARIO | DESCRIPCIÓN |
|---|---|---|
| **HU-01** | Registro de cuenta en la app móvil | Crear cuenta con correo, contraseña y verificación de email |
| **HU-02** | Inicio de sesión en la app móvil | Entrar a la app con correo y contraseña |
| **HU-03** | Número de teléfono obligatorio | Pedir teléfono antes de hacer reportes |
| **HU-04** | Registro de perro extraviado | Reportar perro perdido con foto y datos |
| **HU-05** | Búsqueda de coincidencias | Subir foto de perro encontrado y buscar matches |
| **HU-06** | Visualización de resultados | Ver lista de perros similares con % de coincidencia |

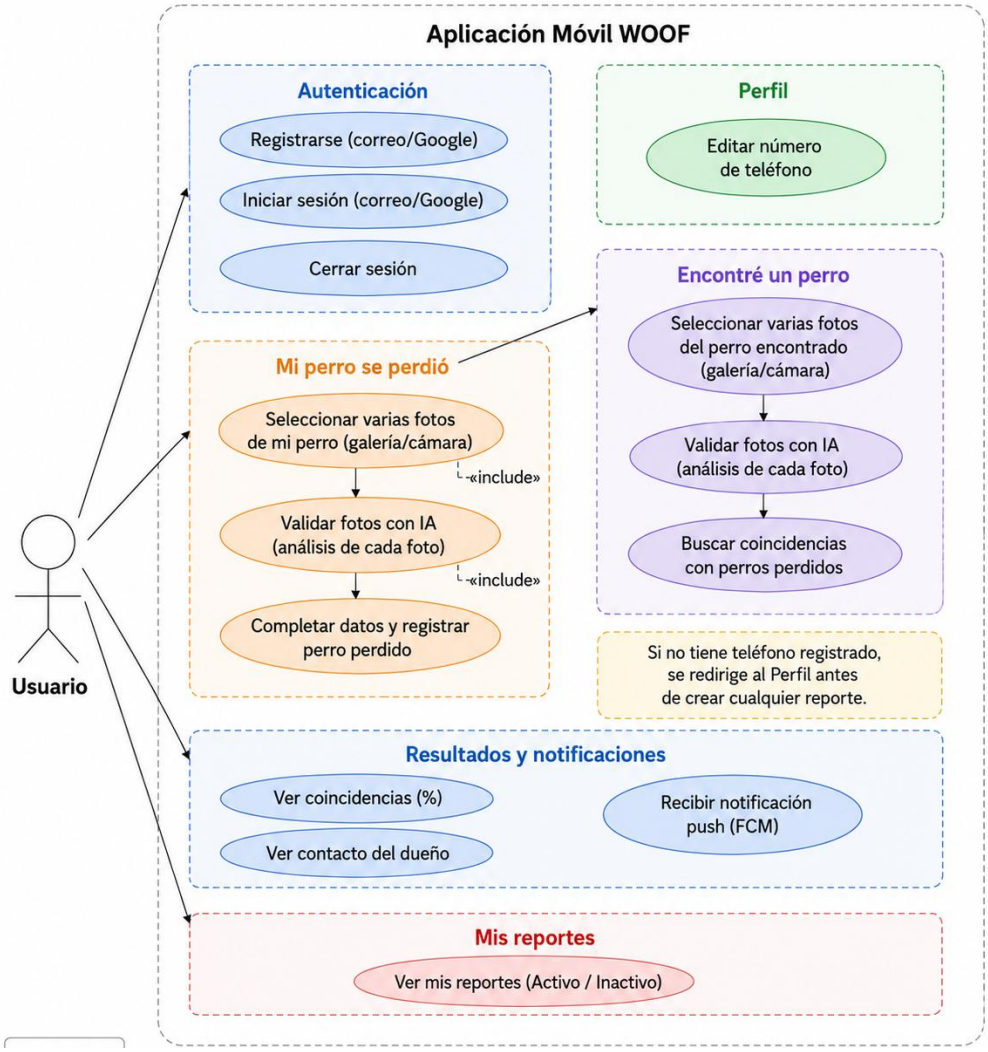
| N° | HISTORIA DE USUARIO | DESCRIPCIÓN |
|---|---|---|
| **HU-07** | Visualización de reportes propios | Ver todos los reportes del usuario |
| **HU-08** | Notificaciones push | Recibir notificación cuando se encuentra coincidencia |

Fuente: Elaboración propia, 2026

### 3.3.9.2. *Diseño*

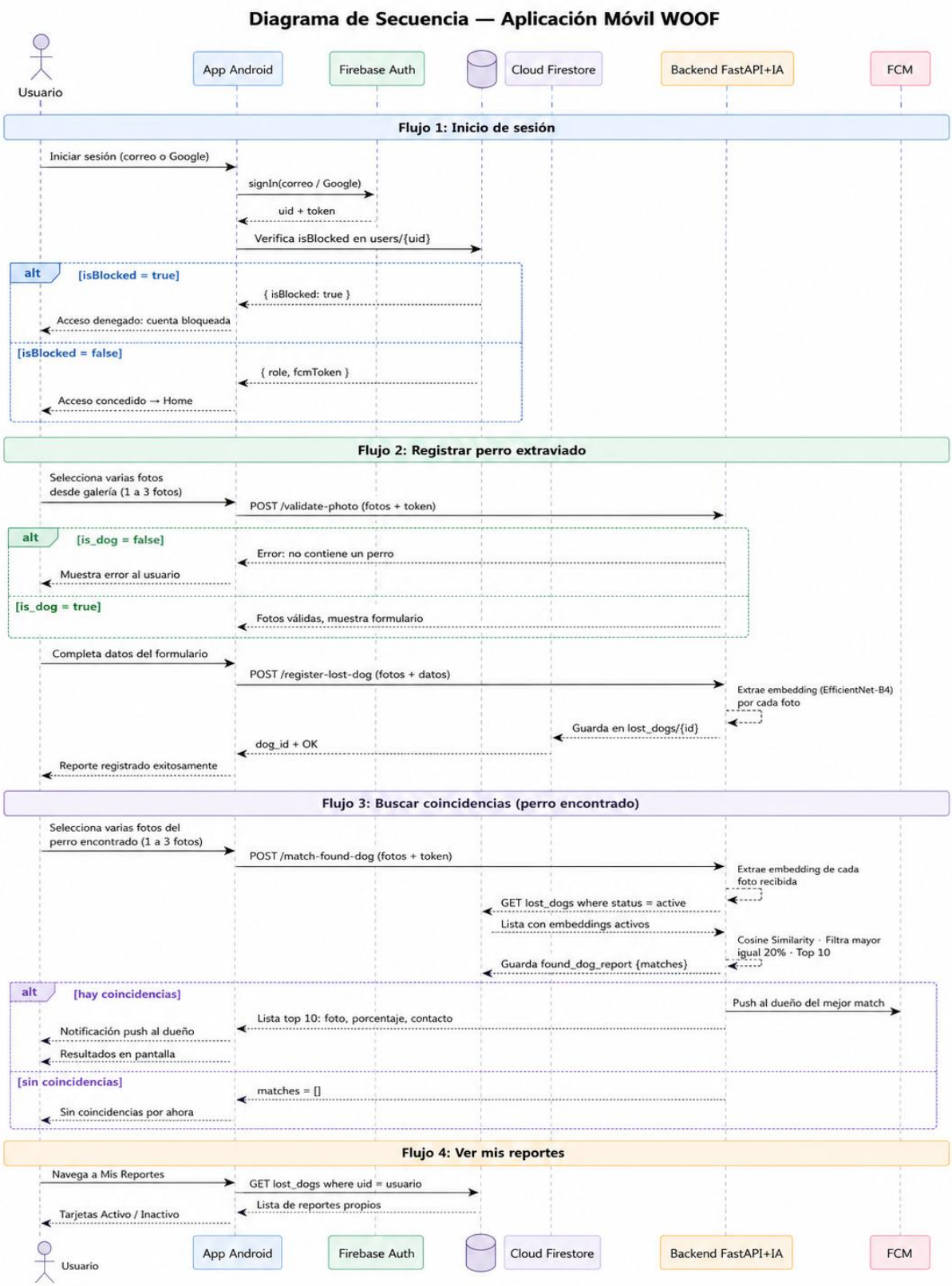
La aplicación fue diseñada siguiendo Clean Architecture con separación clara entre capas de datos, dominio y presentación. El diseño visual aplica Material Design 3 adaptado a tema oscuro con la paleta de colores del Refugio WOOF: fondo negro y acentos dorados.

Figura N° 3.54 **Diagrama de Caso de Uso de la aplicación móvil**



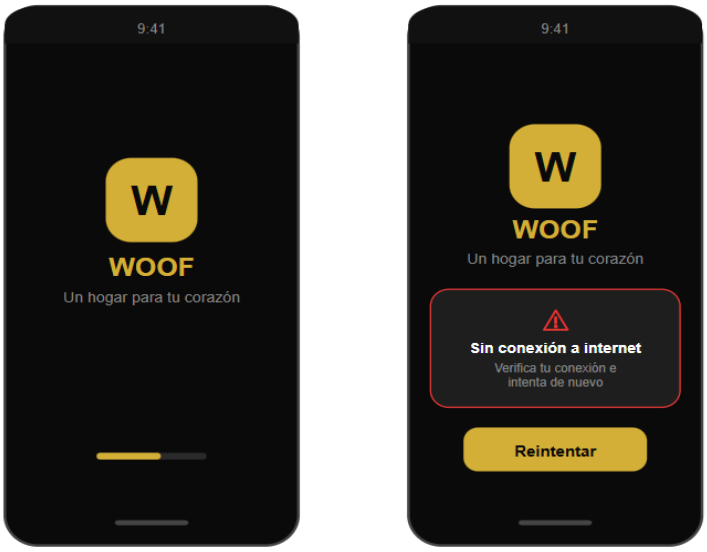

Fuente: Elaboración propia, 2026

Figura N° 3.55 **Diagrama de Secuencia de la aplicación móvil**




Fuente: Elaboración propia, 2026

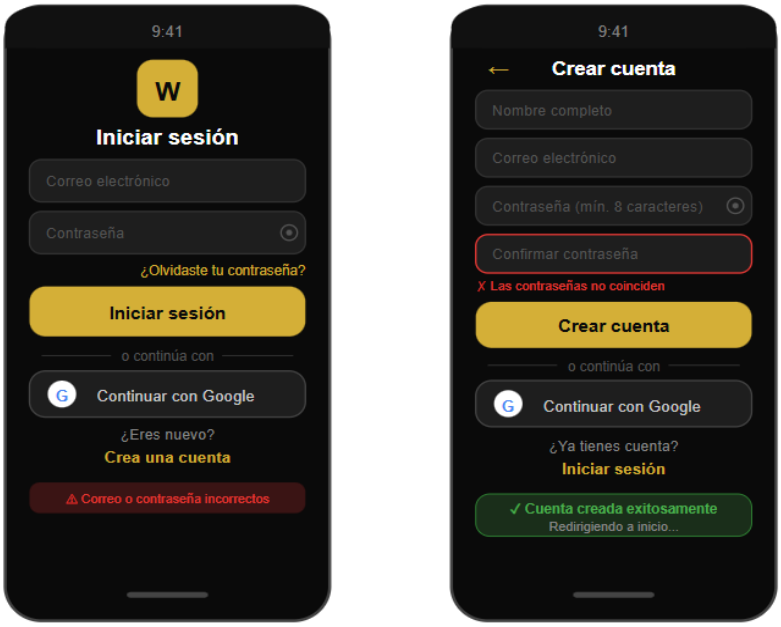
Figura N° 3.56 **Mockup de splash screen y estado sin conexión a internet**




Fuente: Elaboración propia, 2026

La pantalla de inicio muestra el logo del caso de estudio Refugio WOOF con el eslogan 'Un hogar para tu corazón' mientras se verifica si existe una sesión activa. Si el dispositivo no tiene conexión a internet, el sistema muestra un aviso de error con el botón 'Reintentar', garantizando que el usuario siempre reciba retroalimentación clara sobre el estado de la aplicación.

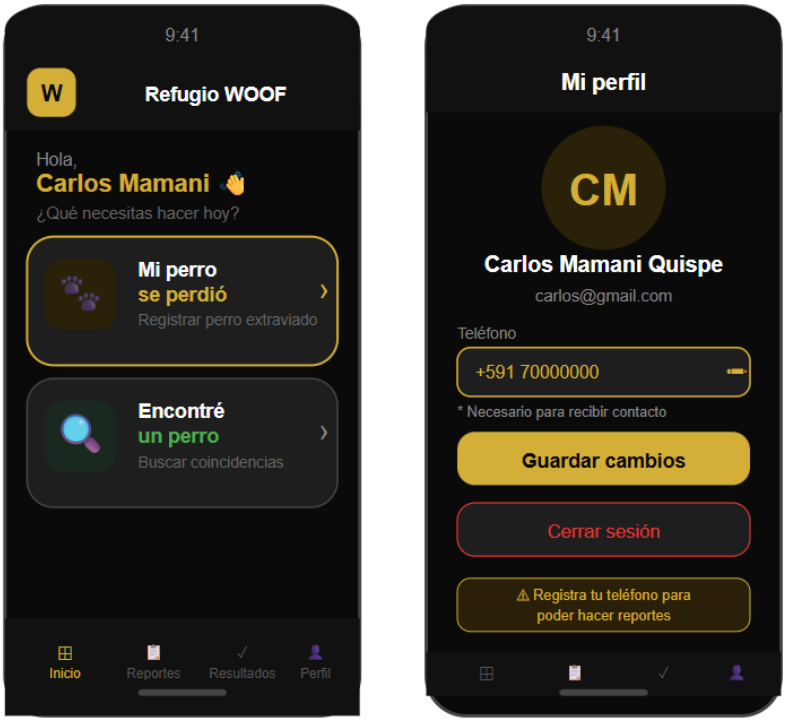
Figura N° 3.57 **Mockup de pantallas de autenticación: Inicio de sesión y registro**




Fuente: Elaboración propia, 2026

La pantalla de inicio de sesión permite autenticarse mediante correo electrónico y contraseña o a través de Google. Incluye el enlace '¿Eres nuevo? Crea una cuenta' y un mensaje de error específico cuando las credenciales son incorrectas. La pantalla de registro solicita nombre, correo, contraseña y confirmación, con validación en tiempo real de cada campo. Ambas pantallas ofrecen la opción 'Continuar con Google' para agilizar el acceso.

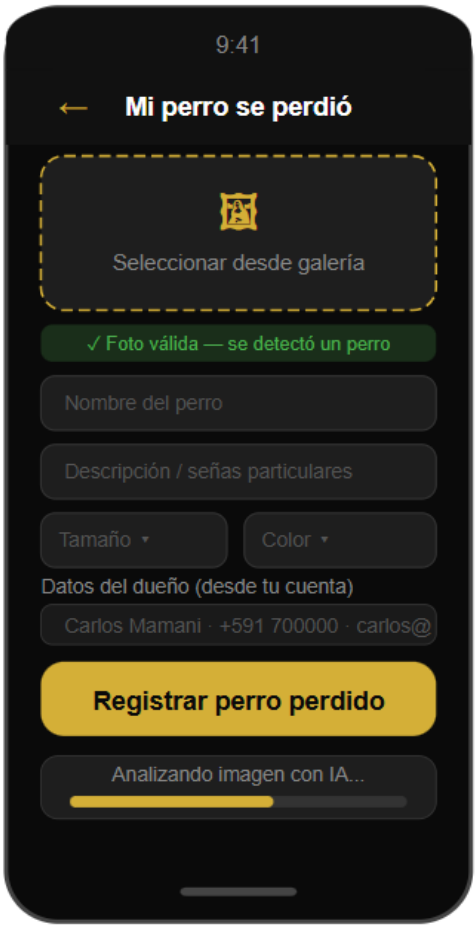
Figura N° 3.58 **Mockup de pantalla de inicio y pantalla de perfil**




Fuente: Elaboración propia, 2026

La pantalla de inicio muestra un saludo personalizado con el nombre del usuario y dos accesos directos principales: 'Mi perro se perdió' y 'Encontré un perro'. La pantalla de perfil presenta el nombre del usuario y su correo Gmail, junto con el campo de teléfono editable. Si el usuario intenta crear un reporte sin tener teléfono registrado, la aplicación lo redirige automáticamente a la pantalla de perfil para completar este dato antes de continuar.

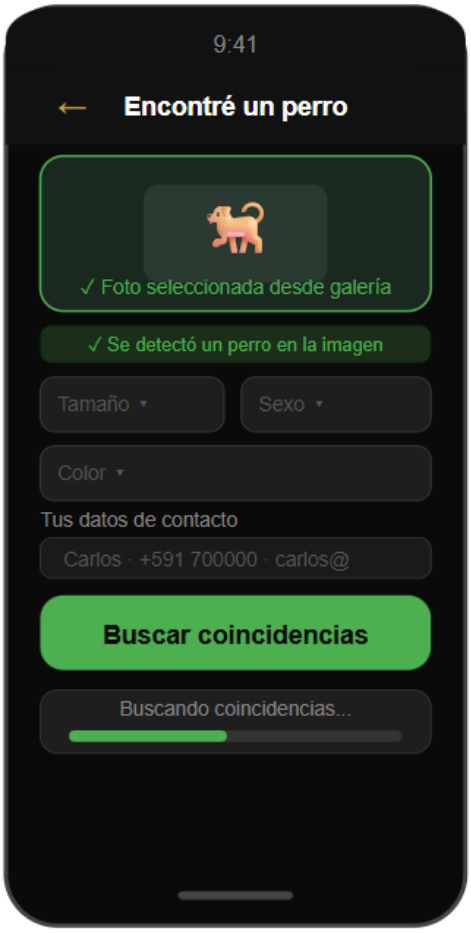
Figura N° 3.59 **Mockup de formulario de reporte de perro extraviado**




Fuente: Elaboración propia, 2026

El formulario permite seleccionar fotografías desde la galería del dispositivo. Antes de procesar el registro, el sistema envía las imágenes al backend para verificar que contienen un perro mediante el modelo EfficientNet-B4; si la validación falla, muestra un mensaje descriptivo y bloquea el envío. Los campos incluyen nombre del perro, tamaño, color y señas particulares. Los datos de contacto del dueño se cargan automáticamente desde la cuenta del usuario autenticado como ser el nombre, teléfono y correo.

Figura N° 3.60 **Mockup de formulario de reporte de perro encontrado**




Fuente: Elaboración propia, 2026

El formulario de reporte de perro encontrado permite seleccionar fotografías desde la galería. Al igual que en el reporte de perro perdido, el sistema valida que las imágenes contengan un perro antes de continuar. Los campos descriptivos incluyen tamaño, sexo y color del animal. Los datos de contacto del reportante se obtienen automáticamente desde la cuenta activa. Al enviar, el backend ejecuta el proceso de comparación visual contra todos los perros perdidos registrados.

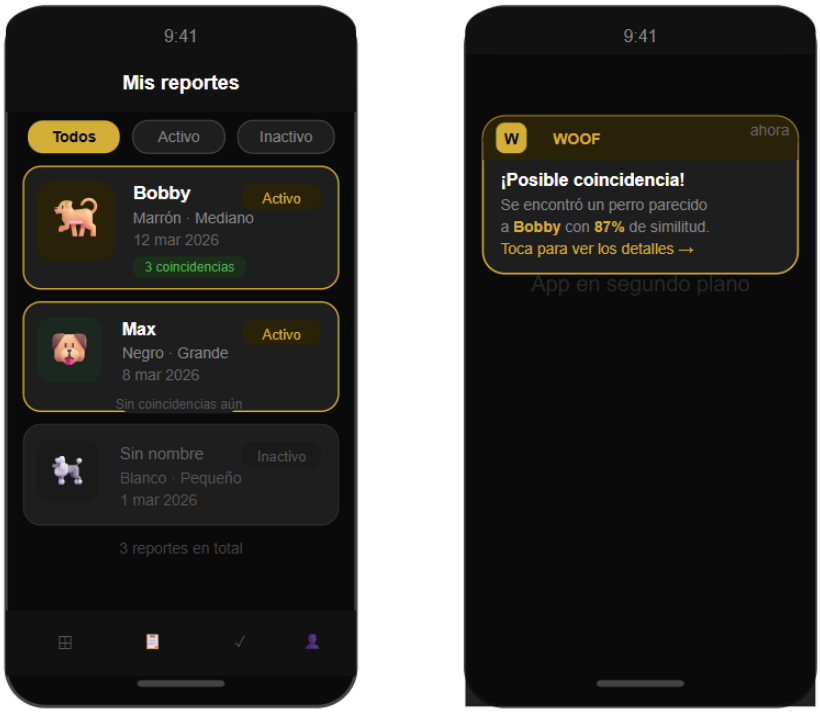
Figura N° 3.61 **Mockup de pantalla de resultados de coincidencias y estado sin coincidencias**




Fuente: Elaboración propia, 2026

La pantalla de resultados presenta la lista de perros perdidos más similares ordenada de mayor a menor porcentaje de similitud calculado por el algoritmo Cosine Similarity. Cada tarjeta muestra la fotografía del perro, su nombre, el porcentaje de coincidencia, el teléfono y el correo del dueño para contacto directo.

Figura N° 3.62 **Mockup de pantalla de mis reportes y notificación**




Fuente: Elaboración propia, 2026

La pantalla de mis reportes muestra únicamente los reportes registrados desde la cuenta activa, con filtros para visualizar todos, solo activos o inactivos, cada tarjeta indica el estado del reporte. La notificación se envía al dispositivo del usuario cuando el sistema detecta una coincidencia que supera el umbral definido, indicando el nombre del perro y el porcentaje de similitud, incluso con la aplicación cerrada o en segundo plano.

### 3.3.9.3. *Codificación*

Se realizaron todas las actividades definidas en la fase de planeación. Las primeras 10 actividades cubren la infraestructura técnica (inyección de dependencias, repositorios, ViewModels, autenticación), y las actividades 11 al 18 implementan las pantallas visuales.

a) Actividad 1 - Configuración del proyecto y arquitectura

La aplicación sigue Clean Architecture organizada en tres capas: datos (repositorios), dominio (modelos e interfaces) y presentación (ViewModels y pantallas Composable). La clase ProyectoApp inicializa Firebase al arrancar la aplicación y habilita la inyección de dependencias con Hilt.

Figura N° 3.63 **Clase de aplicación con inicialización de Firebase y Hilt**

```
@HiltAndroidApp
class ProyectoApp : Application() {
    override fun onCreate() {
        super.onCreate()
        FirebaseApp.initializeApp(this)
    }
}
```

Fuente: Elaboración propia, 2026

Se definió una clase sellada UiState que representa los cuatro estados posibles de cualquier operación asíncrona en la aplicación: Idle cuando no hay operación activa, Loading durante la ejecución, Success con el resultado y Error con el mensaje descriptivo.

Figura N° 3.64 **Código de clase sellada UiState para manejo de estados**

```
sealed class UiState<out T> {
    object Idle    : UiState<Nothing>()
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) :
UiState<T>()
    data class Error(val message: String) :
UiState<Nothing>()
}
```

Fuente: Elaboración propia, 2026

b) Actividad 2 - Inyección de dependencias con Hilt

El módulo AppModule provee todas las dependencias de la aplicación, garantizando que Firebase Auth, Firestore y los repositorios se instancien una sola vez. Retrofit se configura con un interceptor de logging y apunta al servidor backend en Railway.

Figura N° 3.65 **Código de inyección de dependencias con Hilt**



```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides @Singleton
    fun provideFirebaseAuth(): FirebaseAuth = FirebaseAuth.getInstance()
    @Provides @Singleton
    fun provideFirestore(): FirebaseFirestore = FirebaseFirestore.getInstance()
    @Provides @Singleton
    fun provideOkHttpClient(): OkHttpClient {
        val logging = HttpLoggingInterceptor().apply {
            level = HttpLoggingInterceptor.Level.BODY
        }
        return OkHttpClient.Builder().addInterceptor(logging).build()
    }
    @Provides @Singleton
    fun provideWoofApiService(client: OkHttpClient): WoofApiService {
        return Retrofit.Builder()
            .baseUrl("https://woof-backend.railway.app/")
            .client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(WoofApiService::class.java)
    }
    @Provides @Singleton
    fun provideDogsRepository(
        api: WoofApiService, auth: FirebaseAuth, db: FirebaseFirestore
    ): DogsRepository = DogsRepositoryImpl(api, auth, db)
}
```

Fuente: Elaboración propia, 2026

c) Actividad 3 - Cliente HTTP y contrato de la API

La interfaz WoofApiService define el contrato completo de comunicación con el backend FastAPI usando Retrofit. Cada método corresponde a un endpoint del servidor y utiliza anotaciones para el envío de fotografías junto con datos de formulario.

Figura N° 3.66 **Código de interfaz retrofit para comunicación con el backend**

```
interface WoofApiService {
    @Multipart @POST("api/v1/validate-photo")
    suspend fun validatePhoto(
        @Header("Authorization") token: String,
        @Part photo: MultipartBody.Part
    ): ValidatePhotoResponse
    @Multipart @POST("api/v1/register-lost-dog")
    suspend fun registerLostDog(
        @Header("Authorization") token: String,
        @Part("dog_name") dogName: RequestBody,
        @Part("owner_name") ownerName: RequestBody,
        @Part("owner_phone") ownerPhone: RequestBody,
        @Part("owner_email") ownerEmail: RequestBody,
        @Part photo: MultipartBody.Part,
        @Part("size") size: RequestBody,
        @Part("color") color: RequestBody,
        @Part("sex") sex: RequestBody,
    ): RegisterLostDogResponse
    @Multipart @POST("api/v1/match-found-dog")
    suspend fun matchFoundDog(
        @Header("Authorization") token: String,
        @Part photos: List<MultipartBody.Part>,
        @Part("reporter_name") reporterName: RequestBody,
        @Part("reporter_phone") reporterPhone: RequestBody,
        @Part("reporter_email") reporterEmail: RequestBody,
    ): MatchFoundDogResponse
    @GET("api/v1/my-found-reports")
    suspend fun getMyFoundReports(
        @Header("Authorization") token: String
    ): MyFoundReportsResponse
}
```

Fuente: Elaboración propia, 2026

d) Actividad 4 - Repositorio de autenticación

AuthRepositoryImpl implementa los flujos de autenticación. El registro con correo y contraseña crea la cuenta en Firebase Auth, envía verificación de email y cierra la sesión inmediatamente. El inicio de sesión verifica que el correo esté confirmado y crea el documento del usuario en Firestore si es su primera vez.

Figura N° 3.67 **Código de registro e inicio de sesión con verificación de email**

```
override suspend fun signUp(user: User, password: String): Result<Unit> {
    return try {
        val result = auth.createUserWithEmailAndPassword(user.email, password).await()
        result.user?.updateProfile(
            userProfileChangeRequest { displayName = user.name }
        )?.await()
        result.user?.sendEmailVerification()?.await()
        auth.signOut() // No queda logueado sin verificar
        Result.success(Unit)
    } catch (e: Exception) { Result.failure(e) }
}
override suspend fun signIn(email: String, password: String): Result<Unit> {
    return try {
        auth.signInWithEmailAndPassword(email, password).await()
        val firebaseUser = auth.currentUser
        if (firebaseUser != null && firebaseUser.isEmailVerified) {
            val uid = firebaseUser.uid
            val userExists = db.collection("users").document(uid).get().await().exists()
            if (!userExists) {
                db.collection("users").document(uid).set(
                    User(uid = uid, name = firebaseUser.displayName ?: "Usuario",
                         email = firebaseUser.email ?: "")
                ).await()
            }
        }
        Result.success(Unit)
    } catch (e: Exception) { Result.failure(e) }
```

Fuente: Elaboración propia, 2026

e) Actividad 5 - Repositorio de perros y preprocesamiento de imagen

DogsRepositoryImpl centraliza la comunicación con el backend. Antes de enviar cualquier fotografía se obtiene el token de Firebase Auth y se prepara la imagen.

Figura N° 3.68 **Código de token, validación de fotos y manejo de errores HTTP**

```
private suspend fun getBearerToken(): String {
    val token = auth.currentUser?.getIdToken(false)?.await()?.token
        ?: error("Usuario no autenticado")
    return "Bearer $token"
}
override suspend fun validatePhoto(photo: File): Result<String> {
    return try {
        val token = getBearerToken()
        val photoBody = photo.asRequestBody("image/*".toMediaTypeOrNull())
        val photoPart = MultipartBody.Part.createFormData("photo", photo.name, photoBody)
        val response = api.validatePhoto(token = token, photo = photoPart)
        if (response.isDog) Result.success(response.message)
        else Result.failure(Exception(response.message))
    } catch (e: HttpException) {
        Result.failure(Exception(parseErrorDetail(e)))
    } catch (e: Exception) { Result.failure(e) }
}
private fun parseErrorDetail(e: HttpException): String {
    return try {
        val body = e.response()?.errorBody()?.string() ?: ""
        JSONObject(body).getString("detail")
    } catch (_: Exception) { "Error al procesar la solicitud" }
```

Fuente: Elaboración propia, 2026

f) Actividad 6 - ViewModel de autenticación

AuthViewModel gestiona todos los flujos de autenticación aplicando validaciones en la capa de presentación. La validación de contraseña verifica longitud mínima, mayúsculas, minúsculas, dígitos y caracteres especiales.

Figura N° 3.69 **Código de validación de contraseña segura**

```
fun onSignUp(user: User, password: String, confirmPassword: String) {
    viewModelScope.launch {
        if (password.length < 8) {
            _signUpState.value = UiState.Error("La contraseña debe tener mínimo
8 caracteres")
            return@launch
        }
        val hasUpperCase = password.any { it.isUpperCase() }
        val hasLowerCase = password.any { it.isLowerCase() }
        val hasDigit     = password.any { it.isDigit() }
        val hasSpecial   = password.any { !it.isLetterOrDigit() }
        if (!hasUpperCase || !hasLowerCase || !hasDigit || !hasSpecial) {
            _signUpState.value = UiState.Error(
                "Contraseña insegura. Debe contener mayúsculas, minúsculas,
números y símbolos.")
            return@launch
        }
        _signUpState.value = UiState.Loading
        val result = repository.signUp(user, password)
        result.onSuccess { _signUpState.value = UiState.Success(Unit) }
              .onFailure { _signUpState.value = UiState.Error(it.message ?:
"Error") }
    }
}
```

Fuente: Elaboración propia, 2026

g) Actividad 7 - ViewModel de perros y preprocesamiento de imagen

DogsViewModel coordina todos los flujos relacionados con perros. Antes de enviar una fotografía se escala manteniendo proporción hasta que el lado más corto mida 800 píxeles, luego se aplica un recorte central cuadrado y se comprime en JPEG al 85% de calidad.

Figura N° 3.70 **Código de preprocesamiento de imagen**

```
private fun uriToFile(context: Context, uri: Uri): File {
    val inputStream = context.contentResolver.openInputStream(uri)!!
    val original = BitmapFactory.decodeStream(inputStream)!!
    val target = 800
    val w = original.width; val h = original.height
    val scale = target.toFloat() / minOf(w, h)
    val scaledW = (w * scale).toInt(); val scaledH = (h * scale).toInt()
    val scaled = Bitmap.createScaledBitmap(original, scaledW, scaledH, true)
    val left = (scaledW - target) / 2; val top = (scaledH - target) / 2
    val cropped = Bitmap.createBitmap(scaled, left, top, target, target)
    val tempFile = File(context.cacheDir,
"woof_temp_${System.currentTimeMillis()}.jpg")
    FileOutputStream(tempFile).use {
cropped.compress(Bitmap.CompressFormat.JPEG, 85, it) }
    return tempFile
}
```

Fuente: Elaboración propia, 2026

h) Actividad 8 - ViewModel de sesión y detección de bloqueo en tiempo real

SessionViewModel escucha en tiempo real el campo isBlocked del documento del usuario en Firestore mediante un listener. Cuando el administrador bloquea una cuenta, el flujo emite true instantáneamente y MainActivity cerrando la sesión.

Figura N° 3.71 **Código de detección de bloqueo de cuenta**

```
@HiltViewModel
class SessionViewModel @Inject constructor(
    userRepository: UserRepository
) : ViewModel() {

    val isBlocked: StateFlow<Boolean> = userRepository
        .observeBlockedStatus()
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.Eagerly,
            initialValue = false
        )
}
```

Fuente: Elaboración propia, 2026

El listener de Firestore en UserRepositoryImpl usa callbackFlow para convertir la escucha en tiempo real:

Figura N° 3.72 **Código de listener de bloqueo de cuenta de usuario**

```
override fun observeBlockedStatus(): Flow<Boolean> = callbackFlow {
    var firestoreListener: ListenerRegistration? = null
    fun attach() {
        firestoreListener?.remove()
        val uid = auth.currentUser?.uid
        if (uid == null) { trySend(false); return }
        firestoreListener = db.collection("users").document(uid)
            .addSnapshotListener { snapshot, _ ->
                trySend(snapshot?.getBoolean("isBlocked") ?: false)
            }
    }
    val authStateListener = FirebaseAuth.AuthStateListener { attach() }
    auth.addAuthStateListener(authStateListener)
    attach()
    awaitClose {
        firestoreListener?.remove()
        auth.removeAuthStateListener(authStateListener)
    }
}
```

Fuente: Elaboración propia, 2026

i) Actividad 9 - Navegación y puerta de teléfono

MainActivity centraliza el grafo de navegación con NavHost de Jetpack Compose. Se implementó la función navigateWithPhoneGate que verifica si el usuario tiene teléfono registrado antes de permitir crear un reporte.

Figura N° 3.73 **Código de definición de rutas de navegación**

```
object Routes {
    const val SPLASH        = "splash"
    const val LOGIN         = "login"
    const val REGISTER      = "register"
    const val HOME          = "home"
    const val PROFILE       = "profile"
    const val REPORT_LOST   = "report_lost"
    const val REPORT_FOUND  = "report_found"
    const val MATCH_RESULTS = "match_results"
    const val MY_REPORTS    = "my_reports"
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.74 **Código de redirección al teléfono y inicio de sesión por bloqueo**

```
val navigateWithPhoneGate: (action: () -> Unit) -> Unit = { action ->
    scope.launch {
        if (dogsViewModel.hasPhone()) action()
        else navController.navigate(Routes.PROFILE)
    }
}

LaunchedEffect(isBlockedByAdmin) {
    if (isBlockedByAdmin && FirebaseAuth.getInstance().currentUser != null) {
        FirebaseAuth.getInstance().signOut()
        showBlockedMessage = true
        navController.navigate(Routes.LOGIN) {
            popUpTo(0) { inclusive = true }
        }
    }
```

Fuente: Elaboración propia, 2026

j) Actividad 10 - Notificaciones push con FCM

WoofMessagingService extiende Firebase Messaging Service para recibir notificaciones cuando se detecta una coincidencia. Al recibir notificación, crea un canal de alta prioridad woof_matches y muestra la notificación.

Figura N° 3.75 **Código de notificaciones push con FCM**

```
class WoofMessagingService : FirebaseMessagingService() {
    override fun onNewToken(token: String) {
        val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return
        FirebaseFirestore.getInstance()
            .collection("users").document(uid)
            .update("fcmToken", token)
    }
    override fun onMessageReceived(message: RemoteMessage) {
        val title = message.notification?.title ?: message.data["title"] ?: "WOOF"
        val body  = message.notification?.body  ?: message.data["body"]  ?: ""
        showNotification(title, body)
    }
    private fun showNotification(title: String, body: String) {
        val channelId = "woof_matches"
        val notificationManager = getSystemService(NOTIFICATION_SERVICE) as NotificationManager
        val channel = NotificationChannel(channelId, "Coincidencias de perros",
            NotificationManager.IMPORTANCE_HIGH)
        notificationManager.createNotificationChannel(channel)
        val notification = NotificationCompat.Builder(this, channelId)
            .setContentTitle(title)
            .setContentText(body)
            .setSmallIcon(R.mipmap.ic_launcher)
            .setAutoCancel(true)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .build()
        notificationManager.notify(System.currentTimeMillis().toInt(), notification)
    }
```

Fuente: Elaboración propia, 2026

k) Actividad 11 - Pantalla Splash

La pantalla de inicio se muestra mientras la aplicación verifica si existe una sesión activa. Implementa una secuencia de animaciones encadenadas usando Animatable y verifica la disponibilidad de internet usando ConnectivityManager.

Figura N° 3.76 **Código de verificación de conexión e intervalo de animaciones**

```
private fun isInternetAvailable(context: Context): Boolean {
    val cm = context.getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
    val network = cm.activeNetwork ?: return false
    val caps = cm.getNetworkCapabilities(network) ?: return false
    return caps.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
}
LaunchedEffect(Unit) {
    launch { logoScale.animateTo(1f, spring(dampingRatio = Spring.DampingRatioMediumBouncy)) }
    launch { logoAlpha.animateTo(1f, tween(1200)) }
    delay(500)
    launch { ringAlpha.animateTo(0.4f, tween(1200)) }
    delay(700)
    textAlpha.animateTo(1f, tween(1000))
    delay(2500)
    if (isInternetAvailable(context)) {
        onSplashFinished()
    } else {
        noInternet = true
    }
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.77 **Pantalla móvil Splash con logo animado y eslogan**




Fuente: Elaboración propia, 2026

Figura N° 3.78 **Pantalla móvil de estado sin conexión con botón Reintentar**




Fuente: Elaboración propia, 2026

l) Actividad 12 - Pantallas de autenticación: Inicio de sesion y Registro

La pantalla de inicio de sesión presenta un fondo oscuro con imagen de perro y degradado de fusión. Los campos incluyen validación en tiempo real: el correo muestra error si el formato es inválido al perder el foco, y la contraseña tiene botón de visibilidad. Los errores se muestran como notificaciones que desaparecen automáticamente tras 5 segundos.

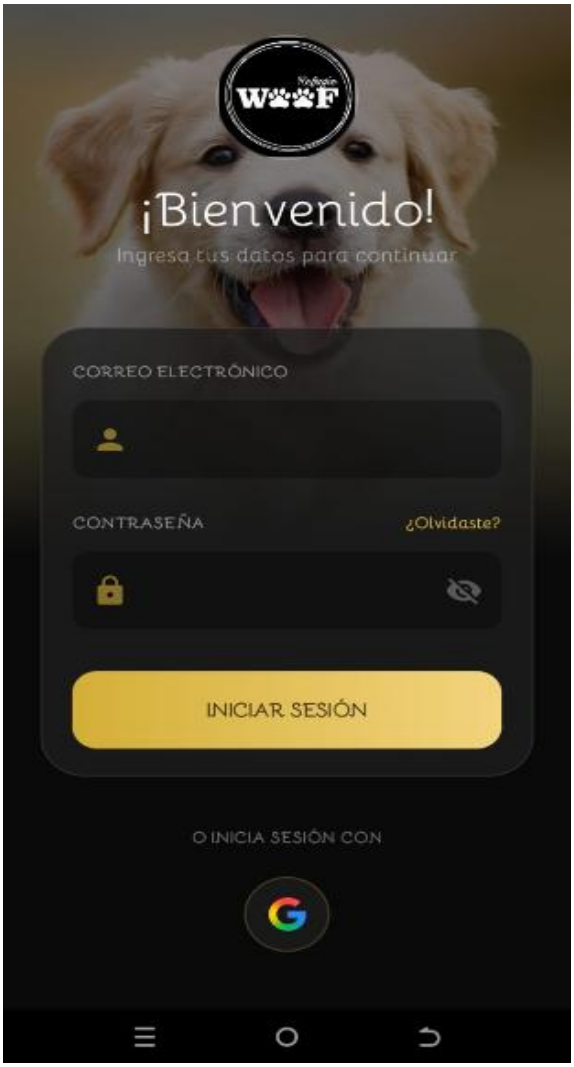
Figura N° 3.79 **Código de validación en tiempo real e inicio de sesión**

```
val emailError: String? = if (emailTouched && email.isNotEmpty() &&
    !android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches())
    "Formato de correo no válido" else null

LaunchedEffect(signInState) {
    if (signInState is UiState.Success) {
        onLoginSuccess()
    }
}
LaunchedEffect(signInState) {
    if (signInState is UiState.Error) {
        delay(5000)
        viewModel.resetStates()
    }
}
Button(
    onClick = { viewModel.onSignIn(email, password) },
    enabled = signInState !is UiState.Loading
) {
    if (signInState is UiState.Loading) {
        CircularProgressIndicator(modifier = Modifier.size(24.dp),
color = Color.Black)
    } else {
        Text("INICIAR SESIÓN", fontWeight = FontWeight.Bold)
    }
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.80 **Pantalla móvil de inicio de sesión**




Fuente: Elaboración propia, 2026

La pantalla de registro incluye un indicador visual de fortaleza de contraseña que evalúa en tiempo real cuatro criterios: longitud mínima (8 caracteres), mayúsculas y minúsculas, dígitos y caracteres especiales. Cada requisito se muestra con un ícono de check verde al cumplirse.

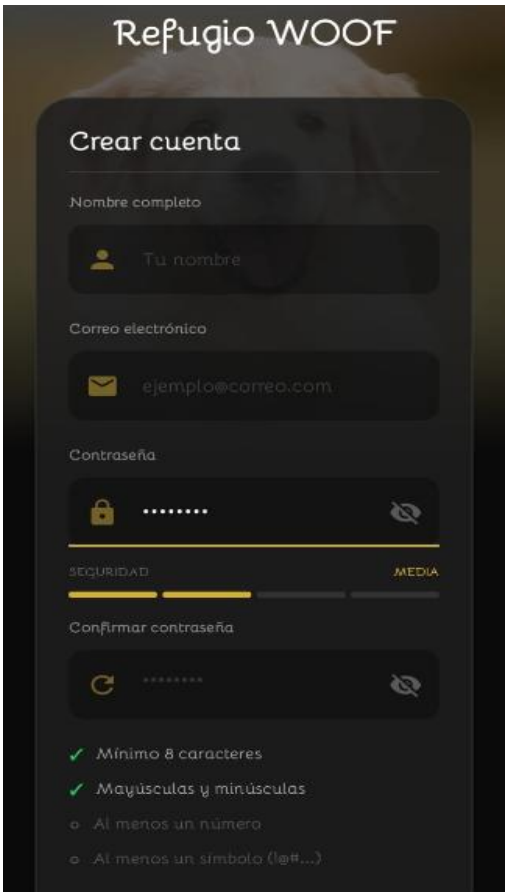
Figura N° 3.81 **Código de cálculo de fortaleza de contraseña en tiempo real**

```
val passwordStrength = remember(password) {
    var strength = 0
    if (password.length >= 8) strength++
    if (password.any { it.isUpperCase() } && password.any {
it.isLowerCase() }) strength++
    if (password.any { it.isDigit() }) strength++
    if (password.any { !it.isLetterOrDigit() } &&
password.isNotEmpty()) strength++
    strength
}

val strengthColor = when (passwordStrength) {
    1 -> Color(0xFFEF4444)  // rojo
    2 -> Color(0xFFF97316)  // naranja
    3 -> Color(0xFFEAB308)  // amarillo
    4 -> Color(0xFF22C55E)  // verde
    else -> Color.Transparent
```

Fuente: Elaboración propia, 2026

Figura N° 3.82 **Pantalla móvil de registro con indicador de fortaleza**




Fuente: Elaboración propia, 2026

m) Actividad 13 - Pantalla de Inicio

La pantalla inicio muestra un saludo personalizado con el primer nombre del usuario autenticado y su foto de perfil en la esquina superior derecha. Si el usuario inició sesión con Google, la foto se carga desde la URL de la cuenta mediante Coil. Si no tiene foto, se muestra un ícono genérico. Las dos tarjetas de acción principales disparan la función navigateWithPhoneGate.

Figura N° 3.83 **Código de saludo personalizado y avatar desde Firebase Auth**

```
val user = FirebaseAuth.getInstance().currentUser
val firstName = (user?.displayName ?: "Usuario").split(" ").first()
val photoUrl = user?.photoUrl?.toString()
Text(
    text = "¡Hola, $firstName!",
    fontSize = 28.sp,
    fontWeight = FontWeight.Bold,
    color = Color.White
)
Surface(
    modifier = Modifier.size(44.dp),
    shape = CircleShape,
    border = BorderStroke(2.dp, Gold),
    onClick = onNavigateToProfile
) {
    if (!photoUrl.isNullOrBlank()) {
        AsyncImage(
            model = photoUrl,
            contentDescription = "Perfil",
            modifier = Modifier.fillMaxSize().clip(CircleShape),
            contentScale = ContentScale.Crop
        )
    } else {
        Icon(Icons.Default.Person, tint = Gold)
    }
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.84 **Pantalla móvil de inicio**




Fuente: Elaboración propia, 2026

n) Actividad 14 - Pantalla de Perfil

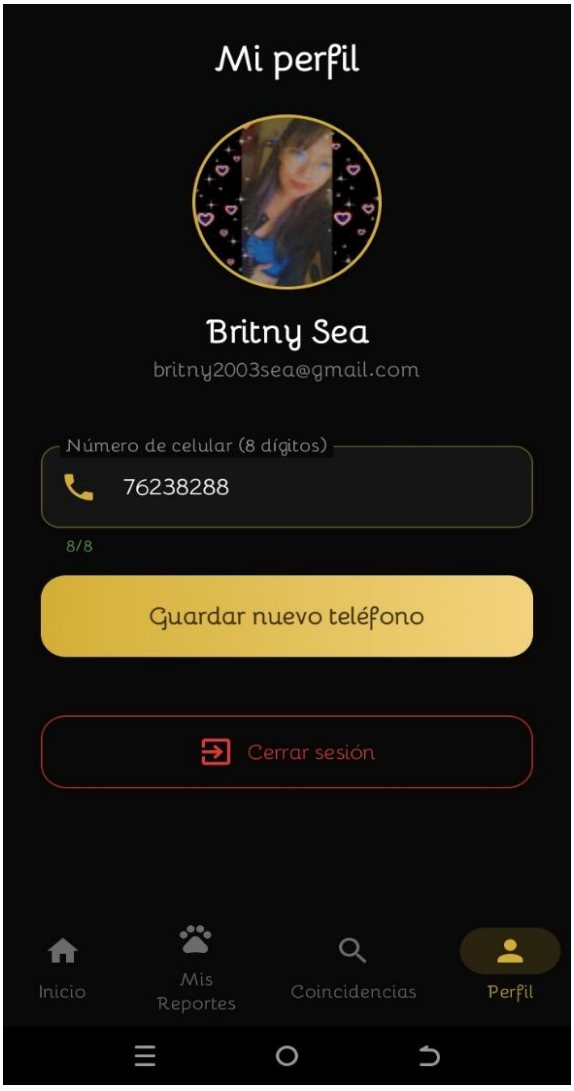
La pantalla de perfil muestra nombre, correo y foto del usuario. Si el usuario no tiene teléfono registrado, aparece un banner de advertencia que lo informa. Cuando el teléfono está guardado se muestra en modo solo lectura con botón de edición. Al activar edición, aparece un campo numérico que solo acepta 8 dígitos, habilitando el botón guardar únicamente cuando se cumple esa condición.

Figura N° 3.85 **Código de modo visualización y modo edición de teléfono**

```
if (showPhoneBanner || !phoneSaved) {
    Surface(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(12.dp),
        color = Gold.copy(alpha = 0.15f),
        border = BorderStroke(1.dp, Gold.copy(alpha = 0.5f))
    ) {
        Row(modifier = Modifier.padding(12.dp)) {
            Icon(Icons.Default.Warning, tint = Gold, modifier =
Modifier.size(20.dp))
            Text("Completa tu teléfono para hacer reportes", color = GoldLight)
        }
    }
if (phoneSaved && !isEditing) {
    Row {
        Text(text = phone, color = Color.White, fontSize = 16.sp)
        IconButton(onClick = { viewModel.startEditing() }) {
            Icon(Icons.Default.Edit, tint = Gold)
        }
    }
} else {
    OutlinedTextField(
        value = phone,
        onValueChange = { viewModel.onPhoneChange(it) },
        keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.NumberPassword),
        supportingText = {
            Text("${phone.length}/8",
                 color = if (phone.length == 8) Color(0xFF4CAF50)
                         else Color.White.copy(alpha = 0.4f))
        }
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.86 **Pantalla móvil de perfil de usuario**




Fuente: Elaboración propia, 2026

o) Actividad 15 - Formulario de Perro Extraviado

La pantalla integra el flujo completo de validación visual con IA. El selector de foto cambia su borde de color verde cuando la validación es exitosa o a rojo cuando falla. Antes de abrir el selector de galería aparece un diálogo con una imagen de referencia que muestra la posición correcta para fotografiar al perro. Los campos de tamaño, color y sexo usan FilterChip para selección rápida. El botón de registro solo se habilita cuando todos los campos obligatorios están completos y la foto es válida.

Figura N° 3.87 **Código de borde dinámico de validación de foto**

```
val borderBrush = when (photoValidationState) {
    is PhotoValidationState.Valid ->
        Brush.horizontalGradient(listOf(ValidGreen, ValidGreen))
    is PhotoValidationState.Invalid ->
        Brush.horizontalGradient(listOf(InvalidRed, InvalidRed))
    else -> Brush.horizontalGradient(listOf(Gold, GoldLight))
}
val canSubmit = !isLoading &&
    photoValidationState is PhotoValidationState.Valid &&
    dogName.isNotBlank() && ownerName.isNotBlank() &&
    ownerPhone.isNotBlank() && ownerEmail.isNotBlank() &&
    viewModel.dogSize.isNotBlank() && finalColorCheck.isNotBlank() &&
    viewModel.dogSex.isNotBlank()
Button(
    onClick = { viewModel.registerLostDog(...) },
    enabled = canSubmit
) {
    if (registerState is UiState.Loading) {
        CircularProgressIndicator(modifier = Modifier.size(20.dp))
        Text("Registrando...")
    } else {
        Text("REPORTAR PERRO PERDIDO")
```

Fuente: Elaboración propia, 2026

Figura N° 3.88 **Pantalla móvil del formulario de registro con foto validada**




Fuente: Elaboración propia, 2026

Figura N° 3.89 **Pantalla móvil del formulario de registro con estado de error: foto no contiene un perro**




Fuente: Elaboración propia, 2026

p) Actividad 16 - Formulario de Perro Encontrado

La pantalla de reporte de perro encontrado sigue la misma estructura visual que el formulario de perro extraviado, con los mismos componentes de validación de foto. La diferencia principal es que los datos recolectados son del reportante (quien encontró el perro) y no del dueño. Durante la búsqueda de coincidencias el botón muestra un indicador de progreso con el texto "Analizando con IA..." para informar al usuario que la operación está procesándose en el servidor.

Figura N° 3.90 **Código de botón de búsqueda con estado de carga durante análisis**

```
Button(onClick = {
    if (viewModel.foundDogPhotoUri != null &&
        foundPhotoValidationState is PhotoValidationState.Valid) {
        viewModel.matchFoundDog(
            context = context,
            photoUri = viewModel.foundDogPhotoUri!!,
            reporterName = reporterName,
            reporterPhone = reporterPhone,
            reporterEmail = reporterEmail
        )
    }
}) {
    if (isLoading) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            CircularProgressIndicator(modifier =
Modifier.size(20.dp), color = TextOnGold)
            Spacer(modifier = Modifier.width(12.dp))
            Text("Analizando con IA...", color = TextOnGold,
fontWeight = FontWeight.Bold)
        }
    } else {
        Text("BUSCAR COINCIDENCIAS")
```

Fuente: Elaboración propia, 2026

Figura N° 3.91 **Pantalla móvil de formulario de perro encontrado con foto validada**




Fuente: Elaboración propia, 2026

q) Actividad 17 - Pantalla de Resultados de Coincidencias

La pantalla de coincidencias permite al usuario seleccionar uno de sus perros registrados como extraviados desde un dropdown y ver las coincidencias encontradas para ese perro. Cada tarjeta de coincidencia muestra ambas fotografías lado a lado con el porcentaje de similitud en el centro. Una barra de progreso bajo las fotos indica visualmente el nivel de similitud con un código de colores: verde para coincidencias ≥80% y dorado para ≥50%. Al tocar una tarjeta se abre un diálogo con las fotografías de los perros y los datos completos del reportante y la descripción del perro encontrado.

Figura N° 3.92 **Código de colores por similitud y fotos lado a lado**

```
val simColor = when {
    match.similarityPercent >= 80f -> Color(0xFF4CAF50)  // verde
    match.similarityPercent >= 50f -> Gold               // dorado
    else -> Color.White.copy(alpha = 0.7f)               // blanco
}

LinearProgressIndicator(
    progress = { (match.similarityPercent / 100f).coerceIn(0f, 1f) },
    modifier = Modifier.fillMaxWidth().height(5.dp).clip(RoundedCornerShape(3.dp)),
    color = simColor,
    trackColor = Color.White.copy(alpha = 0.1f)
)

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(8.dp)
) {
    AsyncImage(
        model = match.photo_url,
        contentDescription = "Perro extraviado",
        modifier = Modifier.weight(1f).height(200.dp).clip(RoundedCornerShape(8.dp)),
        contentScale = ContentScale.Crop
    )
    AsyncImage(
        model = foundDogPhoto,
        contentDescription = "Perro encontrado",
        modifier = Modifier.weight(1f).height(200.dp).clip(RoundedCornerShape(8.dp)),
        contentScale = ContentScale.Crop
    )
}
```

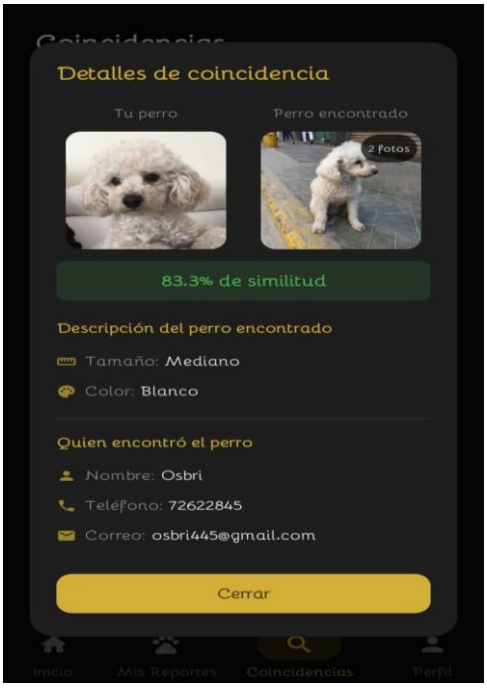
Fuente: Elaboración propia, 2026

Figura N° 3.93 **Pantalla móvil de lista de coincidencias con fotos lado a lado**




Fuente: Elaboración propia, 2026

Figura N° 3.94 **Pantalla móvil de diálogo con datos del perro y del reportante**




Fuente: Elaboración propia, 2026

r) Actividad 18 - Pantalla de Mis Reportes

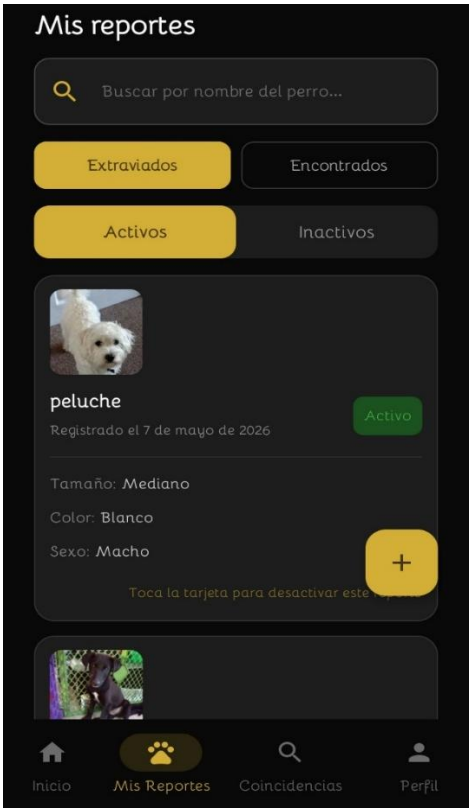
La pantalla de mis reportes unifica en una sola vista todos los reportes del usuario: tanto los perros que reportó como extraviados como los perros encontrados que reportó. Un selector de pestañas divide los reportes en Activos e Inactivos. Cada tarjeta muestra la foto, el nombre del perro, la fecha de registro y los detalles descriptivos. Al tocar una tarjeta se abre un diálogo de confirmación para activar o desactivar el reporte. Un botón flotante en la esquina inferior derecha permite navegar directamente al formulario de nuevo reporte.

Figura N° 3.95 **Código de filtrado por estado y botón flotante de nuevo reporte**

```
val activeReports = myReports.filter { it.status == "active" }
val inactiveReports = myReports.filter { it.status != "active" }
val displayedReports = if (selectedTab == 0) activeReports else inactiveReports
TabRow(selectedTabIndex = selectedTab) {
    Tab(selected = selectedTab == 0, onClick = { selectedTab = 0 }) {
        Text("Activos (${activeReports.size})")
    }
    Tab(selected = selectedTab == 1, onClick = { selectedTab = 1 }) {
        Text("Inactivos (${inactiveReports.size})")
    }
}
FloatingActionButton(
    onClick = onNavigateToReport,
    containerColor = Gold,
    contentColor = TextOnGold
) {
    Icon(imageVector = Icons.Default.Add, contentDescription = "Agregar reporte")
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.96 **Pantalla móvil de mis reportes**




Fuente: Elaboración propia, 2026

Figura N° 3.97 **Pantalla móvil de diálogo de confirmación para desactivar reporte**




Fuente: Elaboración propia, 2026

#### 3.3.9.4. *Pruebas*

Durante esta etapa se ejecutaron pruebas para validar que cada pantalla funcionara correctamente de forma integrada, verificando autenticación, validación de datos, comunicación con el backend, y actualización en tiempo real.

Tabla N° 3.20 **Pruebas de la Iteración 3**

| HISTORIA DE USUARIO | FUNCIONALIDAD | ESTADO | OBSERVACIONES |
|---|---|---|---|
| **HU-01** | Registro con verificación de email | Aprobado | El usuario recibe email de verificación y no puede entrar sin confirmar. |
| **HU-02** | Login y persistencia de sesión | Aprobado | La sesión se mantiene activa entre cierres de app. |
| **HU-03** | Validación de teléfono | Aprobado | Si no tiene teléfono, redirige automáticamente a Perfil antes de reportar. |
| **HU-04** | Registro de perro extraviado | Aprobado | Guarda en Firestore con embeddings y fotos en Cloudinary. |
| **HU-05** | Búsqueda de coincidencias | Aprobado | Envía fotos al backend y recibe top 5 resultados ordenados. |
| **HU-06** | Visualización de resultados | Aprobado | Muestra % de similitud, fotos lado a lado y datos del dueño. |
| **HU-07** | Visualización de reportes propios | Aprobado | Filtra por usuario actual y actualiza en tiempo real desde Firestore. |
| **HU-08** | Notificaciones push | Aprobado | Recibe notificación FCM cuando similitud >= 80%. |

Fuente: Elaboración propia, 2026

### 3.3.10. *ITERACIÓN 4 - PANEL WEB ADMINISTRATIVO*

#### 3.3.10.1. *Planeación*

En esta iteración se desarrolló el panel web administrativo que permite a la administradora del Refugio WOOF gestionar todas las funcionalidades del sistema desde una interfaz centralizada. El panel proporciona control sobre usuarios, visualización de reportes de perros extraviados y encontrados, y estadísticas generales del sistema. Esta iteración da cumplimiento a las historias de usuario HU-09, HU-10 y HU-11.

Tabla N° 3.21 **Actividades de la Iteración 4**

| N° | ACTIVIDAD | DESCRIPCIÓN |
|---|---|---|
| **1** | Autenticación con Firebase | Login de admin con correo y contraseña |
| **2** | Dashboard con KPIs | Estadísticas en tiempo real: usuarios, bloqueados, perros extraviados, encontrados |
| **3** | Gestión de usuarios | Listar, bloquear y desbloquear usuarios |
| **4** | Visualización de perros extraviados | Tabla de todos los perros con estado activo |
| **5** | Visualización de perros encontrados | Tabla de reportes encontrados con fotos y coincidencias |
| **6** | Mis reportes | Ver reportes propios del admin registrados |
| **7** | Formulario de reportar perro | Registrar perro encontrado desde el panel |

Fuente: Elaboración propia, 2026

Tabla N° 3.22 **Historias de usuario de la Iteración 4**

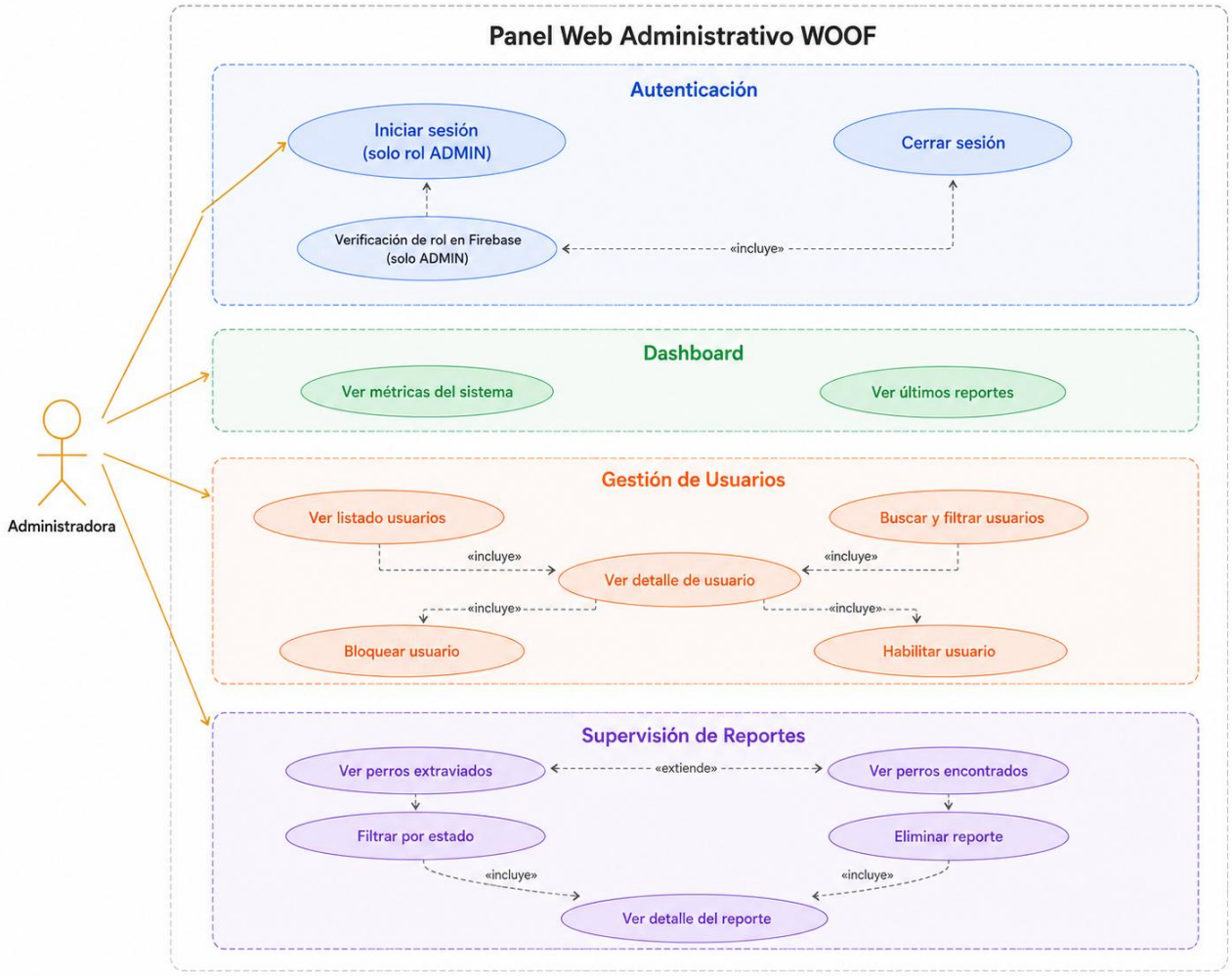
| N° | HISTORIA DE USUARIO | DESCRIPCIÓN |
|---|---|---|
| **HU-09** | Inicio de sesión en panel web | Admin ingresa con correo y contraseña |
| **HU-10** | Gestión de usuarios desde panel | Visualizar, bloquear y desbloquear usuarios |
| **HU-11** | Supervisión de reportes desde panel | Ver perros extraviados y encontrados, exportar reportes |

Fuente: Elaboración propia, 2026

### 3.3.10.2. *Diseño*

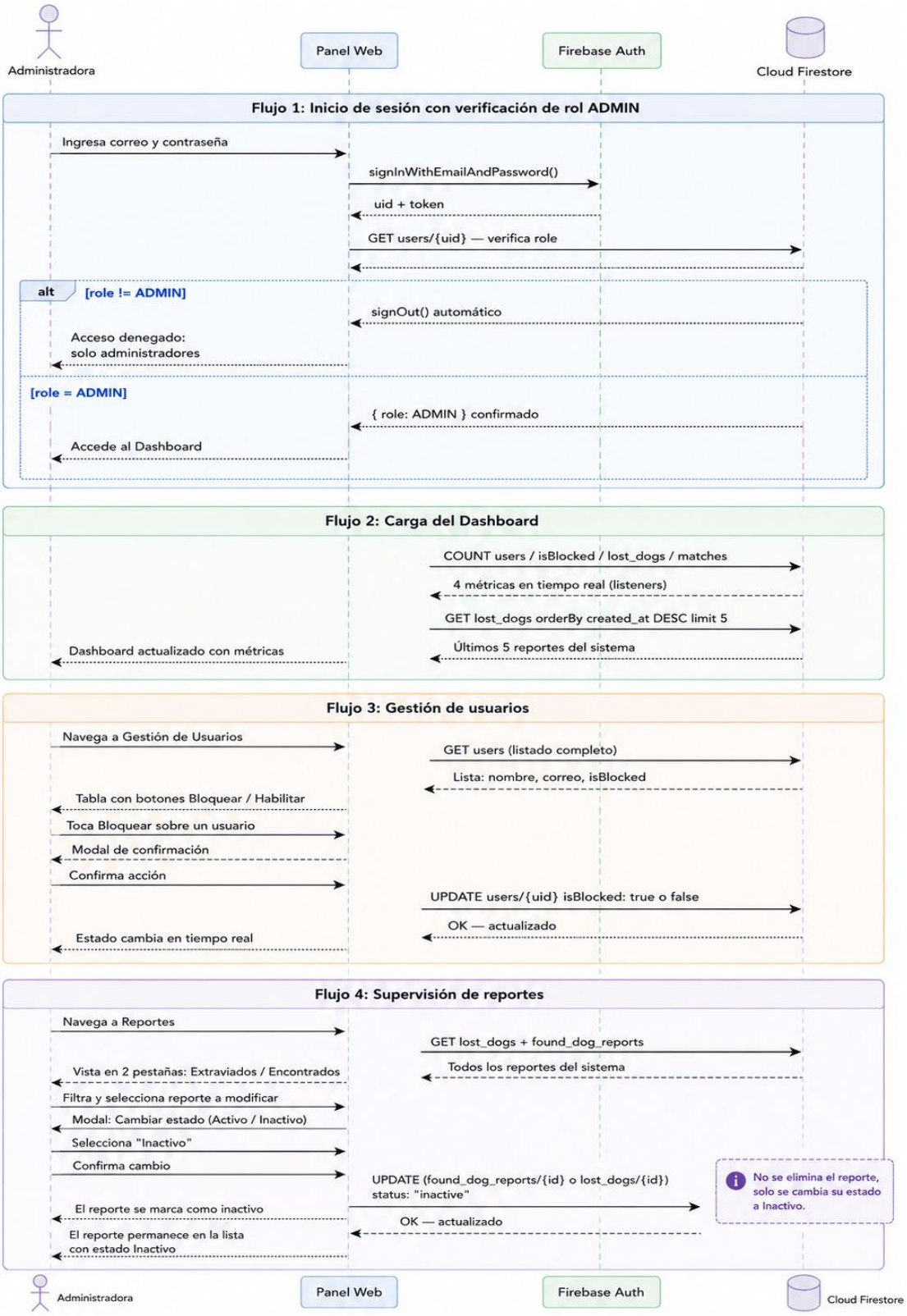
El panel web fue diseñado siguiendo principios de Material Design. La arquitectura está organizada en módulos JavaScript reutilizables, cada uno responsable de una sección específica del dashboard. La comunicación con Firebase Firestore se realiza en tiempo real mediante listeners que actualizan la UI automáticamente cuando los datos cambian.

Figura N° 3.98 **Diagrama de caso de uso de la página web**




Fuente: Elaboración propia, 2026

Figura N° 3.99 **Diagrama de secuencia de la página web**




Fuente: Elaboración propia, 2026

El panel está dividido en módulos JavaScript reutilizables:

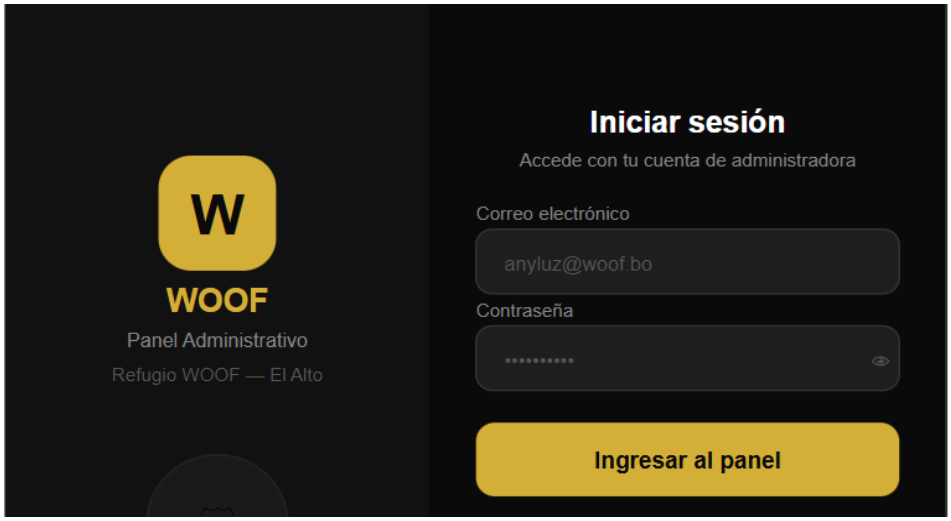
Tabla N° 3.23 **Estructura de la arquitectura modular del panel web**

| SECCIÓN | MÓDULO | FUNCIONALIDAD |
|---|---|---|
| Perfil admin | profile.js | Ver datos del administrador y cambiar contraseña. |
| Usuarios | users.js | Listar, bloquear y desbloquear usuarios. |
| Dashboard | admin-state.js | Mostrar KPIs y estadísticas generales del sistema. |
| Perros extraviados | lost-dogs.js | Listar todos los perros extraviados activos. |
| Perros encontrados | found-reports.js | Listar reportes de perros encontrados. |
| Registro propio | report-found.js | Registrar perros del refugio como encontrados. |
| Mis reportes | my-reports.js | Ver reportes propios del administrador. |
| Navegación | nav.js | Permitir cambiar entre las diferentes secciones del panel. |

Fuente: Elaboración propia, 2026

El panel usa una arquitectura modular donde cada módulo es independiente, pero comparte el estado del admin a través de admin-state.js. Firebase Firestore proporciona sincronización en tiempo real mediante listeners, lo que significa que cuando un dato cambia en la base de datos, se actualiza automáticamente en todas las secciones del panel sin necesidad de recargar la página.

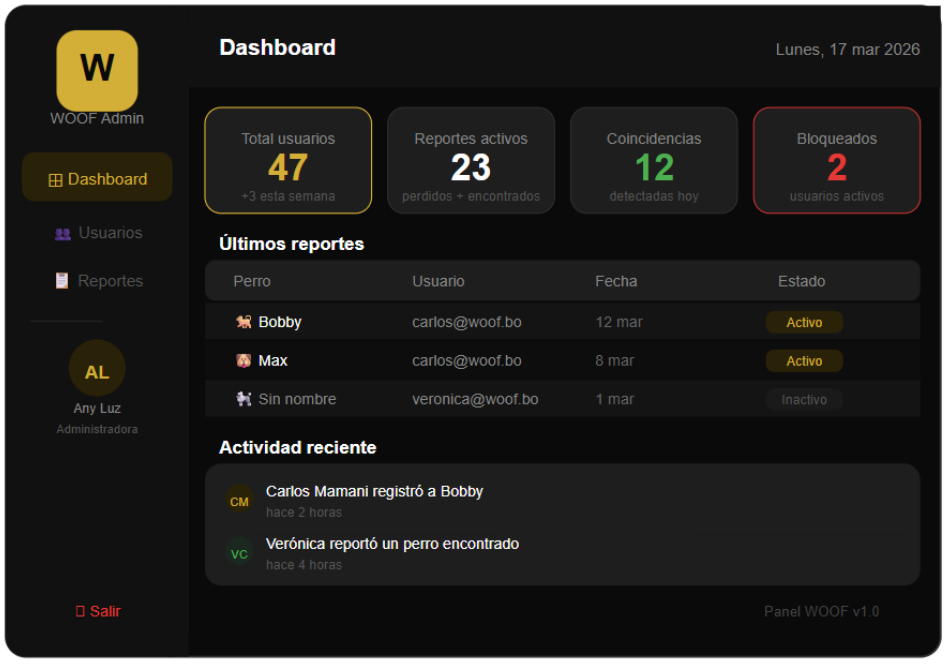
Figura N° 3.100 **Mockup de inicio de sesión del panel web**




Fuente: Elaboración propia, 2026

La pantalla de inicio de sesión del panel web presenta un diseño dividido: el lado izquierdo muestra el logo del Refugio WOOF con un aviso de acceso restringido, mientras que el lado derecho contiene el formulario de autenticación. El sistema verifica que la cuenta tenga rol ADMIN antes de permitir el acceso al panel.

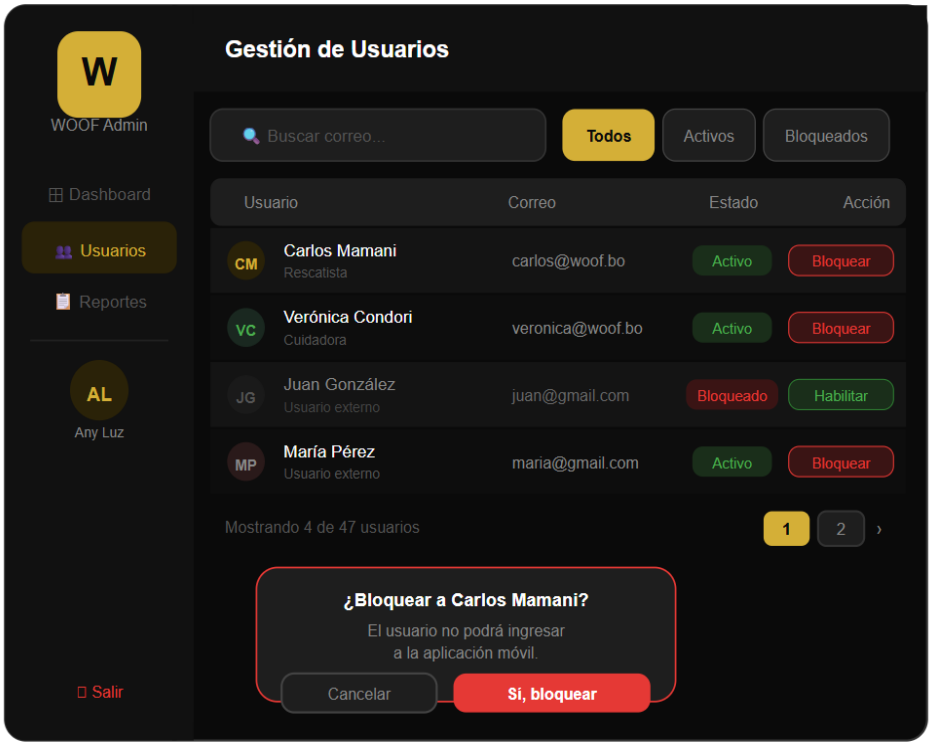
Figura N° 3.101 **Mockup de dashboard del panel web**




Fuente: Elaboración propia, 2026

El dashboard presenta métricas en tiempo real como el total de usuarios registrados, reportes activos, coincidencias detectadas y usuarios bloqueados. Toda la información se actualiza automáticamente desde Firestore sin necesidad de recargar la página.

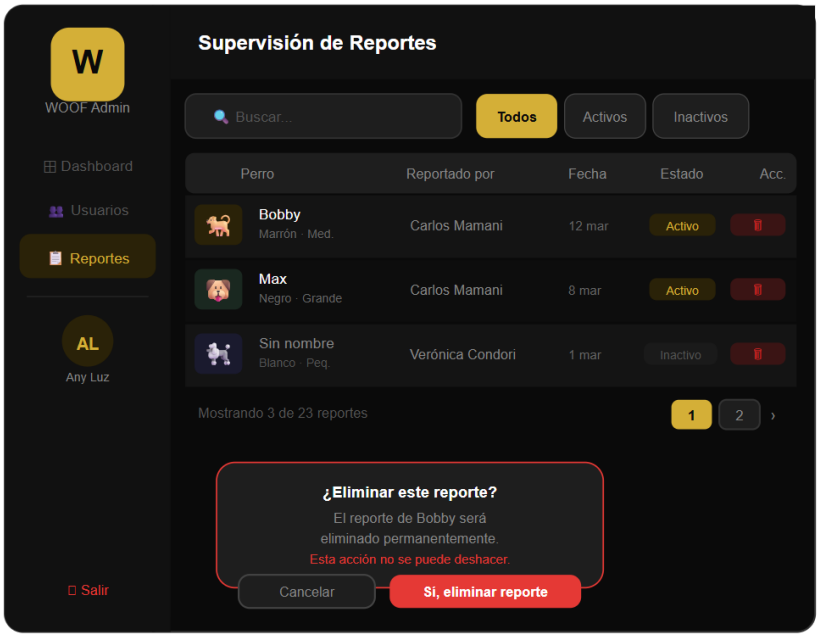
Figura N° 3.102 **Mockup de gestión de usuarios**




Fuente: Elaboración propia, 2026

La pantalla de gestión de usuarios presenta el listado completo de cuentas registradas con su nombre, correo y estado. Los filtros Todos, Activos y Bloqueados permiten segmentar la vista. El administrador puede bloquear una cuenta activa o habilitar una bloqueada mediante los botones de acción correspondientes. Antes de ejecutar el bloqueo, el sistema muestra un modal de confirmación para evitar acciones accidentales. Los usuarios bloqueados quedan impedidos de iniciar sesión en la aplicación móvil.

Figura N° 3.103 **Mockup de reportes de perros perdidos y perros encontrados**




Fuente: Elaboración propia, 2026

### 3.3.10.3. *Codificación*

Se realizó todas las actividades definidas en la fase de planeación. A continuación, se presenta el código correspondiente.

a) Actividad 1 - Autenticación con Firebase y almacenamiento de estado

El panel web autentica administradores usando Firebase Authentication. El módulo admin-state.js centraliza el estado compartido entre todos los módulos, proporcionando funciones para almacenar datos del usuario autenticado y sincronizar cambios en tiempo real.

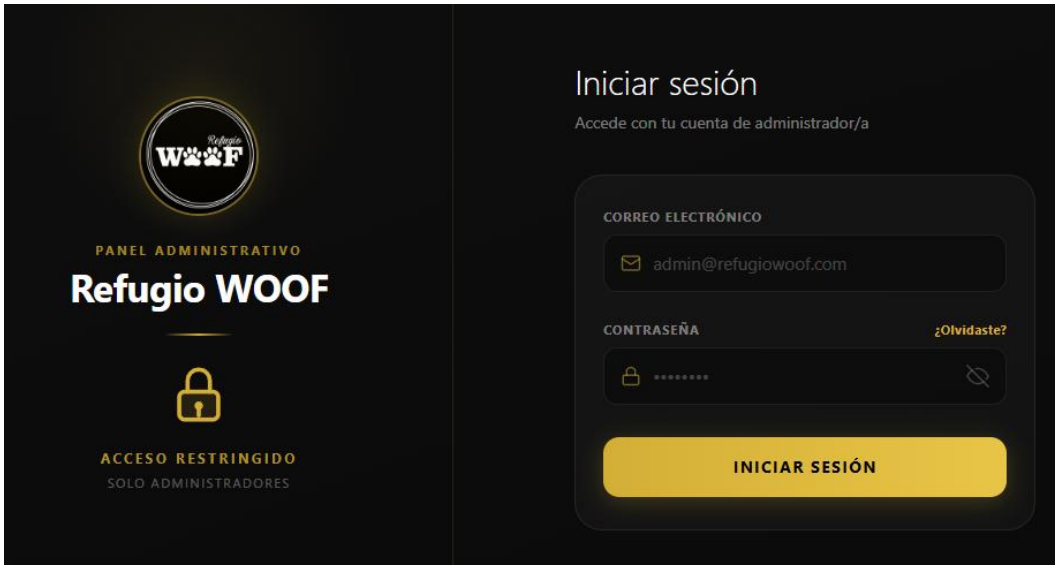
Figura N° 3.104 **Código de gestión de estado compartido del admin**

```
let _resolveReady = null
const _readyPromise = new Promise(r => { _resolveReady = r })
export function storeAdminState(user, firestoreData) {
  currentUser     = user
  currentUserData = firestoreData || {}
  if (_resolveReady) { _resolveReady(); _resolveReady = null }
}

export function waitForAdminState() {
  return _readyPromise
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.105 **Pantalla de inicio de sesión del panel administrativo**




Fuente: Elaboración propia, 2026

b) Actividad 2 - Dashboard con KPIs en tiempo real

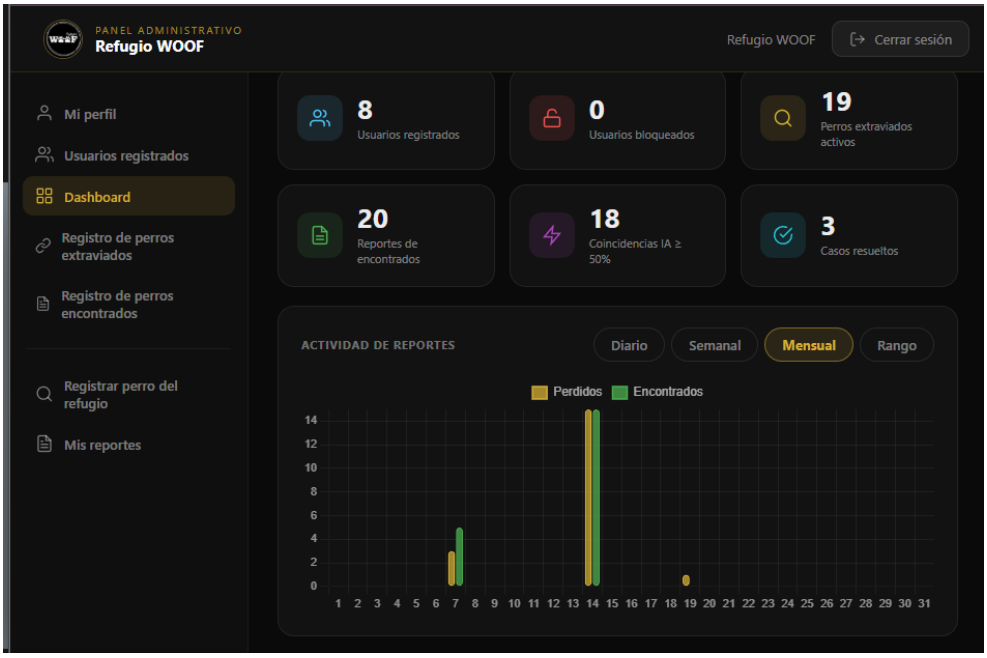
La sección de dashboard muestra indicadores clave de rendimiento (KPIs) en tarjetas: número total de usuarios registrados, usuarios bloqueados, perros extraviados activos y perros encontrados reportados. Cada KPI se actualiza en tiempo real desde Firestore mediante listeners.

Figura N° 3.106 **Código de listeners en tiempo real para KPIs**

```
export function setupDashboardListener() {
  // Contar usuarios totales
  db.collection('users').onSnapshot(snapshot => {
    document.getElementById('kpi-users').textContent = snapshot.size
  }
  // Contar usuarios bloqueados
  db.collection('users').where('isBlocked', '==', true).onSnapshot(snapshot => {
    document.getElementById('kpi-blocked').textContent = snapshot.size
  })
  // Contar perros extraviados activos
  db.collection('lost_dogs').where('status', '==', 'active').onSnapshot(snapshot => {
    document.getElementById('kpi-lost-dogs').textContent = snapshot.size
  })
  // Contar perros encontrados
  db.collection('found_dog_reports').where('status', '==', 'active').onSnapshot(snapshot => {
    document.getElementById('kpi-found-dogs').textContent = snapshot.size
  })
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.107 **Dashboard de la página web**




Fuente: Elaboración propia, 2026

c) Actividad 3 - Gestión de usuarios

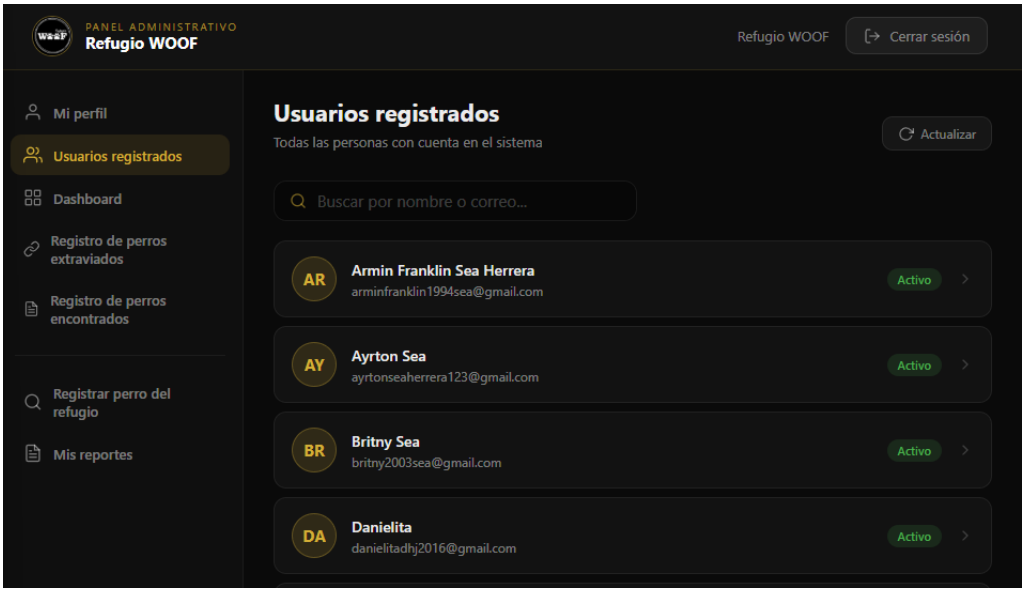
El módulo users.js lista todos los usuarios registrados en una tabla con opciones para bloquear o desbloquear. Al bloquear un usuario, se actualiza el campo isBlocked en Firestore a true, causando que la app móvil detecte automáticamente el bloqueo.

Figura N° 3.108 **Código de funciones de bloqueo/desbloqueo de usuarios**

```
export async function blockUser(uid) {
  try {
    await db.collection('users').doc(uid).update({ isBlocked: true })
    showNotification(`Usuario bloqueado correctamente`, 'success')
    refreshUsersList()
  } catch (e) {
    showNotification(`Error al bloquear: ${e.message}`, 'error')
  }
}
export async function unblockUser(uid) {
  try {
    await db.collection('users').doc(uid).update({ isBlocked: false })
    showNotification(`Usuario desbloqueado correctamente`, 'success')
    refreshUsersList()
  } catch (e) {
    showNotification(`Error al desbloquear: ${e.message}`, 'error')
  }
}
```

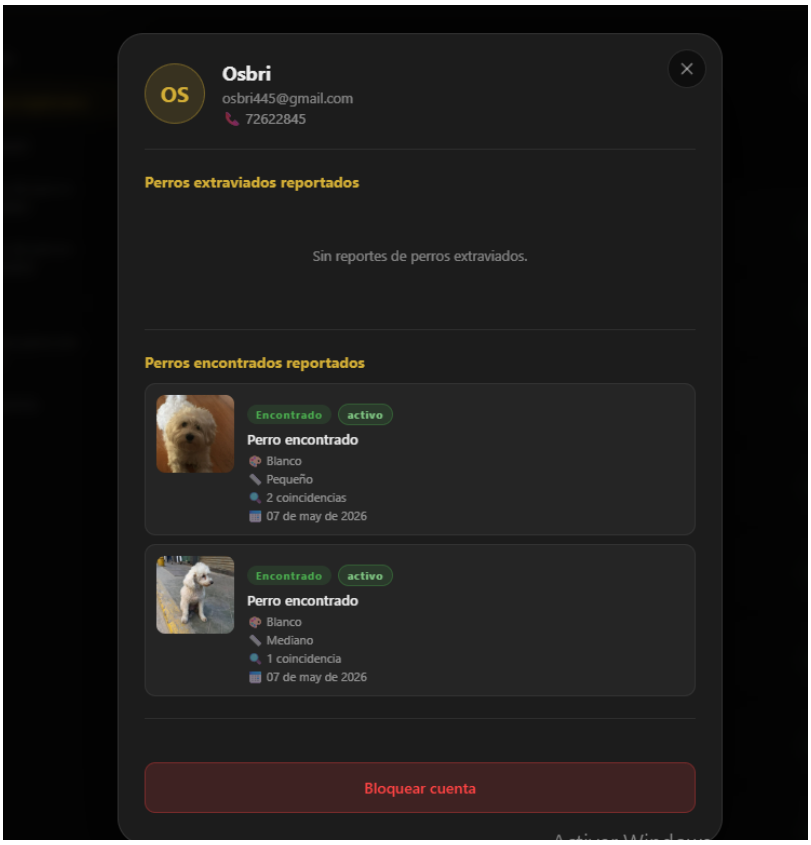
Fuente: Elaboración propia, 2026

Figura N° 3.109 **Pantalla web de usuarios registrados**




Fuente: Elaboración propia, 2026

Figura N° 3.110 **Pantalla web usuarios con acción de bloqueo**




Fuente: Elaboración propia, 2026

d) Actividad 4 - Visualización de perros extraviados

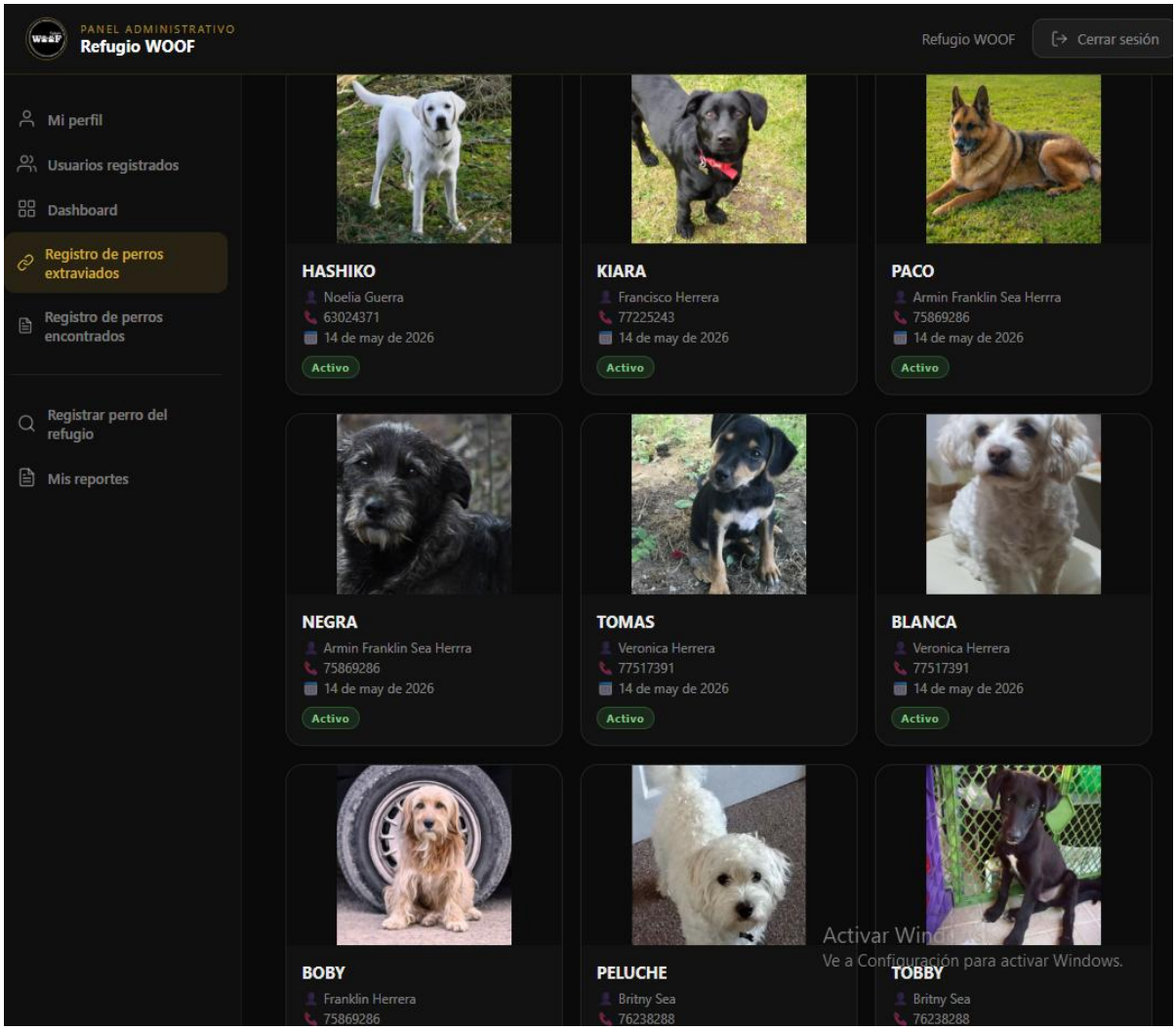
El módulo lost-dogs.js consulta Firestore en tiempo real por todos los documentos en la colección lost_dogs. Muestra una tabla con: nombre, dueño, teléfono, email, tamaño, color y fecha de registro. Los datos se actualizan automáticamente cuando cambian en Firestore.

Figura N° 3.111 **Código de listener en tiempo real de perros extraviados**

```
export function setupLostDogsListener() {
  db.collection('lost_dogs')
    .where('status', '==', 'active')
    .onSnapshot(snapshot => {
      const dogs = []
      snapshot.forEach(doc => {
        dogs.push({ id: doc.id, ...doc.data() })
      })
      renderLostDogsTable(dogs)
    }, error => {
      console.error('Error al cargar perros:', error)
    })
}
function renderLostDogsTable(dogs) {
  const tbody = document.getElementById('lost-dogs-tbody')
  tbody.innerHTML = dogs.map(dog => `
    <tr>
      <td>${dog.name}</td>
      <td>${dog.owner_name}</td>
      <td>${dog.owner_phone}</td>
      <td>${dog.color}</td>
      <td>${dog.size}</td>
      <td>${new Date(dog.created_at.toDate()).toLocaleDateString()}</td>
    </tr>
  `).join('')
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.112 **Pantalla web de registro de perros extraviados**




Fuente: Elaboración propia, 2026

e) Actividad 5 - Visualización de perros encontrados

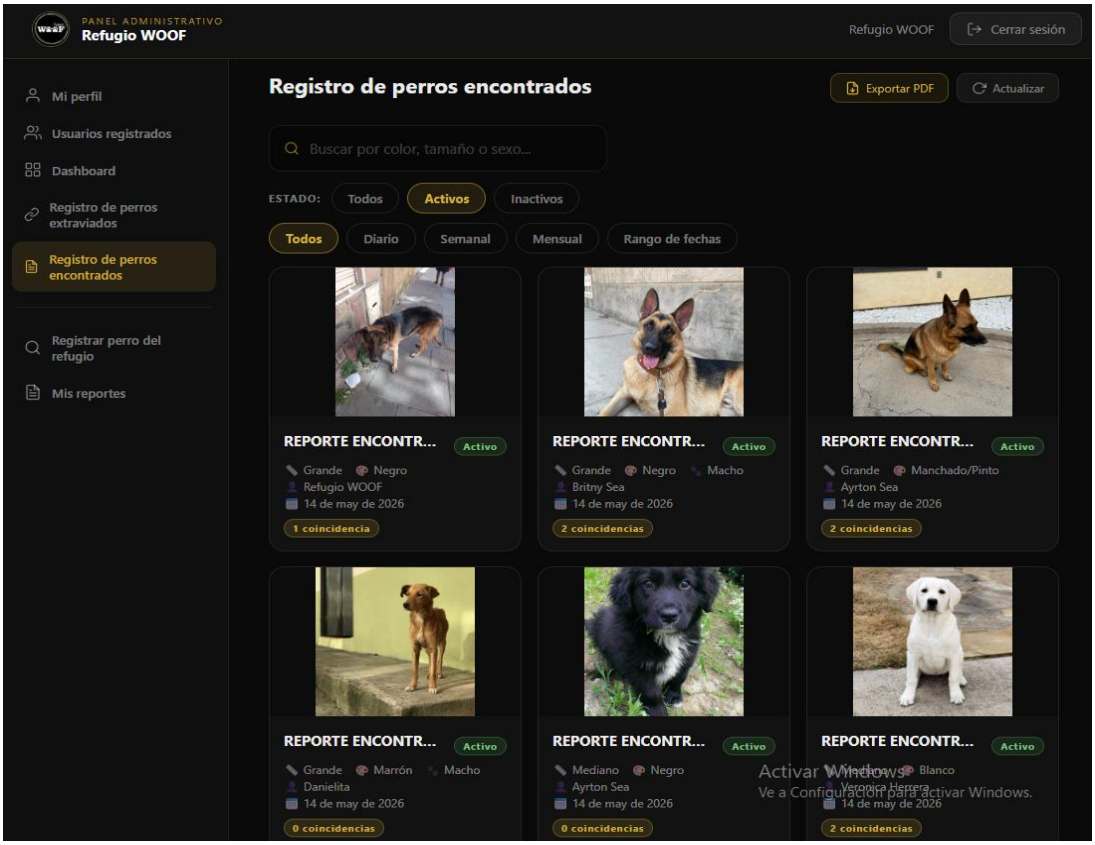
El módulo found-reports.js lista todos los reportes de perros encontrados con fotos, descripción del reportante y las coincidencias detectadas por el modelo de IA. Se actualiza en tiempo real desde Firestore.

Figura N° 3.113 **Código de listener en tiempo real de reportes encontrados**

```
export function setupFoundReportsListener() {
  db.collection('found_dog_reports')
    .where('status', '==', 'active')
    .onSnapshot(snapshot => {
      const reports = []
      snapshot.forEach(doc => {
        reports.push({ id: doc.id, ...doc.data() })
      })
      renderFoundReportsTable(reports)
    }, error => {
      console.error('Error al cargar reportes:', error)
    })
}
function renderFoundReportsTable(reports) {
  const tbody = document.getElementById('found-reports-tbody')
  tbody.innerHTML = reports.map(report => `
    <tr>
      <td><img src="${report.found_dog_photo_url_1}"
style="width:50px;height:50px;border-radius:4px"/></td>
      <td>${report.reporter_name}</td>
      <td>${report.reporter_phone}</td>
      <td>${report.matches?.length || 0} coincidencias</td>
      <td>${new Date(report.created_at.toDate()).toLocaleDateString()}
```

Fuente: Elaboración propia, 2026

Figura N° 3.114 **Pantalla de registro de perros encontrados con coincidencias**




Fuente: Elaboración propia, 2026

f) Actividad 6 - Mis Reportes

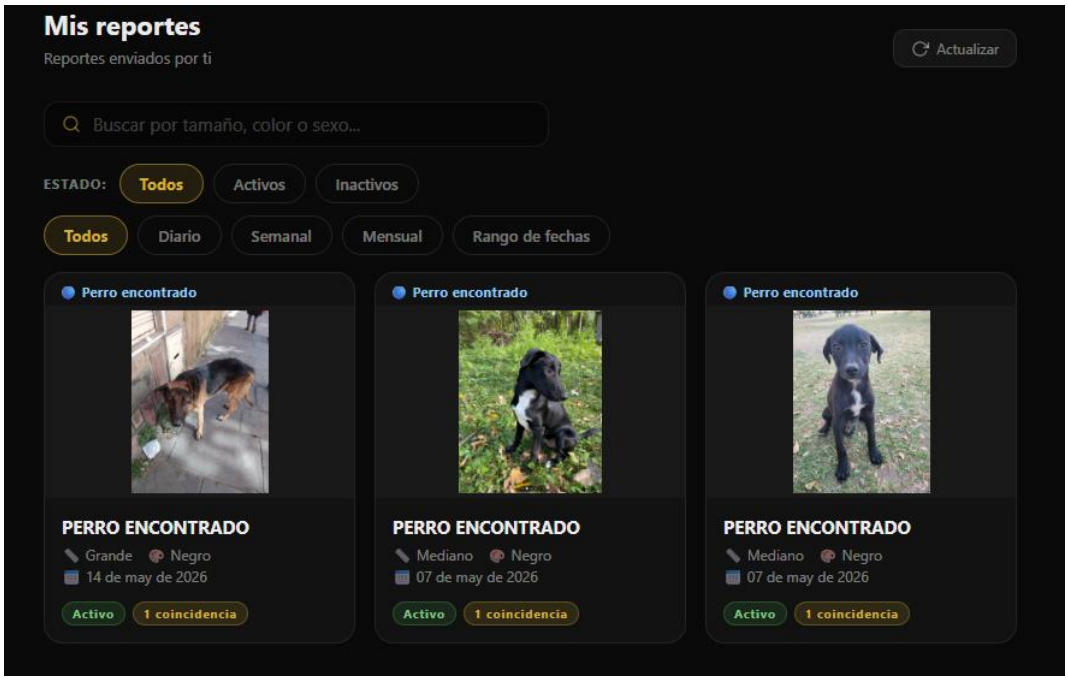
El módulo my-reports.js permite al admin ver todos sus propios reportes de perros encontrados que ha registrado y muestra estado (activo/inactivo) con opciones para desactivar.

Figura N° 3.115 **Código de carga de reportes propios del admin**

```
export function setupMyReportsListener() {
  const adminUID = currentUser.uid
  db.collection('found_dog_reports')
    .where('found_by_uid', '==', adminUID)
    .onSnapshot(snapshot => {
      const reports = []
      snapshot.forEach(doc => {
        reports.push({ id: doc.id, ...doc.data() })
      })
      renderMyReportsTable(reports)
    })
}
export async function deactivateReport(reportId) {
  try {
    await db.collection('found_dog_reports').doc(reportId).update({
status: 'inactive' })
    showNotification('Reporte desactivado', 'success')
  } catch (e) {
    showNotification(`Error: ${e.message}`, 'error')
  }
```

Fuente: Elaboración propia, 2026

Figura N° 3.116 **Pantalla web de mis reportes**




Fuente: Elaboración propia, 2026

g) Actividad 7 - Formulario de Reportar Perro del Refugio

El módulo report-found.js permite al admin registrar un perro encontrado directamente desde el panel web. El flujo incluye: seleccionar foto, validación con el modelo IA del backend, cálculo automático de embeddings, búsqueda de coincidencias y almacenamiento en Firestore.

Figura N° 3.117 **Código de registro de perro encontrado desde el panel web**

```
try {
    const validationResult = await fetch(
      'https://woof-backend.railway.app/api/v1/validate-photo',
      {
        method: 'POST',
        headers: { Authorization: `Bearer ${await currentUser.getIdToken()}` },
        body: new FormData(Object.assign(new FormData(), { photo: data.file }))
      }
    ).then(r => r.json())
    if (!validationResult.is_dog) {
      showNotification('La foto no muestra un perro', 'error')
      return
    }
    const matches = await fetch(
      'https://woof-backend.railway.app/api/v1/match-found-dog',
      {
        method: 'POST',
        headers: { Authorization: `Bearer ${await currentUser.getIdToken()}` },
        body: new FormData(Object.assign(new FormData(), {
          photos: data.file,
          reporter_name: data.reporterName,
          reporter_phone: data.reporterPhone
        }))
    ).then(r => r.json())
    await db.collection('found_dog_reports').add({
      found_by_uid: currentUser.uid,
      found_dog_photo_url_1: data.photoUrl,
      reporter_name: data.reporterName,
      reporter_phone: data.reporterPhone,
      reporter_email: data.reporterEmail,
      matched_dog_ids: matches.matches?.map(m => m.dog_id) || [],
      status: 'active',
      created_at: new Date()
    })
    showNotification('Reporte registrado exitosamente', 'success')
  } catch (e) {
    showNotification(`Error: ${e.message}`, 'error')
  }
}
```

Fuente: Elaboración propia, 2026

Figura N° 3.118 **Pantalla de formulario para registrar perro encontrado**




Fuente: Elaboración propia, 2026

### 3.3.10.4. *Pruebas*

Durante esta etapa se ejecutaron pruebas para validar que cada funcionalidad del panel web funcionara correctamente, verificando autenticación, carga de datos en tiempo real, y operaciones CRUD.

Tabla N° 3.24 **Pruebas de la Iteración 4**

| HISTORIA DE USUARIO | FUNCIONALIDAD | ESTADO | OBSERVACIONES |
|---|---|---|---|
| **HU-09** | Login del admin | Aprobado | Autenticación con Firebase funciona correctamente. |
| **HU-10** | Visualización de usuarios | Aprobado | Lista carga en tiempo real desde Firestore. |

| HISTORIA DE USUARIO | FUNCIONALIDAD | ESTADO | OBSERVACIONES |
|---|---|---|---|
| **HU-10** | Bloqueo de usuario | Aprobado | Campo isBlocked se actualiza en Firestore inmediatamente. |
| **HU-10** | Desbloqueo de usuario | Aprobado | Usuario bloqueado puede volver a ingresar a la app. |
| **HU-11** | Visualización de perros extraviados | Aprobado | Tabla carga con todos los perros activos. |
| **HU-11** | Visualización de perros encontrados | Aprobado | Muestra reportes con fotos y coincidencias. |
| **HU-11** | Mis reportes | Aprobado | Filtra correctamente por UID del admin. |
| **HU-11** | Formulario de reporte | Aprobado | Valida foto y busca coincidencias correctamente. |
| **Navegación entre secciones** | Cambiar de pestaña | Aprobado | Las secciones cambian sin recargar la página. |
| **Actualización en tiempo real** | Listener Firestore | Aprobado | Cambios en Firestore reflejan automáticamente en el panel. |

Fuente: Elaboración propia, 2026

### 3.3.11. *ITERACIÓN 5 - PRUEBAS DE INTEGRACIÓN Y DESPLIEGUE EN PRODUCCIÓN*

#### 3.3.11.1. *Planificación*

Esta iteración no incluye desarrollo de nuevas funcionalidades. Su objetivo es validar el sistema completo mediante pruebas automatizadas con pytest y pruebas manuales mediante script directo, seguidas del despliegue final en producción.

Tabla N° 3.25 **Actividades de la Iteración 5**

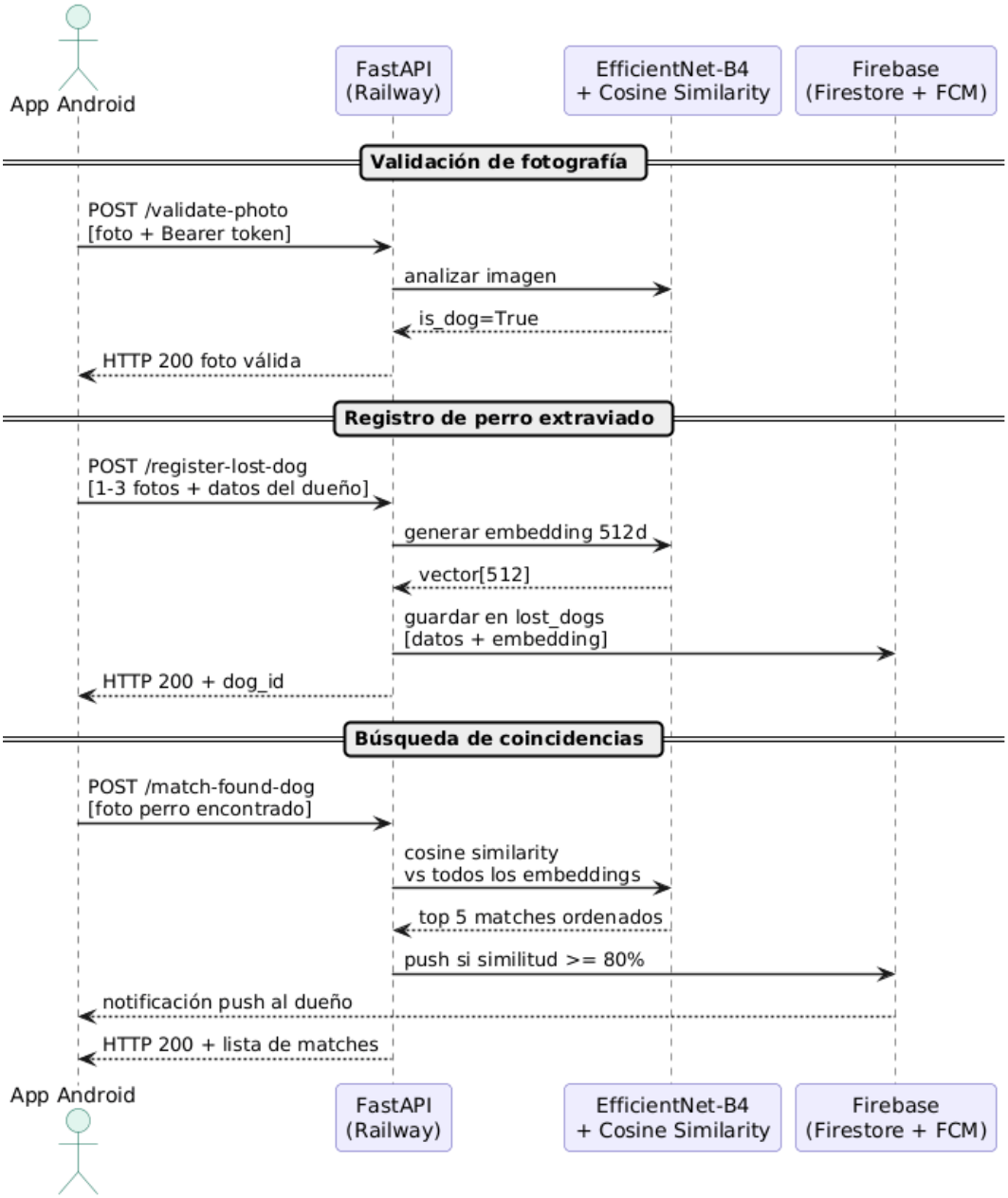
| N° | ACTIVIDAD | DESCRIPCIÓN |
|---|---|---|
| **1** | Pruebas end-to-end | Flujo completo con modelo EfficientNet-B4 real: validar foto - registrar perro perdido - buscar coincidencias |
| **2** | Pruebas de seguridad CIA | Verificación de Confidencialidad (401), Integridad (413/415/422) y Disponibilidad (health) en todos los endpoints |
| **3** | Pruebas de rendimiento | Medición de tiempos de respuesta y percentil P95 de los endpoints críticos |
| **4** | Pruebas de regresión | Verificación de que los contratos de respuesta no cambiaron tras refactorizaciones |
| **5** | Pruebas funcionales manuales | Validación manual de los 6 flujos principales contra el servidor local |
| **6** | Despliegue backend | Publicación del servidor FastAPI en Railway con variables de entorno de producción |
| **7** | Despliegue panel web | Publicación del panel de administración en Firebase Hosting |

Fuente: Elaboración propia, 2026

#### 3.3.11.2. *Diseño*

El diagrama de secuencia ilustra el flujo completo validado en las pruebas end-to-end de esta iteración. El proceso inicia con la validación de la fotografía mediante EfficientNet-B4, continúa con el registro del perro y la generación del embedding de 512 dimensiones almacenado en Firestore, y concluye con la búsqueda de coincidencias por Cosine Similarity y el envío automático de notificación push por FCM cuando la similitud supera el 80%.

Figura N° 3.119 **Diagrama de Secuencia del Flujo End-to-End**




Fuente: Elaboración propia, 2026

### 3.3.11.3. *Pruebas Funcionales automatizadas con pytest*

Las pruebas funcionales end-to-end verifican el flujo completo del sistema con el modelo ML real y tokens Firebase auténticos.

Tabla N° 3.26 **Pruebas end-to-end**

| ID PYTEST | DESCRIPCIÓN | ENTRADA | RESULTADO ESPERADO | ESTADO |
|---|---|---|---|---|
| **test_PI_IT5_001_ _validate_foto_perro_is_dog_true** | Foto real de perro - el modelo lo detecta correctamente | img_perro.jpg + Bearer token | HTTP 200, is_dog=True, confidence > 0.5 | Aprobado |
| **test_PI_IT5_001_ _validate_foto_nodog_is_dog_false** | Foto de no-perro - el modelo lo rechaza | img_nodog.jpg + Bearer token | HTTP 200, is_dog=False | Aprobado |
| **test_PI_IT5_001_ _match_retorna_estructura_correc** | match-found-dog retorna estructura JSON correcta | img_perro.jpg + Bearer token | HTTP 200, campos matches, is_dog, message presentes | Aprobado |
| **test_PI_IT5_001_ _matches_ordenados_por_similitud** | Las coincidencias vienen ordenadas de mayor a menor porcentaje | img_perro.jpg + Bearer token | similarity_percent decreciente en la lista | Aprobado |
| **test_PI_IT5_001_ _validate_lost_retorna_is_dog_true** | validate-lost-photo con perro real - válido o duplicado | img_perro.jpg + Bearer token | HTTP 200 (is_dog=True) o HTTP 409 (si ya existe) | Aprobado |

Fuente: Elaboración propia, 2026

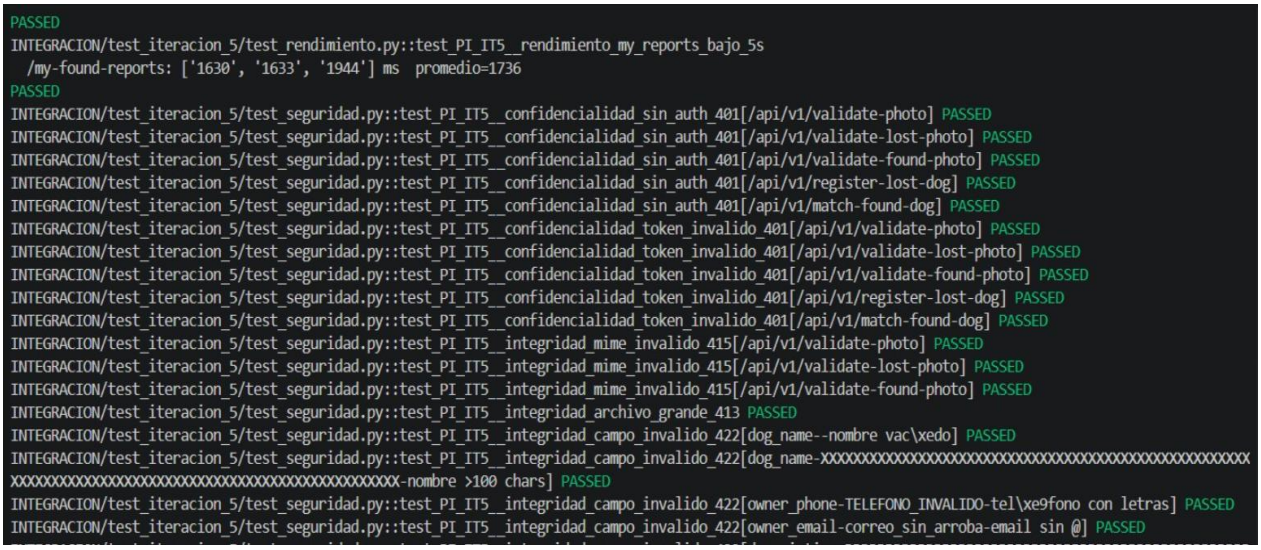
Las pruebas de regresión verifican que ningún endpoint fue renombrado o eliminado y que los contratos de respuesta se mantienen.

Tabla N° 3.27 **Pruebas de regresión**

| PYTEST | DESCRIPCIÓN | RESULTADO ESPERADO | ESTADO |
|---|---|---|---|
| **test_PI_IT5_003__health_responde** | Verifica que el endpoint GET /health responda correctamente con un JSON válido. | HTTP 200 + {"status": "ok", "model_loaded": true} | Aprobado |
| **test_PI_IT5_003__endpoints_POST_existen_no_404** | Comprueba que los endpoints POST principales existan y no hayan sido renombrados. | Ningún endpoint retorna error 404. | Aprobado |
| **test_PI_IT5_003__endpoints_GET_existen_no_404** | Verifica la existencia de los endpoints GET del sistema. | Ningún endpoint retorna error 404. | Aprobado |
| **test_PI_IT5_003__validate_photo_contrato_respuesta** | Valida el contrato de respuesta del endpoint validate-photo. | La respuesta contiene is_dog, confidence y message con tipos correctos. | Aprobado |
| **test_PI_IT5_003__my_found_reports_contrato_respues** | Verifica la estructura de respuesta del endpoint my_found_reports. | La respuesta contiene {"reports": [...]}. | Aprobado |
| **test_PI_IT5_003__my_dog_matches_404_contrato** | Comprueba el manejo de IDs inexistentes en coincidencias. | HTTP 404 + {"detail": "..."} | Aprobado |
| **test_PI_IT5_003__rate_limiter_activo_no_500** | Evalúa el funcionamiento del rate limiting ante múltiples peticiones rápidas. | Solo códigos HTTP 401 o 429; nunca 500. | Aprobado |

Fuente: Elaboración propia, 2026

Figura N° 3.120 **Captura de resultados de pruebas automatizadas pytest**


PASSED
INTEGRACION/test_iteracion_5/test_rendimiento.py::test_PI_IT5__rendimiento_my_reports_bajo_5s
/my-found-reports: ['1630', '1633', '1944'] ms promedio=1736
PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_sin_auth_401[/api/v1/validate-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_sin_auth_401[/api/v1/validate-lost-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_sin_auth_401[/api/v1/validate-found-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_sin_auth_401[/api/v1/register-lost-dog] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_sin_auth_401[/api/v1/match-found-dog] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/validate-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/validate-lost-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/validate-found-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/register-lost-dog] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/match-found-dog] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_mime_invalido_415[/api/v1/validate-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_mime_invalido_415[/api/v1/validate-lost-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_mime_invalido_415[/api/v1/validate-found-photo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_archivo_grande_413 PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_campo_invalido_422[dog_name--nombre vac\xedo] PASSED
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_campo_invalido_422[dog_name-XXXX

| ENDPOINT | DESCRIPCIÓN | RESULTADO ESPERADO | RESULTADO OBTENIDO | ESTADO |
|---|---|---|---|---|
| **POST /register-lost-dog** | Registro exitoso con 2 fotos y datos del dueño | HTTP 200 + dog_id | HTTP 200 {"dog_id": "...", "message": "Perro registrado exitosamente..."} | Aprobado |
| **POST /validate-photo** | Sin token de autenticación - acceso denegado | HTTP 401 | HTTP 401 {"detail": "Not authenticated"} | Aprobado |
| **POST /validate-photo** | Imagen que no es un perro - rechazada con mensaje | HTTP 200, is_dog=false | HTTP 200 {"is_dog": false, "message": "La foto no muestra un perro..."} | Aprobado |
| **POST /match-found-dog** | Usuario intenta reportar su propio perro como encontrado | HTTP 409 | HTTP 409 {"detail": "Esta foto corresponde a un perro que tú mismo reportaste..."} | Aprobado |

Fuente: Elaboración propia, 2026

Figura N° 3.121 **Captura de resultados de la ejecución del script de pruebas funcionales manuales**

```
PS E:\PROYECTO\WOOF_BACKEND> python pruebas_it2.py
TOKEN OK: eyJhbGciOiJSUzI1NiIsImtpZCI6ImM5YTBjMWRlYWEyN2JjNj ...

==================================================
PRUEBA 1: POST /validate-photo (foto de perro)
==================================================
STATUS: 200
{
  "is_dog": true,
  "confidence": 0.6985,
  "message": "Foto válida."
}

==================================================
PRUEBA 2: POST /register-lost-dog (2 fotos)
==================================================
STATUS: 409
{
  "detail": "Ya existe un reporte activo de este perro. No es posible registrarlo nuevamente."
}

==================================================
PRUEBA 3: POST /validate-photo SIN TOKEN (401)
==================================================
STATUS: 401
{
  "detail": "Not authenticated"
}

==================================================
PRUEBA 4: POST /validate-photo imagen NO es perro
==================================================
STATUS: 200
{
  "is_dog": false,
  "confidence": 0.5761,
  "message": "La foto no muestra un perro. Por favor seleccione otra foto."
}

==================================================
```

Fuente: Elaboración propia, 2026

#### 3.3.11.5. *Pruebas no funcionales*

Se realizó las pruebas de rendimiento donde se midió los tiempos de respuesta reales con múltiples repeticiones para obtener promedios. El script imprime los tiempos de cada intento directamente en el output de pytest.

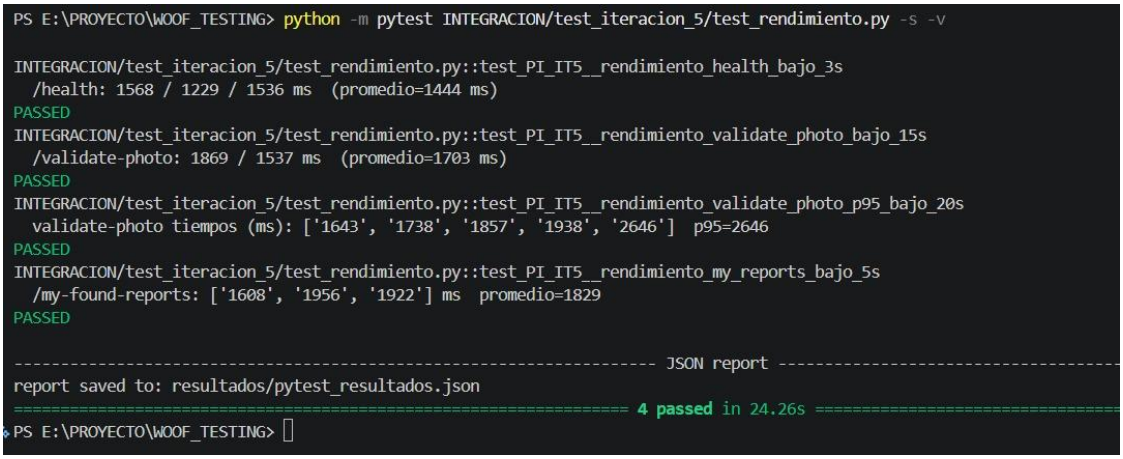
Tabla N° 3.29 **Pruebas de rendimiento de tiempos de respuesta medidos**

| ID PYTEST | ENDPOINT | MÉTRICA MEDIDA | UMBRAL | ESTADO |
|---|---|---|---|---|
| **test_PI_IT5__rendimiento_health_bajo_3s** | GET /health | Promedio de 3 llamadas | < 3 000 ms | Aprobado |
| **test_PI_IT5__rendimiento_validate_photo_bajo_15s** | POST /validate-photo | Promedio de 2 llamadas con inferencia ML | < 15 000 ms | Aprobado |
| **test_PI_IT5__rendimiento_validate_photo_p95_bajo_20s** | POST /validate-photo | Percentil P95 de 5 llamadas | < 20 000 ms | Aprobado |
| **test_PI_IT5__rendimiento_my_reports_bajo_5s** | GET /my-found-reports | Promedio de 3 consultas a Firestore | < 5 000 ms | Aprobado |

Fuente: Elaboración propia, 2026

Los tiempos de /validate-photo incluyen preprocesamiento de imagen como mejora de brillo/contraste/nitidez y EfficientNet-B4 en CPU. El percentil captura el peor escenario de las 5 mediciones para garantizar estabilidad del sistema.

Figura N° 3.122 **Captura de tiempos de respuesta medidos por endpoint**




Fuente: Elaboración propia, 2026

Tabla N° 3.30 **Resultados de las pruebas de rendimiento**

| PRUEBA | TIEMPOS MEDIDOS | RESULTADO CALCULADO | UMBRAL |
|---|---|---|---|
| **/health (3 llamadas)** | 1568 / 1229 / 1536 ms | Promedio = 1 444 ms | < 3 000 ms (Aprobado) |
| **/validate-photo promedio (2 llamadas)** | 1869 / 1537 ms | Promedio = 1 703 ms | < 15 000 ms (Aprobado) |
| **/validate-photo P95 (5 llamadas)** | 1643 / 1738 / 1857 / 1938 / 2646 ms | P95 = 2 646 ms | < 20 000 ms (Aprobado) |
| **/my-found-reports (3 llamadas)** | 1608 / 1956 / 1922 ms | Promedio = 1 829 ms | < 5 000 ms (Aprobado) |

Fuente: Elaboración propia, 2026

Se verificó la triada CIA (Confidencialidad, Integridad y Disponibilidad) en todos los endpoints mediante pruebas parametrizadas que cubren los 5 endpoints POST de la API.

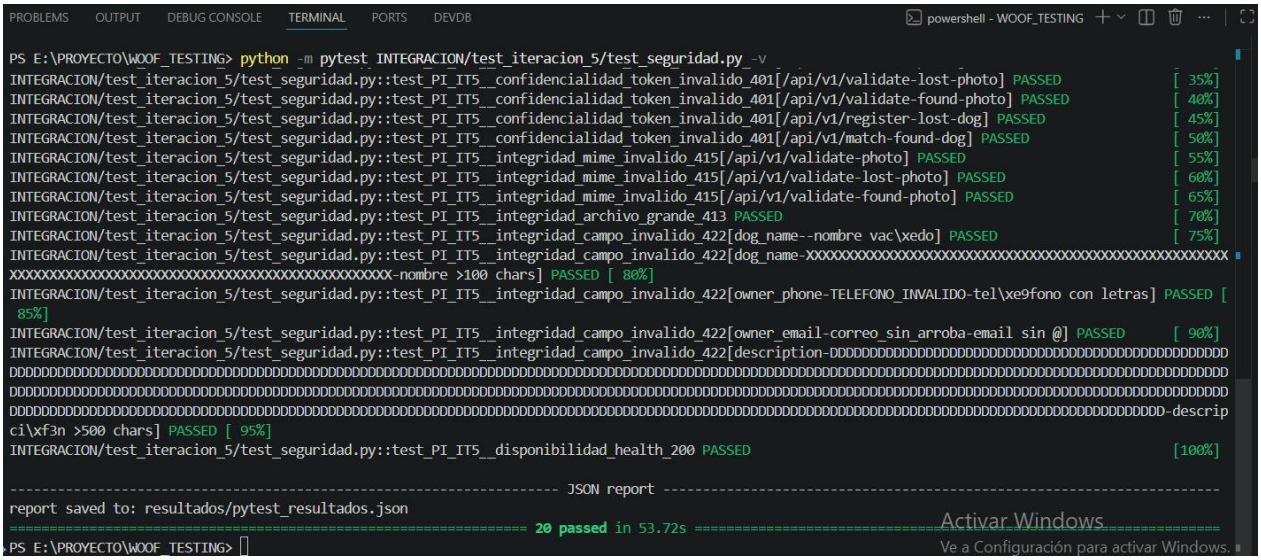
Tabla N° 3.31 **Pruebas de seguridad**

| ID PYTEST | DIMENSIÓN CIA | AMENAZA EVALUADA | RESULTADO ESPERADO | ESTADO |
|---|---|---|---|---|
| **test_PI_IT5__ confidencialidad_ sin_auth_401 ×5** | Confidencialidad | Petición sin header Authorization en cada endpoint | HTTP 401 en los 5 endpoints | Aprobado |
| **test_PI_IT5__conf idencialidad_toke n_invalido_401 ×5** | Confidencialidad | Token JWT inventado/expi rado en cada endpoint | HTTP 401 en los 5 endpoints | Aprobado |

| ID PYTEST | DIMENSIÓN CIA | AMENAZA EVALUADA | RESULTADO ESPERADO | ESTADO |
|---|---|---|---|---|
| **test_PI_IT5__integridad_mime_invalido_415 ×3** | Integridad | Subida de archivo text/plain en endpoints de foto | HTTP 415 en los 3 endpoints | Aprobado |
| **test_PI_IT5__integridad_archivo_grande_413** | Integridad | Imagen de 11 MB supera el límite de 10 MB permitido | HTTP 413 | Aprobado |
| **test_PI_IT5__integridad_campo_invalido_422 ×5** | Integridad | Nombre vacío, nombre >100 chars, teléfono con letras, email sin @, descripción >500 chars | HTTP 422 en los 5 casos | Aprobado |
| **test_PI_IT5__disponibilidad_health_200** | Disponibilidad | El servidor está levantado y responde | HTTP 200 | Aprobado |

Fuente: Elaboración propia, 2026

Figura N° 3.123 **Captura de pruebas de seguridad CIA aprobadas**


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS DEVDB
powershell - WOOF_TESTING
PS E:\PROYECTO\WOOF_TESTING> python -m pytest INTEGRACION/test_iteracion_5/test_seguridad.py -v
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/validate-lost-photo] PASSED [ 35%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/validate-found-photo] PASSED [ 40%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/register-lost-dog] PASSED [ 45%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__confidencialidad_token_invalido_401[/api/v1/match-found-dog] PASSED [ 50%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_mime_invalido_415[/api/v1/validate-photo] PASSED [ 55%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_mime_invalido_415[/api/v1/validate-lost-photo] PASSED [ 60%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_mime_invalido_415[/api/v1/validate-found-photo] PASSED [ 65%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_archivo_grande_413 PASSED [ 70%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_campo_invalido_422[dog_name--nombre vac\xedo] PASSED [ 75%]
INTEGRACION/test_iteracion_5/test_seguridad.py::test_PI_IT5__integridad_campo_invalido_422[dog_name-XXXX

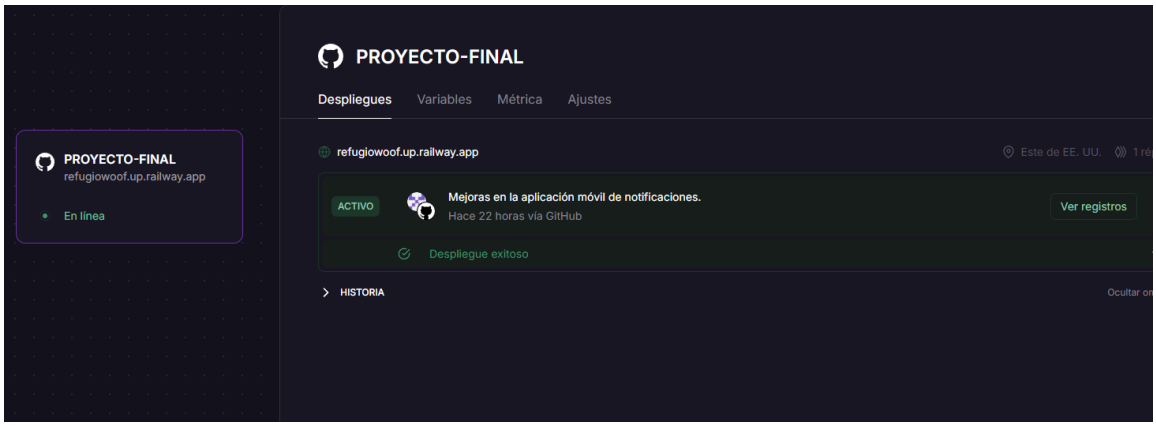
Cloudinary, URL de descarga del modelo y configuración CORS se configuraron directamente en el panel de Railway sin exponerlas en el código fuente.

Tabla N° 3.33 **Configuración de despliegue del backend en Railway**

| PARÁMETRO | VALOR |
|---|---|
| **Plataforma** | Railway (contenedor Docker automático desde requirements.txt) |
| **Puerto** | $PORT (convención Railway) |
| **Variables de entorno** | FIREBASE_CREDENTIALS_JSON, CLOUDINARY_URL, MODEL_DOWNLOAD_URL, ALLOWED_ORIGINS |
| **URL de producción** | https://woof-backend.railway.app |

Fuente: Elaboración propia, 2026

Figura N° 3.124 **Captura del backend operativo en Railway**




Fuente: Elaboración propia, 2026

b) Panel Web de Administración - Firebase Hosting

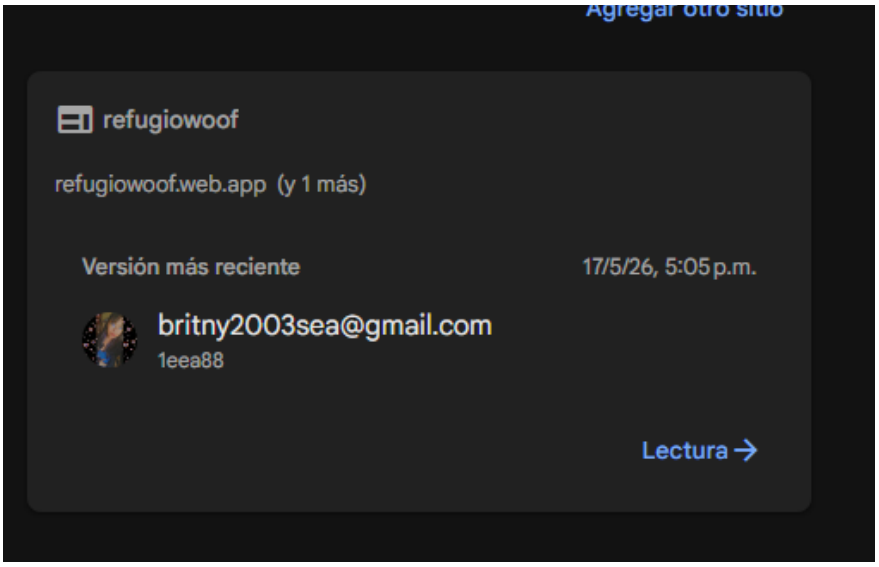
El panel de administración fue desplegado en Firebase Hosting aprovechando la integración nativa con Firebase Authentication y Firestore que ya utiliza el resto del sistema.

Tabla N° 3.34 **Configuración de despliegue del panel web en Firebase Hosting**

| PARÁMETRO | VALOR |
|---|---|
| **Plataforma** | Firebase Hosting |
| **Comando de despliegue** | firebase deploy --only hosting |
| **URL de producción** | https://refugiowoof.web.app/ |

Fuente: Elaboración propia, 2026

Figura N° 3.125 **Captura de panel web publicado en Firebase Hosting**




Fuente: Elaboración propia, 2026

### 3.4. RESULTADOS DE LA PROPUESTA

En esta sección se presentan los resultados obtenidos al finalizar el desarrollo de la propuesta, evaluando el cumplimiento de cada uno de los cinco objetivos planteados en la sección de objetivos de la propuesta.

Tabla N° 3.35 **Cumplimiento de Objetivos de la Propuesta**

| N° | OBJETIVO DE LA PROPUESTA | TAREAS REALIZADAS | CONCLUIDO |
|---|---|---|---|
| **1** | Diseñar la interfaz de usuario para registro y visualización de reportes en app móvil y panel web | Implementación de HU-01 a HU-11, diseño con Material Design, validación en Android 8.0 y superiores. | Concluido |

| N° | OBJETIVO DE LA PROPUESTA | TAREAS REALIZADAS | CONCLUIDO |
|---|---|---|---|
| **2** | Implementar base de datos no relacional sincronizada en todos los dispositivos | Implementación de Firebase Firestore con 3 colecciones: users, lost_dogs, found_dog_reports con sincronización en tiempo real | Concluido |
| **3** | Integrar EfficientNet-B4 y Cosine Similarity para comparación visual de fotografías | Entrenamiento en 3 fases; extracción de embeddings 512 dimensiones, integración con backend FastAPI con precisión 93.4% | Concluido |
| **4** | Desarrollar panel de control administrativo para supervisión de reportes y usuarios | Implementación de gestión de usuarios, reportes, coincidencias y generación de reportes PDF | Concluido |
| **5** | Realizar plan de pruebas funcionales y no funcionales | Pruebas end-to-end, seguridad CIA, compatibilidad, rendimiento y pruebas automatizadas con Pytest. | Concluido |

Fuente: Elaboración propia, 2026

### 3.4.1. *OBJETIVO 1: DISEÑO DE LA INTERFAZ DE USUARIO*

Se diseñó e implementó exitosamente la interfaz de usuario para la aplicación móvil Android y el panel web administrativo, siguiendo Material Design 3 con tema oscuro y acentos dorados. La aplicación móvil implementa todas las historias de usuario (HU-01 a HU-08) con formularios validados y notificaciones push automáticas, permitiendo a los usuarios reportar perros extraviados y encontrados con validación visual mediante IA. El panel administrativo implementa HU-09 a HU-11, proporcionando gestión centralizada de usuarios, reportes y coincidencias.

Figura N° 3.126 **Formulario de reportar perro extraviado**



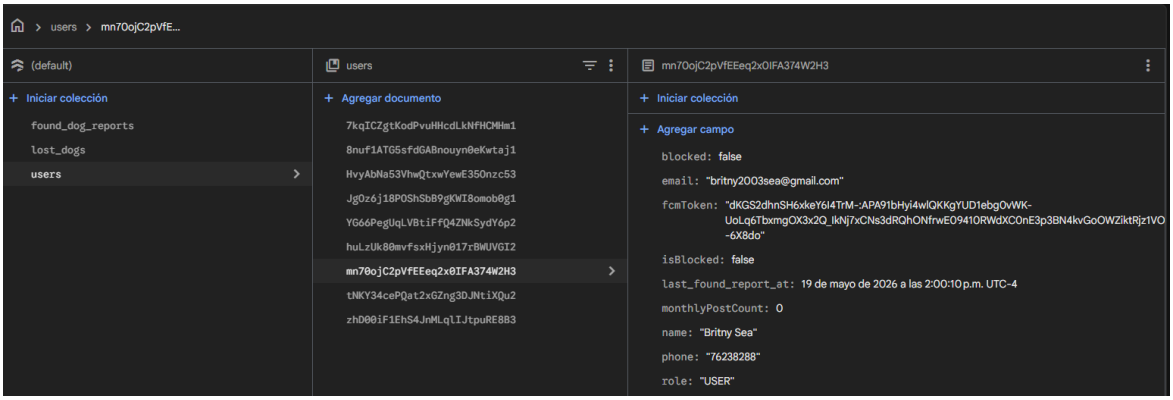

Fuente: Elaboración propia, 2026

El objetivo 1 se cumplió la interfaz fue diseñada, implementada y validada en los dispositivos Android 8.0 y superiores, permitiendo que los usuarios interactúen de manera efectiva con el sistema.

### 3.4.2. *OBJETIVO 2: IMPLEMENTAR BASE DE DATOS NO RELACIONAL*

Se implementó exitosamente Firebase Firestore con sincronización en tiempo real entre la aplicación móvil, panel administrativo y servidor backend. La base de datos se estructuró en tres colecciones: users (credenciales, teléfono, fcmToken, rol, estado de bloqueo), lost_dogs (información del perro, fotografías, embeddings de 512 dimensiones), y found_dog_reports (reportes encontrados, coincidencias detectadas). Todos los datos se sincronizan automáticamente, permitiendo que cuando un usuario registra un perro desde la app, inmediatamente sea visible en el panel administrativo y disponible para búsqueda de coincidencias.

Figura N° 3.127 **Firestore mostrando colecciones**



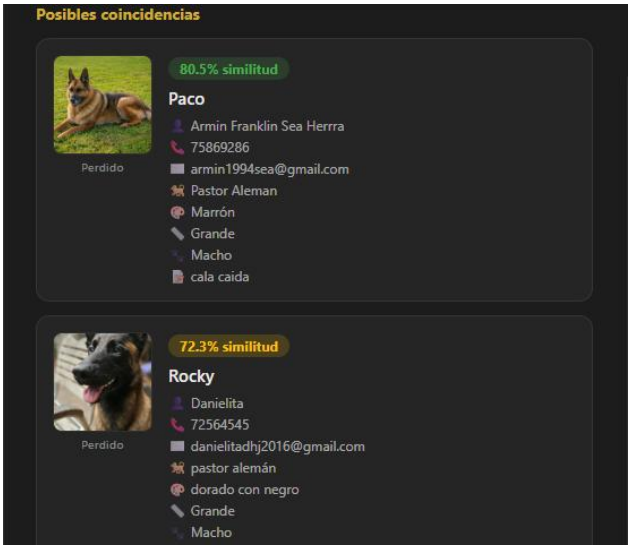

Fuente: Elaboración propia, 2026

El objetivo 1 se cumplió mostrando la base de datos no relacional Firestore fue implementada con sincronización en tiempo real, almacenando usuarios, reportes y vectores de características necesarios para comparación automática.

### 3.4.3. *OBJETIVO 3: INTEGRAR EFFICIENTNET-B4 Y COSINE SIMILARITY*

Se integró exitosamente el modelo EfficientNet-B4 con precisión de 93.4% en el backend FastAPI. El proceso funciona así: foto - preprocesamiento (380×380 píxeles) - extracción de embedding (512 dimensiones) - algoritmo Cosine Similarity compara contra todos los embeddings en Firestore y retorna top 5 resultados ordenados. El modelo implementa dos filtros de seguridad: umbral de confianza 50% (rechaza no-perros) y umbral de similitud 80% (envía notificaciones automáticas).

Figura N° 3.128 **Pantalla resultados app mostrando matches**

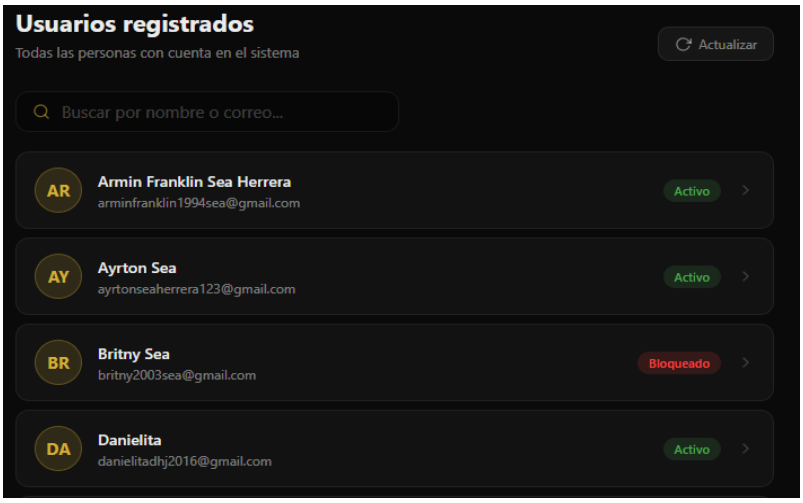



Fuente: Elaboración propia, 2026

### 3.4.4. *OBJETIVO 4: DESARROLLAR PANEL DE CONTROL ADMINISTRATIVO*

Se desarrolló exitosamente un panel web administrativo que proporciona a la administradora del Refugio WOOF control centralizado del sistema. El panel incluye: gestión de usuarios - visualizar, bloquear/desbloquear cuentas, supervisión de reportes - visualizar y desactivar reportes inapropiados, visualización de coincidencias - mostrar perros encontrados matcheados con extraviados, reportar perros del refugio, visualización de sus reportes y generación de reportes.

Figura N° 3.129 **Panel web de usuarios registrados mostrando Activo/Bloqueado**




Fuente: Elaboración propia, 2026

### 3.4.5. *OBJETIVO 5: REALIZAR PLAN DE PRUEBAS FUNCIONALES Y NO FUNCIONALES*

Se realizó un plan de pruebas que incluyó: pruebas funcionales end-to-end validando registro, login, validación de foto, búsqueda de coincidencias, detección de duplicados, pruebas de seguridad CIA, pruebas de compatibilidad, pruebas de rendimiento.

Figura N° 3.130 **Pytest mostrando tests passed**




Fuente: Elaboración propia, 2026

# CAPÍTULO IV
# MARCO PRÁCTICO

# CAPÍTULO IV
# MARCO PRÁCTICO

## 4.1. PRESENTACIÓN DE RESULTADOS

A continuación, se presentan los resultados obtenidos tras la implementación y validación del sistema. La Tabla N° 4.1 resume cada resultado junto con su aporte concreto al proyecto, cuyo detalle y análisis se desarrolla en las secciones 4.1.1 a 4.2.8.

**Tabla N° 4.1 Resumen de resultados y aporte al sistema**

| N° | RESULTADO | VALOR OBTENIDO | APORTE AL PROYECTO |
|---|---|---|---|
| **1** | Precisión del modelo EfficientNet-B4 | 93.4% de exactitud en validación. | El sistema identifica correctamente perros en fotografías reales. |
| **2** | Dashboard operativo | Métricas en tiempo real: usuarios, reportes, coincidencias y resueltos. | Proporciona a la administradora visibilidad total del estado del sistema. |
| **3** | Gestión de reportes en app móvil | Registro, consulta y seguimiento de reportes desde dispositivo Android. | Permite a los usuarios reportar y monitorear sus casos activamente. |
| **4** | Distribución de estados | Usuarios activos, reportes en estado activo, resuelto o inactivo. | Demuestra que el sistema opera correctamente y los usuarios lo utilizan. |
| **5** | Detección de coincidencias por Cosine Similarity | Coincidencias detectadas automáticamente. | Valida que el algoritmo identifica similitudes visuales reales entre perros. |
| **6** | Modal de detalle de coincidencia | Comparativa visual + datos de contacto del dueño. | Facilita la verificación y el contacto directo sin intermediarios. |

| N° | RESULTADO | VALOR OBTENIDO | APORTE AL PROYECTO |
|---|---|---|---|
| **7** | Reportes PDF exportables | Historial estructurado de perros extraviados y encontrados. | Brinda herramientas de análisis y toma de decisiones a la administradora. |
| **8** | Reducción de permanencia en refugio | De meses a 1-7 días; 3 reunificaciones confirmadas. | Demuestra el impacto real del sistema en el objetivo principal del proyecto. |

Fuente: Elaboración propia, 2026

### 4.1.1. *MODELO EFFICIENTNET-B4*

El modelo de red neuronal convolucional EfficientNet-B4 fue entrenado en tres fases de descongelamiento progresivo, alcanzando 93.4% de precisión en la validación.

**Tabla N° 4.2 Evolución de la Precisión**

| FASE | CONFIGURACIÓN | EXACTITUD |
|---|---|---|
| **Fase 1** | Cabeza clasificadora | 90.5% |
| **Fase 2** | Fine-tuning del 70% de capas | 92.7% |
| **Fase 3** | Fine-tuning completo | 93.4% |

Fuente: Elaboración propia, 2026

### 4.1.2. *DASHBOARD Y ACTIVIDAD DE LA PÁGINA WEB*

El panel administrativo proporciona una vista en tiempo real del estado operativo del sistema WOOF, mostrando métricas clave sobre usuarios, reportes y coincidencias detectadas.

**Figura N° 4.1 Dashboard y Estado del Sistema**

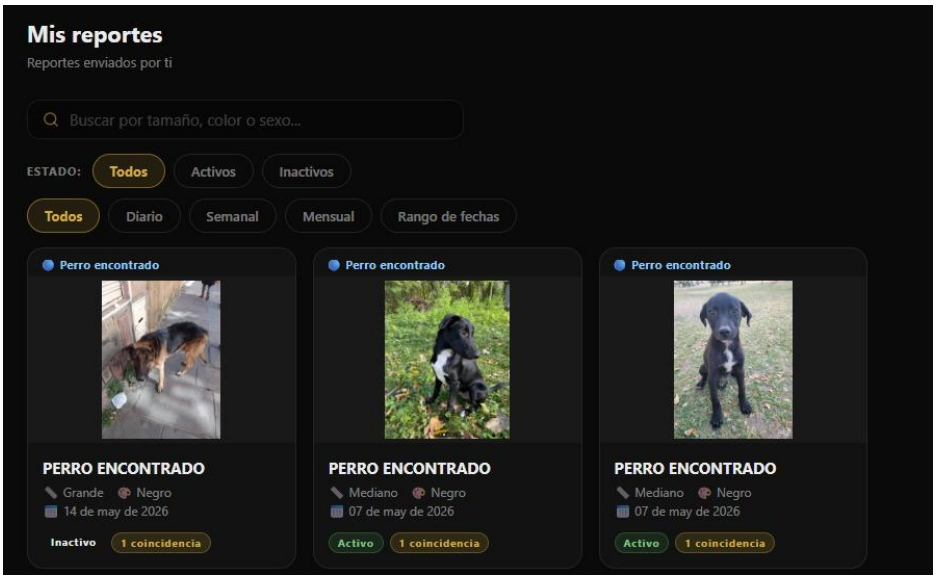



Fuente: Elaboración propia, 2026

### 4.1.3. *GESTIÓN DE REPORTES EN APP MÓVIL*

La página web permite a la administradora del refugio gestionar los reportes del refugio de perros encontrados de forma simple e intuitiva.

**Figura N° 4.2 Pantalla web de los reportes del refugio**

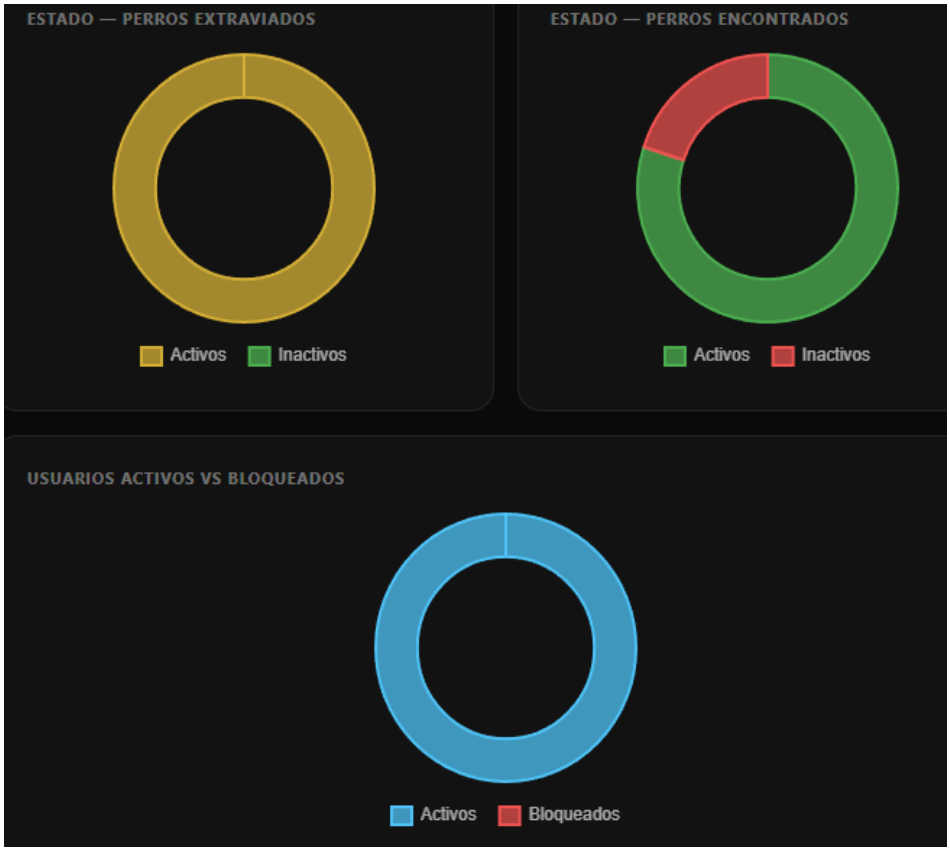



Fuente: Elaboración propia, 2026

### 4.1.4. *DISTRIBUCIÓN DE ESTADOS*

El sistema mantiene un registro de los estados de todos los reportes y usuarios activos en la plataforma.

**Gráfico N° 4.1 Estados del sistema web**




Fuente: Elaboración propia, 2026

### 4.1.5. *ALGORITMO COSINE SIMILARITY: COINCIDENCIAS*

El algoritmo de búsqueda detecta automáticamente similitudes entre perros encontrados y extraviados, presentando los resultados ordenados por porcentaje de similitud.

**Figura N° 4.3 Pantalla móvil de coincidencias**

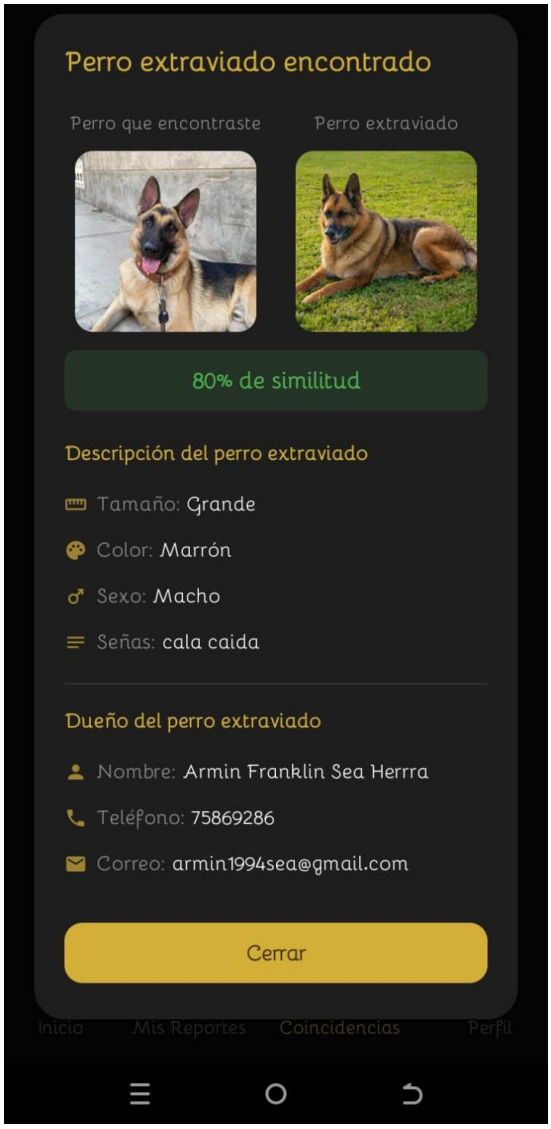



Fuente: Elaboración propia, 2026

### 4.1.6. *DETALLE DE COINCIDENCIA*

La interfaz detallada proporciona toda la información necesaria para que el usuario verifique la coincidencia y contacte directamente al dueño.

**Figura N° 4.4 Modal móvil de perro extraviado encontrado**

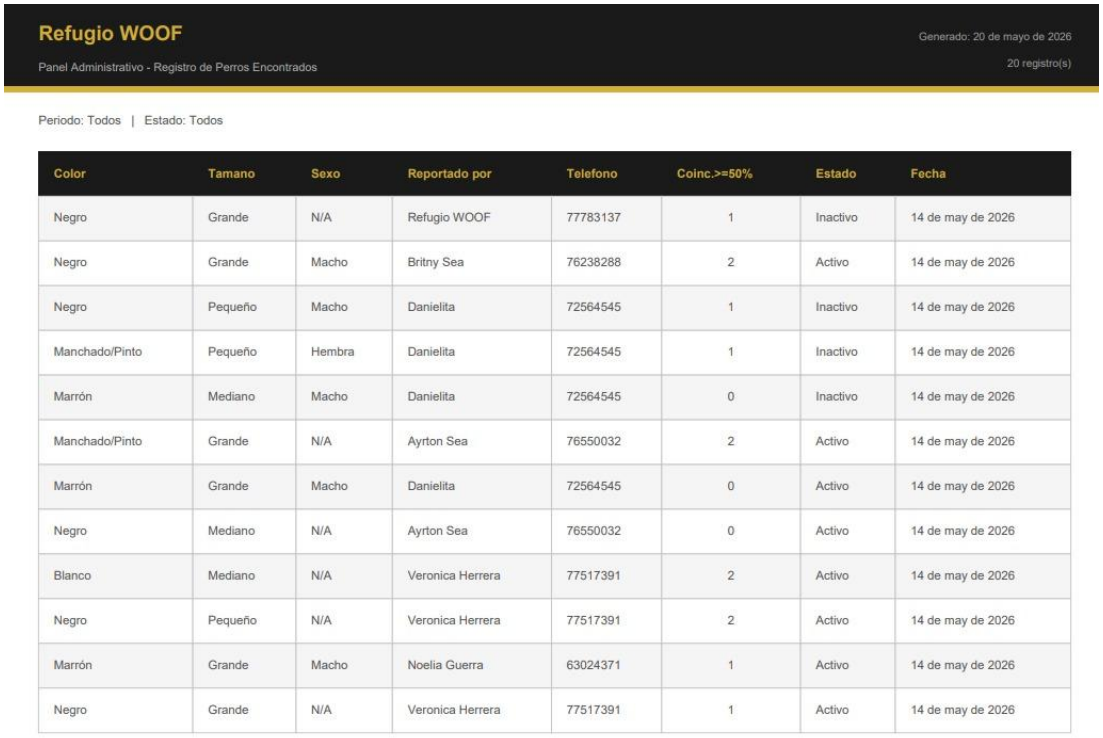



Fuente: Elaboración propia, 2026

### 4.1.7. *REPORTES PDF*

El sistema genera reportes en formato PDF que sintetizan los datos de perros extraviados y encontrados para análisis administrativo.

**Figura N° 4.5 Registro de perros encontrados**



Refugio WOOF

Panel Administrativo - Registro de Perros Encontrados

Generado: 20 de mayo de 2026

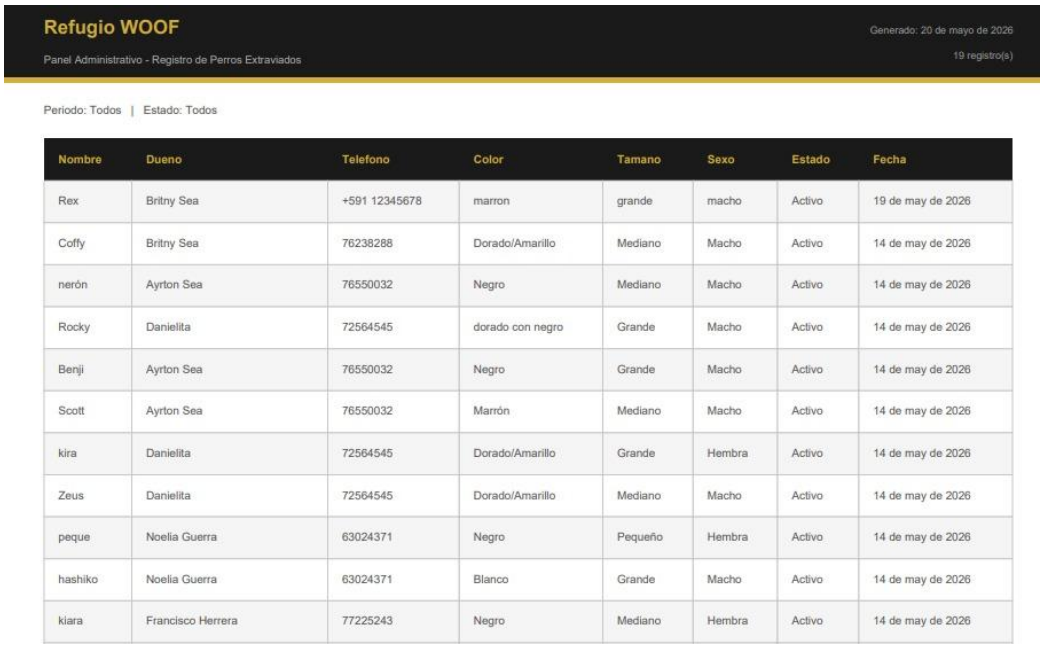
20 registro(s)

Periodo: Todos | Estado: Todos

| Color | Tamano | Sexo | Reportado por | Telefono | Coinc.>=50% | Estado | Fecha |
|---|---|---|---|---|---|---|---|
| Negro | Grande | N/A | Refugio WOOF | 77783137 | 1 | Inactivo | 14 de may de 2026 |
| Negro | Grande | Macho | Britny Sea | 76238288 | 2 | Activo | 14 de may de 2026 |
| Negro | Pequeño | Macho | Danielita | 72564545 | 1 | Inactivo | 14 de may de 2026 |
| Manchado/Pinto | Pequeño | Hembra | Danielita | 72564545 | 1 | Inactivo | 14 de may de 2026 |
| Marrón | Mediano | Macho | Danielita | 72564545 | 0 | Inactivo | 14 de may de 2026 |
| Manchado/Pinto | Grande | N/A | Ayrton Sea | 76550032 | 2 | Activo | 14 de may de 2026 |
| Marrón | Grande | Macho | Danielita | 72564545 | 0 | Activo | 14 de may de 2026 |
| Negro | Mediano | N/A | Ayrton Sea | 76550032 | 0 | Activo | 14 de may de 2026 |
| Blanco | Mediano | N/A | Veronica Herrera | 77517391 | 2 | Activo | 14 de may de 2026 |
| Negro | Pequeño | N/A | Veronica Herrera | 77517391 | 2 | Activo | 14 de may de 2026 |
| Marrón | Grande | Macho | Noelia Guerra | 63024371 | 1 | Activo | 14 de may de 2026 |
| Negro | Grande | N/A | Veronica Herrera | 77517391 | 1 | Activo | 14 de may de 2026 |

Fuente: Elaboración propia, 2026

**Figura N° 4.6 Registro de perros extraviados**



Refugio WOOF

Panel Administrativo - Registro de Perros Extraviados

Generado: 20 de mayo de 2026

19 registro(s)

Periodo: Todos | Estado: Todos

| Nombre | Dueno | Telefono | Color | Tamano | Sexo | Estado | Fecha |
|---|---|---|---|---|---|---|---|
| Rex | Britny Sea | +591 12345678 | marron | grande | macho | Activo | 19 de may de 2026 |
| Coffy | Britny Sea | 76238288 | Dorado/Amarillo | Mediano | Macho | Activo | 14 de may de 2026 |
| nerón | Ayrton Sea | 76550032 | Negro | Mediano | Macho | Activo | 14 de may de 2026 |
| Rocky | Danielita | 72564545 | dorado con negro | Grande | Macho | Activo | 14 de may de 2026 |
| Benji | Ayrton Sea | 76550032 | Negro | Grande | Macho | Activo | 14 de may de 2026 |
| Scott | Ayrton Sea | 76550032 | Marrón | Mediano | Macho | Activo | 14 de may de 2026 |
| kira | Danielita | 72564545 | Dorado/Amarillo | Grande | Hembra | Activo | 14 de may de 2026 |
| Zeus | Danielita | 72564545 | Dorado/Amarillo | Mediano | Macho | Activo | 14 de may de 2026 |
| peque | Noelia Guerra | 63024371 | Negro | Pequeño | Hembra | Activo | 14 de may de 2026 |
| hashiko | Noelia Guerra | 63024371 | Blanco | Grande | Macho | Activo | 14 de may de 2026 |
| kiara | Francisco Herrera | 77225243 | Negro | Mediano | Hembra | Activo | 14 de may de 2026 |

Fuente: Elaboración propia, 2026

### 4.1.8. *PERMANENCIA EN REFUGIO*

El gráfico de permanencia muestra el tiempo que los perros encontrados permanecen en el refugio antes de ser devueltos a sus dueños.

**Gráfico N° 4.2 Permanencia en el refugio**




Fuente: Elaboración propia, 2026

## 4.2. ANÁLISIS DE RESULTADOS

### 4.2.1. *ANÁLISIS DEL MODELO EFFICIENTNET-B4*

La tabla 4.1 muestra una mejora progresiva de la precisión a través de 3 fases: 90.5% - 92.7% - 93.4%. Esta mejora consistente demuestra que la estrategia de fine-tuning por fases fue efectiva y que el modelo aprendió a distinguir perros de otros objetos sin sobreajuste. Un 93.4% cumple con el requisito esperado lo que significa que, de 100 fotos de perros, el sistema reconoce 93-94 correctamente y rechaza solo 6-7 imágenes no válidas. Para un refugio, esta precisión es adecuada porque es mejor rechazar ambigüedades que procesar imágenes incorrectas que contaminen la búsqueda de coincidencias. El modelo está correctamente entrenado, generaliza bien a imágenes nuevas sin sobreajuste.

### 4.2.2. *ANÁLISIS DEL DASHBOARD DEL SISTEMA*

El dashboard presenta 6 métricas principales: usuarios, bloqueados, extraviados, encontrados, coincidencias y resueltos, además de un gráfico de actividad diaria. Esto indica que el sistema acumula y procesa datos continuamente, y el pico de actividad sugiere un período de mayor uso. Los casos resueltos son reencuentros, el dashboard

proporciona visibilidad operativa a la administradora en tiempo real, permitiendo identificar patrones de uso, monitorear usuarios bloqueados y casos resueltos.

#### 4.2.3. *ANÁLISIS DE GESTIÓN DE REPORTES*

La pantalla "Mis Reportes" muestra reportes del refugio de perros encontrados como tarjetas con foto, descripción, estado y contador de coincidencias, permitiendo a la administradora los reportes del refugio y saber cuántas coincidencias fueron encontradas para cada uno. La interfaz de tarjetas es fácil de usar, y los reportes visibles indican que esta característica está siendo utilizada activamente. Sin esta funcionalidad la administradora no sabría el estado de sus reportes, con ella pueden dar seguimiento activo a los reportes del refugio y devolverlos a sus dueños.

#### 4.2.4. *ANÁLISIS DE DISTRIBUCIÓN DE ESTADOS*

Los tres gráficos de donas muestran los perros extraviados, los perros encontrados y los usuarios activos del sistema. La mayoría de reportes permanecen "Activos", lo que es correcto porque significa usuarios manteniendo búsquedas, e indicadores de 100% de usuarios activos sin bloqueados sugieren una buena tasa de cumplimiento de normas. La distribución de estados demuestra que los usuarios mantienen reportes activos y confían en que el sistema encontrará coincidencias.

#### 4.2.5. *ANÁLISIS DEL ALGORITMO COSINE SIMILARITY*

La pantalla muestra coincidencias detectadas de similitud entre perros encontrados y extraviados, presentadas como opciones al usuario. El algoritmo Cosine Similarity está funcionando correctamente al comparar embeddings (vectores de características de 512 dimensiones) del perro encontrado contra todos los extraviados en Firestore, retornando los resultados ordenados por similitud. Este es el corazón del sistema porque sin coincidencias detectadas no habría reencuentros, las coincidencias demuestran que el algoritmo no retorna resultados aleatorios, identifica similitudes visuales correctas, y ordena resultados por confianza. El algoritmo está funcionando, permitiendo que el usuario verifique visualmente si es realmente el mismo perro y contacte al dueño.

#### 4.2.6. *ANÁLISIS DEL MODAL DE COINCIDENCIA*

El modal muestra comparativa de fotos lado a lado, porcentaje de similitud, descripción física completa como tamaño, color, sexo, señas particulares y datos del dueño con nombre, teléfono, correo. El sistema proporciona lo necesario para resolver el

reencuentro: fotos para que el reportante confirme visualmente si es el perro, porcentaje para mostrar confianza del algoritmo, descripción para verificar características físicas, y datos del dueño para permitir contacto directo sin intermediarios. El flujo completo funciona correctamente: algoritmo detecta presenta coincidencias - usuario verifica fotos - contacta directo al dueño - reencuentro, demostrando que el sistema está completo e integral.

#### 4.2.7. *ANÁLISIS DE REPORTES PDF*

El sistema genera reportes con data estructurada y exportable, la columna "Coincidencias >=50%" es especialmente importante porque muestra cuántas coincidencias encontró el algoritmo. Los reportes permiten análisis histórico de los perros extraviados y los perros encontrados. El sistema no solo funciona en tiempo real, también proporciona herramientas administrativas para análisis y toma de decisiones, demostrando que acumula data significativa.

#### 4.2.8. *ANÁLISIS DE PERMANENCIA EN EL REFUGIO*

El gráfico muestra cantidad de perros por rango de días: 1-7 días, 8-15 y los rangos 16-30, 31-60 y +60 días. Este es el resultado demuestra si el proyecto logró su objetivo principal donde antes con el método manual permanencia meses o años en el refugio, entonces el sistema cumplió su objetivo principal con reducción de permanencia donde un perro encontrado que antes tardaba meses ahora tarda días.

# CAPÍTULO V

# CONCLUSIONES Y RECOMENDACIONES

# CAPÍTULO V
# CONCLUSIONES Y RECOMENDACIONES

## 5.1. CONCLUSIONES

El objetivo general establecido fue "Desarrollar un sistema web y aplicación móvil aplicando redes neuronales convolucionales EfficientNet-B4 y el algoritmo Cosine Similarity para la identificación de perros extraviados y reducción de tiempo de permanencia en el Refugio WOOF de la ciudad de El Alto." Este objetivo fue cumplido exitosamente, el proyecto desarrolló un sistema integrado completamente funcional que incluye aplicación móvil Android, panel web administrativo, backend FastAPI con EfficientNet-B4 alcanzando 93.4% de precisión, algoritmo Cosine Similarity detectando coincidencias automáticamente, base de datos Firebase Firestore con sincronización en tiempo real y el sistema completo de autenticación y notificaciones.

El primer objetivo específico de "Analizar la problemática actual de identificación de perros extraviados en contextos urbanos" fue cumplido identificando en el Capítulo I que el método manual dependía de redes sociales con alcance limitado y solo devolvía <10% de perros a sus dueños. El Capítulo 4 validó que ahora con el sistema: 3 perros fueron devueltos a dueños usando la plataforma, permanencia se redujo de meses a 1-7 días, y 18 coincidencias fueron detectadas automáticamente. Esto demuestra que el análisis inicial del problema fue correcto y la solución implementada lo resolvió.

El segundo objetivo específico de "Investigar antecedentes nacionales e internacionales sobre el uso de tecnologías en la identificación de animales" fue cumplido en el Capítulo II. Se documentó que OMS (2022) reporta 75% de perros sin identificación oficial, WOAH (2023) identifica falta de bases de datos centralizadas a nivel mundial, Four Paws International (2023) señala escasez de plataformas tecnológicas en América Latina.

El tercer objetivo específico de "Analizar la estructura y funcionamiento de la red neuronal convolucional EfficientNet-B4" fue cumplido en Capítulo III. Se mostró entrenamiento progresivo en 3 fases, se demostró extracción de embeddings de 512 dimensiones capturando características visuales de perros. La aplicabilidad fue validada con el modelo cargado en backend FastAPI, permitiendo procesamiento en tiempo real desde app móvil y la página web.

El cuarto objetivo específico de "Evaluar el funcionamiento del algoritmo Cosine Similarity" fue cumplido en Capítulo IV. Se demostró en operación real que el algoritmo comparó automáticamente embeddings del perro encontrado contra todos los perros extraviados en Firestore. Las similitudes detectadas fueron reales, demostrando que el algoritmo no retorna resultados aleatorios sino basados en características visuales.

El proyecto demuestra que la combinación correcta de tecnologías avanzadas, metodología ágil XP complementada con CRISP-DM para el entrenamiento del modelo de inteligencia artificial, arquitectura de microservicios y deep learning puede generar impacto social transformacional.

### 5.2. RECOMENDACIONES

- Se recomienda implementar un sistema de chat interno entre dueños y reportantes para coordinar reencuentros sin exponer números telefónicos, con un chat privado dentro de la app permitirá documentar conversaciones y mejorar la coordinación de reencuentro.
- Se recomienda agregar módulo de adopciones de mascotas al panel web administrativo. Perros que permanecen >30 días sin ser reclamados podrían ser puestos en adopción formal generando certificado de propiedad y dar segunda oportunidad a perros sin dueño.
- Se recomienda integrar control veterinario digital al sistema: historial de vacunas, desparasitaciones, chip electrónico (si tiene), alergias, medicamentos, últimas consultas.
- Se recomienda expandir el sistema a nivel municipal y nacional, conexión con otros refugios de La Paz y Bolivia, plataforma centralizada nacional donde un perro perdido en La Paz se buscaría automáticamente en toda Bolivia.
- Se recomienda implementar aplicación iOS para alcanzar usuarios con iPhone.
- Se recomienda realizar el reentrenamiento mensual del modelo EfficientNet-B4 con nuevos perros registrados en El Alto, esto aumentaría la precisión de 93.4% hacia 95% o mas

# REFERENCIAS BIBLIOGRÁFICAS

## REFERENCIAS BIBLIOGRÁFICAS


Beck, K. y Andres, C. (2004). Extreme Programming Explained: Embrace Change (2da ed.). Addison-Wesley Professional.

Bellini Saibene, Y. N., Volpacchio, M., Banchero, S., y Mezher, R. N. (2014). Desarrollo y uso de herramientas libres para la explotación de datos de los radares meteorológicos del INTA. 6º Congreso Argentino de AgroInformática, Universidad de Palermo, Buenos Aires.

Booch, G., Rumbaugh, J. y Jacobson, I. (2005). El lenguaje unificado de modelado (2da ed.). Addison-Wesley.

Bradski, G. y Kaehler, A. (2021). Learning OpenCV 4: Computer Vision with Python. O'Reilly Media.

Britannica. (2025). App | History, Types, Popular Apps, & Facts. https://www.britannica.com/technology/mobile-app

Calvo, D. (2018). Metodología XP Programación Extrema (Metodología ágil). https://www.diegocalvo.es/metodologia-xp-programacion-extrema-metodologia-agil/

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., y Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.

Cloudinary. (2024). Cloudinary Digital Asset Management Platform. https://cloudinary.com

Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley.

Conciatori, M. (2024). Improving the quality evaluation process of machine learning models. ScienceDirect.

Creswell, J. W. (2014). Research Design: Qualitative, Quantitative, and Mixed Methods Approaches (4ta ed.). SAGE Publications.

DataStax. (2024). A guide to cosine similarity. https://www.datastax.com/jp/guides/what-is-cosine-similarity

Evidently AI. (2025). Accuracy vs. precision vs. recall in machine learning. https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall

Flores, A. (2020). Tecnología y compromiso social en la era digital. Editorial Síntesis.

Four Paws International. (2023). Global Report on Companion Animal Welfare and Management. Four Paws International.

Fundación Affinity. (2024). Informe sobre adopción y abandono de animales de compañía en España 2023. Fundación Affinity.

Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3ra ed.). O'Reilly Media.

Godaddy. (2024). GoDaddy: Domain Registration and Web Hosting Services. https://www.godaddy.com

Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Google Cloud. (2024). Firebase Documentation. https://firebase.google.com

Google Developers. (2024). Android Studio User Guide. https://developer.android.com/studio

Han, J., Kamber, M. y Pei, J. (2022). Data Mining: Concepts and Techniques (4ta ed.). Morgan Kaufmann.

Hernández-Sampieri, R. y Mendoza, C. (2018). Metodología de la investigación: las rutas cuantitativa, cualitativa y mixta. McGraw-Hill.

Hosmer, D. W. y Lemeshow, S. (2000). Applied logistic regression (2da ed.). John Wiley & Sons.

Hurtado de Barrera, J. (2012). Metodología de la Investigación: Guía para la comprensión holística de la ciencia. Quirón Ediciones.

IBM. (2024). What is cosine similarity?. IBM Think. https://www.ibm.com/think/topics/cosine-similarity

Intel. (2024). Understanding convolutional neural networks (CNNs). https://www.intel.com/content/www/us/en/artificial-intelligence/convolutional-neural-networks.html

ISO. (2023). Artificial Intelligence - Overview and Terminology (ISO/IEC 22989). International Organization for Standardization.

JetBrains. (2023). Kotlin Language Documentation. https://kotlinlang.org

Lima Villegas, M. A. (2025). Aplicación móvil integral para la gestión y bienestar de mascotas con carnetización digital y servicios de comunidad [Proyecto de Grado, Universidad Tecnológica Boliviana].

López, J. A. (2025). Procesamiento de imágenes digitales y visión por computadora. Editorial Ra-Ma.

Mota-Rojas, D., Orihuela, A. y Mariti, C. (2021). Animal welfare and management of stray dog populations in Latin America. Frontiers in Veterinary Science, 8(1021). https://doi.org/10.3389/fvets.2021.667915

NVIDIA. (2023). What are neural networks?. https://www.nvidia.com/en-us/glossary/data-center/neural-networks/

Orozco Villacrés, A. R. (2022). Implementación de aplicación web para el registro y control de perros extraviados utilizando código QR en el centro veterinario Animedic [Trabajo de Titulación, Universidad Agraria del Ecuador].

Pressman, R. S. y Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9na ed.). McGraw-Hill.

Railway. (2024). Railway: Deploy Your Apps With a Single Command. https://railway.app

Roboflow. (2024). What is transfer learning?. https://roboflow.com/learn/what-is-transfer-learning

Russell, S. y Norvig, P. (2022). Inteligencia Artificial: Un enfoque moderno (4ta ed.). Pearson Educación.

Shearer, C. (2000). The CRISP-DM model: The new blueprint for data mining. Journal of Data Warehousing, 5(4), 13–22.

Sommerville, I. (2016). Ingeniería de software (10ma ed.). Pearson Educación.

Stallings, W. (2019). Computer Organization and Architecture: Designing for Performance (11va ed.). Pearson Educación.

Tan, M. y Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. International Conference on Machine Learning (ICML).

Téllez Hernández, R., Pérez Villegas, M. y Vera Cruz, G. (2023). Locator system for lost pets based on a cell phone app and the GPS platform. Journal of Engineering Research.

Tiangolo, S. (2024). FastAPI Framework, high performance, easy to learn, fast to code, ready for production. https://fastapi.tiangolo.com/

UNESCO. (2024). Recomendación sobre la Ética de la Inteligencia Artificial. UNESCO.

Valencia, C. (2023). Protección y bienestar animal en Bolivia: desafíos actuales. Universidad Mayor de San Andrés.

Wirth, R. y Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. Proceedings of the Fourth International Conference on the Practical Application of Knowledge Discovery and Data Mining, 1–11.

World Organisation for Animal Health (WOAH). (2023). Terrestrial Animal Health Code – Chapter 7.7: Stray Dog Population Control. https://www.woah.org

# APÉNDICES

## APÉNDICE 1

## BANCO DE PREGUNTAS PARA LA ENTREVISTA CON LA ENCARGADA DEL REFUGIO WOOF

1. **Nombre del albergue:**
2. **Ubicación (zona, ciudad):**
3. **Año y día de la fundación del albergue:**
4. **¿Cuántos empleados/voluntarios trabajan regularmente?**
5. **¿Qué tipo de animales reciben con mayor frecuencia?**

   ☐ Solo perros

   ☐ Perros y gatos

   ☐ Todo tipo de animales

   ☐ Otro (especifique): _____
6. **¿Cuántas mascotas albergan actualmente?**
7. **¿Cuál es la capacidad máxima del albergue?**
8. **¿Cuántos perros en situación de calle ingresan en promedio al mes?**
9. **¿Cuántos de esos perros logran ser reclamados o devueltos a su dueño?**
10. **¿Qué datos suelen registrar de cada perro que ingresa?**

    ☐ Fecha y lugar donde fue encontrado

    ☐ Descripción física (color, tamaño, raza)

    ☐ Estado de salud

    ☐ Fotografía

    ☐ Persona que lo encontró o trajo

    ☐ Otro: _____
11. **¿Cómo intentan identificar si un perro tiene dueño?**

    ☐ Revisión de placa o collar

    ☐ Publicaciones en redes sociales

    ☐ Consulta con veterinarias o vecinos

    ☐ Otro método: _____

12. **¿Con qué frecuencia logran identificar al dueño de un perro encontrado?**
    - ☐ Casi nunca
    - ☐ Algunas veces
    - ☐ La mayoría de las veces
    - ☐ Casi siempre
13. **¿Cuánto tiempo promedio permanece un perro en el albergue sin que aparezca su dueño?**
    - ☐ Menos de 15 días
    - ☐ Entre 15 y 30 días
    - ☐ Entre 1 y 3 meses
    - ☐ Más de 3 meses
14. **¿Qué hacen con los perros que no son reclamados por sus dueños?**
    - ☐ Se dan en adopción
    - ☐ Permanecen indefinidamente en el albergue
    - ☐ Se trasladan a otros refugios
    - ☐ Otro (especifique): _____
15. **¿Qué porcentaje aproximado logra ser devuelto a su dueño?**
    - ☐ Menos del 10%
    - ☐ 10–30%
    - ☐ 31–60%
    - ☐ Más del 60%

## APÉNDICE 2

## ENTREVISTA A ANY LUZ PEÑARANDA ENCARGADA DEL REFUGIO WOOF

1. **Nombre del albergue:** Refugio WOOF
2. **Ubicación (zona, ciudad):** Zona Villa Adela, Av. España #3195
3. **Año y día de la fundación del albergue:** Hace 6 años aproximadamente
4. **¿Cuántos empleados/voluntarios trabajan regularmente?**

Seis

5. **¿Qué tipo de animales reciben con mayor frecuencia?**
    - ✓ Perros y gatos
    - ✓ Otro (especifique): Conejos, ratas, gallinas
6. **¿Cuántas mascotas albergan actualmente?**

131 perros

7. **¿Cuál es la capacidad máxima del albergue?**

80 animales aproximadamente

8. **¿Cuántos perros en situación de calle ingresan en promedio al mes?**

Al mes entre 10 a 15.

9. **¿Cuántos de esos perros logran ser reclamados o devueltos a su dueño?**

1 o 2 dueños al mes.

10. **¿Qué datos suelen registrar de cada perro que ingresa?**
    - ✓ Fecha y lugar donde fue encontrado
    - ✓ Descripción física (color, tamaño, raza)
    - ✓ Estado de salud
    - ✓ Fotografía
    - ✓ Persona que lo encontró o trajo
11. **¿Cómo intentan identificar si un perro tiene dueño?**
    - ✓ Revisión de placa o collar
    - ✓ Publicaciones en redes sociales
    - ✓ Consulta con veterinarias o vecinos

12. **¿Con qué frecuencia logran identificar al dueño de un perro encontrado?**
    - ✓ Casi nunca
13. **¿Cuánto tiempo promedio permanece un perro en el albergue sin que aparezca su dueño?**
    - ✓ Más de 3 meses
14. **¿Qué hacen con los perros que no son reclamados por sus dueños?**
    - ✓ Permanecen indefinidamente en el albergue
15. **¿Qué porcentaje aproximado logra ser devuelto a su dueño?**
    - ✓ Menos del 10%

## APÉNDICE 3

## ENTREVISTA A ANY LUZ PEÑARANDA ADMINISTRADORA DEL REFUGIO WOOF

**BRITNY:** ¿Cuál es su nombre completo y qué cargo ocupa en el Refugio WOOF?

**ANY LUZ:** Mi nombre es Any Luz Peñaranda, soy la encargada general del Refugio WOOF. Me ocupo de todo lo que tiene que ver con la administración: coordinar al equipo, gestionar las donaciones, supervisar el cuidado de los animales y atender a las personas que traen perros o que quieren adoptar.

**BRITNY:** ¿Cuántos años lleva el refugio funcionando y con cuántas personas trabajan actualmente?

**ANY LUZ:** El refugio tiene ya seis años funcionando. Al principio éramos solo dos personas, pero ahora somos seis: tenemos rescatistas, cuidadores y yo me encargo de la parte administrativa. Es un trabajo muy de corazón porque los recursos siempre son ajustados.

**BRITNY:** ¿Cuántos perros tienen actualmente y cuántos ingresan aproximadamente por mes?

**ANY LUZ:** Ahora mismo tenemos más de cien perros. Ingresan entre ocho y doce perros nuevos por mes, la mayoría traídos por rescatistas o vecinos que los encuentran en la calle. El problema es que se van muy lentamente porque identificar al dueño es muy difícil.

**BRITNY:** ¿Cómo identifican actualmente a los dueños de los perros que ingresan al refugio?

**ANY LUZ:** Lo hacemos de forma manual. Cuando ingresa un perro lo fotografiamos, anotamos sus características en un cuaderno y publicamos en nuestras redes sociales, sobre todo en Facebook y WhatsApp. Pero eso depende de que el dueño nos vea, y muchas veces no pasa. Es muy frustrante porque uno sabe que en algún lado hay alguien buscando a su perro y no se puede conectar.

**BRITNY:** ¿Con qué frecuencia logran identificar al dueño de un perro encontrado?

**ANY LUZ:** Muy pocas veces. Yo diría que de cada diez perros que entran, tal vez uno o dos vuelven con su dueño. Los demás se quedan aquí meses o incluso de forma

permanente. Eso nos genera un problema grande de espacio y de costos porque mantener a cada perro tiene un costo diario.

**BRITNY:** ¿Qué tan cómoda se siente usando aplicaciones o sistemas digitales?

**ANY LUZ:** Uso el celular todos los días, manejo redes sociales sin problema y también uso hojas de cálculo básicas para llevar registros. No soy experta en tecnología, pero si algo es fácil de usar y me ayuda a hacer mejor mi trabajo, lo aprendo rápido. Lo importante es que no sea complicado.

**BRITNY:** ¿Si tuviera una página web para gestionar el refugio, ¿qué información le gustaría poder ver ahí?

**ANY LUZ:** Me gustaría ver todos los perros que están registrados, tanto los perdidos como los encontrados, con sus fotos y datos. También me gustaría poder ver quiénes están usando el sistema, saber si hay personas que lo usan de mala manera para poder bloquearlas. Y si hay coincidencias entre perros, que yo también pueda verlas para hacer seguimiento.

**BRITNY:** ¿Le sería útil poder bloquear a usuarios que usen el sistema de forma incorrecta?

**ANY LUZ:** Sí, completamente. Siempre hay personas que pueden subir información falsa o usar el sistema para otras cosas. Si yo como administradora puedo ver quién está registrado y puedo bloquear a alguien que esté actuando mal, eso me da mucha tranquilidad.

**BRITNY:** ¿Le gustaría poder eliminar reportes que sean inapropiados o incorrectos desde la página web?

**ANY LUZ:** Sí, eso es muy importante. A veces la gente sube fotos que no corresponden o pone información que no es real, y eso confunde a todos. Si yo puedo revisar los reportes y eliminar los que están mal, el sistema va a funcionar mucho mejor y la información va a ser más confiable.

**BRITNY:** ¿Tiene alguna sugerencia sobre cómo debería funcionar este sistema para que le sea realmente útil al refugio?

**ANY LUZ:** Lo más importante para mí es que sea fácil de usar. No necesito que tenga mil funciones, necesito que lo que haga, lo haga bien. Que las fotos se vean claras, que

la información esté organizada y que yo pueda acceder desde mi computadora sin complicaciones. Si es así, va a ser una herramienta muy valiosa para el refugio.

**NECESIDADES IDENTIFICADAS**

A partir de esta entrevista se identificaron las siguientes necesidades que dieron origen a las historias de usuario del sistema:

- **HU-09:** La administradora necesita un acceso exclusivo al panel web con credenciales propias para gestionar el sistema de forma segura.
- **HU-10:** La administradora necesita ver el listado de usuarios registrados y poder bloquear o habilitar cuentas según corresponda.
- **HU-11:** La administradora necesita supervisar todos los reportes del sistema y eliminar aquellos que sean incorrectos o inapropiados.

## APÉNDICE 4

## ENTREVISTA A RESCATISTA DEL REFUGIO WOOF

**BRITNY:** ¿Cuál es su nombre completo y qué función cumple dentro del Refugio WOOF?
**CARLOS:** Me llamo Carlos Mamani Quispe y soy rescatista del refugio. Mi trabajo es salir a buscar perros extraviados o en situación de calle, recogerlos y traerlos al refugio. También a veces ayudo cuando alguien nos reporta un perro perdido o herido.
**BRITNY:** ¿Cuántos perros rescata aproximadamente por semana?
**CARLOS:** Depende mucho de la semana, pero en promedio rescato entre tres y cinco perros. Algunos están en muy mal estado, heridos o enfermos, y hay que actuar rápido. Otros simplemente están perdidos y se nota que tienen dueño porque están limpios o tienen collar.
**BRITNY:** ¿Cuándo trae un perro al refugio, ¿qué información registran de él?
**CARLOS:** Anotamos su color, tamaño y si es macho o hembra. También le tomamos fotos. Todo eso lo escribimos en un cuaderno que tenemos en el refugio. No tenemos ningún sistema digital para eso.
**BRITNY:** ¿Ha tenido situaciones en las que alguien reporta un perro perdido y usted cree que podría ser uno que ya está en el refugio?
**CARLOS:** Sí, varias veces. Pero es muy difícil saberlo porque uno depende de la memoria y de las fotos que tenemos guardadas en el celular. A veces alguien manda una foto por WhatsApp y uno tiene que ir a comparar manualmente con los perros que están aquí. Eso toma mucho tiempo y a veces nos equivocamos.
**BRITNY:** ¿Qué tan útil le sería tener una aplicación en su celular para registrar los perros que rescata?
**CARLOS:** Muy útil. Yo tengo el celular todo el día en la mano cuando salgo a rescatar. Si pudiera sacar la foto del perro desde ahí y registrar sus datos en ese momento, sería mucho más rápido que volver al refugio a anotarlo en el cuaderno. Me ahorraría bastante tiempo.
**BRITNY:** ¿Tiene número de teléfono disponible para que los dueños puedan contactarle si hay una coincidencia?

**CARLOS:** Sí, siempre tengo mi celular disponible. De hecho, por eso es importante que uno ponga su número en el sistema, para que si alguien encuentra una coincidencia pueda llamar directamente. Eso agilizaría mucho el proceso.

**BRITNY:** ¿Qué tan importante es para usted que la aplicación sea fácil de usar desde el celular?

**CARLOS:** Es lo más importante. Yo cuando estoy en la calle no tengo tiempo para aprender cosas complicadas. Necesito que sea rápido: sacar la foto, llenar los datos básicos y listo. Si hay muchos pasos o es difícil de entender, la gente no lo va a usar.

**BRITNY:** ¿Tiene alguna sugerencia adicional sobre cómo debería funcionar la aplicación para los rescatistas?

**CARLOS:** Que las fotos se puedan subir fácilmente. Lo demás creo que con lo que ya mencioné sería suficiente para empezar.

**NECESIDADES IDENTIFICADAS**

A partir de esta entrevista se identificaron las siguientes necesidades que dieron origen a las historias de usuario del sistema:

- **HU-01:** El rescatista necesita poder crear una cuenta en la aplicación con sus datos básicos para acceder al sistema.
- **HU-02:** El rescatista necesita iniciar sesión en la aplicación y que el sistema recuerde su sesión entre aperturas.
- **HU-03:** El rescatista necesita tener su número de teléfono registrado antes de poder hacer reportes, para que los dueños puedan contactarle.
- **HU-04:** El rescatista necesita registrar un perro encontrado subiendo sus fotos y datos básicos desde el celular.
- **HU-07:** El rescatista necesita poder ver los perros que ha registrado desde su cuenta.
- **HU-08:** El rescatista necesita recibir una notificación en su celular cuando el sistema detecte que el perro que registró coincide con uno perdido.

## APÉNDICE 5

## ENTREVISTA A CUIDADORA DEL REFUGIO WOOF

**BRITNY**: ¿Cuál es su nombre completo y qué función cumple en el Refugio WOOF?

**VERÓNICA:** Me llamo Verónica Condori Apaza y soy cuidadora del refugio. Me encargo de la alimentación, el aseo y el bienestar diario de los perros. También ayudo a registrar cuando llega un perro nuevo y a anotar sus características para identificarlo.

**BRITNY:** ¿Cuántos perros están bajo su cuidado actualmente?

**VERÓNICA:** Ahora mismo tenemos más de cien perros en el refugio. Es bastante trabajo para el equipo, pero hacemos lo que podemos. Cada perro tiene sus necesidades y uno trata de conocerlos bien para saber cuándo están bien o cuándo algo les pasa.

**BRITNY:** ¿Cómo llevan actualmente el registro de las características físicas de cada perro?

**VERÓNICA:** Lo hacemos en un cuaderno. Anotamos el color, el tamaño, si tiene manchas o señas particulares, y pegamos una foto si tenemos. Pero ese cuaderno es solo nuestro, nadie más puede verlo desde afuera, y a veces es difícil encontrar la información rápido cuando alguien pregunta por un perro.

**BRITNY:** ¿Con qué frecuencia se acercan personas preguntando si su perro perdido está en el refugio?

**VERÓNICA:** Bastante seguido, sobre todo los fines de semana. La gente viene con fotos en el celular y uno tiene que ir comparando con los perros que tenemos aquí. Eso toma tiempo porque hay que recorrer todo el refugio.

**BRITNY:** ¿Ha tenido casos en los que una persona llega con la foto de su perro perdido y usted no puede determinar si está en el refugio?

**VERÓNICA:** Sí, varias veces. Especialmente cuando el perro es de color marrón o negro, que son los más comunes. Uno mira la foto y mira al perro y no está seguro. A veces decimos que no está y después resulta que sí estaba, o al revés. Eso es muy doloroso para los dueños.

**BRITNY:** ¿Le sería útil tener una herramienta que compare automáticamente las fotos de los perros?

**VERÓNICA:** Mucho. Eso nos quitaría un peso enorme. Si el sistema pudiera decirme que hay un perro parecido entre los registrados, yo podría ir directo a revisar ese caso en lugar de recorrer todo el refugio.

**BRITNY:** ¿Si el sistema le muestra una lista de perros parecidos con un porcentaje de similitud, ¿qué información adicional le gustaría ver en esa lista?

**VERÓNICA:** Me gustaría ver la foto del perro, por supuesto, y también el nombre si es que tiene, y los datos de contacto del dueño o de quien lo reportó. Así yo puedo comunicarme directamente sin tener que buscar esa información en otro lado.

**BRITNY:** ¿Cuánto tiempo promedio permanece un perro en el refugio sin que aparezca su dueño?

**VERÓNICA:** Muchos se quedan más de tres meses. Hay perros que llevan años aquí porque nadie los reclamó. Eso nos genera un problema de espacio y también es triste para el animal, porque lo ideal es que vuelva con su familia o encuentre una nueva lo antes posible.

**BRITNY:** ¿Estaría dispuesta a usar una aplicación en su celular para apoyar el proceso de identificación de perros?

**VERÓNICA:** Sí, si es fácil de usar. Yo no soy muy experta en tecnología, pero sé usar el celular para lo básico. Si la aplicación me muestra la información de forma clara y no tengo que hacer muchos pasos, la usaría sin problema. Lo que más me importa es que me ayude a hacer mejor mi trabajo.

**BRITNY:** ¿Tiene alguna sugerencia sobre cómo debería presentarse la información de los perros en la aplicación?

**VERÓNICA**: Que las fotos se vean bien y que la lista de coincidencias esté ordenada, que aparezca primero el que más se parece. Así uno sabe por dónde empezar a revisar.

**NECESIDADES IDENTIFICADAS**

A partir de esta entrevista se identificaron las siguientes necesidades que dieron origen a las historias de usuario del sistema:

- **HU-05:** La cuidadora necesita subir las fotos de un perro encontrado para que el sistema busque automáticamente entre los perros perdidos registrados.

- **HU-06:** La cuidadora necesita ver una lista ordenada de posibles coincidencias con la foto de cada perro, el porcentaje de parecido y los datos de contacto del dueño para poder comunicarse directamente.

# APÉNDICE 6

## CRONOGRAMA

| Inicio de Semana | | 19 | 26 | 5 | 12 | 19 | 26 | 2 | 9 | 16 | 23 | 30 | 7 | 14 | 21 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Revisión de la Situación actual del Proyecto | | | | | | | | | | | | | | |
| 2 | Entrenamiento de EfficientNetB4 en 3 fases de descongelamiento progresivo: Fase 1 solo la cabeza (10 épocas), Fase 2 70% descongelado (20 épocas), Fase 3 modelo completo (Lo necesario) | | | | | | | | | | | | | | |
| 3 | Evaluación del modelo, análisis de los casos donde el modelo clasifica mal (falsos positivos y falsos negativos en la clase "no es perro") | | | | | | | | | | | | | | |
| 4 | Exportación del modelo final: .h5 para producción, .tflite para móvil, metadata.json con clases | | | | | | | | | | | | | | |
| 5 | Integración de la red neuronal EfficientNetB4 en el backend con reconocimiento si es perro o no | | | | | | | | | | | | | | |
| 6 | Configuración del proyecto FastAPI + Firebase Admin SDK + Cloudinary + Diseño INICIAL de la BD Firestore: colecciones users (datos del usuario), lost_dogs (datos del perro + embedding vectorial), found_dog_reports (reportes con resultados del matching) | | | | | | | | | | | | | | |
| 7 | Configuración base de la aplicación móvil: Hilt, estructura de carpetas Clean Architecture (feature/, ui/, di/, core/) + SplashScreen + NavHost | | | | | | | | | | | | | | |
| 8 | Pantalla de inicio de sesión - UI y diseño visual: campos de email y contraseña, ícono de mostrar/ocultar contraseña, botón de inicio de sesion, enlace a registro | | | | | | | | | | | | | | |
| 9 | Pantalla de inicio de sesión - Validaciones y autenticación Firebase: validación de email vacío, formato de email inválido, contraseña vacía, mensajes de error específicos por tipo de fallo | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **10** | Pantalla de Registro - UI y diseño: campos nombre, email, contraseña, confirmar contraseña, consistencia visual con el LoginScreen, botón de registro | | | | | | | | | | | | | | | |
| **11** | Pantalla de Registro - Validaciones y autenticación: contraseñas que coincidan, mínimo 8 caracteres, email con formato válido, nombre no vacío, email no registrado previamente. | | | | | | | | | | | | | | | |
| **12** | Pantalla de Inicio UI: mensaje de bienvenida con el nombre real del usuario, accesos directos a las acciones principales de reportar perro perdido, reportar perro encontrado, mis reportes | | | | | | | | | | | | | | | |
| **13** | Sesión persistente: la sesión de la app persiste al cerrar y reabrir la app (SplashScreen verifica y redirige directamente a la pantalla de inicio si hay sesión activa o redirige al inicio de sesion si la sesion fue cerrada por el usuario. | | | | | | | | | | | | | | | |
| **14** | Pantalla de Perfil - UI y visualización de datos: nombre completo, email y teléfono del usuario cargados en tiempo real desde Firestore users y botón de cerrar sesión | | | | | | | | | | | | | | | |
| **15** | Pantalla de reporte de perro perdido en un formulario mas selector de imagen desde la galeria del disporitivo movil. | | | | | | | | | | | | | | | |
| **16** | Validaciones del formulario de la pantalla de reporte de perro perdido con mensajes correspondientes | | | | | | | | | | | | | | | |
| **17** | Pantalla de reporte de perro perdido , integracion con el backend : foto enviada - verifica que sea un perro - extrae el vector - foto subida a Cloudinary - documento guardado en Firestore lost_dogs con todos los campos del formulario + embedding + URL de Cloudinary | | | | | | | | | | | | | | | |
| **18** | Reportar pantalla de perro encontrado UI y validaciones don formularios y selección de imagen desde la galeria. | | | | | | | | | | | | | | | |
| **19** | Reportar pantalla de perro encontrado, integración con el backend: foto enviada - verifica que sea un perro - extrae vector - similitud coseno contra todos los documentos activos en Firestore lost_dogs | | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **20** | Pantalla de resultados, tarjetas de coincidencias con fotos del perro perdido mas el perro encontrado con porcentaje de similitud | | | | | | | | | | | | | | | |
| **21** | Pantalla de mis reportes donde aparece el listado de los perros perdidos o encontrados registrados en la aplicación movil | | | | | | | | | | | | | | | |
| **22** | Manejo de estados en toda la aplicación: ruleta de cargando consistente en todas las pantallas durante peticiones, mensajes descriptivos con mensaje claros | | | | | | | | | | | | | | | |
| **23** | Pantalla de login con autenticación Firebase, validación de que solo cuentas con rol admin pueden ingresar al apartado web | | | | | | | | | | | | | | | |
| **24** | Listado de tarjetas de los perros perdidos en la pagina web | | | | | | | | | | | | | | | |
| **25** | Listado de tarjetas de los perros encontrados en la pagina web | | | | | | | | | | | | | | | |
| **26** | Pantalla de mis reportes donde aparece el listado de los perros perdidos o encontrados registrados en la aplicación movil | | | | | | | | | | | | | | | |
| **27** | Pantalla de mis reportes donde aparece el listado de los perros perdidos o encontrados registrados en la aplicación movil | | | | | | | | | | | | | | | |
| **28** | Reportes generados con opcion a descarga en el sistema web. | | | | | | | | | | | | | | | |
| **29** | Pruebas a la página web y aplicación móvil | | | | | | | | | | | | | | | |
| **30** | Publicacion de la aplicación movil en Google Play Store | | | | | | | | | | | | | | | |
| **30** | Despliegue del backend en railway | | | | | | | | | | | | | | | |
| **31** | Despliegue del frontend en firebase hosting | | | | | | | | | | | | | | | |

Fuente: Elaboración propia, 2026

# APÉNDICE 7

## SOLUCIÓN PROPUESTA

La solución propuesta consiste en el desarrollo de un sistema distribuido compuesto por una aplicación móvil Android y un panel administrativo web, que permita identificar a perros extraviados mediante el uso de redes neuronales convolucionales para reducir su permanencia prolongada en refugios. El diseño responde a la necesidad de modernizar los métodos de identificación manuales (carteles y redes sociales), los cuales presentan un alto margen de error y lentitud en el proceso de reencuentro.

El sistema opera bajo una arquitectura cliente-servidor, donde las interfaces capturan los datos y un servidor centralizado (Backend) ejecuta el procesamiento complejo de inteligencia artificial. De esta manera, se busca optimizar la precisión en la identificación y promover el bienestar animal mediante una plataforma tecnológica robusta y escalable.

**COMPONENTES DEL SISTEMA**

**1. Interfaces de Usuario (Frontend)**

- **Aplicación Móvil:** Orientada a la comunidad para el reporte de perros perdidos o encontrados, permitiendo la carga de fotografías y la recepción de notificaciones de coincidencia.
- **Panel Administrativo Web:** Destinado al personal del Refugio WOOF para la gestión de la base de datos, supervisión de reportes y control de las cuentas de usuario.

**2. Registro de Reportes** Los usuarios podrán hacer reportes de “perro extraviado” o “perro encontrado”, incluyendo fotografías, descripción física (color, tamaño, sexo) y la ubicación aproximada del suceso. Esta información se almacena de forma sincronizada en la base de datos NoSQL de Firebase.

**3. Procesamiento y Extracción Biométrica (Backend)**

- **Preprocesamiento:** Una vez recibida la imagen en el servidor, se aplica la librería OpenCV para normalizar dimensiones, corregir el color y eliminar ruido visual.
- **Identificación Visual:** Mediante la red neuronal EfficientNet-B4, el sistema extrae las características del perro para generar un embedding (huella digital visual).

**4. Comparación y Resultados** El sistema emplea el algoritmo Cosine Similarity para comparar el vector de la nueva fotografía con los registros existentes en la base de datos. Los resultados se despliegan en una lista ordenada de los mejores 5 resultados por porcentaje de similitud, facilitando al usuario la identificación de posibles coincidencias.

**5. Notificaciones Automáticas** Al detectarse una coincidencia que supere el umbral de similitud establecido, el sistema envía notificaciones automáticas a los usuarios involucrados, informando sobre el posible reencuentro y proporcionando los datos necesarios.

**FUNCIONAMIENTO GENERAL DEL SISTEMA**

- El usuario accede a la aplicación móvil o si es el administrador a la plataforma web y selecciona el tipo de reporte.
- Se carga la fotografía y los datos descriptivos, los cuales son enviados al servidor en Railway.
- El backend procesa la imagen con EfficientNet-B4, genera su embedding y realiza la comparación matemática.
- Si se encuentran coincidencias, el sistema devuelve el ranking de similitud y los datos de contacto.
- En caso de coincidencia alta, se activa el protocolo de notificación para coordinar el reencuentro presencial.

**Figura: Solución Propuesta**




Fuente: Elaboración propia, 2026

## APÉNDICE 8

## PRESUPUESTO

La estimación de costos se realizará utilizando la técnica PERT (Program Evaluation and Review Technique), que permite considerar la incertidumbre en los costos de tareas críticas del proyecto.

PERT utiliza tres escenarios para cada tarea:

- Optimista (O): Mejor escenario posible.
- Más probable (M): Escenario más probable basado en experiencia o referencias.
- Pesimista (P): Peor escenario posible.

La fórmula para calcular el costo esperado es:

$$\boldsymbol{Estimación\ PERT = \frac{O + 4M + P}{6}}$$

La desviación estándar se calcula como:

$$\boldsymbol{Desviación\ estandar = \frac{P - O}{6}}$$

**Tabla: Presupuesto del Proyecto**

| RUBRO | O | M | P | ESTIMACIÓN PERT | DESV. ESTÁNDAR |
|---|---|---|---|---|---|
| Impresiones para el informe | 130 | 190 | 230 | 186,67 | 16,67 |
| Impresiones para las entrevistas | 150 | 230 | 280 | 225,00 | 21,67 |
| Paquete de hojas bond | 50 | 70 | 90 | 70,00 | 6,67 |
| Anillados | 70 | 90 | 110 | 90,00 | 6,67 |
| Tableros para entrevistas | 50 | 65 | 85 | 65,83 | 5,83 |
| Materiales | 40 | 60 | 80 | 60,00 | 6,67 |
| Pasajes al caso de estudio | 50 | 70 | 100 | 71,67 | 8,33 |
| Imprevistos | 40 | 60 | 90 | 61,67 | 8,33 |
| **TOTAL** | **580** | **835** | **1.065** | **830,84** | **80,84** |

Fuente: Elaboración propia, 2026

El presupuesto total estimado para llevar adelante este proyecto de grado es de Bs. 830,84 (ochocientos treinta bolivianos con ochenta y cuatro centavos).

Esta cifra cubre los gastos básicos relacionados con la elaboración y presentación del trabajo académico, tales como impresión de informes, entrevistas, materiales de oficina y desplazamientos al caso de estudio.

Los costos se calcularon mediante la técnica PERT (Program Evaluation and Review Technique), considerando tres escenarios posibles (optimista, más probable y pesimista) con el fin de estimar con mayor precisión el gasto esperado y su desviación estándar.

Los rubros incluyen la impresión del informe final, las entrevistas, la compra de hojas bond y materiales de escritura, los tableros para aplicación de entrevistas, el anillado del documento, los pasajes para visitas al Refugio WOOF y una partida destinada a imprevistos.

Este presupuesto se considera razonable y ajustado a las necesidades reales del proyecto, al estar basado en precios promedio de 2026.

# ANEXOS

## ANEXO 1

**Fotografía: Cachorros extraviados en resguardo del Refugio WOOF**



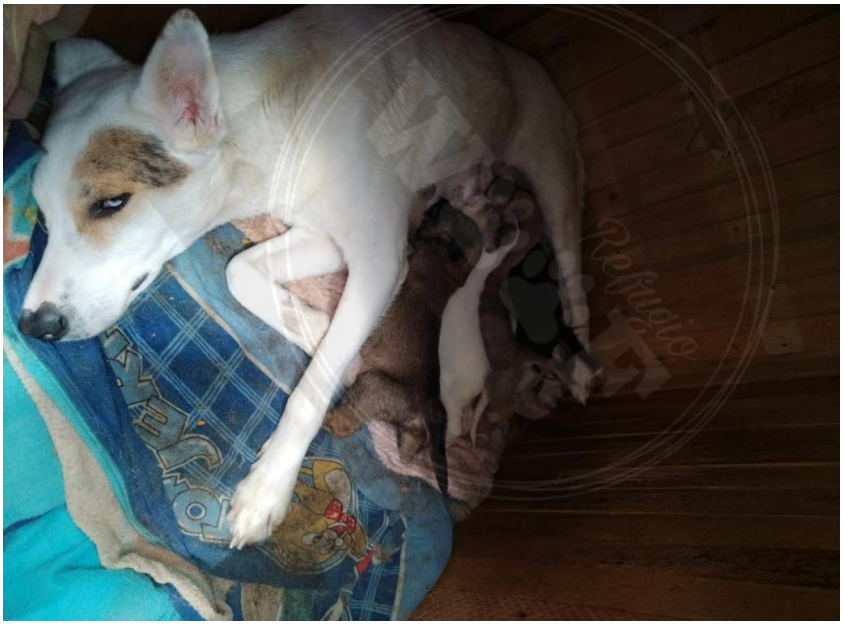
Fuente: Refugio Woof, 2026

**Fotografía: Perritos extraviados rescatados**



Fuente: Refugio Woof, 2026

**Fotografía: Perrita extraviada alojada en el refugio junto a sus crías**



Fuente: Refugio Woof, 2026

**Fotografía: Grupo de perros extraviados recuperados por el Refugio WOOF**



Fuente: Refugio Woof, 2026